O'REILLY®

Aprendendo Git

Um guia prático e visual para os fundamentos do Git



novatec

Anna Skoulikari

Aprendendo Git

Um guia prático e visual para os fundamentos do Git

Anna Skoulikari

O'REILLY
Novatec

Authorized Portuguese translation of the English edition of Learning Git ISBN 9781098133917 © 2023 Anna Skoulikari. This translation is published and sold by permission of O'Reilly Media, Inc., the owner of all rights to publish and sell the same.

Tradução em português autorizada da edição em inglês da obra Learning Git ISBN 9781098133917 © 2023 Anna Skoulikari. Esta tradução é publicada e vendida com a permissão da O'Reilly Media, Inc., detentora de todos os direitos para publicação e venda desta obra.

© Novatec Editora Ltda. [2024].

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor e da Editora.

Editor: Rubens Prates
Assistente editorial: Karina Tsuchiya
Tradução: Aldir Coelho Corrêa da Silva
Revisão gramatical: Tássia Carvalho

ISBN do impresso: 978-85-7522-882-1

ISBN do ebook: 978-85-7522-883-8

Histórico de impressões:

Janeiro/2024 Primeira edição

Novatec Editora Ltda.
Rua Luís Antônio dos Santos 110
02460-000 – São Paulo, SP – Brasil
Tel.: +55 11 2959-6529
E-mail: *novatec@novatec.com.br*
Site: *https://novatec.com.br*
Twitter: *twitter.com/novateceditora*
Facebook: *facebook.com/novatec*
LinkedIn: *linkedin.com/in/novatec*
LIS20231213

Dedico este livro aos meus pais.
Obrigado à minha mãe e ao meu pai por todo o apoio nessa jornada insana que chamamos de vida.

Sumário

Prefácio

1 O Git e a linha de comando

- O que é o Git?
- A interface gráfica de usuário e a linha de comando
- Abrindo uma janela de linha de comando
- Executando comandos na linha de comando
- Instalando o Git
- Opções e argumentos de comandos
- Limpando a linha de comando
- Abrindo a janela do sistema de arquivos
- Trabalhando com diretórios
- Fechando a linha de comando
- Definindo configurações do Git
- Preparando um editor de texto
- Terminais integrados
- Resumo

2 Repositórios locais

- Configuração atual
- Introduzindo os repositórios
- Inicializando um repositório local
- Áreas do Git
- Adicionando um arquivo a um projeto Git
- Resumo

3 Fazendo um commit

- Configuração atual
- Por que fazemos commits?
- As duas etapas para se fazer um commit
- Visualizando uma lista de commits
- Resumo

4 Branches

- Estado do repositório local
- Por que usamos branches?
- Arquivos não modificados e modificados
- Fazendo commits em um branch
- Criando um branch
- O que é HEAD?
- Mudando de branches
- Trabalhando em um branch separado
- Resumo

5 Merging

- Estado do repositório local

Introdução ao merging
Tipos de merges
Fazendo um merge fast-forward
Fazendo o checkout de commits
Criando um branch e mudando para ele ao mesmo tempo
Resumo

6 Serviços de hospedagem e autenticação

Serviços de hospedagem e repositórios remotos
Definindo uma conta de serviço de hospedagem
Definindo credenciais de autenticação
Resumo

7 Criando um repositório remoto e fazendo push para ele

Estado do repositório local
As duas maneiras de começar a trabalhar em um projeto Git
Interação entre repositórios locais e remotos
Por que usar repositórios remotos?
Criando um repositório remoto com dados
Trabalhando em um repositório remoto diretamente em um serviço de hospedagem
Resumo

8 Clonando e buscando

Estado dos repositórios local e remoto
Clonando um repositório remoto
Excluindo branches
Colaboração no Git e branches
Incorporando alterações do repositório remoto
Excluindo branches (continuação)
Resumo

9 Merges three-way

Estado dos repositórios locais e remoto
Por que os merges three-way são importantes?
Definindo um cenário de merge three-way
Definindo branches upstream
Editando o mesmo arquivo várias vezes entre commits
Trabalhando ao mesmo tempo com outras pessoas em arquivos diferentes
Merge three-way na prática
Fazendo o pull de alterações de um repositório remoto
Estado dos repositórios locais e remoto
Resumo

10 Conflitos de merge

Estado dos repositórios locais e remoto
Introdução aos conflitos de merge
Como resolver conflitos de merge

Definindo um cenário de conflito de merge
Processo de solução de conflitos de merge
Resolvendo conflitos de merge na prática
Ficando atualizado com um repositório remoto
Sincronizando os repositórios
Estado dos repositórios locais e remoto
Resumo

11 Rebasing

Estado dos repositórios locais e remoto
Integrando alterações no Git
Por que o rebasing é útil?
Definindo o exemplo de rebasing
Preparando arquivos e removendo-os da preparação
Preparando-se para o rebasing
Os cinco estágios do processo de rebasing
Rebasing e conflitos de merge
Rebasing de um branch na prática
Regra de ouro do rebasing
Sincronizando os repositórios
Estado dos repositórios locais e remoto
Resumo

12 Pull Requests (Merge Requests)

Estado dos repositórios locais e remoto
Introdução aos pull requests
Particularidades dos serviços de hospedagem
Por que usar Pull Requests?
Entendendo como os Pull Requests são integrados
Preparando-se para fazer um pull request
Uma maneira mais fácil de definir branches upstream
Criando um pull request em um serviço de hospedagem
Revisando e aprovando um pull request
Fazendo o merge de um pull request
Excluindo branches remotos
Sincronizando os repositórios locais e fazendo a limpeza
Estado dos repositórios locais e remoto
Resumo

Epílogo

A Pré-requisitos dos capítulos

Pré-requisitos de configuração para todos os capítulos
Pré-requisitos de configuração do Capítulo 2
Pré-requisitos de configuração do Capítulo 3
Pré-requisitos de configuração do Capítulo 4
Pré-requisitos de configuração do Capítulo 5
Pré-requisitos de configuração dos capítulos 6 e 7
Pré-requisitos de configuração do Capítulo 8
Pré-requisitos de configuração do Capítulo 9
Pré-requisitos de configuração do Capítulo 10
Pré-requisitos de configuração do Capítulo 11
Pré-requisitos de configuração do Capítulo 12

B Consulta rápida de comandos

C Referência sobre a linguagem visual

Commits
Diagrama Git
Diagrama Repositório
Sobre a autora
Colofão

Prefácio

Nunca pensei que fosse escrever um livro que ensinasse a usar o Git. No entanto, graças a uma feliz sequência de eventos, tive uma ideia bastante original de como poderia ensinar de maneira simples como essa tecnologia funciona.

Minha jornada começou quando participei de um bootcamp[1] de codificação para aprender sobre desenvolvimento web. Os instrutores do bootcamp apresentaram brevemente o Git para os alunos, mas, como todos os nossos projetos eram feitos individualmente, ele não foi muito usado.

Após o bootcamp de codificação, consegui um emprego como desenvolvedor front-end júnior para trabalhar em um site de uma grande empresa. Minha jornada real de aprendizado do Git começou no primeiro dia em meu novo emprego. Nos primeiros meses, atuando como parte de uma equipe em uma empresa de grande porte, fiquei apavorado. Sempre que precisava fazer algo que parecia, mesmo que remotamente, complicado usando o Git, achava que iria destruir o repositório ou que cometeria algum erro grave.

Para trabalhar apropriadamente com meus colaboradores, decidi aprender por conta própria as vantagens e desvantagens do Git. Contudo, enquanto consultava vários recursos online, rapidamente ficou claro que a maioria dos materiais disponíveis não tinha sido projetada para iniciantes. Depois que entendi os aspectos básicos, começou a se formar em minha mente a ideia de que eu poderia ensinar essa tecnologia de maneira mais simples usando imagens e cores.

Acabei criando um curso online e fazendo seu upload na web. Enquanto preparava o curso, aos poucos fui pensando na possibilidade de algum dia também escrever um livro sobre o assunto.

Recebi muitos feedbacks positivos sobre o curso, e finalmente, no verão de 2021, decidi que era hora de começar esse projeto. O livro que você está lendo agora é resultado dessa decisão, e espero que ele o ajude em sua

jornada de conhecimento do Git!

Para quem é este livro

Este livro é para qualquer pessoa que queira aprender os aspectos básicos de como o Git funciona. Ele foi projetado principalmente para pessoas que estejam começando a aprender habilidades técnicas, ou que trabalhem em funções não técnicas, mas precisem usar o Git para colaborar com seus colegas da área técnica. Alguns exemplos de pessoas que podem se beneficiar do livro seriam (mas não apenas) os alunos de bootcamps de codificação, alunos de ciência da computação, escritores da área técnica, gerentes de produto, designers, desenvolvedores juniores, cientistas de dados e programadores autodidatas.

O livro foi escrito para pessoas que não têm nenhuma, ou têm pouca, experiência no uso do Git. Se você não tem experiência no uso do Git, isso não será um problema, porque este livro começa a partir do nível mais básico. Inicialmente, instalaremos o Git, veremos como usar a linha de comando, e prosseguiremos a partir daí.

Se você já tem alguma experiência no uso do Git ou da linha de comando, o primeiro capítulo pode parecer uma revisão. No entanto, recomendo que não o salte porque ele definirá o projeto Rainbow que usaremos no restante do livro.

Usando o livro

Este livro é uma experiência de aprendizado prática, na qual você fará exercícios em seu computador enquanto aprende os conceitos básicos do Git. No decorrer do livro, você encontrará dois projetos: o projeto Rainbow e o projeto Book.

O projeto Rainbow é um projeto de treinamento prático no qual você trabalhará fazendo os exercícios fornecidos no livro. É um projeto simplificado pensado apenas para fins de aprendizado. O projeto Book é um projeto imaginário que usarei para demonstrar como certos recursos do Git podem ser utilizados em um projeto mais realista. Veremos cada um deles com mais detalhes e examinaremos a maneira como o livro foi estruturado.

[NOTA]

Não se preocupe se encontrar neste prefácio termos com os quais ainda não está familiarizado, como *repositório* e *commit*. Explicarei todos esses conceitos nos próximos capítulos.

PROJETO RAINBOW

No decorrer deste livro, você trabalhará no projeto Rainbow para aprender os conceitos básicos do Git. Para fazer o acompanhamento, deve ler o livro de maneira linear do Capítulo 1 ao Capítulo 12 e fazer todos os exercícios no computador. Por exemplo, os exercícios do Capítulo 4 presumirão que você já fez os exercícios dos capítulos 1, 2 e 3.

Repositórios

Em um nível básico, um *repositório* é uma cópia de um projeto Git. Inicialmente, você criará um único repositório local chamado rainbow para trabalhar no projeto Rainbow. Depois (no Capítulo 7), criará um repositório remoto chamado rainbow-remote. E, para concluir, no Capítulo 8 simulará que está colaborando com um amigo no projeto, e criará um segundo repositório local chamado friend-rainbow. Do Capítulo 8 em diante, sempre que for feita uma referência ao seu “amigo” estar fazendo algo, você terá de executar a ação no repositório friend-rainbow.

[NOTA]

Quando mencionar o projeto Rainbow com R maiúsculo, estarei me referindo ao projeto inteiro que começa com um repositório e termina com três. Quando me referir ao diretório de projeto rainbow ou ao repositório rainbow (com r minúsculo), estarei indicando o repositório local específico que faz parte do projeto Rainbow.

Commits

No projeto Rainbow, você criará e editará arquivos para listar as cores do arco-íris e algumas cores que não fazem parte dele. Esse exemplo não foi pensado para ser um caso realista de um projeto controlado com versões pelo Git. É um projeto simplificado que permitirá que você dê ênfase ao aprendizado, e não a criar algo complicado.

No decorrer do livro, usarei diagramas para ilustrar o que está ocorrendo no projeto Rainbow. Sempre que você adicionar uma cor ao projeto Rainbow, fará um *commit* em um repositório. Basicamente, um commit representa uma versão do projeto. Nos diagramas, o commit será representado por um círculo com a cor que você adicionou, e também usaremos o nome da cor como o nome do commit. Por exemplo, a primeira cor que você adicionará ao seu projeto Rainbow será o vermelho; logo, o círculo que representará esse commit terá a cor red (vermelha) e nos referiremos a ele como o commit red (vermelho).

Para tornar o livro acessível para leitores com deficiência na visão de cores, incluirei o nome do commit (ou uma abreviação do nome) nos diagramas. Consulte a Figura P-1 para ver um exemplo do commit red (vermelho).



Figura P-1: Exemplo de um commit com seu nome completo.

A Tabela P-1 contém uma lista de todos os commits que você fará no projeto Rainbow deste livro, mostrando seus nomes completos e abreviações.

Tabela P-1: Lista completa de commits feitos no projeto Rainbow

Nome completo do Commit	Abreviação do Commit
red (vermelho)	R
orange (laranja)	O

Nome completo do Commit	Abreviação do Commit
yellow (amarelo)	Y
green (verde)	G
blue (azul)	B
brown (marrom)	Br
commit de mesclagem (merge) 1	M1
indigo	I
violet (violeta)	V
commit de mesclagem (merge) 2	M2
gray (cinza)	Gr
black (preto)	Bl
arco íris (rainbow)	Ra
pink (rosa)	P
commit de mesclagem (merge) 3	M3

A Figura P-2 é um diagrama de todos os commits.

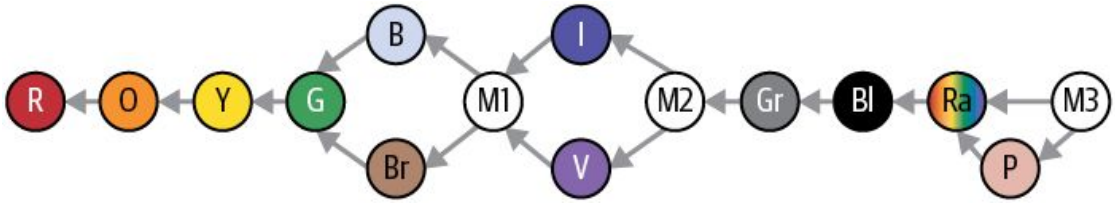


Figura P-2: Os 15 commits que você terá feito no projeto Rainbow ao final deste livro.

Embora o livro tenha sido projetado para ser lido do Capítulo 1 ao Capítulo 12 de maneira linear, podem ocorrer algumas situações em que você talvez queira ou precise começar a leitura a partir de um capítulo específico. Por exemplo:

- Você fez uma vez os exercícios do livro inteiro e deseja rever o trabalho a partir de um capítulo específico.
- Algo que não foi possível solucionar deu errado no projeto Rainbow em um capítulo anterior e você deseja começar do zero em um novo capítulo.

Nesse caso, pode usar as instruções do Apêndice A para recriar como seria o projeto Rainbow no início do capítulo a partir do qual deseja começar.

O Apêndice B contém um guia de consulta rápida com os comandos introduzidos em cada capítulo.

O Apêndice C é um guia sobre a linguagem visual usada nos diagramas do livro.

PROJETO BOOK

O projeto Book é um projeto imaginário que usarei para demonstrar como o Git pode ser usado em projetos mais realistas. Nesse projeto, simularemos que estou escrevendo um livro e desejo usar o Git para controlar as versões dos arquivos. O livro será composto de 10 capítulos representados por 10 arquivos de texto, um para cada capítulo: `chapter_one.txt`, `chapter_two.txt` e assim por diante. Em alguns momentos, também simularei como seria trabalhar no projeto Book com um coautor e/ou um editor. Essas discussões ocorrerão nas seções *Exemplo do projeto Book*.

Você *não* trabalhará ativamente no projeto Book ou o criará. Ele só será usado para fornecer exemplos e descrições adicionais de como certos recursos do Git são utilizados.

Além das seções Exemplo do projeto Book do livro, você também encontrará outras seções. Elas estão descritas a seguir.

SEÇÕES DO LIVRO

Aqui está um guia rápido dos diferentes tipos de seções que você encontrará neste livro:

Exemplo do projeto Book

Como mencionado anteriormente, as seções Exemplo do projeto Book fornecem contexto e exemplos adicionais sobre o uso de recursos e comandos do Git, baseados no projeto Book.

Acompanhe

As seções Acompanhe apresentarão listas numeradas de etapas que você terá de executar em seu computador. Se uma etapa incluir um comando em negrito, será preciso inserir e executá-lo na linha de comando. Exemplos de saída serão fornecidos para todos os comandos que produzirem saídas. Essa saída será baseada no projeto Rainbow em que trabalhei ao criar este livro; ela foi gerada pelo sistema operacional

macOS, mas a saída de comandos do Git deve ser igual no Microsoft Windows. Quando existirem diferenças significativas entre a saída de um comando no Microsoft Windows e no macOS, isso será mencionado no texto.

Salve o comando

As seções Salve o comando introduzirão comandos úteis, alguns dos quais você usará nas seções Acompanhe.

Uma lista completa de todos os comandos importantes agrupados por capítulo também está disponível para consulta no Apêndice B.

Visualize

As seções Visualize mostrarão diagramas do que está ocorrendo no projeto Rainbow. Dois diagramas importantes que serão usados nessas seções (assim como nas figuras) são o *Diagrama Git* e o *Diagrama Repositório*. Introduzirei o Diagrama Git no Capítulo 2 e começarei a usar o Diagrama Repositório no Capítulo 4. Cada diagrama do livro será criado por etapas, com explicações no texto.

Um resumo da linguagem visual usada em todo o livro também está disponível para consulta no Apêndice C.

Nota

As seções Nota fornecerão informações úteis relacionadas ao material do texto.

REPOSITÓRIO DO LIVRO

Embora minha intenção fosse fornecer dentro do próprio livro a maioria das informações necessárias para você viver a experiência de *Aprendendo Git*, algumas tecnologias e processos mudam com muita frequência para poderem ser documentados no livro. Criei o repositório público *Aprendendo Git* (https://github.com/gitlearningjourney/learning-git) no GitHub para oferecer informações atualizadas sobre essas tecnologias e processos. Entre outras coisas, o repositório contém:

- Informações sobre o download do Git.
- Links para recursos relacionados ao trabalho com serviços de hospedagem.

- Informações sobre a definição de acesso HTTPS ou SSH a repositórios remotos.

No decorrer do livro, mencionarei quando ocorrerem situações em que talvez você precise consultar o repositório de *Aprendendo Git* para obter mais orientações.

O QUE ESTE LIVRO NÃO É

Este não é um livro de consulta. Não abordarei cada um dos comandos do Git (existem muitos!). Ele também não é um guia avançado do Git. Existem muitos recursos do Git que não serão discutidos neste livro por não serem necessários para a execução das ações básicas que desejo ensiná-lo a executar. Fui bastante seletiva em relação ao que incluiria no livro. Minha intenção é fornecer um modelo mental claro dos aspectos básicos do Git para que, tendo uma base sólida já estabelecida, você possa continuar o aprendizado e conhecer qualquer recurso adicional de que precisar.

O livro também não lhe dirá qual deve ser seu fluxo de trabalho do Git ou como você deve usar os recursos do Git. Tentei o máximo possível evitar dar qualquer opinião sobre essas questões e em vez disso me concentrei em ensinar como a ferramenta funciona. Dependendo de seu contexto e preferências individuais, a ferramenta poderá ser usada de diferentes maneiras.

Onde possível, este livro tenta não ser normativo. Por exemplo, você poderá usar o editor de texto ou o serviço de hospedagem que preferir para fazer os exercícios do livro.

RESUMO DOS CAPÍTULOS

O livro foi dividido em duas partes lógicas. Na primeira parte (capítulos 1 a 5), você aprenderá a trabalhar com repositórios locais em seu computador. Nos capítulos de 6 a 12, também aprenderá a trabalhar com serviços de hospedagem e repositórios remotos. A seguir temos um breve resumo do que abordaremos em cada capítulo:

- No Capítulo 1, você se preparará para trabalhar em um projeto com o uso do Git instalando-o, aprendendo alguns aspectos básicos da linha de comando, preparando configurações do Git, criando um dos diretórios que usará no restante do livro e preparando um editor de texto.

- No Capítulo 2, transformará seu diretório de projeto em um repositório Git. Introduzirei o Diagrama Git para representar as diferentes áreas do Git, incluindo o diretório de trabalho, a área de preparação (staging area), o histórico de commits e o repositório local. No fim do capítulo, você criará o primeiro arquivo em seu diretório de projeto.
- No Capítulo 3, você conhecerá e executará as duas etapas principais para fazer o primeiro commit em seu repositório local.
- No Capítulo 4, conhecerá os branches (ramificações): o que são eles, como criá-los, como se alternar entre branches e como identificar em que branch você está.
- No Capítulo 5, você conhecerá os dois tipos de merge e executará um merge fast-forward.
- No Capítulo 6, se preparará para trabalhar com repositórios remotos selecionando um serviço de hospedagem e definindo detalhes de autenticação para se conectar com esses repositórios pelo HTTPS ou SSH.
- No Capítulo 7, discutiremos as diferentes maneiras de trabalhar com repositórios locais e remotos. Você aprenderá a criar um repositório remoto e a fazer o upload de dados nele.
- No Capítulo 8, começaremos a simular como é trabalhar com outras pessoas em um projeto Git. Para fazê-lo, você criará um segundo repositório local; simularemos que o repositório pertence a um amigo que o ajudará no projeto Rainbow e está instalado no computador dele. Durante o processo, você aprenderá a clonar repositórios remotos e a buscar dados.
- No Capítulo 9, você executará um merge three-way e conhecerá a diferença entre buscar dados (fetching data) e extrair dados (pulling data).
- No Capítulo 10, percorreremos um exemplo de solução de conflitos de merge durante um merge three-way.
- No Capítulo 11, você conhecerá o rebasing. Trata-se de uma maneira alternativa ao uso de merge para incorporação de alterações de um branch em outro branch.

- No Capítulo 12, conheceremos as pull requests (também conhecidas como merge requests) e veremos como elas facilitam a colaboração em projetos Git.
- O livro também inclui três apêndices, cujo conteúdo é discutido em "Apêndices" na página 13.

Convenções usadas neste livro

As convenções tipográficas a seguir foram usadas no livro:

Itálico

Indica novos termos, URLs, endereços de email e ênfase.

Largura constante

Usada para comandos, saídas de comandos, nomes de arquivo, nomes de diretório, nomes de branches, nomes de repositórios e outros textos exibidos na tela.

Largura constante em negrito

Mostra comandos ou outros tipos de texto que devem ser digitados ou inseridos pelo usuário.

<Largura constante em colchetes angulares>

Mostra texto que deve ser substituído por valores fornecidos pelo usuário ou determinados pelo contexto.

Como entrar em contato conosco

Envie comentários e dúvidas sobre este livro para: *novatec@novatec.com.br*.

Temos uma página da web para este livro, na qual incluímos a lista de erratas, exemplos e qualquer outra informação adicional.

- Página da edição em português

  http://novatec.com.br/livros/aprendendo-git
- Página da edição original, em inglês

  https://oreil.ly/learning-git
- Página do livro no GitHub

  https://github.com/gitlearningjourney/learning-git

Para obter mais informações sobre livros da Novatec, acesse nosso site em:

https://novatec.com.br

Agradecimentos

Gostaria de agradecer a todas as pessoas que me apoiaram durante o processo de escrever este livro. Isso inclui meus amigos e família, alguns dos quais forcei a testar como usuários versões anteriores do livro. Também inclui toda a equipe da O'Reilly, que me ajudou no decorrer desse projeto de criação e, para concluir, todos os revisores técnicos e usuários testadores que leram várias versões do livro e me forneceram inestimável feedback. Ah, sim, agradeço também ao meu parceiro, que foi paciente nas diversas vezes em que posterguei a data de publicação!

---

1 N.T.: Um bootcamp é um tipo de treinamento intensivo e acelerado, em um curto intervalo de tempo, com imersão completa no aprendizado, para adquirir habilidades específicas.

1

O Git e a linha de comando

Neste capítulo, explicarei o que é o Git e para que ele é usado, e você o instalará em seu computador. Também definiremos algumas de suas variáveis de configuração (também chamadas de settings). Além disso, introduzirei a interface gráfica de usuário e a linha de comando, que são as duas ferramentas que utilizaremos para interagir com o Git e com o exemplo de projeto Rainbow. Para que você se sinta à vontade para trabalhar na linha de comando, veremos como executar algumas ações básicas como visualizar a localização do diretório atual, entrar e sair de diretórios, e criar diretórios. Para concluir, no fim do capítulo, você preparará o editor de texto que usará para trabalhar no projeto Rainbow no Capítulo 2.

Se já tem experiência em trabalhar na linha de comando, pode já conhecer algumas informações que serão fornecidas no capítulo. No entanto, não recomendo que o salte porque ele preparará a configuração que usaremos no resto do livro.

[NOTA]

Para saber como este livro deve ser utilizado, você precisa ler “Usando o livro”, na página 10. Se não leu essa seção, é altamente recomendável que o faça agora.

O que é o Git?

O Git é uma tecnologia que pode ser usada para rastrear as alterações feitas em um projeto e permitir que várias pessoas forneçam suas colaborações. Em um nível básico, a versão de um projeto controlada pelo Git é composta de uma pasta com arquivos, e o Git rastreia as alterações que são feitas nos arquivos do projeto. Isso nos permite salvar diferentes versões do trabalho que estamos executando e é o que faz o Git ser chamado de sistema de controle de versões.

O Git foi criado por Linus Torvalds a fim de que ele pudesse controlar com versões o trabalho feito em um grande projeto de desenvolvimento de software chamado kernel Linux. Contudo, já que consegue rastrear alterações feitas em todos os tipos de arquivos, pode ser empregado em uma grande variedade de projetos.

É uma ferramenta poderosa, e como fornece muitos recursos – além do fato de ter sido projetado originalmente para uso na linha de comando – utilizá-lo pode ser um pouco mais complicado do que simplesmente selecionar Arquivo→Salvar em seu computador.

Resumindo, o Git é um sistema de controle de versões que você pode baixar em seu computador e que nos permite rastrear o histórico de um projeto e colaborar com outras pessoas. A seguir, examinaremos o Exemplo 1-1 do Projeto Book para ver uma demonstração de como eu poderia usar o Git para o meu projeto Book.

Exemplo 1-1 do Projeto Book

Suponhamos que eu estivesse escrevendo um livro e quisesse usar o Git para controlar com versões todos os arquivos desse projeto, chamado Book. Sempre que fizer alterações no livro, poderei salvar uma versão dele utilizando o Git. Por exemplo, digamos que eu fizesse alterações na segunda-feira, na quarta-feira e na sexta-feira e salvasse uma versão em cada um desses dias. Isso significa que terei pelo menos três versões de meu projeto. Uma versão de um projeto no Git é chamada de *commit*. No Capítulo 2, você aprenderá mais sobre os commits. Por enquanto, só precisa saber que em meu exemplo tenho pelo menos três commits.

Esses três commits me permitem examinar as diferentes versões do livro obtidas no fim de

minha sessão de trabalho na segunda-feira, na sessão de trabalho de quarta e na sessão de sexta. O Git também permite comparar qualquer um desses commits (ou versões salvas de meu projeto) para verificar o que mudou entre as diferentes versões. Isso ilustra como o Git me ajudaria a monitorar o histórico do projeto.

Agora suponhamos que eu decidisse trabalhar no projeto Book com um coautor. O Git permitiria que eu e meu coautor trabalhássemos no projeto ao mesmo tempo e juntássemos nossos trabalhos ao terminar. Por exemplo, eu trabalharia no Capítulo 1 enquanto o coautor trabalharia no Capítulo 2, e quando estivéssemos prontos, uniríamos os trabalhos.

Se um editor fosse revisar o livro, ele também poderia fazer edições em todos os capítulos que foram escritos, e essas alterações seriam integradas à versão principal. Isso mostra como o Git é uma ferramenta útil para a colaboração.

Conheceremos agora algumas das outras ferramentas que serão usadas nessa experiência de aprendizado e como você interagirá com o Git.

A interface gráfica de usuário e a linha de comando

As duas principais maneiras de interagir com um computador são usando a interface gráfica de usuário ou a linha de comando.

A GUI (graphical user interface, interface gráfica de usuário) é o conjunto de representações gráficas de objetos (ícones, botões etc.) que nos permite interagir com o computador. Podemos considerá-la uma interface de apontar e clicar. Por exemplo, os ícones de pasta que representam as pastas em seu desktop fazem parte da GUI do computador.

A linha de comando – também conhecida como CLI (command line interface, interface de linha de comando), terminal ou shell – é um local em que podemos digitar comandos baseados em texto para interagir com o computador.

A maneira padrão de trabalhar com o Git é por meio da linha de comando. No entanto, também existem maneiras de trabalhar usando uma GUI: por exemplo, utilizando um cliente Git com GUI (Git GUI client) ou um editor de texto que tenha integrações com o Git. Isso significa que você pode executar ações do Git clicando em botões e selecionando opções em vez de inserir comandos na linha de comando.

Neste livro, você aprenderá a usar o Git na linha de comando porque isso lhe permitirá desenvolver um modelo mental sólido de como ele funciona e

dará acesso a todas as suas funcionalidades. A GUI de seu computador só será utilizada para outras ações, por exemplo, examinar arquivos no sistema de arquivos ou trabalhar no editor de texto para gerenciar arquivos. Conheceremos a linha de comando com mais detalhes na próxima seção.

[NOTA]

No decorrer do livro, fornecerei instruções específicas para usuários do macOS e do Microsoft Windows. Se você é usuário do Linux, presumirei que já conhece alguns aspectos básicos da linha de comando.

Abrindo uma janela de linha de comando

Para usar a linha de comando, você deve abrir uma janela de linha de comando utilizando uma aplicação de linha de comando. A qualquer momento, em uma janela de linha de comando, você estará em um diretório específico, que chamamos de *diretório atual*. Um diretório, para nossa finalidade, será o mesmo que uma pasta.

Quando você abrir uma janela de linha de comando, haverá um prompt de comando no canto superior esquerdo. Trata-se de um texto pequeno cujo conteúdo exato varia dependendo do sistema operacional e das configurações do computador. Por padrão, entretanto, o prompt de comando indica o local do diretório que está na linha de comando (em outras palavras, seu diretório atual). Na abertura de uma janela de linha de comando, o local do diretório começa no diretório de usuário atual (também conhecido como pasta home), que é representado pelo símbolo til (~). Essa localização de diretório será a única parte importante do prompt de comando que você terá de identificar para os exercícios deste livro. Depois do prompt de comando, aparecerá um cursor para indicar onde você está digitando na linha de comando.

Consulte a Figura 1-1 para ver um exemplo de um prompt de comando genérico com anotações. Nos exemplos do livro, usaremos um cifrão ($) no fim do prompt de comando, mas esse é apenas um modo de um prompt de comando terminar. Talvez seu prompt de comando termine com um caractere ou símbolo diferente.

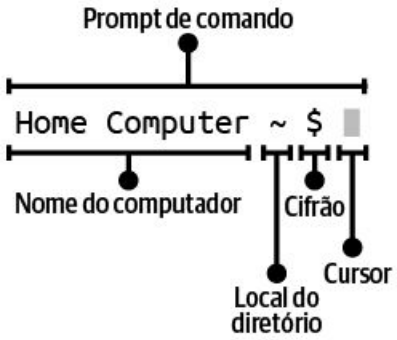


Figura 1-1: Exemplo de um prompt de comando.

A aplicação de linha de comando que você usará para fazer os exercícios do livro vai depender do sistema operacional que estiver sendo utilizado:

- macOS – a aplicação de linha de comando se chama Terminal.
- Microsoft Windows – a aplicação de linha de comando se chama Git Bash. Ela só estará disponível se o Git estiver instalado em seu computador.

[NOTA]

Se você é usuário do Microsoft Windows e não está com o Git instalado, terá de ir até o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) e seguir as etapas de instalação do Git para Microsoft Windows para ter acesso ao Git Bash antes de poder percorrer o restante do capítulo.

Se é usuário do macOS e não está com o Git instalado, pode ler o restante da seção. Você instalará o Git em “Instalando o Git”, na página 25.

Para abrir uma janela de linha de comando, você pode usar a função de busca de seu computador e procurar, selecionar e abrir a aplicação de linha de comando. Consulte Acompanhe 1-1 para abrir uma janela de linha de comando e visualizar o prompt de comando.

[ACOMPANHE 1-1]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ Veja o prompt de comando na janela de linha de comando.

O que deve ser observado:

- O prompt de comando indica a localização do diretório.

Agora que você abriu uma janela de linha de comando, veremos como executará seu primeiro comando na linha de comando.

Executando comandos na linha de comando

No fim do prompt de comando, na janela de linha de comando, é exibido o cursor, que indica onde digitaremos os comandos. Por padrão, no Terminal (macOS), o cursor fica na mesma linha do prompt de comando, enquanto no Git Bash (Microsoft Windows) ele fica na linha de baixo. Depois que você digitar um comando, para executá-lo terá de pressionar a tecla Enter (Return).

Se uma etapa em uma seção Acompanhe deste livro incluir um comando em negrito após um cifrão (`$`), você deve executá-lo na linha de comando. Se ele produzir saída, esta será exibida abaixo do comando (sem estar em negrito). Se o comando tiver de ser executado em um diretório que não seja o diretório de usuário atual, o local será indicado antes do cifrão.

A Figura 1-2 mostra um exemplo do que você verá quando executar um comando em uma seção Acompanhe. Nesse caso, estou executando o comando `pwd` a partir do diretório `desktop`. Você aprenderá o que o comando `pwd` faz posteriormente neste capítulo; por enquanto, consulte a Figura 1-2 para identificar onde é mostrada a localização do diretório, o comando a ser executado e a saída.

Na versão impressa deste livro, alguns comandos mais longos têm quebra de linha devido a restrições de largura de página. Se você encontrar uma linha de comando como esta:

```
rainbow $ git remote add origin https://github.com/gitlearningjourney/rainbow-remote.git
```

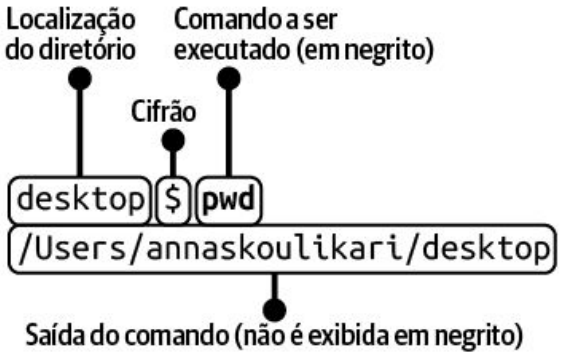


Figura 1-2: Como executar um comando incluído em uma seção Acompanhe.

deve inserir o comando inteiro na mesma linha, com um único espaço antes da parte separada (nesse caso, a URL). Lembretes serão incluídos nos locais apropriados. Se você estiver lendo uma versão do livro que não seja afetada por essa restrição de tamanho da linha, ignore os lembretes.

SAÍDA DOS COMANDOS

Alguns comandos produzem saída e outros não. Para os comandos que produzirem saída, fornecerei um exemplo de saída baseado no projeto Rainbow em que trabalhei ao criar este livro. Essa saída terá sido produzida pelo sistema operacional macOS, mas a saída para comandos relacionados ao Git não diferem entre os sistemas operacionais. Nos poucos casos em que a saída de comandos não relacionados ao Git diferirem entre os sistemas operacionais de maneira significativa, destacarei isso no texto.

Se, ao fazer os exercícios de uma seção Acompanhe, você gerar uma saída que pareça muito diferente da exibida neste livro ou ocorrer um erro inesperado, pode ter feito algo não condizente com o indicado pelas instruções e talvez precise rever as etapas da seção Acompanhe.

[NOTA]

É possível que o Git passe por atualizações no futuro que afetem um pouco as saídas. Se eu tomar conhecimento de alguma alteração significativa, pretendo documentar essas situações na página de errata do repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*).

De agora em diante, sempre que você encontrar um comando em um exercício da seção Acompanhe, deve inserir o comando que está em negrito na linha de comando e executá-lo.

EXECUTANDO O PRIMEIRO COMANDO NA LINHA DE COMANDO

O primeiro comando que você tentará executar na linha de comando é `git version`. Se o Git estiver instalado em seu computador, o comando fornecerá o número da versão que você instalou. Se não estiver com o Git instalado no computador, a saída fornecerá uma mensagem indicando que ele não está instalado.

Para ter acesso a todos os comandos usados nos exercícios deste livro, recomendo que você utilize uma versão do Git superior à versão 2.28. Vá até a seção Acompanhe 1-2 para verificar se está com o Git instalado e saber qual é a versão.

[ACOMPANHE 1-2]

❶ $ **git version**
git version 2.35.1

O que deve ser observado:

- Se você estiver com o Git instalado, verá a versão que está sendo usada em seu computador.
- Se o Git não estiver instalado, aparecerá uma mensagem de erro.

Se a saída do comando `git version` indicar que você tem uma versão do Git instalada que é maior que a 2.28, se quiser salte a seção “Instalando o Git” e vá para a seção “Opções e argumentos de comandos”.

Se a saída do comando `git version` indicar que você não está com o Git instalado em seu computador ou que a versão do Git é mais antiga que a 2.28, passe para a próxima seção para instalar uma versão atualizada do Git no computador.

Instalando o Git

Se você ainda não tiver o Git (versão 2.28 ou superior) instalado, consulte Acompanhe 1-3. Caso contrário, passe para a próxima seção.

[ACOMPANHE 1-3]

❶ Acesse o repositório de *Aprendendo Git* (https://github.com/gitlearningjourney/learning-git) e siga as etapas para baixar o Git para seu sistema operacional.

Agora que você instalou uma versão atualizada do Git no computador, continuaremos aprendendo um pouco mais sobre os comandos usados nas seções Acompanhe deste livro.

Opções e argumentos de comandos

Haverá situações em que você usará comandos com opções e/ou argumentos. Opções são configurações que alteram o comportamento de um comando. Uma opção vem depois de um único traço (-) ou de um traço duplo (--).

Argumentos são valores que fornecem informações para o comando. Eles serão representados por colchetes angulares (<>), indicando que esses itens devem ser substituídos por valores inseridos pelo usuário. Nos exercícios, você terá de passar um valor para o argumento, sem incluir os colchetes angulares.

Um exemplo de um comando com uma opção e um argumento que você usará é git commit -m "<*mensagem*>". Nesse exemplo, -m é a opção e <*mensagem*> é o argumento, como mostrado na Figura 1-3. Abordaremos o que esse comando faz no Capítulo 3.



Figura 1-3: Exemplo de um comando com uma opção e um argumento.

Além da inserção de comandos na linha de comando, também é importante aprender a removê-los.

Limpando a linha de comando

Sempre que você inserir um comando na janela de linha de comando, ele será listado diretamente abaixo do comando que foi inserido anteriormente (ou abaixo de sua saída). Após ter inserido vários comandos, a janela de

linha de comando ficará muito cheia. Para remover o conteúdo da janela, você pode usar o comando clear.

[SALVE O COMANDO]

clear

Limpa a janela de linha de comando.

Consulte a seção Acompanhe 1-4 para ver como o comando clear é usado.

[ACOMPANHE 1-4]

❶ $ **clear**

O que deve ser observado:

- A janela de linha de comando está limpa.

Abordamos a inserção de comandos e sua remoção na janela de linha de comando. É hora de preparar a próxima ferramenta que o ajudará em sua jornada de aprendizado, a janela do sistema de arquivos.

Abrindo a janela do sistema de arquivos

Você usará uma aplicação para abrir a janela do sistema de arquivos, que faz parte da GUI. No decorrer de nossa jornada de aprendizado, interagiremos tanto com a janela do sistema de arquivos quanto com a janela de linha de comando. Logo, será útil termos as duas janelas abertas lado a lado na tela do computador.

A aplicação de sistema de arquivos que você usará vai depender de seu sistema operacional:

- macOS – a aplicação de sistema de arquivos é o Finder.
- Microsoft Windows – a aplicação de sistema de arquivos é o File Explorer (ou Windows Explorer).

Consulte a seção Acompanhe 1-5 para abrir uma janela de sistema de arquivos.

[ACOMPANHE 1-5]

❶ Encontre sua aplicação de sistema de arquivos e abra a janela do sistema de arquivos perto da janela de linha de comando.

Agora que você está com as duas janelas abertas, voltaremos a alguns aspectos básicos da linha de comando.

Trabalhando com diretórios

Como mencionado anteriormente, você sempre estará em um diretório específico (o diretório atual) em uma janela de linha de comando. Supondo que não tenha alterado nenhuma configuração padrão, quando abrir a aplicação de linha de comando pela primeira vez, inicialmente estará no diretório de usuário atual (sua pasta home), indicado pelo símbolo til (`~`) no prompt de comando.

Quando navegar para outros diretórios na linha de comando, o prompt de comando mudará para indicar o diretório em que você está. Você também pode usar o comando `pwd`, que é a abreviação de “print working directory” (exibir diretório de trabalho), para ver o caminho do diretório atual.

[SALVE O COMANDO]

pwd

Mostra o caminho do diretório atual.

Consulte a seção Acompanhe 1-6 para ver como o comando pwd é usado.

[ACOMPANHE 1-6]

❶ $ **pwd**

/Users/annaskoulikari

O que deve ser observado:

- Você está em seu diretório de usuário atual.

[NOTA]

Na seção Acompanhe 1-6, a saída do comando pwd para usuários do Microsoft Windows será semelhante a /c/Users/annaskoulikari, que é um pouco diferente da saída para usuários do macOS. Lembre-se disso ao percorrer os outros exercícios deste capítulo.

A saída do comando pwd exibe o caminho do diretório atual. Na seção Acompanhe 1-6, /Users/annaskoulikari é um exemplo de caminho. Meu nome é Anna Skoulikari e annaskoulikari é meu nome de usuário no computador. Users e annaskoulikari são dois diretórios. Os diretórios de um caminho são separados por uma barra (/). O diretório annaskoulikari está dentro do diretório Users.

É útil conhecer a localização de um diretório em uma janela de linha de comando porque muitos comandos exibem informações sobre ou afetam o diretório atual quando os executamos. Também ajuda a navegar pelo sistema de arquivos, o que examinaremos posteriormente neste capítulo.

Agora que abordamos como identificar o diretório atual, aprenderemos a visualizar o conteúdo real de um diretório.

VISUALIZANDO O CONTEÚDO DE DIRETÓRIOS

Você pode visualizar o conteúdo de um diretório na GUI e na janela de linha de comando. No entanto, antes de fazermos isso, preciso mencionar que existem dois tipos de arquivos e diretórios no sistema de arquivos: arquivos e diretórios visíveis e arquivos e diretórios ocultos. Os arquivos e diretórios visíveis sempre são vistos no sistema de arquivos. Os arquivos e diretórios *ocultos* só podem ser vistos no sistema de arquivos quando alteramos nossas configurações para vê-los. Geralmente são arquivos e diretórios que armazenam informações que, como usuários, não precisamos acessar, por exemplo, configurações de aplicações e diversas configurações do sistema.

Não recomendo modificar ou excluir arquivos ou diretórios ocultos, a menos que você saiba realmente o que está fazendo. Uma vez que alterar as configurações para visualizar arquivos e diretórios ocultos, eles aparecerão parcialmente transparentes (desativados com a cor cinza). Seus nomes costumam começar com um ponto (.).

Em sua jornada de aprendizado do Git, você tomará conhecimento da existência de alguns arquivos e diretórios ocultos; logo, é preciso saber

como visualizá-los, tanto na GUI quanto na linha de comando.

Na GUI, para visualizar arquivos e diretórios ocultos em uma janela de sistema de arquivos, você terá de torná-los visíveis explicitamente:

- macOS – para alternar-se entre a visualização e a ocultação de arquivos e diretórios ocultos, pressione Cmd-Shift-ponto.
- Microsoft Windows – você deve alterar as configurações do sistema de arquivos para visualizar arquivos e diretórios ocultos. Se necessário, consulte os recursos online para ver instruções passo a passo para seu computador.

Consulte a seção Acompanhe 1-7 para visualizar os arquivos e diretórios ocultos em seu sistema de arquivos.

[ACOMPANHE 1-7]

❶ Torne visíveis os arquivos e diretórios ocultos de seu sistema de arquivos.

Na linha de comando, para visualizar uma lista dos arquivos e diretórios visíveis do diretório atual, use o comando `ls` (que significa “list”).

Para visualizar os arquivos e diretórios visíveis e também os ocultos do diretório atual, use o comando `ls` com a opção `-a`: `ls -a`.

[SALVE O COMANDO]

ls

Lista os arquivos e diretórios visíveis.

ls -a

Lista os arquivos e diretórios ocultos e visíveis.

Consulte a seção Acompanhe 1-8 para ver como esses comandos são usados para listar diferentes tipos de arquivos.

[ACOMPANHE 1-8]

❶ $ **ls**

```
Applications Downloads Music
Desktop Library Pictures
Documents Movies Public
```

❷ $ **ls -a**

```
. .config Desktop
.. .local Documents
..CFUserTextEncoding .npm Downloads
.DS_Store .ssh Library
.Trash .viminfo Movies
.bash_history .vscode Music
.bash_sessions .yarnrc Pictures
.bashrc Applications Public
```

O que deve ser observado:

- Os nomes de muitos arquivos e diretórios ocultos começam com um ponto (.).
- Os arquivos e diretórios visíveis e ocultos mostrados na saída deste livro serão diferentes dos de seu computador porque os conteúdos dos computadores das pessoas são diferentes.

Agora que vimos como identificar o diretório atual e visualizar seu conteúdo na linha de comando, examinaremos como podemos nos mover entre diretórios.

ENTRANDO E SAINDO DE UM DIRETÓRIO

Na GUI, para entrar em um diretório você pode clicar duas vezes nele. Na linha de comando, para entrar em um diretório, use o comando `cd`, que é a abreviação de “change directory” (mudar de diretório), e passe o nome ou o caminho do diretório.

[SALVE O COMANDO]

cd <caminho_diretório>
Muda de diretório.

Consulte a Seção 1-9 para ver como navegar para o diretório desktop.

[ACOMPANHE 1-9]

❶ $ **cd desktop**

❷ desktop $ **pwd**
/Users/annaskoulikari/desktop

O que deve ser observado:

- Na etapa 1, o comando `cd` não produz nenhuma saída.
- Na etapa 2, o prompt de comando e a saída do comando `pwd` indicam que você está no diretório `desktop`.

Anteriormente neste capítulo, mencionei que o prompt de comando exibe o local do diretório. Na seção Acompanhe 1-9, observe que o prompt de comando foi atualizado para exibir que seu diretório atual é o diretório `desktop`. Por padrão, a maneira como o local do diretório é apresentado pode diferir dependendo do sistema operacional usado:

- macOS – no Terminal, o nome do diretório atual é exibido.
- Microsoft Windows – no Git Bash, o caminho do diretório atual é exibido.

[NOTA]

Navegar para dentro e para fora de diretórios na linha de comando não afetará o que você está visualizando no sistema de arquivos. Por exemplo, navegar para o diretório desktop na linha de comando não fará seu sistema de arquivos exibir automaticamente o conteúdo do diretório desktop.

Na GUI, para voltar para o diretório pai, você pode selecionar o botão voltar (back). Na linha de comando, para voltar para o diretório pai, passe dois pontos (..) para o comando cd. Dois pontos representam o diretório pai do diretório atual. Consulte a seção Acompanhe 1-10 para ver como funciona.

[ACOMPANHE 1-10]

❶ desktop $ **cd ..**

❷ $ **pwd**
/Users/annaskoulikari

O que deve ser observado:

- Na etapa 2, o prompt de comando e a saída de pwd indicam que você voltou para o diretório de usuário atual.

Agora que vimos como navegar para dentro e para fora de diretórios, examinaremos como criar um diretório novo.

CRIANDO UM DIRETÓRIO

Na GUI, você pode criar um diretório selecionando ou clicando com o botão direito na opção de menu ou no ícone desejado. Na linha de comando, para criar um diretório você usará um comando chamado mkdir, que é a abreviação de “make directory” (criar diretório). O diretório será criado dentro do diretório atual quando você executar o comando.

[SALVE O COMANDO]

mkdir <nome_diretório>
Cria um diretório.

Para simplificar, evite incluir espaços nos nomes de seus diretórios. Se o nome de um diretório tiver espaços, você terá de mudar a maneira de usar certos comandos na linha de comando, o que complicará suas tarefas.

[NOTA]

Ao trabalhar na linha de comando, você deve evitar usar espaços nos nomes de arquivos, diretórios ou outros itens que criar porque isso pode causar complicações quando usar comandos.

Como mencionado no Prefácio, no decorrer deste livro você trabalhará em um projeto em que criará e editará arquivos para listar as cores do arco-íris e algumas cores que não fazem parte dele. Esse não é um exemplo realista de um projeto normalmente controlado com versões por meio do Git; é um exemplo simplificado que permitirá que você dê ênfase a aprender como o Git funciona.

Já que o principal objetivo do exemplo de projeto é listar as cores do arco-íris, você dará o nome rainbow (arco-íris) ao diretório de seu projeto. Você criará esse diretório de projeto no diretório desktop para poder vê-lo facilmente na área de trabalho na tela de seu computador. Consulte a seção Acompanhe 1-11 para criar seu diretório.

[ACOMPANHE 1-11]

❶ $ **cd desktop**

❷ desktop $ **mkdir rainbow**

❸ desktop $ **ls**
rainbow

O que deve ser observado:

- Na etapa 3, a saída de ls exibirá o diretório de projeto rainbow que você acabou de criar e outros diretórios ou arquivos que estiverem em seu diretório desktop. Esses últimos não estão sendo exibidos no exemplo de saída, já que o conteúdo em seu computador será diferente do existente no meu.

Se você examinar a área de trabalho de seu computador, deve ver o diretório de projeto rainbow que acabou de criar, como mostrado na Figura 1-4.

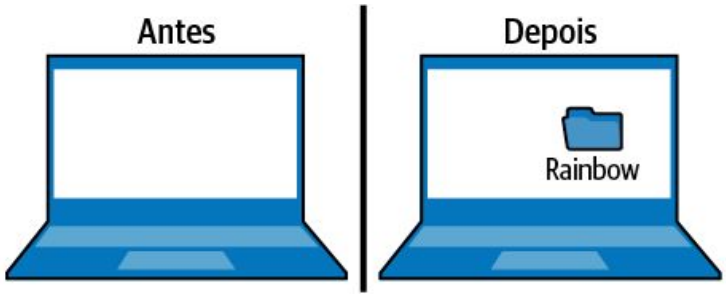


Figura 1-4: Exemplo de sua área de trabalho antes e depois de você criar o diretório de projeto Rainbow.

Você notará que, mesmo tendo criado o diretório de projeto rainbow, isso não significa que navegou para ele na linha de comando. Consulte a seção Acompanhe 1-12 para navegar explicitamente para o diretório de projeto rainbow na linha de comando.

[ACOMPANHE 1-12]

❶ desktop $ **cd rainbow**

❷ rainbow $ **pwd**
/Users/annaskoulikari/desktop/rainbow

O que deve ser observado:

- Na etapa 2, o prompt de comando e a saída do comando pwd indicam que você está no diretório Rainbow.

Você criou e acessou o diretório rainbow. No entanto, o que aconteceria se fechasse a janela de linha de comando e a abrisse novamente?

Fechando a linha de comando

Por padrão, se você fechar a janela de linha de comando e a abrir novamente, o local do diretório será redefinido para o diretório de usuário atual. Logo, você terá de navegar de novo para o diretório em que deseja trabalhar. Consulte a seção Acompanhe 1-13 para ver como funciona.

[ACOMPANHE 1-13]

❶ rainbow $ **pwd**
/Users/annaskoulikari/desktop/rainbow

❷ Feche sua janela de linha de comando e abra uma nova.

❸ $ **pwd**
/Users/annaskoulikari

❹ $ **cd desktop**

❺ desktop $ **cd rainbow**

❻ rainbow $ **pwd**
/Users/annaskoulikari/desktop/rainbow

Examinamos vários aspectos da linha de comando. Agora você definirá algumas configurações básicas do Git.

Definindo configurações do Git

As configurações do Git são definições que permitem personalizar como o Git funcionará. Elas são compostas de variáveis e seus valores e são armazenadas em alguns arquivos diferentes. Para trabalhar com o Git, você deve definir certas variáveis de configuração relacionadas às configurações de usuário.

Antes de definir qualquer variável, você verificará se existe em seu sistema de arquivos um arquivo de configuração global do Git e, se existir, quais variáveis foram definidas. Para fazer isso, usará o comando `git config`, passando as opções `--global` e `--list`.

O comando `git config` com a opção `--global` é um exemplo de comando em que não importa qual é nosso diretório atual quando o executamos; ele só exibe ou altera informações do arquivo de configuração global do Git, que é um arquivo oculto chamado `.gitconfig`, geralmente criado no diretório de usuário atual.

[SALVE O COMANDO]

git config --global --list

Lista as variáveis do arquivo de configuração global do Git e seus valores.

Consulte a seção Acompanhe 1-14 para ver como funciona.

[ACOMPANHE 1-14]

❶ $rainbow $ **git config --global --list**

fatal: unable to read config file '/Users/annaskoulikari/.gitconfig': No such file or directory

O que deve ser observado:

- A saída mostrada aqui indica que não existe nenhuma variável definida em seu arquivo de configuração global do Git. Nesse caso, você verá um erro indicando que o arquivo não existe. Se você tiver definido variáveis em seu arquivo de configuração global do Git, a saída exibirá as variáveis que foram definidas e seus valores.

As duas variáveis em que estamos interessados neste livro são user.name e user.email. Sempre que alguém salva uma versão de um projeto (ou, em outras palavras, faz um commit), o Git guarda o nome e o endereço de email da pessoa e a associa à versão salva. As variáveis user.name e user.email são usadas para definir o nome e o endereço de email que serão salvos para os commits feitos. Isso significa que você poderá ver quem trabalhou e o que essa pessoa fez em um projeto Git. É necessário configurar essas variáveis para podermos trabalhar com o Git. Lembre-se de que qualquer pessoa que puder ver uma lista de seus commits em um projeto conseguirá visualizar seu endereço de email; portanto, certifique-se de usar um endereço que você não se importe que outras pessoas vejam.

Para definir essas variáveis em seu arquivo de configuração global do Git, passe-as como argumentos para o comando git config, inserindo os valores desejados em aspas (lembre-se, você não deve incluir os valores em colchetes angulares).

[SALVE O COMANDO]

git config --global user.name "<nome>"
Define seu nome no arquivo de configuração global do Git.

git config --global user.email "<email>"
Define seu endereço de email no arquivo de configuração global do Git.

Se na saída da seção Acompanhe 1-14 as variáveis estiverem configuradas com os valores que você quer que elas tenham, pule a seção Acompanhe 1-15. Se as variáveis não aparecerem ou não estiverem configuradas com os valores desejados, decida que nome e endereço de email quer associar ao trabalho que fará no projeto Rainbow e consulte a seção Acompanhe 1-15. Certifique-se de substituir meu nome de usuário e endereço de email pelos valores que deseja usar.

[ACOMPANHE 1-15]

❶ rainbow $ **git config --global user.name "annaskoulikari"**

❷ rainbow $ **git config --global user.email "gitlearningjourney@gmail.com"**

❸ rainbow $ **git config --global --list**
user.name=annaskoulikari
user.email=gitlearningjourney@gmail.com

O que deve ser observado:

- Na etapa 3, na saída de git config --global --list, as variáveis user.name e user.email estarão configuradas com os valores que você inseriu.

Agora que você instalou o Git e preparou suas configurações de usuário, a última ferramenta da qual precisa para trabalhar em um projeto Git é um editor de texto.

Preparando um editor de texto

Um projeto Git é composto de arquivos e diretórios que são controlados com versões. O Git pode controlar com versões qualquer tipo de arquivo. No projeto Rainbow, você usará um editor de texto para trabalhar com arquivos de texto simples (que têm extensão .txt).

Um editor de texto é um programa que permite que o usuário edite texto simples (plain text). Você precisará de um para executar vários exercícios das seções Acompanhe deste livro. Um editor de texto é diferente de um processador de texto, que é usado principalmente para editar rich text. Alguns exemplos de processadores de texto seriam o Microsoft Word e o Google Docs; eles não podem ser utilizados para gerenciar os arquivos de um projeto Git. Rich text é um texto com estilos anexados ou embutidos dentro dele. Se você notar que o texto está em negrito ou itálico, saberá que se trata de um rich text.

Alguns editores de texto facilitam mais do que outros trabalhar em projetos Git. Este livro foi escrito de maneira a permitir que você use o editor de texto que preferir. Se já tiver um editor de texto instalado que utilizou para trabalhar em projetos Git, fique à vontade para usá-lo nos exercícios do livro. Se não tiver certeza de qual editor de texto deve empregar, recomendo o Visual Studio Code (*https://code.visualstudio.com*): é um editor de texto popular

e foi ele que usei ao trabalhar no projeto Rainbow apresentado aqui. Consulte a seção Acompanhe 1-16 para preparar um editor de texto.

[ACOMPANHE 1-16]

❶ Selecione seu editor de texto preferido. Se ainda não tiver um instalado, baixe um editor de texto.

❷ Abra o diretório de projeto rainbow em uma janela do editor de texto.

Terminais integrados

Alguns editores de texto avançados (também chamados de ambientes de desenvolvimento integrado, ou IDEs), como o Visual Studio Code, incluem uma versão de uma linha de comando, geralmente chamada de terminal integrado, em que podemos executar os comandos que normalmente usaríamos na janela de linha de comando.

Quando usamos um terminal integrado, costuma ser mais fácil gerenciar o espaço na tela, porque o terminal já faz parte da janela do editor de texto. No entanto, isso depende de preferências pessoais. Se seu editor de texto tiver um terminal integrado, talvez você prefira usá-lo em vez de utilizar uma janela de linha de comando separada para executar os comandos do Git nos outros exercícios deste livro. As duas abordagens funcionarão bem; logo, cabe a você decidir o que prefere.

No restante do livro, sempre que eu quiser me referir ao local em que você terá de executar comandos, mencionarei a janela de linha de comando; contudo, lembre-se de que isso inclui o terminal integrado.

Agora, com o Git instalado e seu editor de texto pronto para ser usado, você já pode começar a trabalhar no projeto Rainbow!

Resumo

Neste capítulo, você conheceu os aspectos básicos do Git e viu que ele é uma ferramenta útil para o rastreamento do histórico de um projeto e a colaboração com outras pessoas. Além disso, se preparou para trabalhar em um projeto com o uso do Git instalando uma versão atualizada em seu computador e definindo algumas variáveis de configuração básicas.

Você também conheceu alguns aspectos básicos da linha de comando, por exemplo, como visualizar o conteúdo de diretórios, como entrar e sair deles e principalmente como criá-los. Durante esse processo, criou um diretório

de projeto chamado rainbow que usará no decorrer dessa jornada de aprendizado.

Por fim, você preparou um editor de texto para criar e editar arquivos para qualquer projeto Git em que quiser trabalhar.

Você está pronto para passar para o Capítulo 2, em que transformará realmente seu projeto rainbow em um repositório Git e começará a aprender sobre as áreas mais importantes do trabalho com o Git.

2

Repositórios locais

No capítulo anterior, você conheceu alguns aspectos básicos da linha de comando e se preparou para trabalhar com o Git baixando-o e definindo algumas configurações.

Neste capítulo, transformará o diretório de projeto rainbow que criou no Capítulo 1 em um repositório Git. Também conhecerá quatro áreas importantes do trabalho com o Git: o diretório de trabalho, a área de preparação, o histórico de commits e o repositório local. Para ajudá-lo a ver como essas áreas funcionam em conjunto, criaremos um Diagrama Git que incluirá uma representação de cada área.

Para concluir, no fim do capítulo você adicionará o primeiro arquivo ao diretório de projeto rainbow e, durante o processo, conhecerá os arquivos não rastreados (untracked) e rastreados (tracked). Comecemos, então!

Configuração atual

No início deste capítulo, é preciso que você tenha:

- Baixado ou atualizado o Git no computador. (Você precisará de uma versão igual ou superior à 2.28.)
- Criado um diretório de projeto vazio chamado rainbow na área de trabalho.
- Aberto uma janela de linha de comando e navegado para o diretório rainbow.
- Escolhido um editor de texto e aberto o diretório de projeto rainbow em uma janela dele.
- Definido as variáveis de configuração globais do Git user.name e user.email com seu nome e endereço de email.

Introduzindo os repositórios

Repositório (também chamado de *repo*) é o termo que usamos para nos referir a uma versão do projeto controlada pelo Git. Existem dois tipos de repositórios:

- Um *repositório local* é aquele armazenado em um computador.
- Um *repositório remoto* é aquele que reside em um serviço de hospedagem.

Um *serviço de hospedagem* é uma empresa que fornece hospedagem para projetos usando o Git. Quando este texto foi escrito, os principais serviços de hospedagem eram o GitHub (*https://github.com*), o GitLab (*https://about.gitlab.com*) e o Bitbucket (*https://bitbucket.org/product*).

Na primeira parte deste livro, até o Capítulo 5, você conhecerá os repositórios locais e só trabalhará com eles. Na segunda parte, do Capítulo 6 em diante, também aprenderá a trabalhar com repositórios remotos.

Agora que vimos a diferença entre repositórios locais e remotos, você aprenderá a inicializar um repositório local.

Inicializando um repositório local

Um repositório local é representado por um diretório oculto chamado `.git` que existe dentro de um diretório de projeto. Ele contém todos os dados sobre as alterações que foram feitas nos arquivos de um projeto.

Para transformar um diretório de projeto em um repositório local, é preciso *inicializar*, ou criar, o repositório. Quando você inicializar um repositório, o diretório `.git` será criado automaticamente dentro do diretório de projeto. Já que o diretório `.git` é um diretório oculto, você não poderá vê-lo a não ser que torne visíveis os arquivos e diretórios ocultos.

[NOTA]

Não devemos nunca mexer em nenhum arquivo ou diretório do diretório .git. Isso pode ter consequências indesejadas para seu repositório. Você não deve excluir esse diretório a não ser que queira excluir o repositório.

Na seção Acompanhe 2-1, você se certificará de que as definições de suas configurações permitam visualizar arquivos e diretórios ocultos. (Se precisar de um lembrete de como fazê-lo, consulte “Visualizando o conteúdo de diretórios”, na página 28). Em seguida, verificará tanto na janela do sistema de arquivos quanto na janela de linha de comando se existe algum diretório ou arquivo visível ou oculto no diretório de projeto rainbow.

[ACOMPANHE 2-1]

❶ Torne visíveis os arquivos e diretórios ocultos em seu sistema de arquivos.

❷ Abra o diretório de projeto rainbow em uma janela de sistema de arquivos e examine seu conteúdo. Não deve haver nenhum arquivo ou diretório visível ou oculto.

❸ rainbow $ **ls -a**

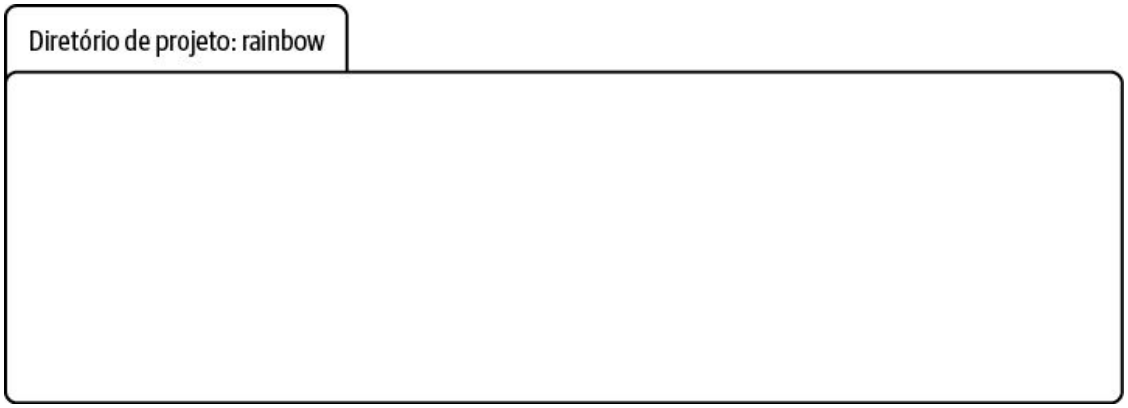
. ..

O que deve ser observado:

- Na etapa 2, você verá na janela do sistema de arquivos que o diretório de projeto rainbow está vazio.
- Na etapa 3, verá na janela de linha de comando que o diretório rainbow está vazio.

Para nos preparar para criar o Diagrama Git, produziremos uma representação do diretório de projeto rainbow vazio como visto em Visualize 2-1.

[VISUALIZE 2-1]



Representação do diretório de projeto rainbow vazio que conterá o Diagrama Git.

Para inicializar um repositório Git, você usará o comando git init. O diretório atual precisa ser o diretório de projeto que você deseja transformar em um repositório ao executar esse comando.

Normalmente os usuários do Git inicializam um repositório Git utilizando apenas o comando git init sem opções adicionais; no entanto, no projeto Rainbow, você inicializará o repositório usando o comando git init com a opção -b (que é a abreviação de --initial-branch) e passará o nome main. Veremos o que são os branches com mais detalhes no Capítulo 4. Por enquanto, só é preciso saber que, por padrão, o Git criará um branch chamado master quando você inicializar um novo repositório local. A partir do Git versão 2.28, o nome do branch inicial é configurável. Optei por usar o nome main em vez de master neste livro, porque o termo “master” (mestre) não é considerado terminologia inclusiva. Discutiremos esse tópico com mais detalhes em “Conhecendo um pouco da história do Git: master e main”, na página 66.

[NOTA]

Se você quiser que o branch inicial de todos os seus repositórios tenha um nome diferente de “master,” pode definir uma variável chamada init.defaultBranch em seu arquivo de configuração global. O processo é semelhante ao de como definimos as variáveis user.name e user.email em “Definindo configurações do Git” na página 34. Se a variável init.defaultBranch for usada, você poderá inicializar repositórios Git com o comando git init, e o branch inicial terá o nome que estiver definido na configuração.

[SALVE O COMANDO]

git init
Inicializa um repositório Git.

git init -b <nome_branch>
Inicializa um repositório Git e define que o nome do branch inicial será *<nome_branch>*.

Consulte a seção Acompanhe 2-2 para transformar seu diretório de projeto rainbow em um repositório Git.

[ACOMPANHE 2-2]

❶ Para ver o diretório .git ser criado, certifique-se de ter uma visualização do conteúdo de seu diretório de projeto rainbow em uma janela de sistema de arquivos com os arquivos e diretórios ocultos sendo exibidos.

❷ rainbow $ **git init -b main**

Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❸ Vá até o diretório de projeto rainbow na janela do sistema de arquivos e examine o diretório .git que acabou de ser criado. Abra o diretório .git para ver seu conteúdo.

O que deve ser observado:

- O Git criou o diretório .git dentro do diretório de projeto rainbow.

A primeira área para a qual adicionaremos uma representação no Diagrama Git será o repositório local, representado pelo próprio diretório .git. Isso pode ser visto em Visualize 2-2.

[VISUALIZE 2-2]

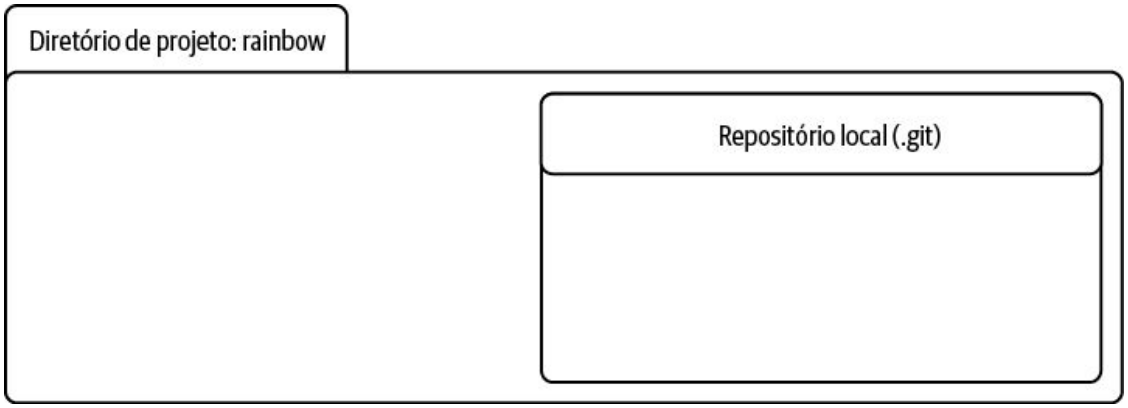


Diagrama Git com uma representação do repositório local.

Dentro do diretório .git existem vários arquivos e diretórios. Alguns deles representam as áreas do Git que você conhecerá a seguir. Na próxima seção, também adicionaremos suas representações ao Diagrama Git. À medida que você continuar trabalhando no repositório rainbow, mais arquivos e diretórios serão criados no diretório .git; conheceremos alguns ao avançar.

A Figura 2-1 mostra um exemplo do conteúdo de um diretório .git em um repositório recém-inicializado.

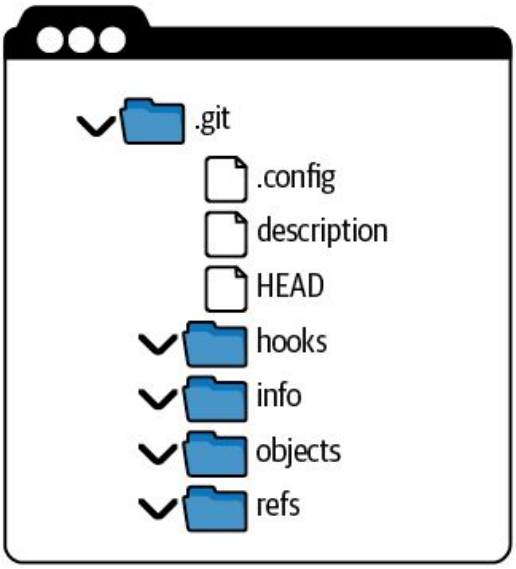


Figura 2-1: Conteúdo de um diretório .git logo após a execução do comando git init.

Agora examinaremos as diferentes áreas com as quais você interagirá ao trabalhar com o Git.

Áreas do Git

Existem quatro áreas importantes às quais devemos dar atenção no trabalho com o Git:

- Diretório de trabalho (working directory)
- Área de preparação (staging area)
- Histórico de commits (commit history)
- Repositório local (local repository)

Você conheceu o repositório local na seção anterior. Nesta seção, examinaremos as outras áreas e como elas estão relacionadas, e continuaremos a criar o Diagrama Git. Começaremos com o diretório de trabalho.

INTRODUÇÃO AO DIRETÓRIO DE TRABALHO

O *diretório de trabalho* contém os arquivos e diretórios do diretório de projeto que representam uma versão de um projeto. É como uma bancada de trabalho. É onde adicionamos, editamos e excluímos arquivos e diretórios.

Examinaremos o Exemplo 2-1 do Projeto Book para saber como os arquivos são gerenciados no diretório de trabalho.

Exemplo 2-1 do Projeto Book

Suponhamos que o projeto Book em que estou trabalhando tivesse 10 capítulos e eu tivesse 10 arquivos de texto, um para cada capítulo: chapter_one.txt, chapter_two.txt e assim por diante.

Para adicionar cada um dos capítulos ao meu projeto, eu criaria esses arquivos no diretório de trabalho.

Se quisesse fazer alguma alteração no conteúdo desses capítulos, começaria editando os arquivos no diretório de trabalho.

E, por fim, se quisesse remover um capítulo inteiro de meu livro, a primeira etapa seria excluir o arquivo correspondente do diretório de trabalho.

Podemos concluir do Exemplo 2-1 do Projeto Book que o diretório de trabalho é o local em que fazemos todas as modificações no conteúdo de um projeto.

No caso do repositório rainbow, atualmente o diretório de trabalho está vazio. Para continuar criando o Diagrama Git, adicionaremos a ele uma representação do diretório de trabalho. Isso pode ser visto em Visualize 2-3.

[VISUALIZE 2-3]

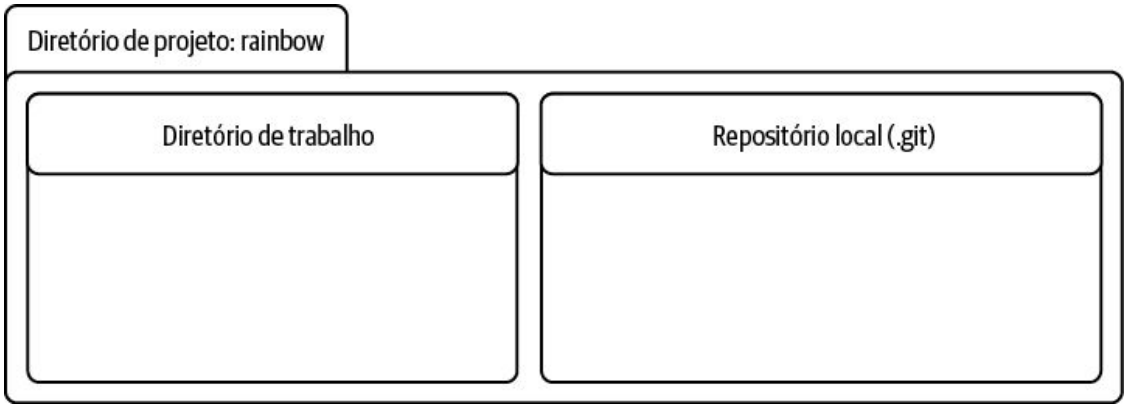


Diagrama Git com uma representação do diretório de trabalho e do repositório local.

Na seção Visualize 2-3, o diretório de projeto rainbow contém o repositório local dentro dele. No entanto, é importante saber que, quando as pessoas se referem a uma versão de projeto controlada com o Git, normalmente se referem ao diretório de projeto como o repositório. Por exemplo, no nosso caso, nos referiríamos ao "repositório rainbow".

Dentro do repositório local, existem duas áreas importantes que queremos examinar melhor: a área de preparação e o histórico de commits. Elas serão explicadas a seguir e também discutiremos o conceito de commit com mais detalhes.

INTRODUÇÃO À ÁREA DE PREPARAÇÃO

A *área de preparação* é semelhante a um espaço de rascunho. É o local em que podemos adicionar e remover arquivos quando estamos preparando o que queremos incluir na próxima versão salva do projeto (o próximo commit). A área de preparação é representada por um arquivo do diretório .git chamado index.

[NOTA]

O arquivo index só será criado se você tiver adicionado pelo menos um arquivo à área de preparação em seu projeto. No diretório de projeto rainbow, anda não adicionamos nenhum arquivo à área de preparação, portanto o arquivo index ainda não pode ser visto no diretório .git. No Capítulo 3, você adicionará um arquivo à área de preparação e verá o arquivo index ser criado.

Você entenderá melhor por que a área de preparação é tão útil quando tentar adicionar um arquivo a ela no Capítulo 3. Por enquanto, adicionaremos uma representação dela dentro do repositório local no Diagrama Git, como mostrado em Visualize 2-4.

[VISUALIZE 2-4]

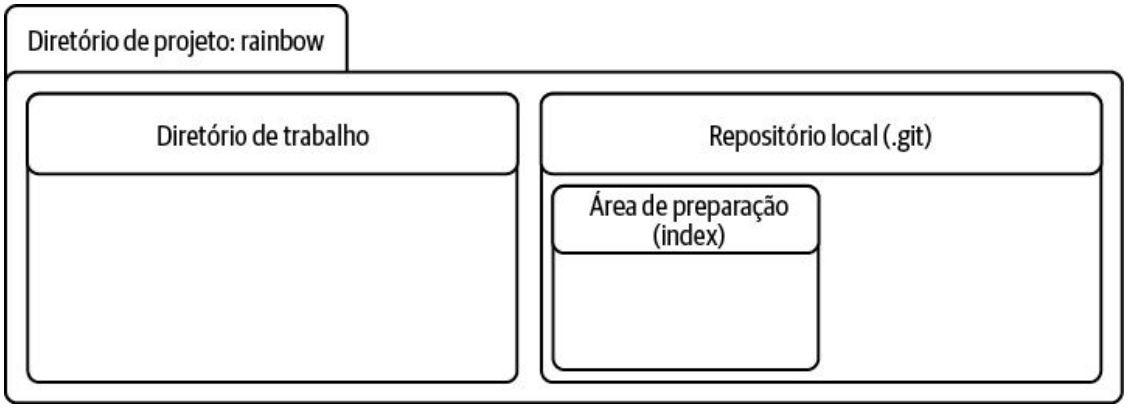


Diagrama Git com uma representação do diretório de trabalho, da área de preparação e do repositório local.

Agora o Diagrama Git inclui representações do diretório de trabalho, da área de preparação e do repositório local. A última área que quero apresentar é o histórico de commits, mas, antes de fazê-lo, examinaremos apropriadamente o conceito de commit.

O QUE É UM COMMIT?

Um *commit* no Git é basicamente uma versão de um projeto. Podemos considerá-lo um snapshot de um projeto, ou uma versão autônoma de um projeto que contém referências a todos os arquivos que fazem parte desse commit.

Cada commit tem um *hash de commit* (também chamado de *ID de commit*). Trata-se de um hash único de 40 caracteres composto de letras e números que age como um nome para o commit, fornecendo uma maneira de o referenciarmos.

Um exemplo de um hash de commit seria 51dc6ecb327578cca503abba4a56e8c18f3835e1.

Na verdade, só precisamos dos sete primeiros caracteres de um hash de commit para referenciar um commit. Logo, no exemplo de hash que acabamos de ver, poderíamos usar apenas 51dc6ec para referenciar o commit.

[NOTA]

Já que os hashes de commit são únicos, seus hashes de commit do projeto Rainbow serão diferentes dos mostrados neste livro. Lembre-se disso ao fazer os exercícios da seção Acompanhe.

Agora que você tem uma noção do que são os commits, veremos a última área que adicionaremos ao nosso Diagrama Git, o histórico de commits.

INTRODUÇÃO AO HISTÓRICO DE COMMITS

Podemos considerar o *histórico de commits* o local em que nossos commits residem. Ele é representado pelo diretório objects dentro do diretório .git. Para entender melhor o que é o histórico de commits, teríamos de nos aprofundar no funcionamento interno do Git, mas esse é um tópico complexo que você não precisa conhecer para aprender os aspectos básicos de como usar o Git. Para nossa finalidade, basta saber que, sempre que fizer um commit, ele será salvo no histórico de commits.

Consulte a seção Acompanhe 2-3 para identificar o histórico de commits no repositório rainbow.

[ACOMPANHE 2-3]

❶ Na janela do sistema de arquivos, dentro do diretório de projeto rainbow, examine o diretório .git e encontre o diretório objects.

Na seção Visualize 2-5, adicionamos uma representação do histórico de commits ao Diagrama Git dentro do repositório local. Isso conclui o Diagrama Git.

[VISUALIZE 2-5]

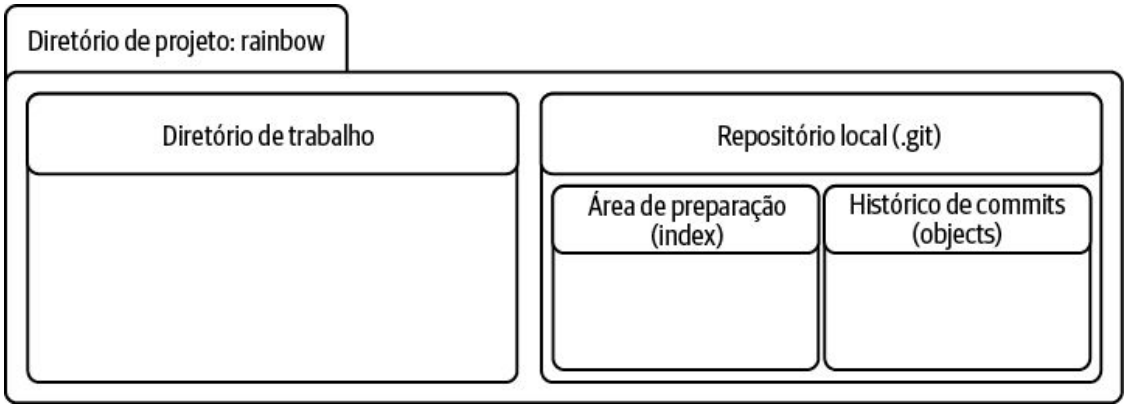


Diagrama Git completo com representações do diretório de trabalho, da área de preparação, do histórico de commits e do repositório local.

Agora que temos um Diagrama Git completo mostrando as áreas mais importantes do trabalho com o Git, adicionaremos o primeiro arquivo ao projeto Rainbow e usaremos um editor de texto para editá-lo.

Adicionando um arquivo a um projeto Git

Como mencionado anteriormente no livro, no decorrer dessa jornada de aprendizado você trabalhará em um projeto em que listará as cores do arco-íris e algumas cores que não fazem parte dele. Sempre que adicionar uma cor, fará um commit para monitorar como o projeto está progredindo.

A primeira etapa será criar um arquivo chamado rainbowcolors.txt, em que você listará as cores. Como mencionado em “Criando um diretório”, na página 32, é importante que o nome do arquivo não inclua espaços. Em seguida, usando o editor de texto que preparamos no Capítulo 1, você adicionará “Red is the first color of the rainbow.” na linha 1 desse arquivo. Consulte a seção Acompanhe 2-4 para criar e editar seu arquivo.

[ACOMPANHE 2-4]

❶ Use seu editor de texto para criar um arquivo chamado rainbowcolors.txt dentro do diretório de projeto rainbow.

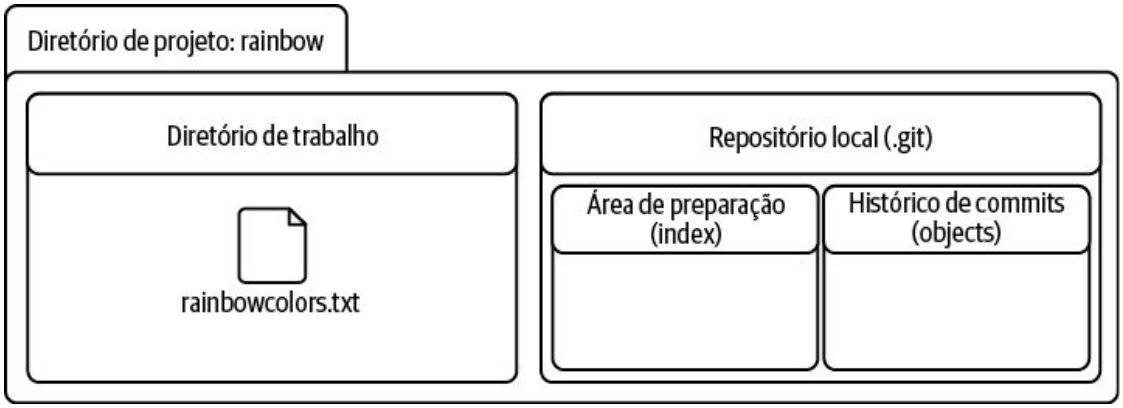
❷ Na linha 1, adicione “Red is the first color of the rainbow.” e salve o arquivo.

O que deve ser observado:

- O arquivo rainbowcolors.txt se encontra dentro do diretório de projeto rainbow, portanto está no diretório de trabalho.

Ainda que o arquivo rainbowcolors.txt esteja no diretório de trabalho, ele não faz parte de seu repositório. Não foi adicionado à área de preparação e ainda não foi incluído em um commit no histórico de commits. Podemos ver isso em Visualize 2-6.

[VISUALIZE 2-6]



Diretório de projeto rainbow após você adicionar o arquivo rainbowcolors.txt ao diretório de trabalho.

Já que o arquivo rainbowcolors.txt ainda não está em seu repositório, ele é um arquivo não rastreado (untracked file). Um *arquivo não rastreado* é um arquivo do diretório de trabalho que o Git não está controlando com versões. Ele não foi adicionado à área de preparação e não foi incluído em um commit, portanto não faz parte do repositório.

Uma vez que você adicionar um arquivo à área de preparação e incluí-lo em um commit, ele passará a ser um *arquivo rastreado*. Será um arquivo controlado com versões (em outras palavras, um arquivo rastreado pelo Git).

Cada arquivo novo de um projeto controlado com versões pelo Git precisa ser adicionado explicitamente à área de preparação e incluído em um commit para se tornar um arquivo rastreado. Você executará essas etapas no Capítulo 3.

[NOTA]

No Prefácio, mencionei que criar e editar arquivos que listam cores do arco-íris e algumas cores que não fazem parte dele não é um exemplo muito realista de um projeto controlado com versões pelo Git. Lembre-se de que o objetivo desse projeto é simplificar a jornada de aprendizado para que você se concentre em como o Git funciona em vez de no conteúdo dos arquivos que está editando. As alterações que fará nos arquivos de projetos reais serão muito diferentes.

Resumo

Neste capítulo, você transformou o diretório de projeto rainbow em um repositório inicializando o repositório. Também criamos um Diagrama Git que exibe as quatro áreas importantes do Git: o diretório de trabalho, a área de preparação, o histórico de commits e o repositório local.

Você adicionou seu primeiro arquivo ao diretório de projeto rainbow e conheceu a importante diferença entre arquivos não rastreados e rastreados.

No próximo capítulo, percorreremos as etapas envolvidas para se fazer um commit. Durante o percurso, examinaremos o papel desempenhado por cada uma das áreas do Diagrama Git no processo de commit.

3

Fazendo um commit

No último capítulo, você conheceu as diferentes áreas com as quais interagirá ao trabalhar com o Git: o diretório de trabalho, a área de preparação, o histórico de commits e o repositório local. Criamos um Diagrama Git dessas áreas e terminamos o capítulo adicionando o primeiro arquivo ao seu diretório de projeto rainbow.

Neste capítulo, percorreremos o processo de fazer um commit no projeto Rainbow e você verá qual é o papel desempenhado por cada uma das áreas do Diagrama Git nessa tarefa. Introduzirei dois comandos importantes que o ajudarão em seu trabalho diário com o Git. O primeiro comando lhe permitirá verificar o estado do diretório de trabalho e da área de preparação e o segundo, visualizar uma lista de commits.

Configuração atual

Atualmente você tem um diretório de projeto chamado rainbow com um diretório .git dentro dele, e também tem um arquivo, chamado rainbowcolors.txt, em seu diretório de trabalho. A área de preparação e o histórico de comandos estão vazios porque você ainda não fez nenhum commit no repositório rainbow. Tudo isso pode ser visto em Visualize 3-1.

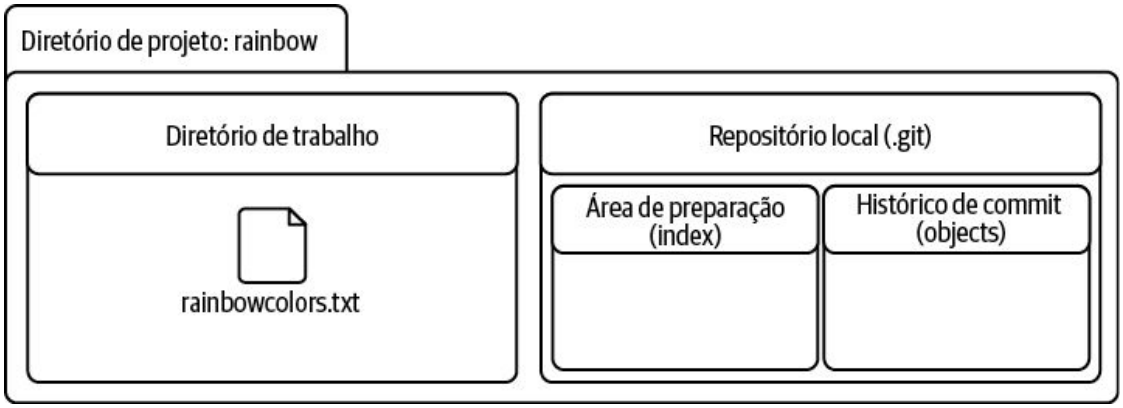
Por que fazemos commits?

No Capítulo 2, você aprendeu que, basicamente, um commit representa uma versão de um projeto. Sempre que você quiser salvar uma nova versão de um projeto, poderá fazer um commit.

É importante fazer um commit porque isso nos permite ter um backup do trabalho e evita a frustração causada pela perda de trabalhos não salvos. Uma vez feito o commit, o trabalho estará salvo, e você poderá voltar e

examinar esse commit para ver qual era a aparência de seu projeto naquele momento.

[VISUALIZE 3-1]



No início do Capítulo 3, o diretório de projeto rainbow tem um arquivo não rastreado no diretório de trabalho.

No que diz respeito a quando devemos fazer commits, não há uma regra absoluta. Isso pode depender de vários fatores, como se você está trabalhando em um projeto sozinho ou com outras pessoas e em que tipo de projeto está trabalhando (por exemplo, se está escrevendo um código que precise de compilação ou de documentação para um recurso). Para concluir, se você estiver trabalhando em equipe, pode depender do fluxo de trabalho usado e das convenções que foram acordadas (por exemplo, como os branches são utilizados, o que discutiremos no Capítulo 4).

Uma frase muito usada no universo Git é “faça commit cedo e com frequência”. Se você for iniciante no Git, esse pode ser um bom conselho. Nesse estágio, é preferível que haja muitos commits em vez de poucos. Além disso, uma vez que você tiver adquirido um bom conhecimento dos aspectos básicos do Git, poderá conhecer as ferramentas adicionais que ele fornece para a limpeza de commits.

As duas etapas para se fazer um commit

Agora que sabemos por que devemos fazer commits, veremos como fazê-los. Fazer um commit é um processo de duas etapas:

1. Adicione à área de preparação todos os arquivos que deseja incluir no próximo commit.
2. Faça um commit com uma mensagem de commit.

No decorrer desse processo você interagirá com as quatro áreas do Git

introduzidas no Capítulo 2: o repositório local, o diretório de trabalho, a área de preparação e o histórico de commits.

Um comando útil do processo de commit é git status. Entre outras coisas, ele informa o estado do diretório de trabalho e da área de preparação. Isso é útil porque, em um projeto com muitos arquivos, é fácil a pessoa perder o controle do estado em que se encontram os arquivos do diretório de trabalho (isto é, quais foram editados) e de quais arquivos foram adicionados à área de preparação.

[SALVE O COMANDO]

git status

Exibe o estado do diretório de trabalho e da área de preparação.

Em um projeto com muitos arquivos, pode ser difícil nos lembrarmos de quais não são rastreados, quais são rastreados e quais foram editados. Atualmente, só temos um arquivo no projeto Rainbow, mas examinaremos o Exemplo 3-1 do Projeto Book para considerar o caso de um projeto com mais arquivos.

Exemplo 3-1 do Projeto Book

O projeto Book em que estou trabalhando é composto de 10 arquivos, um para cada capítulo do livro. Após trabalhar um pouco no livro entre os commits, posso me esquecer de quais foram os arquivos de capítulos que editei. Sei que trabalhei nos capítulos 2 e 3, mas fiz alguma alteração no Capítulo 4? Se fiz, não quero perdê-la.

Nesse caso, o comando git status pode me ajudar informando que arquivos de capítulos editei, quais adicionei à área de preparação e quais ainda preciso adicionar.

O comando git status só fornece informações; ele nunca altera nada no repositório. Sinta-se à vontade para usá-lo a qualquer momento enquanto trabalha em um projeto Git para saber mais sobre o estado do diretório de trabalho e da área de preparação.

Consulte a seção Acompanhe 3-1 para usar o comando git status e verificar o estado do diretório de trabalho e da área de preparação no repositório rainbow. Para executar as etapas desta seção Acompanhe, você precisa estar no diretório de projeto rainbow na linha de comando, como indicado pelo prompt de comando.

[ACOMPANHE 3-1]

❶ rainbow $ **git status**
On branch main
No commits yet
Untracked files:
(use "git add <file>..." to include in what will be committed)
rainbowcolors.txt
nothing added to commit but untracked files present (use "git add" to track)

O que deve ser observado:

- A saída de git status informa que ainda não existem commits (No commits yet). Em outras palavras, atualmente o histórico de commits não contém nenhum commit.
- O arquivo rainbowcolors.txt é um arquivo não rastreado.
- O Git fornece as instruções necessárias para a inclusão do arquivo não rastreado na área de preparação: use "git add <file>..." to include in what will be committed.

No Capítulo 2, mencionei que rainbowcolors.txt é um arquivo não rastreado e que para ele se tornar um arquivo rastreado deve ser adicionado à área de preparação e incluído em um commit. Executaremos então a primeira etapa do processo de fazer um commit, que é adicionar à área de preparação qualquer arquivo que quisermos incluir no commit.

ADICIONANDO ARQUIVOS À ÁREA DE PREPARAÇÃO

Para adicionar arquivos à área de preparação, você usará o comando git add. Se só quiser adicionar à área de preparação os arquivos individuais que tiver editado, passe o nome ou os nomes dos arquivos para o comando git add como argumentos. Para adicionar todos os arquivos que editou ou alterou em seu diretório de trabalho, use o comando git add com a opção -A (que significa “all”). Isso pode ser útil se você tiver editado muitos arquivos e não quiser escrever cada nome de arquivo na linha de comando.

[SALVE O COMANDO]

git add <nome_arquivo>
Adiciona um arquivo à área de preparação.

git add <nome_arquivo> <nome_arquivo> ...
Adiciona vários arquivos à área de preparação.

git add -A
Adiciona à área de preparação todos os arquivos do diretório de trabalho que foram editados ou alterados.

Como mencionado no Capítulo 2, a área de preparação nos permite selecionar que arquivos atualizados (ou alterações) serão incluídos no próximo commit. A regra geral é agrupar alterações que estejam relacionadas. Isso permite manter os commits mais organizados. Como você verá na próxima seção, cada commit também tem uma mensagem de commit associada. Ela pode ser usada para fornecer uma descrição do que foi atualizado em um commit específico.

Essa primeira etapa do processo de commit nos permite ser muito específicos sobre o que incluiremos em um commit. Ou seja, você pode editar muitos arquivos em seu projeto, mas não precisa salvar todos no mesmo commit. Veremos isso no Exemplo 3-2 do Projeto Book .

Exemplo 3-2 do Projeto Book

O projeto Book em que estou trabalhando tem 10 capítulos representados por 10 arquivos. Imagine um cenário em que eu estivesse trabalhando nos capítulos 1, 2 e 3 (portanto, estaria editando chapter_one.txt, chapter_two.txt e chapter_three.txt).

Se eu decidir que o trabalho que fiz no Capítulo 2 já está pronto para entrar em um commit (ser salvo), mas não quiser incluir o trabalho dos capítulos 1 e 3 no próximo commit, posso adicionar a versão atualizada do arquivo do Capítulo 2 à área de preparação *sem* adicionar a versão atualizada dos arquivos dos capítulos 1 e 3. Isso significa que *só* as alterações do arquivo do Capítulo 2 serão incluídas em meu próximo commit e "salvas" oficialmente no repositório local.

O Exemplo 3-1 do Projeto Book mostra que a área de preparação nos permite ter um grande controle sobre a aparência das versões salvas do projeto (nossos commits).

No projeto Rainbow, rainbowcolors.txt é o primeiro arquivo que você adicionará à área de preparação. Isso significa que, quando você adicionar o arquivo, o arquivo index (que representa a área de preparação no diretório .git) será criado. Como vimos no Capítulo 2, esse arquivo não existirá até você adicionar um arquivo à área de preparação. O arquivo index é um

arquivo binário, ou seja, seu conteúdo não faz sentido para os humanos e não é facilmente entendido. Para nossa finalidade, só temos de saber que ele representa a área de preparação. Consulte a seção Acompanhe 3-2 para adicionar o arquivo rainbowcolors.txt à área de preparação e criar o arquivo index.

[ACOMPANHE 3-2]

❶ Para ver o arquivo index ser criado, certifique-se de ter uma visualização de seu diretório de projeto rainbow na janela do sistema de arquivos e de que os arquivos e diretórios ocultos e o conteúdo do diretório .git estejam visíveis.

❷ rainbow $ **git add rainbowcolors.txt**

❸ rainbow $ **git status**

On branch main

No commits yet

Changes to be committed:

(use "git rm --cached <file>..." to unstage)

new file: rainbowcolors.txt

❹ Usando a janela do sistema de arquivos, examine o conteúdo do diretório .git e identifique o arquivo index recém-criado.

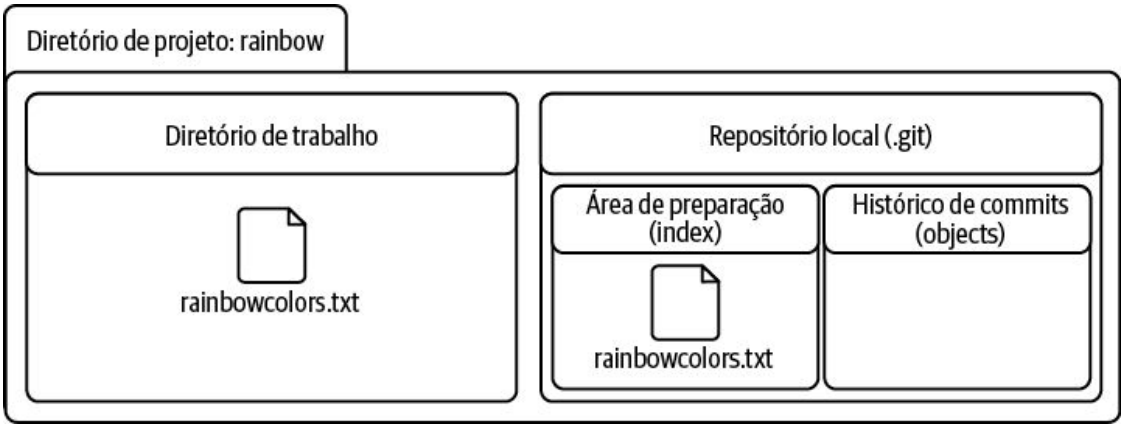
O que deve ser observado:

- Você adicionou o arquivo rainbowcolors.txt à área de preparação, e na etapa 3 ele foi listado na seção Changes to be committed:. Isso é mostrado em Visualize 3-2.

Agora o arquivo rainbowcolors.txt está tanto no diretório de trabalho quando na área de preparação. Isso está ocorrendo porque o comando git add não *move* um arquivo do diretório de trabalho para a área de preparação. Ele *copia* o arquivo do diretório de trabalho para a área de preparação.

Com o arquivo rainbowcolors.txt na área de preparação, você está pronto para passar para a segunda etapa do processo de commit, que é fazer realmente um commit com uma mensagem de commit.

[VISUALIZE 3-2]



Diretório de projeto rainbow depois que você adicionou o arquivo rainbowcolors.txt à área de preparação.

FAZENDO UM COMMIT

É importante notar que, em inglês, *commit*, além de ser um verbo, é um substantivo. No Git, o verbo *to commit* significa salvar algo, e o substantivo (*um commit*) significa uma versão do projeto. Portanto, fazer um commit significa salvar uma versão de um projeto.

Para fazer um commit, você usará o comando git commit e passará a opção -m (que significa “mensagem”), digitando uma mensagem entre aspas. A mensagem deve ser uma breve descrição das alterações que você fez nessa versão do projeto.

[SALVE O COMANDO]

git commit -m "<mensagem>"

Cria um novo commit com uma mensagem.

Vejamos como poderíamos criar uma mensagem de commit no Exemplo 3-3 do Projeto Book.

Exemplo 3-3 do Projeto Book

Em meu projeto Book, se eu trabalhasse no Capítulo 2, adicionasse somente esse arquivo à área de preparação e quisesse fazer um commit apenas com as atualizações desse capítulo, poderia usar a mensagem de commit “Capítulo 2 atualizado”.

Lembre-se de que outras pessoas e equipes podem ter regras diferentes do que incluir em uma mensagem de commit. Se você estiver trabalhando em um projeto Git com outras pessoas, deve verificar com seus colaboradores para saber o que eles esperam que seja incluído nessas mensagens.

No projeto Rainbow, você passará apenas o nome da cor que adicionou ao projeto como a mensagem de commit, para podermos representar facilmente os commits nos diagramas da seção Visualize. Já que a primeira cor que você adicionou à lista de cores do arco-íris foi red (vermelho), na seção Acompanhe 3-3 faremos um commit com a mensagem “red”.

[ACOMPANHE 3-3]

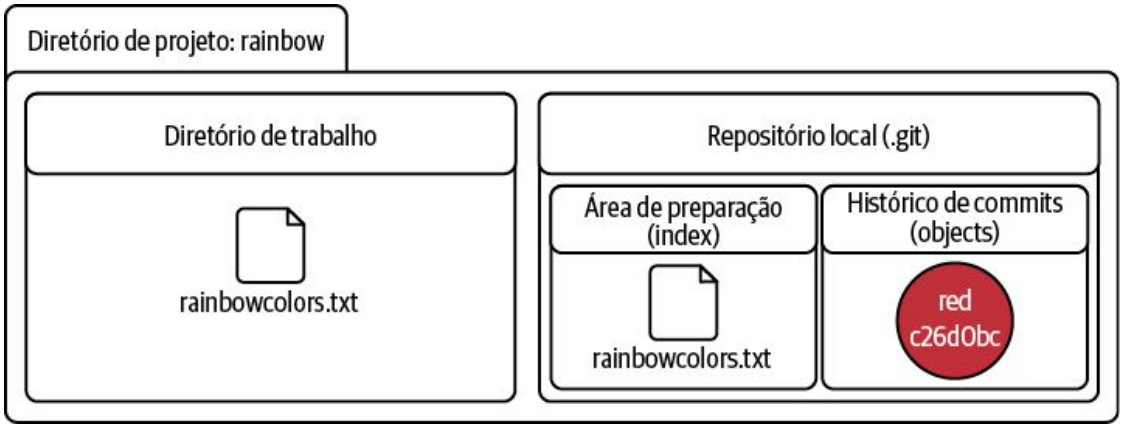
❶ rainbow $ **git commit -m "red"**
[main (root-commit) c26d0bc] red
1 file changed, 1 insertion(+)
create mode 100644 rainbowcolors.txt

O que deve ser observado:

- A saída do comando `git commit` exibe os sete primeiros caracteres do hash do commit red, que neste livro é `c26d0bc`. Os sete primeiros caracteres do hash de seu commit serão diferentes, já que, como vimos em “O que é um commit?”, na página 46, os hashes de commit são únicos.

Em Visualize 3-3, podemos ver o commit red no histórico de commits com os sete primeiros caracteres do hash do commit.

[VISUALIZE 3-3]



Diretório de projeto rainbow depois que você fez o commit de red.

Você também aprendeu no capítulo anterior que, quando um novo arquivo que era não rastreado é adicionado à área de preparação e incluído em um commit, ele passa a ser um arquivo rastreado, porque o Git saberá da sua existência. Portanto, agora rainbowcolors.txt é um arquivo rastreado.

O restante da saída após o comando git commit não é importante para nossa finalidade. Apenas fornece mais informações sobre o que mudou nesse commit, e, pelo que aprendi com minha experiência, isso não é algo que a maioria dos usuários do Git precise examinar com detalhes.

Agora que você fez o primeiro commit no repositório rainbow, veremos as informações que encontrará sobre esse commit no histórico de commits.

Visualizando uma lista de commits

Para ver uma lista de commits no histórico de commits, você usará o comando git log. Esse comando lista os commits de um repositório local na ordem cronológica inversa. Ele exibe quatro informações sobre cada commit:

1. Hash do commit
2. Nome e endereço de email do autor
3. Data e hora em que o commit foi feito
4. Mensagem do commit

[SALVE O COMANDO]

git log

Exibe uma lista de commits na ordem cronológica inversa.

[NOTA]

Se a saída do comando git log ultrapassar o tamanho da janela de linha de comando, para visualizar o restante dos commits você deve pressionar Enter (Return) ou usar a seta para baixo. Para sair do comando, insira Q. Atualmente você tem apenas um commit; logo, a saída de git log será muito curta. No entanto, à medida que commits forem adicionados ao projeto Rainbow, a saída de git log será mais longa e você precisará usar essas teclas.

Consulte a seção Acompanhe 3-4 para treinar como listar commits.

[ACOMPANHE 3-4]

❶ rainbow $ **git log**

```
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5 (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red
```

O que deve ser observado:

- A saída de git log exibe que no momento você só tem um commit, o commit red.
- O hash completo do commit red neste livro é c26d0bc371c3634ab49543686b3c8f10e9da63c5. O hash de commit em seu projeto será diferente.
- O autor do commit red deve coincidir com o nome e o endereço de email que você forneceu no Capítulo 1 quando definiu as variáveis de configuração do Git user.name e user.email.
- A data e a hora em que o commit foi feito são exibidos.
- Abaixo da data fica a mensagem de commit, que nesse caso é red.

Nos diagramas da seção Visualize, representarei os commits como círculos. Cada commit será exibido na cor usada na mensagem de commit e rotulado com seu nome ou abreviação (por exemplo, R para o commit red). Lembre-se de que cada um desses círculos representa um commit individual que você verá listado na saída de git log. Isso fica claro na Figura 3-1.



Figura 3-1: Neste livro, os commits são representados por círculos nos diagramas.

Há mais uma coisa que aparece na saída de git log na seção Acompanhe 3-4 que ainda não abordamos. Perto do hash do commit, entre parênteses, podemos ver o texto HEAD -> main. main é um branch; você aprenderá o que são os branches e como trabalhar com eles no próximo capítulo, e também aprenderá o que é HEAD.

Resumo

Este capítulo introduziu as duas etapas necessárias para fazermos um commit, que são adicionar arquivos à área de preparação e fazer um commit com uma mensagem, e você fez seu primeiro commit no repositório `rainbow`. Para ajudar no processo, introduzi o comando `git status`, que fornece informações sobre o estado dos arquivos do diretório de trabalho e da área de preparação.

Você também aprendeu a listar os commits de um repositório local usando o comando `git log`. Na saída de `git log`, vimos uma menção a `main` entre parênteses perto do hash do commit. No Capítulo 4, você aprenderá que `main` é um *branch* e saberemos por que e como os branches são usados no Git.

4

Branches

No capítulo anterior, você conheceu o processo de fazer um commit e fez seu primeiro commit no repositório rainbow.

Neste capítulo, aprenderá o que são os branches e por que temos de usá-los. Você continuará fazendo commits no repositório rainbow e verá como isso afeta os branches em seu projeto. Para concluir, criará um novo branch e aprenderá a passar (ou mudar) para ele. No processo de fazer novos commits no repositório rainbow, você também conhecerá o conceito de arquivos não modificados e modificados e como os commits são vinculados uns aos outros.

Estado do repositório local

No Capítulo 2, criamos um Diagrama Git com as quatro áreas importantes do Git: o diretório de trabalho, a área de preparação, o histórico de commits e o repositório local. A seção Visualize 4-1 usa o Diagrama Git para mostrar o estado do repositório rainbow no início deste capítulo.

[VISUALIZE 4-1]

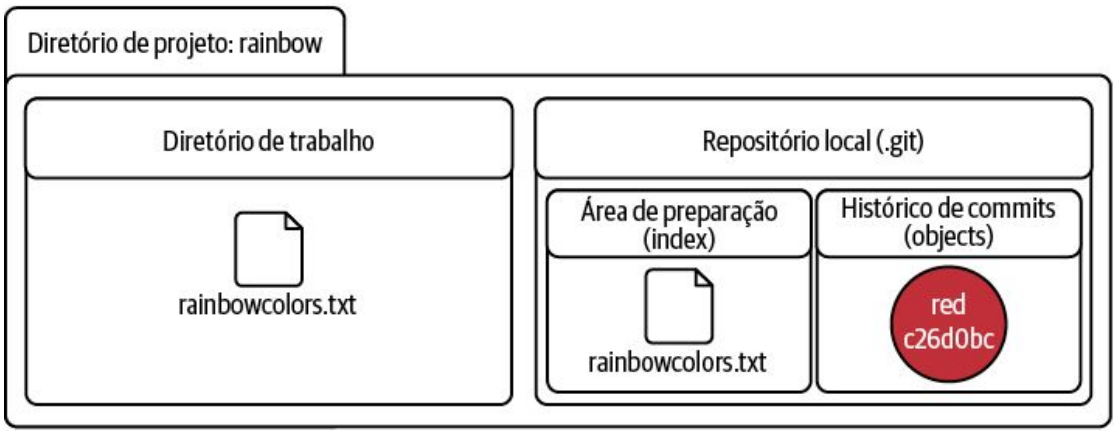


Diagrama Git mostrando o estado do repositório rainbow no início do Capítulo 4, com um commit, o commit red (vermelho).

[NOTA]

Desse ponto em diante, os diagramas da seção Visualize só exibirão o nome da cor no commit, ou uma abreviação do nome. Não incluirei mais os sete primeiros caracteres do hash do commit.

Para nos concentrarmos no histórico de commits, introduzirei agora um novo diagrama chamado Diagrama Repositório. O Diagrama Repositório só inclui uma representação do histórico de commits de um repositório e os branches e referências relevantes. O repositório local é representado por um retângulo, com seu nome sendo exibido no canto superior esquerdo. A seção Visualize 4-2 mostra o estado atual do repositório rainbow na forma de um Diagrama Repositório.

[VISUALIZE 4-2]

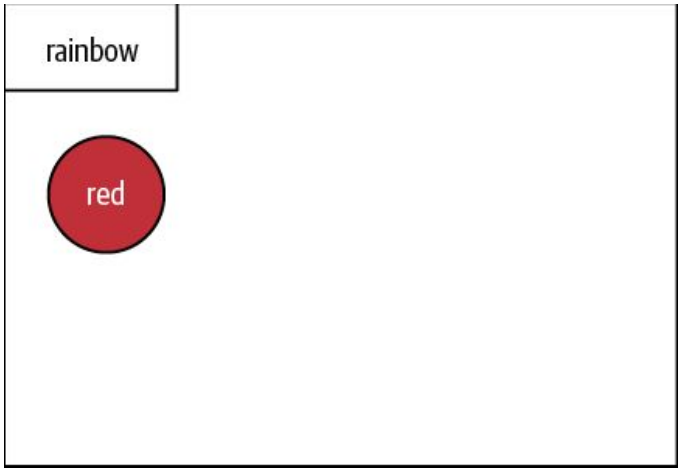


Diagrama Repositório exibindo o estado atual do repositório rainbow com um commit, o commit red.

Por que usamos branches?

Antes de examinarmos as particularidades dos branches no Git, explicarei por que eles são tão úteis. Existem duas razões principais para usarmos branches:

- Trabalhar no mesmo projeto de diferentes maneiras.
- Ajudar várias pessoas a trabalhar no mesmo projeto ao mesmo tempo.

Podemos considerar um branch uma linha de desenvolvimento. Um projeto Git pode ter vários branches (ou linhas de desenvolvimento). Cada um desses branches é uma versão autônoma do projeto. Diferentes projetos Git podem usar os branches de maneiras distintas, dependendo das necessidades das pessoas que estiverem trabalhando no projeto.

Um padrão comum para o trabalho com branches é usarmos uma linha de desenvolvimento primária oficial – o branch principal (main) ou primário – e a partir dela criarmos branches secundários, chamados de *branches de tópicos (topic branches)* ou *branches de recursos (feature branches)*, que são utilizados para o trabalho em uma parte específica do projeto. Esses branches de tópicos têm vida curta; eles acabam sendo reunidos ou incorporados novamente ao branch principal e depois são excluídos. Os dois processos que você pode usar para integrar um branch a outro se chamam merging e rebasing. Eles serão abordados com mais detalhes nos

capítulos 5, 9, 10, 11 e 12 (sim, são tópicos grandes!).

Esse padrão de empregar uma linha de desenvolvimento primária é a abordagem que você usará no projeto Rainbow. Para tornar a ideia um pouco mais concreta, examinaremos o Exemplo 4-1 do Projeto Book a fim de ver como os branches podem ser usados no projeto Book.

Exemplo 4-1 do Projeto Book

> Suponhamos que o branch oficial de meu projeto Book fosse o branch main (principal). Não quero adicionar trabalho à linha de desenvolvimento oficial até meu editor tê-lo revisado e aprovado. Logo, sempre que trabalho em um capítulo, crio um branch secundário, trabalho nesse branch, e então o envio para ser revisado pelo editor. Após ele aprovar o trabalho que fiz no branch secundário, posso integrar esse branch ao branch main.
>
> Se em algum momento eu decidir trabalhar no livro com um coautor, cada um de nós poderá trabalhar em seu próprio branch secundário. Só quando o trabalho em um branch secundário específico tiver sido aprovado tanto pelo outro autor quanto pelo editor é que ele será considerado pronto para ser integrado ao branch main.

Agora que você tem uma noção de por que usamos branches, tentaremos entender melhor como eles funcionam no Git.

O QUE SÃO EXATAMENTE OS BRANCHES NO GIT?

Os branches no Git são ponteiros móveis que apontam para commits. Quando listamos os commits de um repositório local usando o comando git log, vemos informações sobre que branches apontam para quais commits. Consulte a seção Acompanhe 4-1 para ver isso explicitamente no repositório rainbow.

[ACOMPANHE 4-1]

❶ rainbow $ **git log**

commit c26d0bc371c3634ab49543686b3c8f10e9da63c5 (**HEAD -> main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red

O que deve ser observado:

- Na saída de git log, perto do hash do commit vemos entre parênteses HEAD -> main.

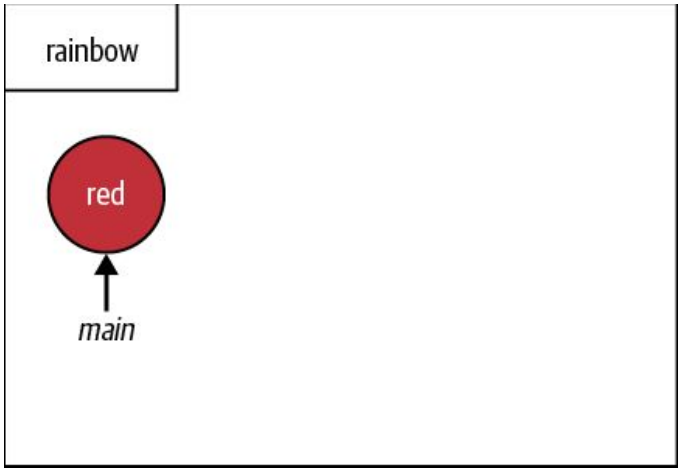
O branch ou os branches que aparecem dentro dos parênteses perto de um hash de commit específico na saída de git log são os que apontam para esse commit.

[NOTA]

HEAD (em letras maiúsculas) não é um branch. Discutiremos o que esse termo representa em “O que é HEAD?”, na página 73.

No repositório rainbow, o branch main aponta para o commit red (vermelho). Isso pode ser visto em Visualize 4-3.

[VISUALIZE 4-3]



O branch main aponta para o commit red no repositório rainbow.

Posteriormente neste capítulo, você continuará fazendo commits no branch main e verá que ele se move para apontar para os novos commits.

[NOTA]

O branch main do repositório rainbow é um branch local. Na primeira parte deste livro, quando você estiver trabalhando apenas com um repositório local no projeto Rainbow, só trabalhará com branches locais. Na segunda parte do livro, após a introdução dos repositórios remotos, aprenderá o conceito de branches remotos e branches de rastreamento remoto.

Para entender melhor o conceito de branches como ponteiros móveis que apontam para commits, consulte a seção Acompanhe 4-2.

[ACOMPANHE 4-2]

❶ Em uma janela do sistema de arquivos, vá até o diretório de projeto rainbow.

❷ Exiba todos os arquivos e diretórios ocultos do diretório de projeto rainbow. Para obter mais informações sobre como visualizar arquivos e diretórios ocultos, consulte “Visualizando o conteúdo de diretórios”, na página 28.

❸ Dentro do diretório de projeto rainbow na janela do sistema de arquivos, acesse .git > refs > heads > main.

❹ Abra o arquivo main. No macOS, o arquivo será aberto automaticamente em um editor de texto básico chamado TextEdit. No Microsoft Windows, você pode optar por abrir o arquivo com um editor de texto básico chamado Bloco de notas (Notepad).

Você só usará o TextEdit e o Bloco de notas para visualizar o conteúdo de alguns dos arquivos do diretório .git. Esses editores de texto ficam separados do editor de texto que você está utilizando para gerenciar os arquivos do projeto Rainbow.

O que deve ser observado:

- Na etapa 4, dentro do arquivo main você verá o hash do commit red no repositório rainbow.

Na etapa 3 da seção Acompanhe 4-2, para chegar ao arquivo main, você navegou para o diretório .git, para o diretório refs e depois para o diretório heads. O termo “refs” significa “referências”. O diretório heads armazena um arquivo para cada branch local do repositório local. Atualmente você só tem um branch local, o branch main; logo, só há um arquivo nesse diretório. Podemos considerar esse arquivo como armazenando a “ponta” (head) desse branch; em outras palavras, o último commit do branch.

Agora que você entendeu melhor o que são os branches e como eles funcionam, examinaremos resumidamente as convenções de nomenclatura do branch primário.

CONHECENDO UM POUCO DA HISTÓRIA DO GIT: MASTER E MAIN

Normalmente, quando inicializamos um repositório local usando apenas o comando git init sem opções, em segundo plano o Git cria um branch chamado master. No entanto, o termo “master” não é considerado terminologia inclusiva; logo, nos últimos anos, uma grande parte da comunidade Git decidiu passar a usar main (ou outros nomes) como o nome do branch padrão.

É por isso que no Capítulo 2, quando você inicializou o repositório rainbow, usou o comando git init com a opção -b e passou o valor main. A palavra *main*

não é especial; você poderia ter optado por dar ao primeiro branch qualquer outro nome passando outro valor para o comando `git init -b`. Quando fez o primeiro commit no repositório `rainbow`, isso atualizou o branch `main` para apontar para o primeiro commit. No projeto Rainbow, o branch `main` é a linha primária de desenvolvimento.

Em sua jornada de aprendizado do Git, você encontrará muitos recursos de aprendizado que ainda se referem ao branch `master`. É importante entender que não há nada de especial nesse branch; é apenas o nome padrão para o primeiro branch criado no Git.

Você está pronto para adicionar uma nova cor ao arquivo `rainbowcolors.txt` e fazer outro commit. Contudo, antes de o fazermos, gostaria de introduzir alguns estados adicionais em que um arquivo de um projeto Git pode estar.

Arquivos não modificados e modificados

No Capítulo 2, introduzi o conceito de arquivos não rastreados e rastreados. O Git sabe da existência do arquivo `rainbowcolors.txt` porque ele foi incluído em um commit e, portanto, é um arquivo rastreado.

Os arquivos rastreados do diretório de trabalho podem estar em um entre dois estados. Os *arquivos não modificados* são os arquivos do diretório de trabalho que não foram editados desde o último commit. Quando um arquivo do diretório de trabalho tiver sido editado (e salvo no editor de texto), ele se tornará um *arquivo modificado*. Desde o último commit, você não editou o arquivo `rainbowcolors.txt`; logo, ele é um arquivo não modificado.

[NOTA]

> Para o Git saber que um arquivo foi editado, este *precisa* ter sido salvo no editor de texto. Se você fez edições em um arquivo, mas não salvou essas alterações em seu editor de texto, o Git o considerará um arquivo não modificado.

No Capítulo 3, você conheceu o comando git status, que exibe o estado do diretório de trabalho e da área de preparação e a diferença entre os dois. Na verdade, o comando git status exibe uma lista de todos os arquivos *modificados* e informa se eles foram ou não adicionados à área de preparação. No entanto, ele não lista arquivos não modificados.

Consulte a seção Acompanhe 4-3 para usar o comando git status e ver se o arquivo rainbowcolors.txt é um arquivo não modificado; em seguida, edite e salve o arquivo, e use git status novamente para ver a alteração do seu estado.

[ACOMPANHE 4-3]

❶ rainbow $ **git status**
On branch main
nothing to commit, working tree clean

❷ No diretório de projeto rainbow em seu editor de texto, abra o arquivo rainbowcolors.txt, adicione “Orange is the second color of the rainbow.” na linha 2 e salve o arquivo.

❸ rainbow $ **git status**
On branch main
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: rainbowcolors.txt
no changes added to commit (use "git add" and/or "git commit -a")

O que deve ser observado:

- Na etapa 1, o arquivo rainbowcolors.txt é um arquivo não modificado. Ele ainda não está sendo listado na saída de git status.
- Na etapa 3, o arquivo rainbowcolors.txt é um arquivo modificado. Agora ele está sendo listado na saída de git status.
- O arquivo rainbowcolors.txt não foi preparado para o commit; em outras palavras, ele não foi adicionado à área de preparação.

Você acabou de ver que o arquivo rainbowcolors.txt passou de um arquivo não modificado para um arquivo modificado quando você o editou e salvou suas alterações. Consulte a Figura 4-1 para ver um exemplo de qual é a aparência atual de seu arquivo rainbowcolors.txt.

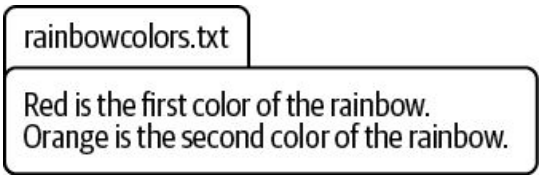


Figura 4-1: Arquivo rainbowcolors.txt depois que você adicionou uma frase sobre a cor orange (laranja).

Na seção Acompanhe 4-4, você adicionará o arquivo rainbowcolors.txt à área de preparação para que ele seja incluído em seu próximo commit; verá então que a saída de git status mudou:

[ACOMPANHE 4-4]

❶ rainbow $ **git add rainbowcolors.txt**

❷ rainbow $ **git status**

On branch main

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

modified: rainbowcolors.txt

O que deve ser observado:

- O arquivo rainbowcolors.txt está preparado para o commit; em outras palavras, ele foi adicionado à área de preparação.

Você concluiu a primeira etapa do processo de commit, que é a inclusão de arquivos na área de preparação. Agora fará outro commit e verá como isso afeta seu branch main.

Fazendo commits em um branch

Você está pronto para fazer seu segundo commit no repositório rainbow. Dessa vez, adicionará a cor orange (laranja); logo, a mensagem do commit será “orange”. Consulte a seção Acompanhe 4-5 para fazer seu commit.

[NOTA]

Se a saída do comando git log ultrapassar o tamanho da janela de linha de comando, você deve pressionar Enter (Return) ou usar a seta para baixo para visualizar os outros commits. Para sair do comando, digite Q.

[ACOMPANHE 4-5]

❶ rainbow $ **git commit -m "orange"**
[main 7acb333] orange
1 file changed, 2 insertions(+), 1 deletion(-)

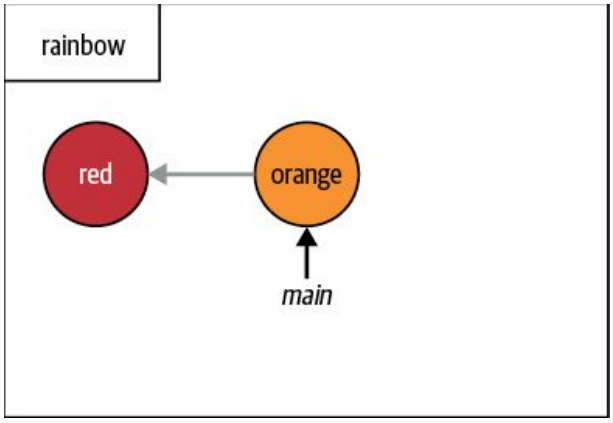
❷ rainbow $ **git log**
commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD -> main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red

O que deve ser observado:

- Você fez um novo commit, o commit orange. No repositório rainbow deste livro, o hash do commit orange é 7acb333f08e12020efb5c6b563b285040c9dba93. O hash de seu commit será diferente.
- O texto HEAD -> main aparece entre parênteses perto do commit orange.

A seção Visualize 4-4 exibe o estado do repositório rainbow após o exercício da seção Acompanhe 4-5.

[VISUALIZE 4-4]



Repositório rainbow após você fazer o commit de orange.

O que deve ser observado:

- Há um segundo commit, o commit orange.
- O commit orange aponta para trás para o commit red.
- O branch main aponta para o commit orange.

Em Visualize 4-4, podemos ver que existe uma seta cinza apontando do commit orange para trás para o commit red. Essa seta cinza representa o link pai. Todo commit, que não seja o primeiro de um repositório, tem um commit pai (alguns commits podem ter mais de um pai; abordaremos isso no Capítulo 5). O commit pai do commit orange é o commit red; é por isso que o commit orange aponta para trás para o commit red.

Esses links pais descrevem como os commits estão associados uns aos outros. Conhecê-los permitirá a você visualizar o histórico de commits e rastrear que trabalho foi feito em quais branches.

[NOTA]

Nos diagramas da seção Visualize, setas cinza são usadas para representar os links pais e setas pretas representam os ponteiros de branches.

Para verificar que commit é o pai de um commit específico, você pode usar o comando `git cat-file` com a opção `-p` e passar um hash de commit: `git cat-file -p <hash_de_commit>`. A maioria dos usuários do Git provavelmente não usará esse comando em seu trabalho diário, mas já que ele é uma boa ferramenta de aprendizado, vamos vê-lo em ação.

Consulte a seção Acompanhe 4-6 para recuperar o hash do pai do commit orange. Para fazê-lo, você terá de passar o hash do commit orange para o comando `git cat-file -p`. A maneira mais fácil de recuperar o hash de um commit específico é examinando a lista de commits produzida pelo comando `git log` e copiando o hash de commit inteiro na linha de comando. Alternativamente, você pode examinar novamente a saída de `git commit` na seção Acompanhe 4-5 e passar os sete primeiros caracteres do hash de commit, que é mostrado lá.

[ACOMPANHE 4-6]

❶ Recupere o hash do commit orange (você pode copiá-lo da saída de git log na seção Acompanhe 4-5). Você deve passar esse hash de commit como argumento para o comando git cat-file -p na etapa 2 desta seção Acompanhe. Pode copiar e colar o hash de commit inteiro ou inserir apenas os sete primeiros caracteres, como mostrado aqui.

❷ rainbow $ **git cat-file -p 7acb333**

```
tree 407fe6a858cd7f157405e013a088fdc1c61f0a40
parent c26d0bc371c3634ab49543686b3c8f10e9da63c5
author annaskoulikari <gitlearningjourney@gmail.com> 1645260127 +0100
committer annaskoulikari <gitlearningjourney@gmail.com> 1645260127 +0100
orange
```

O que deve ser observado:

- Na saída de git cat-file -p, podemos ver que perto de parent está sendo referenciado o hash do commit red deste livro. Em sua saída, será referenciado o seu hash do commit red.

Você fez um segundo commit em seu repositório rainbow, e em Visualize 4-4 observou que, à medida que faz commits em um branch, o ponteiro do branch se move para apontar para o último commit. Agora veremos como criar outros branches para que você possa trabalhar em outras linhas de desenvolvimento.

Criando um branch

Atualmente, você só tem um branch local, chamado main, em seu repositório rainbow. Para listar os branches de um repositório local, pode usar o comando git branch. Para criar um novo branch, passe o nome de um branch que ainda não exista para esse comando. Lembre-se de que os nomes dos branches não podem ter espaços.

[SALVE O COMANDO]

git branch
Lista os branches locais.

git branch <nome_novo_branch>
Cria um branch.

Como ocorre com as mensagens de commit, que discutimos no Capítulo 3, se você estiver trabalhando com outras pessoas em um projeto, deve verificar se existe alguma regra que determine como nomear seus branches. Já que não há regras de nomenclatura específicas para branches no projeto Rainbow, você usará o nome genérico feature para o branch que criará na seção Acompanhe 4-7. No entanto, lembre-se de que em um projeto Git real geralmente o nome do branch é mais descritivo, estando relacionado ao recurso ou tópico no qual estamos trabalhando.

[ACOMPANHE 4-7]

❶ rainbow $ **git branch**

* main

❷ rainbow $ **git branch feature**

❸ rainbow $ **git branch**

feature

* main

❹ rainbow $ **git log**

commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD -> main**, **feature**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 09:42:07 2022 +0100

orange

commit c26d0bc371c3634ab49543686b3c8f10e9da63c5

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 09:23:18 2022 +0100

red

❺ Usando a janela do sistema de arquivo, acesse .git > refs > heads para ver que arquivos existem agora nesse local.

❻ Abra o arquivo feature. Ele deve conter um hash de commit.

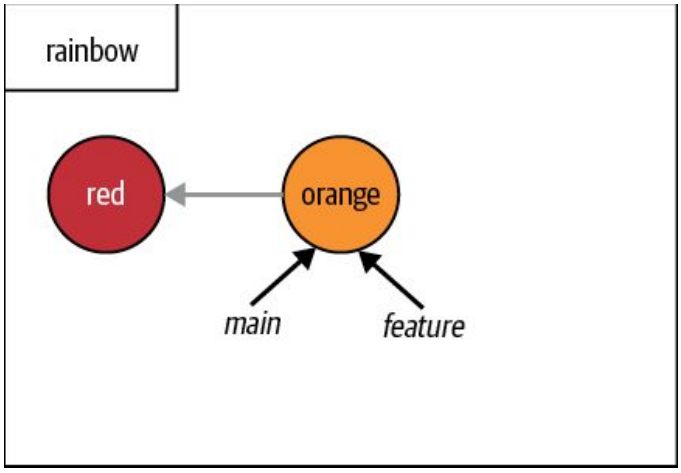
O que deve ser observado:

- Na etapa 2, você criou um novo branch chamado feature que aponta para o commit orange.

O estado do repositório rainbow depois que você concluiu o exercício da seção Acompanhe 4-7 pode ser visto na seção Visualize 4-5.

Na seção Visualize 4-5, podemos ver que agora existem duas setas, que representam os branches main e feature, apontando para o commit orange. Inicialmente um novo branch apontará para o commit em que você estava quando criou o branch. Nesse caso, podemos dizer que você "criou o branch feature a partir do branch main". É por isso que agora tanto o branch feature quanto o branch main apontam para o mesmo commit.

[VISUALIZE 4-5]



Repositório rainbow após você criar o branch feature.

Na saída de git log na seção Acompanhe 4-7, entre parênteses perto do commit orange, você deve ver HEAD —> main, feature. Como sabe, main e feature são branches – mas o que é HEAD?

O que é HEAD?

A qualquer momento, você sempre estará examinando uma versão específica de seu projeto. Portanto, estará em um branch específico que aponta para um commit. HEAD é apenas um ponteiro que informa em que branch estamos. O nome HEAD sempre usa letras maiúsculas, mas isso é somente uma convenção; não é um acrônimo.

[NOTA]

Haverá situações em que você estará em um commit sem que nenhum branch esteja apontando para ele. O Git chama isso de “estado de HEAD desanexado” (detached HEAD state). Esse caso será examinado com mais detalhes em “Fazendo o checkout de commits”, na página 96.

Consulte a seção Acompanhe 4-8 para entender o que é HEAD examinando o diretório .git.

[ACOMPANHE 4-8]

❶ Usando a janela do sistema de arquivos, acesse rainbow > .git > HEAD.

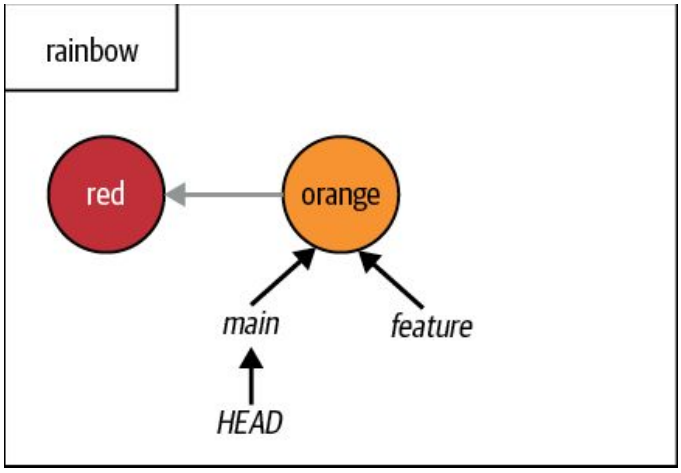
❷ Abra o arquivo HEAD. O conteúdo deve ser ref: refs/heads/main.

O que deve ser observado:

- O arquivo HEAD contém ref: refs/heads/main, que é uma referência ao arquivo main que representa o branch main.

Isso pode ser visto em Visualize 4-6.

[VISUALIZE 4-6]



No repositório rainbow, HEAD está apontando para main.

Na seção Visualize 4-6, uma seta mostra que HEAD está apontando para main. Isso indica que atualmente você está no branch main.

[NOTA]

HEAD (em letras maiúsculas) não deve ser confundido com o diretório heads encontrado em .git > refs > heads. O diretório heads armazena um arquivo para cada branch local do repositório local, enquanto HEAD indica em que branch estamos ao referenciar um dos arquivos do diretório heads. Podemos distingui-los porque sabemos que HEAD está sempre em letras maiúsculas.

Outra maneira de saber em que branch você está atualmente é examinando a saída do comando git branch ou do comando git log. Na saída de git branch, o branch em que você está atualmente terá um asterisco perto dele. Se você examinar a saída da seção Acompanhe 4-7, verá que o asterisco está perto do branch main. Na saída de git log, HEAD apontará entre parênteses para o branch em que você está.

Agora que criou um novo branch, está pronto para começar a usá-lo – mas, por enquanto, ainda está no branch main. A seguir, você se alternará entre branches, movendo o ponteiro HEAD para seu novo branch feature.

Mudando de branches

Para trabalhar em outro branch (ou linha de desenvolvimento) em um projeto Git, você tem de mudar para esse branch. Outra maneira de dizer isso na terminologia do Git seria que você tem de “fazer o checkout” de outro branch.

Atualmente, você tem dois branches, main e feature. No entanto, como acabamos de ver, só porque criou um branch no Git, isso não significa que mudou automaticamente para esse branch. Você deve instruir o Git explicitamente que deseja mudar para um branch. Pode fazer isso usando o comando git switch ou o comando git checkout e passando o nome do branch para o qual deseja mudar.

[NOTA]

Se você tiver uma versão do Git mais antiga que a versão 2.23, não terá acesso ao comando git switch e deve usar o comando git checkout. O comando git checkout está disponível para todos os usuários do Git.

[SALVE O COMANDO]

git switch <nome_branch>
Muda de branches.

git checkout <nome_branch>
Muda de branches.

A única função do comando git switch é mudar de branch, enquanto o comando git checkout executa mais ações. Falaremos sobre git checkout em “Fazendo o checkout de commits”, na página 96.

De agora em diante, nas seções Acompanhe deste livro você usará o comando git switch, já que esse é o comando especializado incluído nas últimas versões do Git para esse fim. Contudo, se quiser poderá optar por utilizar o comando git checkout, porque eles são equivalentes.

O comando git switch (ou git checkout) faz três coisas quando usado para a mudança de branch:

1. Faz o ponteiro HEAD apontar para o branch para o qual estamos mudando.
2. Fornece para a área de preparação um snapshot do commit para o qual estamos mudando.
3. Copia o conteúdo da área de preparação no diretório de trabalho.

[NOTA]

Se precisar de um lembrete do que são a área de preparação e o diretório de trabalho, volte à seção “Áreas do Git”, na página 44.

Resumindo, quando mudamos de branch, acabamos mudando o commit que estamos examinando, se os dois branches apontarem para dois commits diferentes. Atualmente, os dois branches que você tem no repositório rainbow apontam para o mesmo commit; logo, só a primeira ação ocorrerá, e o commit em que estamos não mudará. No Capítulo 5, veremos um exemplo em que todas as três ações listadas aqui ocorrerão.

Consulte a seção Acompanhe 4-9 para mudar para o branch feature.

[ACOMPANHE 4-9]

❶ rainbow $ **git branch**
feature
* main

❷ rainbow $ **git switch feature**
Switched to branch 'feature'

❸ rainbow $ **git branch**
* feature
main

❹ rainbow $ **git log**
commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD -> feature**, **main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red

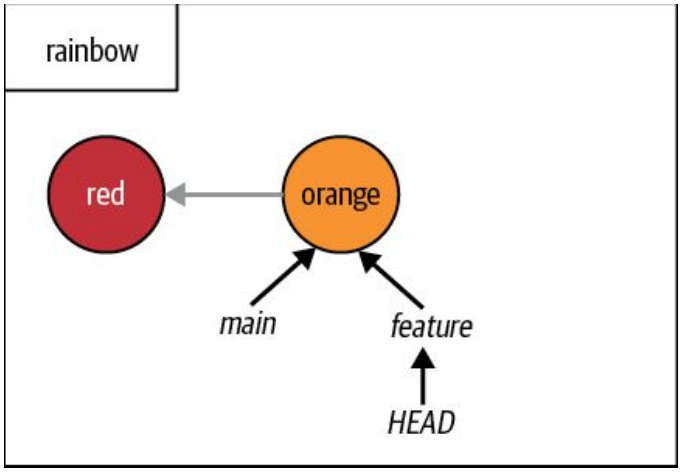
❺ Usando a janela do sistema de arquivos, acesse rainbow > .git > HEAD e abra o arquivo HEAD em uma nova janela. Você verá que o conteúdo do arquivo mudou e agora está referenciando o branch feature, refs/heads/feature.

O que deve ser observado:

- Na etapa 4, a saída de git log mostra que HEAD está apontando para o branch feature.
- Você mudou para o branch feature; logo, é nele que trabalhará agora.

Essas observações são mostradas na seção Visualize 4-7.

[VISUALIZE 4-7]



Repositório rainbow depois que você mudou para o branch feature.

Como você pode ver, a mudança de branch alterou o conteúdo do arquivo HEAD no diretório .git. Isso é mostrado na Figura 4-2.

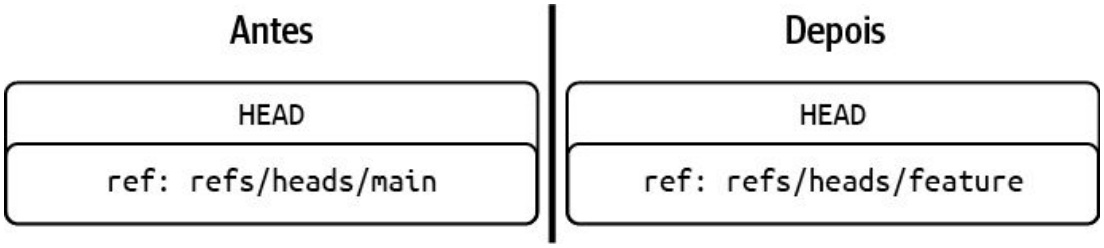


Figura 4-2: Conteúdo do arquivo HEAD antes e depois de você mudar do branch main para o branch feature no repositório rainbow.

Agora que você mudou para o branch feature, veremos o que acontecerá ao trabalhar nele.

Trabalhando em um branch separado

Você está no branch feature, e na seção Acompanhe 4-10 adicionará a cor yellow (amarela) ao repositório rainbow.

[ACOMPANHE 4-10]

❶ No diretório de projeto rainbow em seu editor de texto, no arquivo rainbowcolors.txt, adicione “Yellow is the third color of the rainbow.” na linha 3. Em seguida, salve o arquivo.

❷ rainbow $ **git add rainbowcolors.txt**

❸ rainbow $ **git commit -m "yellow"**

```
[feature fc8139c] yellow
1 file changed, 2 insertions(+), 1 deletion(-)
```

❹ rainbow $ **git log**

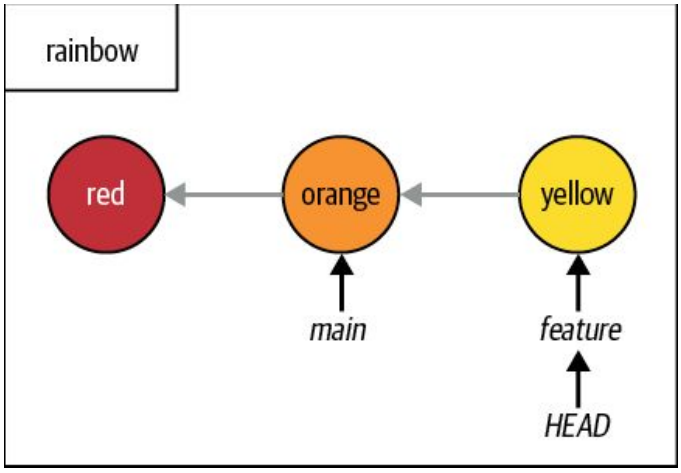
```
commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (HEAD -> feature)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
commit 7acb333f08e12020efb5c6b563b285040c9dba93 (main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red
```

O que deve ser observado:

- O branch feature aponta para o último commit, o commit yellow (amarelo).
- O branch main ainda está apontando para o commit orange (laranja).

Essas observações são mostradas na seção Visualize 4-8.

[VISUALIZE 4-8]



Repositório rainbow depois que você fez o commit yellow.

Como mencionado anteriormente, quando fazemos um commit, o branch em que nos encontramos atualmente é atualizado para apontar para o novo commit. Os branches main e feature não apontam mais para o mesmo commit porque feature foi atualizado para apontar para o novo commit yellow. HEAD continua apontando para o branch feature.

Resumo

Este capítulo introduziu o conceito de branches como diferentes linhas de desenvolvimento e mostrou que, na prática, eles são ponteiros móveis que apontam para commits. Examinamos por que podemos querer usá-los e você aprendeu a listar, criar e mudar branches. Abordamos por que o primeiro branch do repositório rainbow se chama main e por que em outros recursos de aprendizado que ensinam o Git você pode encontrar um branch chamado master. Você também aprendeu que HEAD é um ponteiro que aponta para o branch em que estamos atualmente.

Ao fazer mais commits, você observou que os arquivos rastreados mudam o estado de não modificado para modificado quando são editados. Também observou que é o branch em que estamos que se move para apontar para o último commit feito em um repositório local, e viu como os commits são associados uns aos outros por meio de links pais.

Agora que abordamos como usar branches para trabalhar em diferentes linhas de desenvolvimento simultaneamente, passaremos para o Capítulo 5

para aprender a combinar essas diferentes linhas de desenvolvimento usando o merging.

5

Merging

No último capítulo, você conheceu os branches e viu que eles nos permitem trabalhar no mesmo projeto de diferentes maneiras e colaborar com outras pessoas em um projeto.

Neste capítulo, aprenderá a integrar alterações de um branch em outro branch. No Git, existem duas maneiras de fazer isso, que se chamam merging e rebasing. Abordaremos o rebasing no Capítulo 11; por enquanto, nos concentraremos no merging. Este capítulo introduzirá os dois tipos de merges (merges fast-forward e merges three-way) e você executará um merge fast-forward.

Durante o processo, também aprenderá como o Git evita que percamos alterações que não tenham entrado em um commit, como a mudança de branch pode alterar o conteúdo do diretório de trabalho e como fazer o checkout de commits diretamente.

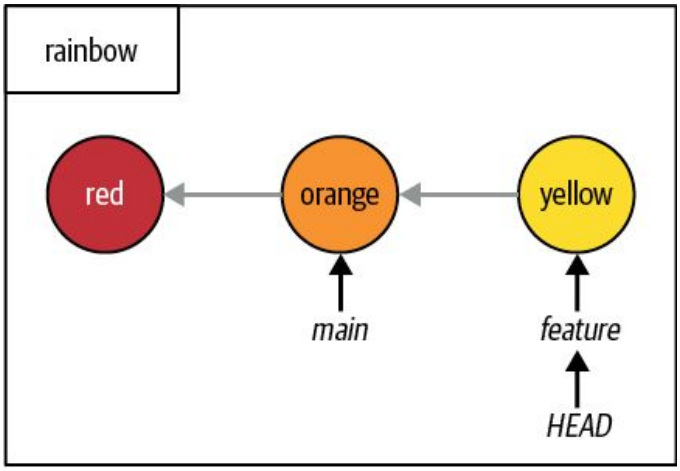
Estado do repositório local

No começo deste capítulo, devem existir três commits e dois branches em seu repositório rainbow e você deve estar no branch feature. O estado atual do repositório rainbow é mostrado na seção Visualize 5-1.

Introdução ao merging

No Capítulo 4, você criou seu primeiro branch, feature, e começou a trabalhar nele. Os branches são um recurso poderoso do Git e graças a eles podemos trabalhar em diferentes partes de um projeto independentemente. No entanto, como integraremos o trabalho que fizemos com o branch main quando estivermos prontos?

[VISUALIZE 5-1]



Repositório rainbow no começo do Capítulo 5, com três commits e dois branches.

O *merging* no Git é uma maneira pela qual podemos passar as alterações de um branch para outro branch. Em um merge, existe um branch que será integrado, chamado *branch de origem*, e um branch no qual ocorrerá o merge, chamado *branch de destino*. O branch de origem é aquele que contém as alterações que serão integradas ao branch de destino. O branch de destino é o que recebe as alterações e, portanto, o único que é alterado nessa operação. Examinaremos como usar o merging no Exemplo 5-1 do Projeto Book.

Exemplo 5-1 do Projeto Book

Suponhamos que eu decidisse junto ao meu editor que, para o projeto Book, sempre que começar a trabalhar em um dos capítulos, criarei um branch secundário fora do branch main. Também concordamos, eu e o editor, que o merge do branch secundário em main só será feito após ele ter revisado meu trabalho no branch.

Por exemplo, digamos que eu decidisse trabalhar no Capítulo 4. Posso criar um branch chamado chapter_four fora do branch main, trabalhar nesse branch e, quando achar que avancei o suficiente, enviar meu trabalho para o editor. Após ele aprovar as alterações, farei o merge do branch chapter_four no branch main em meu repositório local.

O Exemplo 5-1 do Projeto Book mostra que, quando usamos branches, também é essencial aprender a combinar novamente o trabalho como um todo unificado. Agora examinaremos os diferentes tipos de merges que existem.

Tipos de merges

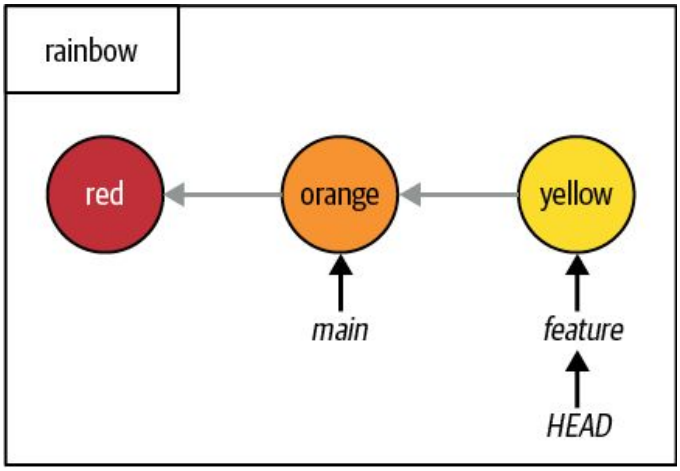
Existem dois tipos de merges:

- Merges fast-forward
- Merges three-way

O fator determinante de qual desses tipos de merges ocorrerá quando você fizer o merge do branch de origem no branch de destino é se os históricos de desenvolvimento dos dois branches divergiram. O histórico de desenvolvimento de um branch pode ser rastreado se seguirmos os links pais dos commits.

No Capítulo 4, expliquei que nos Diagramas Repositório uma seta cinza apontando de um commit para outro indica um link pai (isto é, a seta aponta para trás do commit filho para o commit pai). Em Visualize 5-2, podemos ver que, no repositório rainbow, o commit pai do commit orange (laranja) é o commit red (vermelho). Se usarmos a mesma lógica, o commit orange é o commit pai do commit yellow (amarelo).

[VISUALIZE 5-2]



Links pais no repositório rainbow.

O histórico de desenvolvimento de um branch começa com o commit para o qual ele aponta e se estende para trás ao longo da cadeia de commits. Na seção Visualize 5-2, podemos ver que o histórico de desenvolvimento do branch main é, portanto, composto dos commits orange e red, enquanto o histórico de desenvolvimento do branch feature é composto dos commits yellow, orange e red.

Para ilustrar a diferença entre os merges fast-forward e three-way, começaremos vendo um exemplo de um merge fast-forward no Exemplo 5-2 do Projeto Book.

No Exemplo 5-2 do Projeto Book, veremos que um merge fast-forward é um tipo de merge que ocorre quando os históricos de desenvolvimento dos branches envolvidos no merge não divergem – em outras palavras, quando é possível alcançar o branch de destino seguindo os links pais que compõem o histórico de commits do branch de origem. Durante um merge fast-forward, o Git pega o ponteiro do branch de destino e o move para o commit do branch de origem.

Exemplo 5-2 do Projeto Book

Suponhamos que existissem dois commits, A e B, no branch main do repositório de meu projeto Book, como visto na Figura 5-1.

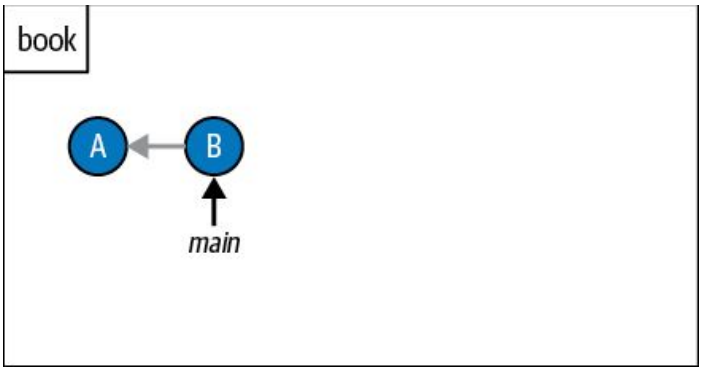


Figura 5-1: Histórico de commits do repositório book com um único branch, o branch main.

Agora, suponhamos que eu criasse o branch chapter_six para trabalhar no Capítulo 6 de meu livro e adicionasse a ele os commits C, D e E, como visto na Figura 5-2.

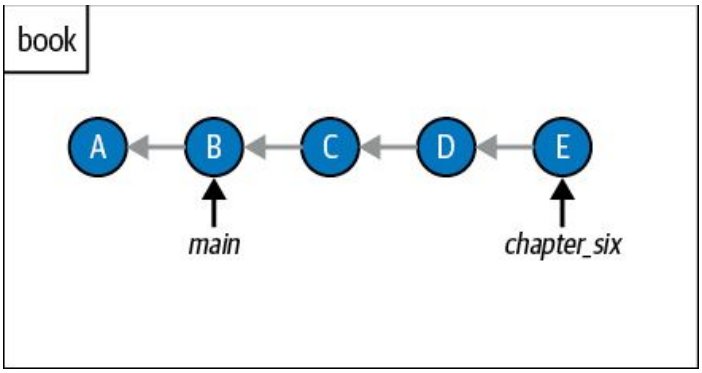


Figura 5-2: Histórico de commits do repositório book depois que trabalhei no branch chapter_six.

Se eu seguir para trás os links pais do branch main, verei que o branch é composto dos commits A e B. Em outras palavras, o histórico de desenvolvimento do branch main é composto dos commits A e B. Por outro lado, o histórico de desenvolvimento do branch chapter_six é composto dos commits A, B, C, D e E.

Quando um branch pode ser alcançado ao longo do histórico de commits de outro branch, dizemos que os históricos de desenvolvimento dos branches *não divergiram*. Se eu seguir para trás os links pais a partir do branch chapter_six, que aponta para o commit E, alcançarei o branch main, que aponta para o commit B. Logo, o branch main e o branch chapter_six não divergiram.

Se eu fizesse o merge do branch chapter_six no branch main, ocorreria um *merge fast-forward*. Durante o merge fast-forward, o ponteiro do branch main se moveria para a frente a fim de apontar para o commit para o qual o branch chapter_six aponta, que é o commit E, como visto na Figura 5-3.

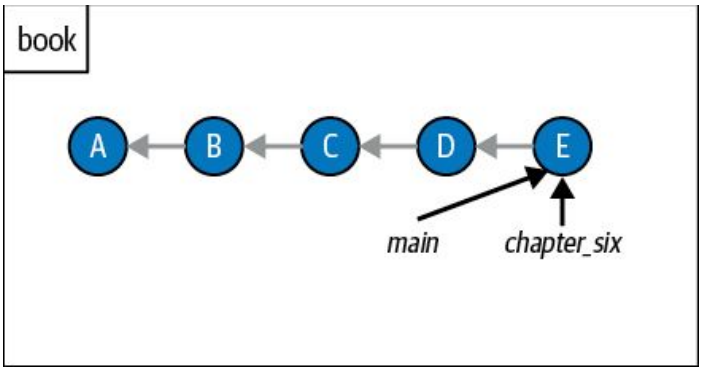


Figura 5-3: Histórico de commits depois que fiz o merge do branch chapter_six no branch main no repositório book.

Nesse exemplo de merge, chapter_six é o branch de origem e main é o branch de destino. A Figura 5-3 mostra que o ponteiro do branch main simplesmente se moveu para a frente, do commit B para o commit E. É por isso que esses tipos de merges são chamados de merges "fast-forward" (de avanço rápido).

Veremos agora um exemplo de um merge three-way no Exemplo 5-3 do Projeto Book.

O Exemplo 5-3 do Projeto Book mostra que um merge three-way é um tipo de merge que ocorre quando os históricos de desenvolvimento dos branches envolvidos divergem. Os históricos de desenvolvimento *terão divergido* quando *não* conseguirmos alcançar o branch de destino seguindo o histórico de commits do branch de origem. Nesse caso, quando você fizer o merge do branch de origem no branch de destino, o Git executará um merge three-way, criando um commit de merge (merge commit) para associar os dois históricos de desenvolvimento; em seguida, ele moverá o ponteiro do branch de destino para o commit de merge.

Exemplo 5-3 do Projeto Book

Suponhamos que os dois últimos commits do branch main em meu repositório book fossem os commits F e G, como mostra a Figura 5-4.

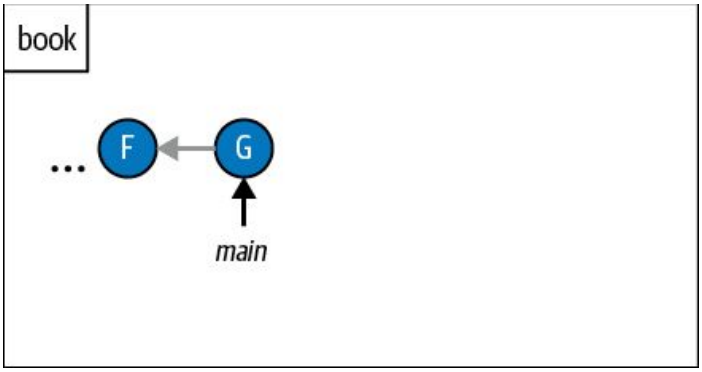


Figura 5-4: Histórico de commits do repositório book com os dois últimos commits no branch main.

Agora suponhamos que eu decidisse criar um branch chapter_eight para trabalhar no Capítulo 8 de meu livro e fizesse os commits H, I e J. Ao mesmo tempo, entretanto, adicionei algum trabalho ao branch main e agora ele está apontando para o commit L. Isso pode ser visto na Figura 5-5.

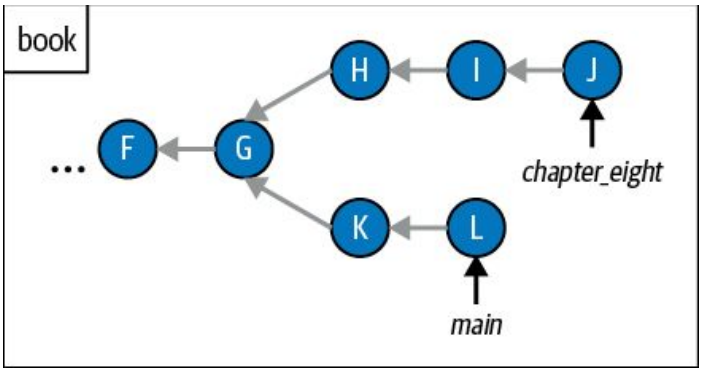


Figura 5-5: Exemplo de um histórico de commits em que algum trabalho foi adicionado ao branch main e ao branch chapter_eight.

Na Figura 5-5, podemos ver que o histórico de desenvolvimento do branch chapter_eight é composto dos commits F, G, H, I e J. Por outro lado, o histórico de commits do branch main é composto dos commits F, G, K e L. Não há como seguirmos para trás os links pais (representados por setas cinza) do branch chapter_eight a fim de alcançar o commit para o qual o branch main aponta, que é o commit L. No Git, para descrever essa situação, dizemos que os históricos de desenvolvimento dos branches *divergiram*.

Se eu fizer o merge do branch chapter_eight no branch main, ele não poderá ser um merge fast-forward porque não há uma maneira de simplesmente movermos o ponteiro do branch para a frente a fim de combinar esses dois históricos de desenvolvimento. Em vez disso, um *commit de merge* (representado pelo commit M) será criado para associar os dois históricos de desenvolvimento, como mostrado na Figura 5-6. Um commit de merge é um commit que tem mais de um pai. Esse é um exemplo de um *merge three-way*.

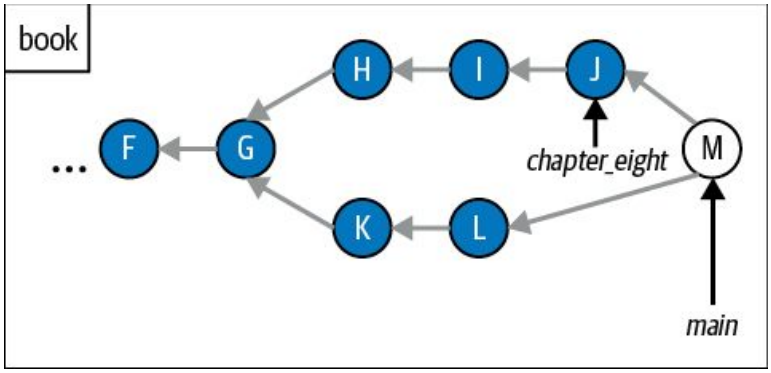


Figura 5-6: Histórico de commits depois que eu fiz o merge do branch chapter_eight no branch main.

Na Figura 5-6, o commit M aponta para trás tanto para o commit J quanto para o commit L. Esse tipo de merge é chamado de merge three-way porque, para executar o merge, o Git se voltará

para os dois commits para os quais os branches envolvidos estão apontando – no caso do repositório book, os commits J e L – e também para o commit que é o ancestral comum desses dois commits, que nesse caso é o commit G. Daí o nome merge “three-way” (de três vias).

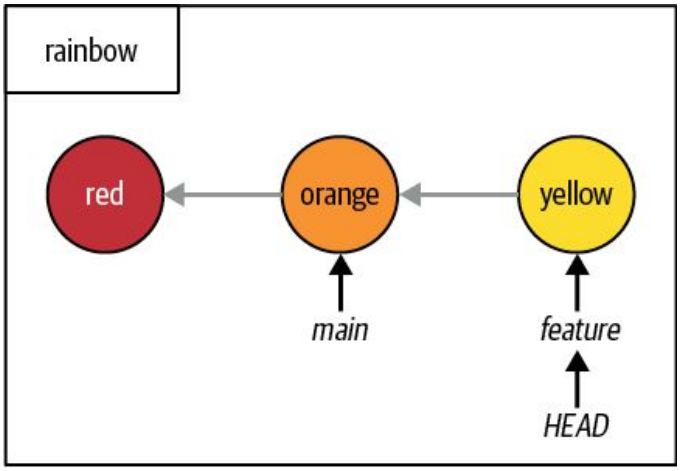
Os merges three-way são um tipo mais complexo de merge em que podemos vivenciar *conflitos de merge*. Esses problemas surgirão se fizermos o merge de dois branches nos quais diferentes alterações tiverem sido feitas nas mesmas partes dos mesmos arquivos ou se um arquivo que foi editado em um branch tiver sido excluído no outro branch.

O Capítulo 9 deste livro foi dedicado a uma explicação mais detalhada dos merges three-way e, no Capítulo 10, você aprenderá mais sobre os conflitos de merge. Veremos um exemplo prático em cada um desses capítulos. No capítulo atual, você tentará executar um merge fast-forward no projeto Rainbow.

Fazendo um merge fast-forward

Para praticar o merging no repositório rainbow, você fará o merge do branch feature no branch main. O branch feature é o branch de origem e o branch main é o branch de destino. Na seção Visualize 5-3, podemos ver que é possível alcançar o branch main seguindo o histórico de commits do branch feature; logo, esse será um merge fast-forward.

[VISUALIZE 5-3]



No repositório rainbow, os históricos de desenvolvimento do branch main e do branch feature não divergem.

Estas são as duas etapas envolvidas em um merge:

1. Passe para o branch para o qual deseja fazer o merge (o branch de destino).
2. Use o comando git merge e passe o nome do branch que está sendo integrado (o branch de origem).

[SALVE O COMANDO]

git merge <nome_branch>
Integra as alterações de um branch em outro branch.

Esta seção o conduzirá pela execução de seu primeiro merge. Durante o processo, você aprenderá mais duas lições importantes sobre o Git: em primeiro lugar, que o Git nos protege de perder os trabalhos que fizemos em arquivos que não inserimos em commits, e, em segundo lugar, que mudar de branch pode fazer o conteúdo do diretório de trabalho mudar.

PASSANDO PARA O BRANCH PARA O QUAL VOCÊ ESTÁ FAZENDO O MERGE

A primeira etapa de um merge é passar para o branch de destino. Você fará o merge de feature em main, logo tem de passar para o branch main. Lembre-se das três coisas que ocorrem quando usamos o comando git switch (ou git checkout) para mudar para um branch:

1. Ele altera o ponteiro HEAD para que aponte para o branch para o qual estamos mudando.
2. Ele preenche a área de preparação com todos os arquivos e diretórios que fazem parte do commit para o qual estamos mudando.
3. Ele copia o conteúdo da área de preparação no diretório de trabalho.

Como essas três etapas indicam, se os branches apontarem para commits diferentes, mudar de branch também mudará o conteúdo do diretório de trabalho. Você verá isso na prática em breve, mas primeiro examinaremos o outro recurso importante que mencionei.

O Git nos impede de perder alterações que não entraram em um commit

Acabei de mencionar que mudar de branch muda o conteúdo do diretório de trabalho. No entanto, e se você tiver arquivos modificados em seu diretório de trabalho (em outras palavras, arquivos que editou) que ainda não inseriu em um commit? Perderá todo o trabalho que fez nesses arquivos se mudar de branch? Felizmente não! O Git não deixará você perder alterações que não entraram em um commit.

Se o Git detectar que mudar de branch o fará perder as alterações do diretório de trabalho que não entraram em um commit, ele o impedirá de mudar de branch e exibirá uma mensagem de erro. Contudo, isso só ocorrerá se os arquivos com alterações não pertencentes a um commit

tiverem alterações conflitantes no branch para o qual você está mudando.

Para mostrar por que isso é importante, examinaremos o Exemplo 5-4 do Projeto Book.

Exemplo 5-4 do Projeto Book

Suponhamos que eu quisesse trabalhar em duas abordagens diferentes para o Capítulo 5 em meu repositório book. Isso significa que teria de trabalhar no arquivo chapter_five.txt. Criei dois branches fora do branch main, um chamado chapter_five_approach_a e o outro com o nome chapter_five_approach_b.

Primeiro, entrei no branch chapter_five_approach_a e trabalhei na abordagem A. Fiz alguns commits nos quais estou editando o arquivo chapter_five.txt.

Em seguida, passei para o branch chapter_five_approach_b para trabalhar na abordagem B em meu editor de texto. Enquanto estava no branch chapter_five_approach_b, fiz várias edições no arquivo chapter_five.txt, mas me esqueci de fazer commits no branch (em outras palavras, não salvei meu arquivo apropriadamente).

Em determinado momento, decidi que queria voltar ao branch chapter_five_approach_a para verificar algo que fiz nele. Se o Git simplesmente me permitisse mudar de branch, a versão do arquivo chapter_five.txt em meu diretório de trabalho, em que trabalhei no branch chapter_five_approach_b, seria substituída pela versão do arquivo no branch chapter_five_approach_a, e *todas* as alterações em que trabalhei na abordagem B seriam perdidas porque me esqueci de inseri-las em um commit.

Felizmente, o Git não permitirá que isso ocorra. Ele me avisará que eu tenho alterações não inseridas em um commit e me lembrará de fazer um commit para não as perder.

Para ver um exemplo mais realista da edição de um arquivo no projeto Rainbow e acompanhar como o Git nos protege da perda de alterações não inseridas em um commit, consulte a seção Acompanhe 5-1.

[ACOMPANHE 5-1]

❶ rainbow $ **git status**
On branch feature
nothing to commit, working tree clean

❷ No diretório de projeto rainbow em seu editor de texto, no arquivo rainbowcolors.txt, adicione “Green is the fourth color of the rainbow.” na linha 4. Em seguida, salve o arquivo.

❸ rainbow $ **git status**
On branch feature
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: rainbowcolors.txt
no changes added to commit (use "git add" and/or "git commit -a")

❹ rainbow $ **git switch main**
error: Your local changes to the following files would be overwritten by checkout:
rainbowcolors.txt
Please commit your changes or stash them before you switch branches.
Aborting

O que deve ser observado:

- Na etapa 1, a saída de git status mostra que não existem arquivos modificados no diretório de trabalho.
- Na etapa 3, a saída de git status indica que rainbowcolors.txt é um arquivo modificado do diretório de trabalho.
- Na etapa 4, podemos ver que o Git *não lhe* permitiu mudar de branch. Ele avisa que Your local changes to the following files would be overwritten by checkout (as alterações locais feitas nos arquivos a seguir seriam sobrepostas pelo checkout) e solicita a você que faça o commit das alterações ou salve-as antes de mudar de branch (Please commit your changes or stash them before you switch branches).

Se o Git lhe tivesse permitido mudar de branch, a versão do arquivo rainbowcolors.txt no diretório de trabalho, que faz menção às cores red (vermelho), orange (laranja), yellow (amarelo) e green (verde), teria sido substituída pela versão do arquivo rainbowcolors.txt do commit orange para o qual o branch main aponta, que só faz menção às cores red e orange. Isso significa que você teria perdido *todo* o trabalho que fez ao adicionar a cor green à sua lista de cores.

Em vez disso, o Git o impediu de perder seu trabalho no arquivo rainbowcolors.txt avisando-o para fazer o commit das alterações.

[NOTA]

O Git não impedirá a mudança de branch se você fizer alterações em arquivos sem salvá-los no editor de texto, porque esses arquivos serão considerados não modificados. Portanto, lembre-se sempre de salvar os arquivos em seu editor de texto quando terminar de trabalhar neles!

Agora daremos prosseguimento à execução do primeiro merge no projeto Rainbow. Nesse caso, presumiremos que você ainda não está pronto para adicionar notas sobre a cor green. Consulte a seção Acompanhe 5-2 e remova a frase sobre essa cor para que não existam arquivos modificados em seu diretório de trabalho.

[ACOMPANHE 5-2]

❶ No diretório de projeto rainbow em seu editor de texto, remova a linha “Green is the fourth color of the rainbow.” do arquivo rainbowcolors.txt e salve-o. Certifique-se de remover qualquer linha ou espaço extra que possa ter adicionado.

❷ rainbow $ **git status**

On branch feature

nothing to commit, working tree clean

O que deve ser observado:

- O comando git status indica que seu diretório de trabalho não tem mais nenhum arquivo modificado.

A seguir, você executará a primeira etapa de um merge, que é mudar para o branch de destino da integração. Ao fazer isso, verá que a mudança de branch alterará os arquivos de seu diretório de trabalho.

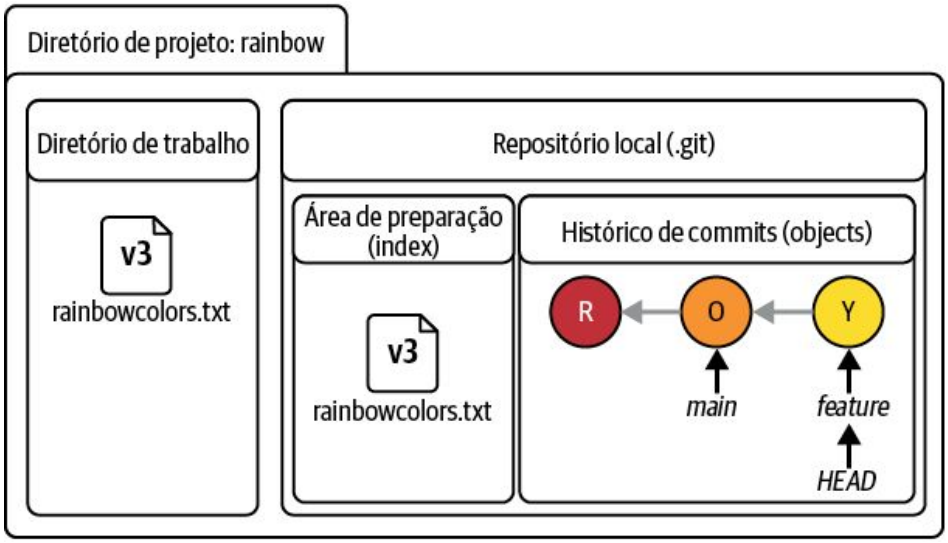
Mudar de branch altera os arquivos do diretório de trabalho

Para ver como a mudança de branch altera os arquivos do diretório de trabalho, consulte a seção Visualize 5-4, que usa o Diagrama Git introduzido no Capítulo 2 para mostrar a aparência atual das diferentes áreas do diretório de projeto rainbow.

[NOTA]

Nos diagramas da seção Visualize em que não houver espaço suficiente para incluir o nome completo de um commit, usarei uma forma abreviada do nome: por exemplo, R para red, O para orange e assim por diante. Para ver uma lista completa das abreviações usadas para cada commit, consulte a Tabela P-1 no Prefácio.

[VISUALIZE 5-4]



Diretório de projeto rainbow antes de você mudar do branch feature para o branch main.

O que deve ser observado:

- A versão do arquivo rainbowcolors.txt no diretório de trabalho e na área de preparação é a que faz menção às cores red, orange e yellow. Ela é representada como a versão 3 (v3) do arquivo.

Agora, consulte a seção Acompanhe 5-3 para mudar do branch feature para o branch main (que aponta para um commit diferente) no repositório rainbow e observe que o conteúdo de seu diretório de trabalho mudou.

[ACOMPANHE 5-3]

❶ Certifique-se de estar com o arquivo rainbowcolors.txt aberto em seu editor de texto e coloque-o perto da janela de linha de comando para ver os dois quando executar os comandos a seguir. Examine o conteúdo do arquivo rainbowcolors.txt.

❷ rainbow $ **git switch main**
Switched to branch 'main'

❸ Examine o conteúdo do arquivo rainbowcolors.txt.

❹ rainbow $ **git log**
commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD -> main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red

O que deve ser observado:

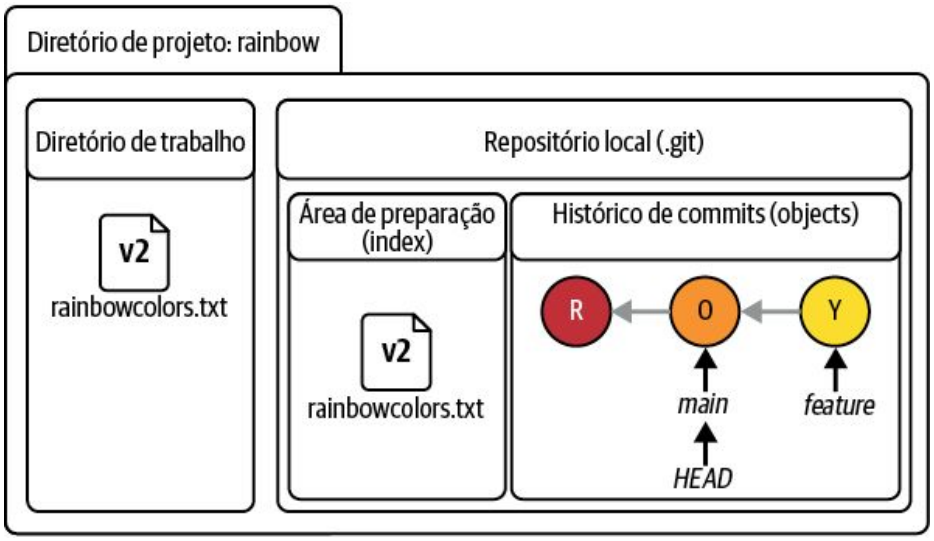
- A versão do arquivo rainbowcolors.txt que estava em seu editor de texto antes de você mudar para o branch main fazia menção às cores red, orange e yellow. A versão que apareceu após você mudar para o branch main só faz menção às cores red e orange. Isso é mostrado na seção Visualize 5-5.
- Na etapa 4, a saída de git log só exibe os commits red e orange. Ela não exibe mais o commit yellow.

O que deve ser observado:

- Você está no branch main e ele está apontando para o commit orange, que inclui a versão do arquivo rainbowcolors.txt que faz menção às cores red e orange. Essa versão é representada como a versão 2 (v2).
- A versão v3 do arquivo rainbowcolors.txt foi substituída pela versão v2 tanto na área de preparação quanto no diretório de trabalho.

Você acabou de ver explicitamente que mudar de branch muda o conteúdo do diretório de trabalho. Antes de passar para a segunda etapa da execução de um merge, examinaremos rapidamente por que a saída de git log na etapa 4 da seção Acompanhe 5-3 só está exibindo os commits red e orange.

[VISUALIZE 5-5]



Diretório de projeto rainbow depois que você mudou do branch feature para o branch main.

Visualizando uma lista de todos os commits

Em “Visualizando uma lista de commits”, na página 58, mencionei que o comando git log exibe uma lista de commits na ordem cronológica inversa. No entanto, na verdade, ele só exibirá uma lista dos commits que você conseguir alcançar ao seguir os links pais a partir do commit em que estiver quando o executar. Para ver uma lista dos commits de *todos* os branches de seu repositório local, você deve usar o comando git log com a opção --all.

[SALVE O COMANDO]

git log --all

Exibe uma lista de commits de todos os branches de um repositório local na ordem cronológica inversa.

Consulte a seção Acompanhe 5-4 para ver uma lista dos commits de todos os branches de seu repositório local.

[ACOMPANHE 5-4]

❶ rainbow $ **git log --all**

```
commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (feature)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
commit 7acb333f08e12020efb5c6b563b285040c9dba93 (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red
```

O que deve ser observado:

- A saída de git log --all exibe os commits red, orange e yellow.

Agora que você está no branch main, pode passar para a segunda etapa da execução de um merge.

USANDO O COMANDO GIT MERGE PARA EXECUTAR UM MERGE

Consulte a seção Acompanhe 5-5 para executar o merge, passando para o comando git merge o nome do branch que você está integrando – o branch de origem, que é feature.

[ACOMPANHE 5-5]

❶ Certifique-se de que a janela de seu editor de texto esteja aberta perto da janela de linha de comando com uma visualização do arquivo rainbowcolors.txt para poder ver os dois ao executar os comandos a seguir. Examine o conteúdo do arquivo rainbowcolors.txt.

❷ rainbow $ **git merge feature**

```
Updating 7acb333..fc8139c
Fast-forward
rainbowcolors.txt | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)
```

❸ Examine o conteúdo do arquivo rainbowcolors.txt.

❹ rainbow $ **git log**

```
commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (HEAD -> main, feature)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
commit 7acb333f08e12020efb5c6b563b285040c9dba93
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red
```

O que deve ser observado:

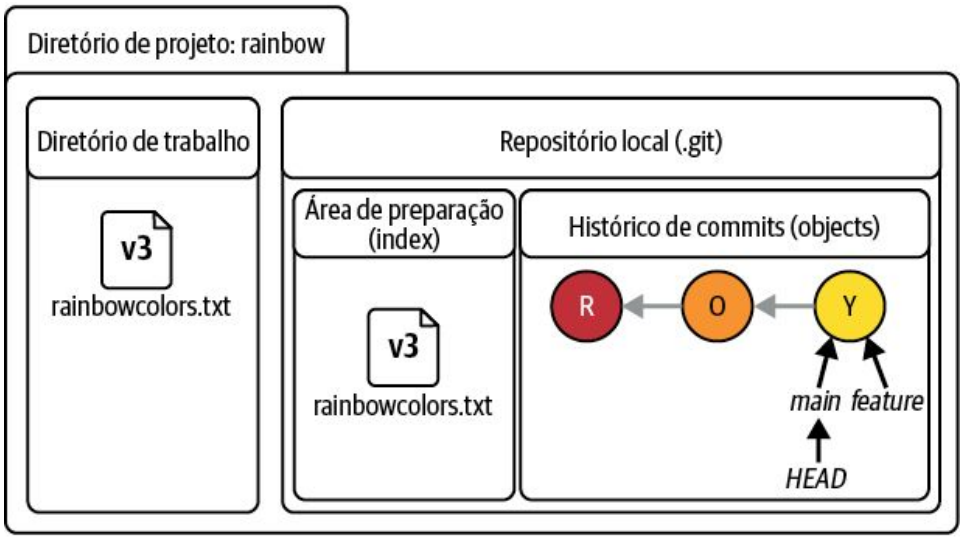
- A saída de git merge menciona Updating 7acb333..fc8139c e Fast-forward. Isso quer dizer que o Git atualizou o commit para o qual o branch main está apontando e que esse foi um merge fast-forward. Lembre-se de que os hashes de commit na sua saída serão diferentes dos exibidos neste livro.
- A saída de git log mostra que main aponta para o commit yellow. Em seu editor de texto, você verá que o diretório de trabalho contém a versão do arquivo rainbowcolors.txt que faz parte do commit yellow.
- Você fez o merge do branch feature no branch main, mas ele ainda existe; não foi excluído automaticamente.

Todas essas observações são mostradas na seção Visualize 5-6.

Você fez o merge de feature em main, mas, no Git, fazer o merge de um branch não o exclui. Você deve excluir explicitamente um branch se não quiser

mais usá-lo. Por enquanto, manteremos o branch feature, e veremos como excluir branches no Capítulo 8. O próximo tópico que desejo abordar aqui é como fazer o checkout de commits.

[VISUALIZE 5-6]



O diretório de trabalho e o repositório local após o merge de feature em main.

Fazendo o checkout de commits

No Capítulo 4, mencionei que o comando git checkout pode ser usado tanto para a mudança de branch quanto para outras ações. Uma das outras coisas que podem ser feitas com o comando git checkout é o checkout de commits.

Você está no branch main, que aponta para o commit yellow. No entanto, e se quisesse examinar uma versão mais antiga de seu projeto? Por exemplo, e se quisesse ver o estado do projeto no commit orange?

Atualmente nenhum branch está apontando para o commit orange; logo, você não pode mudar para ele passando para outro branch. Em vez disso, pode *fazer o checkout* desse commit usando o comando git checkout e passando o hash do commit orange.

[SALVE O COMANDO]

git checkout <hash_commit>
Faz o checkout de um commit.

Quando você fizer isso, o comando git checkout executará três ações semelhantes às que descrevi anteriormente neste capítulo e no Capítulo 4:

1. Alterará o ponteiro HEAD para que aponte para o *commit* para o qual você está mudando.
2. Preencherá a área de preparação com todos os arquivos e diretórios que fazem parte do commit para o qual você está mudando.
3. Copiará o conteúdo da área de preparação no diretório de trabalho.

A principal diferença entre essas etapas e as mencionadas anteriormente é que na etapa 1 o ponteiro HEAD apontará diretamente para um commit em vez de apontar para um branch. Isso significa que você estará em algo que o Git chama (estranhamente) de *estado de HEAD desanexado*, que lhe permitirá examinar qualquer commit – ou, em outras palavras, qualquer versão de seu projeto – em todo o repositório.

Como essas etapas indicam, fazer o checkout de commits muda o conteúdo do diretório de trabalho como ocorre com a mudança de branch.

[NOTA]

Não é recomendável que você faça alterações em um repositório enquanto estiver no estado de HEAD desanexado (isto é, enquanto não estiver em um branch). É preferível fazer commits em branches porque é mais fácil nos lembrarmos de e referenciarmos branches do que hashes de commit, e porque o Git foi projetado para ser usado com branches. Logo, ao fazer o checkout de um commit, geralmente criamos e passamos para um novo branch que aponte para esse commit, para sair do estado de HEAD desanexado.

Consulte a seção Acompanhe 5-6 para fazer o checkout do commit orange e ver como é entrar no estado de HEAD desanexado.

[ACOMPANHE 5-6]

❶ Recupere o hash de seu commit orange (você pode copiá-lo da saída de git log). Certifique-se de usar seu hash de commit na etapa 2 dessa seção Acompanhe.

❷ rainbow $ **git checkout 7acb333f08e12020efb5c6b563b285040c9dba93**

Note: switching to '7acb333f08e12020efb5c6b563b285040c9dba93'.

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using -c with the switch command. Example:

git switch -c <new-branch-name>

Or undo this operation with:

git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 7acb333 orange

❸ rainbow $ **git log --all**

commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**main**, **feature**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 10:09:59 2022 +0100

yellow

commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 09:42:07 2022 +0100

orange

commit c26d0bc371c3634ab49543686b3c8f10e9da63c5

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 09:23:18 2022 +0100

red

❹ Examine o conteúdo do arquivo rainbowcolors.txt.

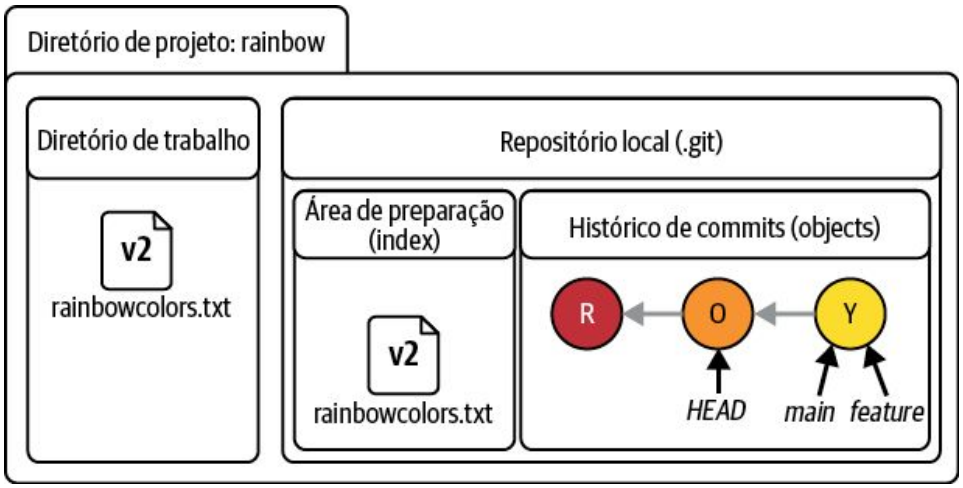
O que deve ser observado:

- Na etapa 2, a saída de git checkout informa que You are in 'detached HEAD' state (Você está no estado de ‘HEAD desanexado’). A saída também faz referência a uma modificação do comando git switch, que discutiremos na próxima seção.
- Na etapa 3, a saída de git log mostra que agora HEAD aponta para o commit orange.
- A versão do arquivo rainbowcolors.txt em seu diretório de trabalho é a que faz parte do commit orange (representada por v2), que só faz menção às

cores red e orange.

Essas observações são mostradas na seção Visualize 5-7.

[VISUALIZE 5-7]



O diretório de trabalho e o repositório local depois que você fez o checkout do commit orange e entrou no estado de HEAD desanexado.

[NOTA]

Pode parecer assustador o fato de o conteúdo do diretório de trabalho mudar sempre que você mudar de branch ou fizer o checkout de um commit diretamente – mas lembre-se de que nada foi perdido. Todos os seus commits estarão seguramente armazenados no histórico de commits e você sempre poderá mudar para outro branch ou fazer o checkout de outro commit para ver o que estava examinando antes.

Você acabou de ver como é fazer o checkout de um commit diretamente em vez de fazer o checkout de um branch. Agora consulte a seção Acompanhe 5-7 para voltar para o branch main e sair do estado de HEAD desanexado.

[ACOMPANHE 5-7]

❶ rainbow $ **git switch main**
Previous HEAD position was 7acb333 orange
Switched to branch 'main'

❷ rainbow $ **git log**
commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> main**, **feature**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
commit 7acb333f08e12020efb5c6b563b285040c9dba93
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red

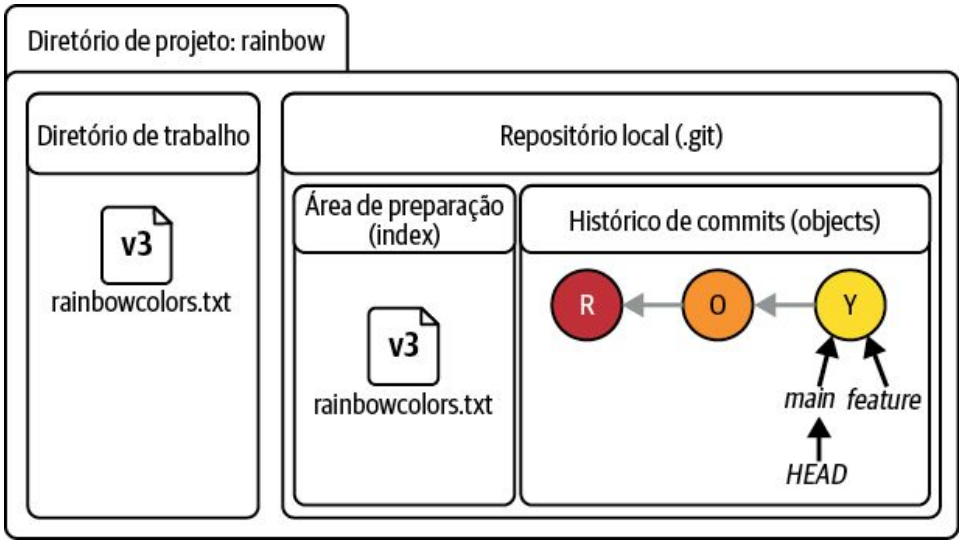
❸ Examine o conteúdo do arquivo rainbowcolors.txt.

O que deve ser observado:

- A versão do arquivo rainbowcolors.txt em seu diretório de trabalho é a que faz parte do commit yellow (representada por v3), que faz menção às cores red, orange e yellow.
- Você voltou ao branch main.

Esses detalhes podem ser vistos na seção Visualize 5-8.

[VISUALIZE 5-8]



O diretório de trabalho e o repositório local depois que você mudou para o branch main e saiu do estado de HEAD desanexado.

Criando um branch e mudando para ele ao mesmo tempo

No Capítulo 4, você aprendeu a criar um novo branch usando o comando git branch e a utilizar o comando git switch (ou git checkout) para mudar para esse branch.

Na etapa 2 da seção Acompanhe 5-6, a saída de git checkout exibiu: If you want to create a new branch to retain commits you create, you may do so (now or later) by using -c with the switch command. Example: git switch -c <*new-branch-name*> (Se você quiser criar um novo branch para reter os commits que gerar, pode fazê-lo (agora ou depois) usando -c com o comando switch. Por exemplo: git switch -c <*nome_novo_branch*>).

Isso ocorreu porque é realmente possível usar o comando git switch ou git checkout para criar um branch e mudar para ele ao mesmo tempo. Se você utilizar o comando git switch, deve empregar a opção -c (que significa “criar”), e se usar o comando git checkout, deve utilizar a opção -b.

[SALVE O COMANDO]

git switch -c <nome_novo_branch>
Cria um novo branch e muda para ele.

git checkout -b <nome_novo_branch>
Cria um novo branch e muda para ele.

Você treinará usar esse comando em “Preparando-se para fazer um pull request”, na página 245, quando criar um novo branch.

Resumo

Neste capítulo, abordamos o que é merging e por que o executamos. Introduzi os dois tipos de merges – fast-forward e three-way – e expliquei que o tipo de merge que será executado vai depender dos históricos de desenvolvimento dos branches envolvidos.

Para concluir, você executou um merge fast-forward por conta própria no repositório rainbow e, durante o processo, entendeu que o Git nos protege de perder alterações que não entraram em um commit, viu que mudar de branch pode mudar o conteúdo do diretório de trabalho e aprendeu a fazer o checkout de commits.

Este capítulo marca o fim da primeira parte do livro, na qual você trabalhou exclusivamente com repositórios locais. A seguir, no Capítulo 6, você preparará sua conta de serviço de hospedagem e detalhes de autenticação para começar a trabalhar com repositórios remotos.

6

Serviços de hospedagem e autenticação

No último capítulo, você conheceu o merging e como esse recurso do Git permite integrar alterações de um branch em outro branch.

Até agora, só discutimos como trabalhar em repositórios locais, e você usou apenas o repositório rainbow, que é um repositório local. O capítulo atual marca o começo da segunda parte do livro, em que você trabalhará com serviços de hospedagem e repositórios remotos.

Neste capítulo, você escolherá um serviço de hospedagem e preparará os detalhes de autenticação que usará para se conectar com repositórios remotos nesse serviço utilizando o protocolo HTTPS (Hypertext Transfer Protocol Secure) ou SSH (Secure Shell). As informações necessárias para executar essas tarefas estão contidas neste capítulo assim como nos recursos do repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*).

[NOTA]

> Se você trabalha em uma empresa que usa o Git e um serviço de hospedagem, talvez já tenha uma conta desse serviço usando o endereço de email da empresa. No entanto, para fazer os exercícios deste livro, recomendo usar uma conta pessoal em um serviço de hospedagem em vez de uma conta da empresa. Isso é importante porque a empresa pode ter definido configurações adicionais na conta de seu serviço de hospedagem, as quais posteriormente podem complicar os exercícios do livro.

Se já trabalha com o serviço de hospedagem que deseja usar para fazer os exercícios do livro e está com os detalhes de autenticação definidos para se conectar com repositórios remotos por um protocolo seguro, pode saltar este capítulo e passar para o Capítulo 7 para aprender mais sobre como criar e fazer um push (envio) para repositórios remotos.

Se ainda não escolheu que serviço de hospedagem deseja usar, não definiu detalhes de autenticação para um protocolo específico ou deseja saber mais sobre os protocolos disponíveis, continue lendo.

Serviços de hospedagem e repositórios remotos

No Capítulo 2, mencionei dois tipos de repositórios: locais e remotos. Os repositórios locais ficam no computador, enquanto os repositórios remotos residem em um serviço de hospedagem na nuvem.

Também mencionei que os serviços de hospedagem são empresas que fornecem hospedagem para projetos usando o Git. Neste livro, darei informações sobre os três principais serviços de hospedagem: GitHub, GitLab e Bitbucket.

Para transferir dados entre um repositório local e um repositório remoto em um serviço de hospedagem, você deve se conectar e se autenticar usando o SSH ou o HTTPS. Do Capítulo 7 em diante, você fará o download e o upload de dados de e para um repositório remoto utilizando comandos como `git push`, `git clone`, `git fetch` e `git pull`. Para usar esses comandos e se conectar com o repositório remoto, precisa ter preparado antecipadamente detalhes de autenticação para o protocolo que quiser empregar.

Neste capítulo, primeiro você escolherá um serviço de hospedagem e depois definirá detalhes de autenticação para se conectar com os repositórios remotos pelo HTTPS ou o SSH.

Definindo uma conta de serviço de hospedagem

Como mencionei anteriormente, se você já trabalha com um serviço de hospedagem e deseja usá-lo para os exercícios do livro, pode simplesmente fazer login em sua conta no site do serviço.

Se nunca trabalhou com um serviço de hospedagem, terá de escolher qual deseja usar e criar uma conta. Os três principais serviços de hospedagem são o GitHub, o GitLab e o Bitbucket. Se não tiver certeza de qual usar, recomendo utilizar o GitHub já que ele é o mais popular e é o que usei para os exemplos de saída do projeto Rainbow neste livro.

Consulte a seção Acompanhe 6-1 para tomar sua decisão sobre o serviço de hospedagem e fazer login.

[ACOMPANHE 6-1]

❶ Selecione o serviço de hospedagem em que deseja criar um repositório remoto e acesse o site. Faça login ou crie uma conta. Nesses exemplos, usaremos o GitHub.

Agora que você selecionou seu serviço de hospedagem e fez login, deve escolher que protocolo deseja usar para se conectar com os repositórios remotos e preparar os detalhes de autenticação para esse protocolo.

Definindo credenciais de autenticação

Quando você criar um repositório remoto, poderá fazer alterações nele de duas maneiras:

1. Fazendo login no serviço de hospedagem por meio do site e executando as alterações diretamente no local.
2. Executando as alterações em seu repositório local e fazendo o upload dessas alterações no repositório remoto no serviço de hospedagem.

Nos dois casos, você terá de se *autenticar*, ou verificar sua identidade. A autenticação é importante para o controle de quem fará login em sua conta no serviço de hospedagem e quem fará o upload de alterações em um repositório remoto.

No primeiro caso, você usará seu nome de usuário ou endereço de email e sua senha para se autenticar e fazer login na conta do serviço de hospedagem.

No entanto, e no segundo caso? Como seu serviço de hospedagem saberá que deve permitir que você faça uploads de um repositório local para um repositório remoto?

A resposta é que você terá de se autenticar por meio do protocolo que decidiu usar. Neste capítulo, abordaremos os protocolos HTTPS e SSH. Só será preciso configurar um deles, e você pode escolher o que preferir.

[NOTA]

No Capítulo 7, você percorrerá o processo de criar um repositório remoto para seu projeto Rainbow, o que produzirá duas URLs: uma do HTTPS e a outra do SSH. Você terá de usar a URL do protocolo que decidiu utilizar.

Primeiro examinaremos o protocolo HTTPS. Se você só estiver interessado em conhecer e usar o protocolo SSH, pode saltar a próxima seção e ir direto para “Usando o SSH”, na página 106. Lembre-se de que não é preciso conhecer bem esses protocolos para poder usá-los.

Se não tiver certeza do protocolo que deve escolher, recomendo configurar o protocolo HTTPS para executar os exercícios deste livro porque o processo é um pouco mais simples e os exemplos do livro usarão o protocolo HTTPS. Seja qual for o protocolo que você configurar, também poderá configurar o outro em seu trabalho futuro com o Git; logo, no momento essa não é uma decisão crítica.

USANDO O HTTPS

O protocolo HTTPS usa um nome de usuário e algum tipo de senha (ou credencial de autenticação) para permitir o estabelecimento de uma conexão segura com repositórios remotos. No passado, todos os serviços de hospedagem permitiam usar na autenticação HTTPS a mesma senha que utilizávamos no login em nossa conta no serviço (que chamaremos de *senha da conta*). No entanto, o GitHub e o Bitbucket não permitem mais isso; eles requerem a criação de outra credencial de autenticação.

No GitHub, a credencial de autenticação se chama *token de acesso pessoal*. No Bitbucket, ela se chama *senha do aplicativo* (app password). Com o GitLab, ainda podemos usar apenas a senha da conta para nos autenticarmos. Consulte a Tabela 6-1 na qual há uma visão geral dos três serviços de hospedagem mais comuns e das credenciais de acesso necessárias para a autenticação pelo HTTPS.

Tabela 6-1: Os serviços de hospedagem e as credenciais de acesso necessárias para a autenticação pelo HTTPS

Serviço de hospedagem	nome de usuário	senha
GitHub	Endereço de email ou nome de usuário	Token de acesso pessoal
GitLab	Endereço de email ou nome de usuário	Senha da conta
Bitbucket	Endereço de email ou nome de usuário	Senha do aplicativo

Consulte a seção Acompanhe 6-2 para preparar sua credencial de autenticação.

[ACOMPANHE 6-2]

❶ Se você estiver usando o GitHub ou o Bitbucket e optou pelo protocolo HTTPS, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) para obter mais informações sobre como definir sua credencial de autenticação e pode ignorar a próxima seção.

Se decidiu usar o protocolo SSH, passe para a seção “Usando o SSH”.

USANDO O SSH

O protocolo SSH utiliza um par de chaves SSH pública e privada para permitir o estabelecimento de uma conexão segura com repositórios remotos. As três etapas principais da definição do acesso SSH são:

1. Criar um par de chaves SSH em seu computador.
2. Adicionar a chave SSH privada ao agente SSH.
3. Adicionar a chave SSH pública à conta do serviço de hospedagem.

Consulte a seção Acompanhe 6-3 para executar as três etapas e definir o acesso SSH.

[ACOMPANHE 6-3]

❶ Se você decidiu usar o protocolo SSH, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) para obter mais informações sobre como definir os detalhes de autenticação para se conectar por meio desse protocolo.

[NOTA]

Compartilhar sua chave SSH privada seria um risco à segurança. Considere-a uma senha e não a compartilhe com ninguém.

Agora que você preparou seus detalhes de autenticação para se conectar com repositórios remotos pelo HTTPS ou SSH, pode passar para o próximo capítulo para criar o primeiro repositório remoto do projeto Rainbow.

Resumo

Neste capítulo, você selecionou que serviço de hospedagem deseja usar para fazer o restante dos exercícios do livro e preparou os detalhes de autenticação necessários para se conectar com repositórios remotos pelo HTTPS ou SSH. No Capítulo 7, aprenderá a criar repositórios remotos e começaremos a examinar como trabalhar com outras pessoas em projetos Git.

7

Criando um repositório remoto e fazendo push para ele

No último capítulo, vimos como escolher o serviço de hospedagem que será usado, criar uma conta e definir detalhes de autenticação para o estabelecimento de uma conexão segura com repositórios remotos pelo HTTPS ou SSH.

Neste capítulo, examinaremos as diferentes maneiras pelas quais você pode usar repositórios locais ou remotos para começar a trabalhar em um projeto Git e por que os repositórios remotos são úteis. Você executará as etapas para criar um repositório remoto para o projeto Rainbow e fará o upload de alguns dados nele. Recursos adicionais para ajudá-lo enquanto percorre o capítulo estão disponíveis no repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*).

Estado do repositório local

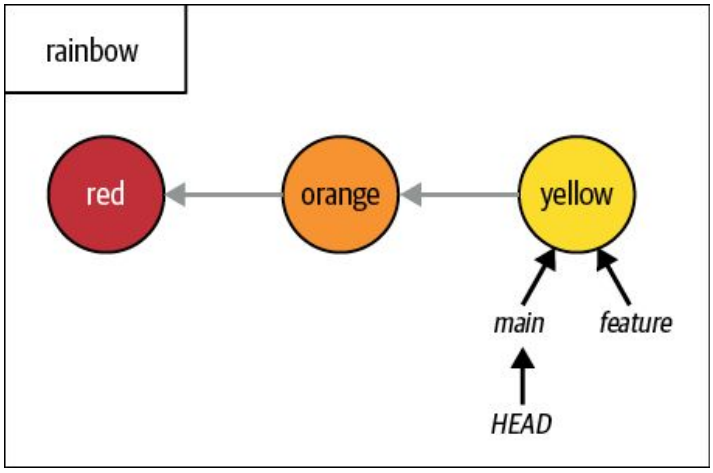
No começo deste capítulo, você deve ter três commits e dois branches em seu repositório rainbow e deve estar no branch main. O estado atual do repositório rainbow é mostrado na seção Visualize 7-1.

As duas maneiras de começar a trabalhar em um projeto Git

No Capítulo 2, você aprendeu que existem dois tipos de repositórios: locais, que são armazenados em um computador, e remotos, que residem em um serviço de hospedagem. Você pode trabalhar em um projeto usando o Git a partir de um repositório local ou remoto. Até agora, no projeto Rainbow, adotamos a primeira abordagem porque você começou a trabalhar em seu projeto a partir de um repositório local. No Capítulo 8, examinaremos um

exemplo prático em que começaremos a trabalhar em um projeto Git a partir de um repositório remoto. Fornecerei uma visão geral de cada abordagem nesta seção.

[VISUALIZE 7-1]



Repositório rainbow no início do Capítulo 7.

COMEÇANDO DE UM REPOSITÓRIO LOCAL

Para começar a trabalhar em um projeto Git a partir de um repositório local, primeiro você deve criar um repositório local em um computador usando o comando git init e fazer pelo menos um commit. Em seguida, deve criar um repositório remoto em um serviço de hospedagem. Poderá então fazer o upload de dados do repositório local para o repositório remoto.

No Git, usamos o termo *push* ou *pushing* para nos referir ao processo de fazer o upload de dados de um repositório local para um repositório remoto, e o comando que utilizamos para isso é git push.

[SALVE O COMANDO]

git push
Faz o upload de dados em um repositório remoto.

Essa é a abordagem que você adotará no projeto Rainbow neste capítulo, e ela é mostrada na Figura 7-1.

Um exemplo de uma situação em que você teria começado a trabalhar em um projeto Git a partir de um repositório local seria se tivesse um projeto em seu computador, que não fosse um repositório, no qual já estivesse trabalhando há algum tempo – em outras palavras, o Git não está sendo usado para fazer o controle com versões – e decidisse começar a controlá-lo por meio de versões com o Git. Nesse caso, usaria o comando `git init` para inicializar o repositório local em seu computador, faria um commit inicial, e executaria o restante das etapas para criar um repositório remoto e fazer o upload de dados nele.



Figura 7-1: Começando a trabalhar em um projeto Git a partir de um repositório local.

A outra maneira de começar a trabalhar em um projeto Git é a partir de um repositório remoto.

COMEÇANDO DE UM REPOSITÓRIO REMOTO

Para começar a trabalhar em um projeto Git a partir de um repositório remoto, você pode encontrar um repositório remoto em que queira trabalhar ou pode criar um novo repositório remoto em um serviço de hospedagem. Em seguida, *clone* (em outras palavras, copie) o repositório remoto em seu computador, o que criará um repositório local. Essa é a abordagem que examinaremos no Capítulo 8 e que é mostrada na Figura 7-2.

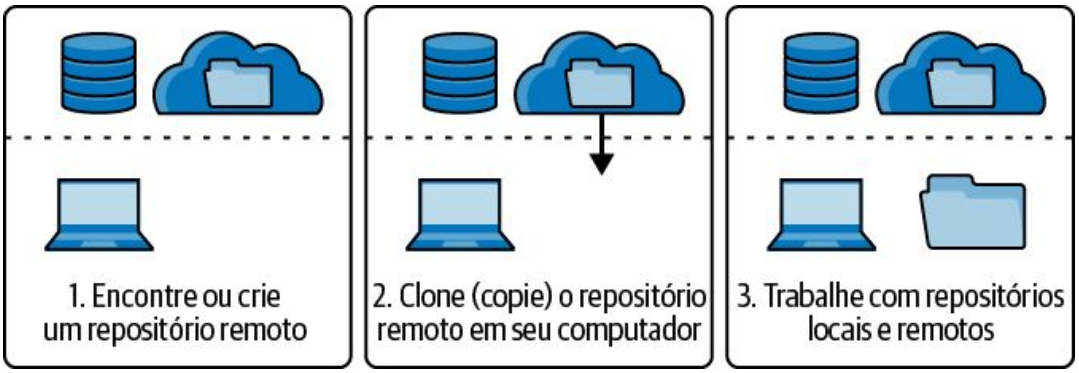


Figura 7-2: Começando a trabalhar em um projeto Git a partir de um repositório remoto.

Um exemplo de uma situação em que você teria começado a trabalhar em um projeto a partir de um repositório remoto seria se um amigo seu estivesse trabalhando em um projeto que tivesse um repositório remoto e lhe pedisse para fazer contribuições. Nesse caso, você teria de pedir ao seu amigo para lhe informar onde fica o repositório remoto e, em seguida, cloná-lo (ou copiá-lo) em seu computador e começar a trabalhar nele.

Até agora, você trabalhou no projeto Rainbow exclusivamente com um repositório local. Posteriormente, neste capítulo, criaremos um repositório remoto e faremos o upload de dados nele. No entanto, primeiro quero falar rapidamente sobre a interação entre repositórios locais e remotos.

Interação entre repositórios locais e remotos

Os repositórios locais e os repositórios remotos agem separadamente. Quando se trabalha com eles, é importante saber que nenhuma interação ocorre automaticamente. Em outras palavras, não ocorrerá nenhuma atualização automática enviada do repositório local para o repositório remoto, e, se invertermos a situação, nenhuma atualização enviada do repositório remoto para o repositório local ocorrerá automaticamente.

Não existe uma conexão dinâmica entre os dois. Qualquer alteração ocorrida em um deles será resultado da execução explícita de comandos. Neste capítulo e no próximo, você aprenderá quais são esses comandos.

Antes de dar prosseguimento e criar um repositório remoto, discutiremos por que você poderia querer criar um.

Por que usar repositórios remotos?

Existem três razões principais para os repositórios remotos serem úteis e importantes no trabalho em um projeto Git. Eles permitem:

- Fazer facilmente o backup do projeto em um lugar que não seja o seu computador.
- Acessar um projeto Git a partir de vários computadores.
- Colaborar com outras pessoas em projetos Git.

Examinaremos o Exemplo 7-1 do Projeto Book para ver por que você pode querer criar um repositório remoto para um projeto Git.

Exemplo 7-1 do Projeto Book

Suponhamos que eu só tivesse um repositório local em meu computador para o projeto Book e nenhum repositório remoto. Se alguém roubasse meu laptop ou a unidade de disco rígido parasse de funcionar, eu perderia todo o trabalho que fiz no livro. Os repositórios remotos são um bom mecanismo de backup.

Um repositório remoto também me permitiria acessar meu projeto a partir de outros computadores. Suponhamos que eu tivesse dois laptops — um em casa e o outro no escritório em que trabalho todos os dias — e o projeto Book estivesse armazenado no laptop em casa. Se eu também tiver um repositório remoto para ele, poderei acessar o projeto Book a partir do laptop do local em que trabalho. Poderei então fazer alterações diretamente no site do serviço de hospedagem ou clonar o repositório remoto no laptop do local de trabalho para criar um repositório local. Um benefício adicional é o de que posso mostrar para meus colaboradores o progresso que estou fazendo no livro.

Além disso, um repositório remoto me permitiria colaborar facilmente com outras pessoas no projeto Book. Posso dar acesso ao repositório remoto a qualquer pessoa que quiser visualizar meu progresso e fazer contribuições. Outras pessoas poderão então clonar o repositório remoto em seu computador para criar um repositório local ou simplesmente visualizar o conteúdo do repositório remoto diretamente no site do serviço de hospedagem. Por exemplo, eu preciso de um repositório remoto para colaborar no projeto Book com meu coautor e meu editor.

O Exemplo 7-1 do Projeto Book ilustra por que é tão útil trabalhar com repositórios remotos. Agora que você sabe disso, veremos como criar um. Suponhamos que você falasse a um amigo sobre sua experiência de trabalho no projeto Rainbow e ele quisesse ver o que está sendo feito. Para tornar isso possível, seria preciso criar um repositório remoto. É o que faremos agora.

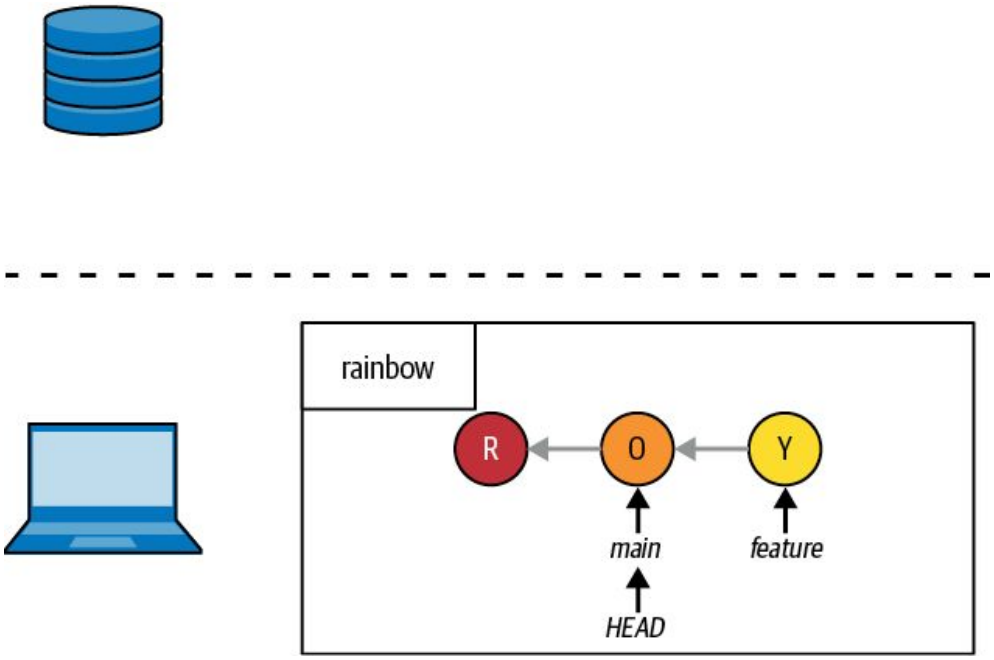
Criando um repositório remoto com dados

Atualmente o projeto Rainbow é composto de um único repositório local: o repositório rainbow, mostrado na seção Visualize 7-2.

[NOTA]

Desse ponto em diante no livro, expandiremos o Diagrama Repositório para incluir um espaço no topo e representar o que está ocorrendo no repositório remoto, como visto na seção Visualize 7-2.

[VISUALIZE 7-2]



Atualmente o projeto Rainbow é composto de um único repositório local, chamado Rainbow.

Seu amigo quer ver em que você está trabalhando. Para permitir que ele faça isso, é preciso criar um repositório remoto e fazer o upload de alguns dados nele. Há três etapas nesse processo:

1. Crie o repositório remoto no serviço de hospedagem.
2. Adicione no repositório local uma conexão com o repositório remoto.
3. Faça o upload (ou o push) do repositório local para o repositório remoto.

Nas próximas seções, percorreremos cada uma dessas etapas, e no decorrer do processo introduzirei alguns tipos novos de branches.

CRIANDO O REPOSITÓRIO REMOTO

Quando criar um repositório remoto em um serviço de hospedagem, você lhe dará um *nome de projeto de repositório remoto* e o serviço de hospedagem fornecerá uma *URL de repositório remoto*. A URL do repositório remoto incluirá automaticamente o nome de projeto de repositório remoto. Como você aprendeu no Capítulo 6, o serviço de hospedagem gerará uma URL do SSH e uma URL do HTTPS para seu repositório. Você deve usar a do protocolo que escolheu.

O processo de criar o repositório remoto é todo feito no site do serviço de hospedagem. Você terá de consultar a documentação do seu serviço de hospedagem para ver detalhes sobre as etapas específicas que deve seguir. Para obter recursos adicionais, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*).

Aqui, darei alguma orientação geral. Como mencionado anteriormente, quando você criar seu repositório remoto, terá de fornecer o nome de projeto. Para o projeto Rainbow, ele será rainbow-remote.

[NOTA]

É comum um repositório remoto e um repositório local compartilharem o mesmo nome. No entanto, para tornar mais fácil para você diferenciar repositórios locais e remotos no projeto Rainbow, daremos nomes distintos para eles.

Você também terá de escolher se o repositório remoto será público ou privado. Essa é uma definição que poderá ajustar em um serviço de hospedagem para cada repositório. Um *repositório público* poderá ser visto por qualquer pessoa que estiver na internet. Um *repositório privado* só poderá ser visto pelas pessoas que receberem acesso a ele. Nos dois casos, você só poderá fazer contribuições no repositório se receber acesso. Já que o repositório em que você está trabalhando neste livro é apenas para fins de aprendizado, recomendo que o torne privado. Dessa forma, terá como controlar quem poderá visualizá-lo (por exemplo, poderá conceder acesso ao amigo que quer ver em que você está trabalhando).

Certos serviços de hospedagem podem perguntar se você deseja incluir algum arquivo ao criar o repositório, como um arquivo README ou .gitignore. Para o que pretendemos com este livro, você não deve incluir nenhum arquivo; se a opção para incluí-los estiver marcada, certifique-se de desmarcá-la. Já que você fará o upload de dados de seu repositório local para o repositório remoto, é melhor que o repositório remoto esteja vazio.

Se o serviço de hospedagem fornecer uma opção de seleção de uma licença, você pode ignorá-la (isto é, não selecione a licença), já que aqui estamos fazendo apenas um exercício de aprendizado.

Para concluir, alguns serviços de hospedagem podem lhe pedir para inserir um valor para o nome do branch padrão. Para os exercícios deste livro, você pode deixar esse valor em branco ou inserir main como valor.

Agora, consulte a seção Acompanhe 7-1 para criar seu repositório remoto.

[ACOMPANHE 7-1]

❶ Faça login em sua conta no serviço de hospedagem que escolheu.

❷ Execute as etapas para criar um repositório remoto. Para obter mais informações sobre isso, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) ou consulte diretamente a documentação de seu serviço de hospedagem.

Ao criar o repositório para esse exercício:

- Para o nome do repositório, use rainbow-remote.
- Você pode optar por tornar o repositório público ou privado. Recomendo torná-lo privado.
- Não inclua nenhum arquivo. Por exemplo, não inclua um arquivo README ou um arquivo .gitignore.
- Se for solicitado a fornecer um nome de branch padrão, pode deixar o campo em branco ou configure-o com main.

❸ Após terminar as etapas para criar o repositório remoto, localize sua URL. Haverás duas versões da URL, uma para o HTTPS e a outra para o SSH. Em um exercício mais à frente, você terá de usar a URL do protocolo que configurou no Capítulo 6. Nos exemplos deste livro, as duas URLs do repositório remoto são:

- HTTPS: https://github.com/gitlearningjourney/rainbow-remote.git
- SSH: git@github.com:gitlearningjourney/rainbow-remote.git

O que deve ser observado:

- Você criou um repositório remoto chamado rainbow-remote; no entanto, não existem dados no repositório remoto.

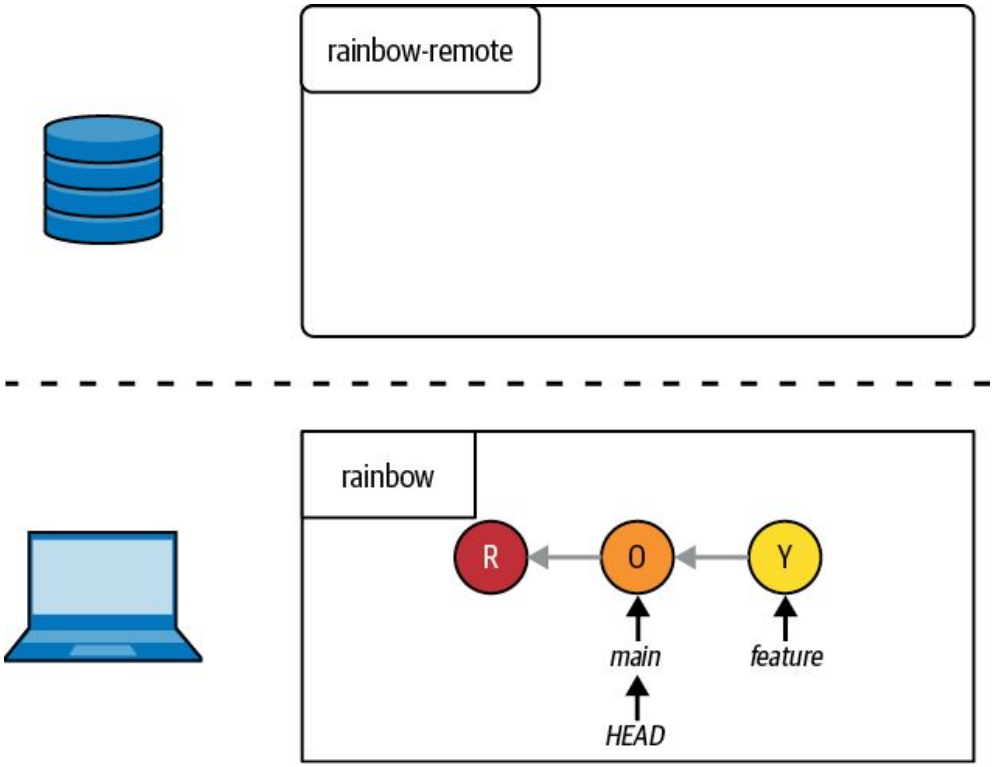
O estado do projeto Rainbow após a conclusão das etapas da seção Acompanhe é mostrado na seção Visualize 7-3.

[NOTA]

Desse ponto em diante, usaremos no Diagrama Repositório um retângulo com cantos comuns de 90 graus para representar um repositório local e um retângulo com cantos arredondados para representar um repositório remoto.

Você criou um repositório remoto em seu serviço de hospedagem, mas, como mostrou a seção Visualize, atualmente ele está vazio. Criar um repositório remoto em um serviço de hospedagem não faz com que ocorra o upload de dados nele. Para fazer o upload de dados, primeiro você precisa adicionar uma conexão com o repositório remoto em seu repositório local. Vejamos como fazer isso.

[VISUALIZE 7-3]



Projeto Rainbow após você criar o repositório remoto vazio rainbow-remote.

ADICIONANDO UMA CONEXÃO COM O REPOSITÓRIO REMOTO

Um repositório local pode se comunicar com um repositório remoto quando há uma conexão com o repositório remoto armazenada dentro dele. Essa conexão terá um nome, que chamaremos de *nome curto do repositório remoto* ou apenas *nome_remoto*. Um repositório local pode ter conexões com vários repositórios remotos, embora isso não seja muito comum.

Se um repositório local for inicializado localmente, para definir uma conexão com um repositório remoto você terá de associar explicitamente a URL do repositório remoto ao *nome_remoto*. Para fazê-lo, use o comando git remote add, passando o *nome_remoto* seguido da URL do repositório remoto.

[SALVE O COMANDO]

git remote add <nome_remoto> <URL>

Adiciona uma conexão com um repositório remoto chamada <nome_remoto> em <URL>.

[NOTA]

Os exemplos deste livro usarão a URL do HTTPS; contudo, você pode optar por usar sua URL do SSH ou do HTTPS, dependendo do protocolo que escolher. Certifique-se de utilizar a URL fornecida por seu serviço de hospedagem, que será diferente da URL usada nos exemplos do livro.

Quando uma conexão com um repositório remoto estiver armazenada em um repositório local, você poderá se conectar com o repositório remoto referenciando o *nome_remoto* em vez da URL na linha de comando.

[NOTA]

Quando você clonar um repositório remoto para criar um repositório local, o Git adicionará automaticamente uma conexão com o repositório remoto com o *nome_remoto* padrão origin. Você verá isso na prática em “Clonando um repositório remoto”, na página 128. Mesmo quando um repositório local é inicializado localmente, é comum o uso de origin para o *nome_remoto*, já que esse é o *nome_remoto* padrão para um repositório quando ele é clonado.

Como mencionado em “Conhecendo um pouco da história do Git: master e main”, na página 66, esse conceito é semelhante a como, quando inicializamos um repositório usando o comando git init sem opções adicionais, o Git cria um branch automaticamente com o nome padrão master. Novamente, não há nada de especial nos termos origin ou master; eles são apenas as convenções de nomenclatura padrão atuais usadas pelo Git.

O repositório rainbow foi inicializado localmente; logo, você terá de adicionar uma conexão com o repositório remoto no repositório local explicitamente. Antes de fazê-lo, veremos mais alguns comandos úteis.

[NOTA]

Na documentação oficial do Git, uma conexão com um repositório remoto armazenada em um repositório local é chamada apenas de *remote*.

Para ver a lista de conexões com repositórios remotos armazenadas em um repositório local pelo *nome_remoto*, você pode usar o comando `git remote`. Se passar a opção `-v` (que é a abreviação de “verboso”) para o comando `git remote`, ele listará as conexões com repositórios remotos armazenadas em um repositório local pelo *nome_remoto* junto com suas URLs.

[SALVE O COMANDO]

git remote

Lista as conexões com repositórios remotos armazenadas no repositório local pelo nome_remoto.

git remote -v

Lista as conexões com repositórios remotos do repositório local com o *nome_remoto* e as URLs.

Consulte a seção Acompanhe 7-2 agora e associe sua URL ao *nome_remoto* origin no repositório rainbow. Na etapa 4, certifique-se de inserir o comando inteiro em uma única linha.

[ACOMPANHE 7-2]

❶ Usando uma janela do sistema de arquivos, acesse rainbow > .git > config. Abra o arquivo config. Você verá que atualmente não existem conexões com repositórios remotos listadas no arquivo. Em outras palavras, você não verá nenhuma menção à URL ou ao *nome_remoto* de seu repositório remoto no arquivo.

❷ Vá até seu serviço de hospedagem e copie a URL de repositório remoto do protocolo que escolheu (SSH ou HTTPS). Você terá de usar essa URL na etapa 4 desta seção Acompanhe.

❸ rainbow $ **git remote**

❹ rainbow $ **git remote add origin** https://github.com/gitlearningjourney/rainbow-remote.git

❺ rainbow $ **git remote**
origin

❻ rainbow $ **git remote –v**
origin https://github.com/gitlearningjourney/rainbow-remote.git (fetch)
origin https://github.com/gitlearningjourney/rainbow-remote.git (push)

❼ Feche o arquivo config e abra-o novamente em uma nova janela. Você verá que existe uma conexão com um repositório remoto listada no arquivo. Ela deve ser semelhante a:
[remote "origin"]
url = https://github.com/gitlearningjourney/rainbow-remote.git
fetch = +refs/heads/*:refs/remotes/origin/*

O que deve ser observado:

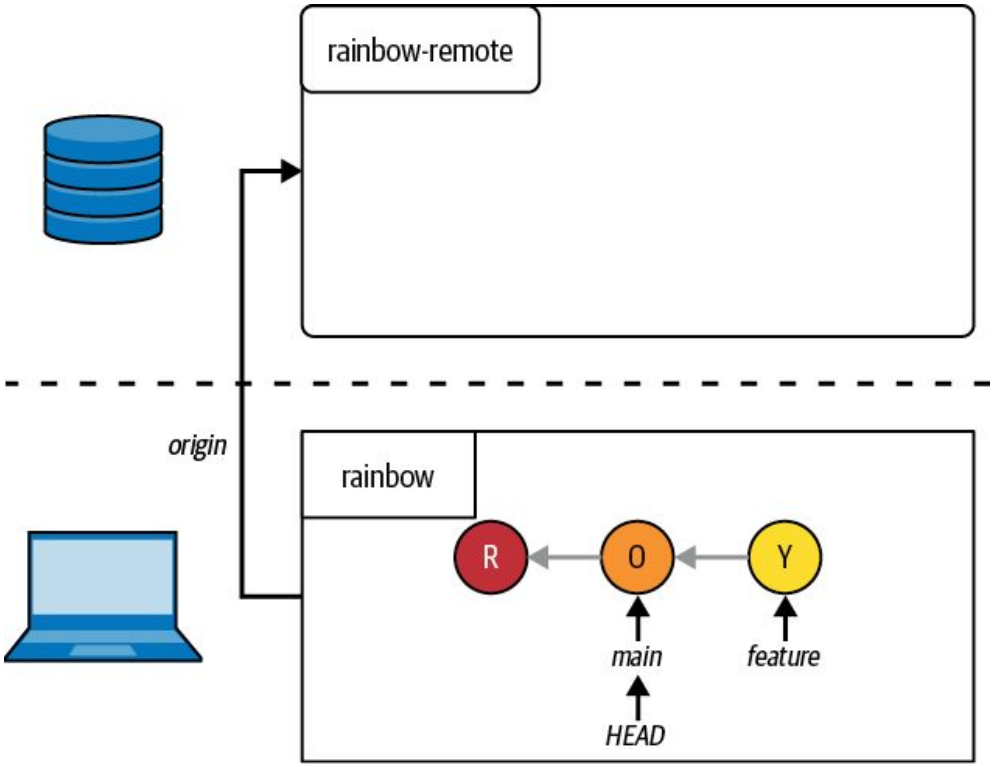
- Na etapa 3, nenhum *nome_remoto* é listado na saída de git remote.
- Na etapa 5, um *nome_remoto* é listado na saída de git remote, que é origin.

A inclusão dessa conexão é mostrada na seção Visualize 7-4.

O que deve ser observado:

- O repositório rainbow tem um *nome_remoto* associado à URL do repositório remoto, chamado origin.
- O repositório rainbow-remote ainda não tem nenhum dado.

[VISUALIZE 7-4]



Projeto Rainbow após você adicionar uma conexão com o repositório rainbow-remote em seu repositório Rainbow.

Na seção Visualize 7-4, podemos ver que a seta que representa o *nome_remoto* armazenado no repositório rainbow, que está relacionado ao repositório rainbow-remote, segue em uma única direção. Isso está ocorrendo porque a conexão entre um repositório local e um remoto vai do repositório local ao repositório remoto, mas não na direção inversa. Em um repositório local, podemos encontrar uma lista de todos os repositórios remotos com os quais ele tem uma conexão armazenada; no entanto, em um repositório remoto não encontraremos uma lista dos repositórios locais que armazenam uma conexão com esse repositório.

Observe também que, só porque você adicionou uma conexão com o repositório remoto em seu repositório local, isso não significa que dados do repositório local foram enviados para o repositório remoto. Para fazer o upload de dados em um repositório remoto, você deve fazer o push de um branch para o repositório. Esse processo fará o upload de todos os commits que fazem parte desse branch.

Para ajudá-lo a entender o que acontece quando executamos essa última etapa, tenho de apresentar mais alguns tipos de branches.

INTRODUZINDO BRANCHES REMOTOS E BRANCHES DE RASTREAMENTO REMOTO

No Capítulo 4, você conheceu os branches, que, como vimos, são ponteiros móveis que apontam para commits. Até agora, só trabalhamos com branches locais. Quando você fizer o push de um branch local para um repositório remoto, criará um *branch remoto*. Um branch remoto é um branch em um repositório remoto.

Os branches remotos *não* são atualizados automaticamente à medida que fazemos mais commits nos branches locais. É preciso fazer o push dos commits explicitamente de um branch local para um branch remoto. Cada branch remoto (do qual o repositório local tem conhecimento) também tem um *branch de rastreamento remoto*. Ele é uma referência em um repositório local ao commit para o qual um branch remoto apontava na última vez em que ocorreu uma comunicação de rede com o repositório remoto. Podemos considerá-lo como um bookmark.

Você pode estabelecer um relacionamento de rastreamento entre um branch local e um branch remoto definindo que branch remoto um branch local deve rastrear. Esse branch é chamado de *branch upstream*. Esses são alguns casos em que o Git definirá o branch upstream automaticamente, mas em outros será preciso defini-lo explicitamente.

Quando você fizer o push de um trabalho de um branch local para um branch remoto, o Git precisará saber para qual branch remoto será feito o push. Se o branch local tiver um branch upstream definido para ele, você poderá usar `git push` sem argumentos; o Git fará então o push do trabalho automaticamente para esse branch. No entanto, se não houver um branch upstream definido para o branch local em que você estiver trabalhando, será preciso especificar para qual branch remoto será o feito o push quando o comando `git push` for inserido (caso contrário, uma mensagem de erro será exibida).

Neste capítulo, mostrarei como especificar o branch remoto para o qual você deseja fazer o push quando usar o comando `git push`. No Capítulo 9, você aprenderá a definir um branch upstream.

Estamos prontos para executar a terceira etapa da criação de um repositório remoto: fazer o push de um branch local para o repositório remoto.

FAZENDO PUSH PARA UM REPOSITÓRIO REMOTO

Para fazer o push de um branch local para seu repositório remoto, você usará o comando git push e passará o *nome_remoto* relacionado ao repositório remoto e o nome do branch que deseja enviar. Para o repositório rainbow, o *nome_remoto* que usaremos é origin e o branch que enviaremos se chama main.

[SALVE O COMANDO]

git push <nome_remoto> <nome_branch>

Faz o upload de conteúdo de *<nome_branch>* para o repositório remoto *<nome_remoto>*.

Duas coisas acontecerão depois que você executar o comando git push:

1. Um branch remoto será criado em seu repositório remoto.
2. Um branch de rastreamento remoto será criado em seu repositório local.

[NOTA]

Para fazer o push de dados para um repositório remoto, você precisa estar conectado à internet e ter acesso SSH ou HTTPS ao serviço de hospedagem que escolheu.

No Capítulo 4, você usou o comando `git branch` para listar todos os branches locais. Agora usará o comando `git branch` com a opção `--all` para listar todos os branches de rastreamento remoto e locais de seu repositório local.

[SALVE O COMANDO]

git branch --all

Lista todos os branches locais e branches de rastreamento remoto.

Consulte a seção Acompanhe 7-3 para fazer o push de seu branch para o repositório remoto.

[ACOMPANHE 7-3]

❶ Usando uma janela do sistema de arquivos, acesse rainbow > .git > refs e examine os diretórios que existem dentro do diretório refs. Deve haver dois diretórios: heads e tags.

❷ rainbow $ **git push origin main**

Enumerating objects: 9, done.

Counting objects: 100% (9/9), done.

Delta compression using up to 4 threads

Compressing objects: 100% (5/5), done.

Writing objects: 100% (9/9), 747 bytes | 373.00 KiB/s, done.

Total 9 (delta 1), reused 0 (delta 0), pack-reused 0

remote: Resolving deltas: 100% (1/1), done.

To github.com:gitlearningjourney/rainbow-remote.git

* [new branch] main -> main

❸ Na janela do sistema de arquivos, examine novamente os diretórios que estão dentro do diretório refs. Agora deve haver três diretórios: heads, tags e remotes.

❹ Vá até seu repositório rainbow-remote no serviço de hospedagem e atualize a página. Acesse a página que lista seus commits. Agora você deve ver seus três commits.

❺ rainbow $ **git branch --all**

feature

* main

remotes/origin/main

❻ rainbow $ **git log**

commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> main, origin/main, feature**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 10:09:59 2022 +0100

yellow

commit 7acb333f08e12020efb5c6b563b285040c9dba93

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 09:42:07 2022 +0100

orange

commit c26d0bc371c3634ab49543686b3c8f10e9da63c5

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 09:23:18 2022 +0100

red

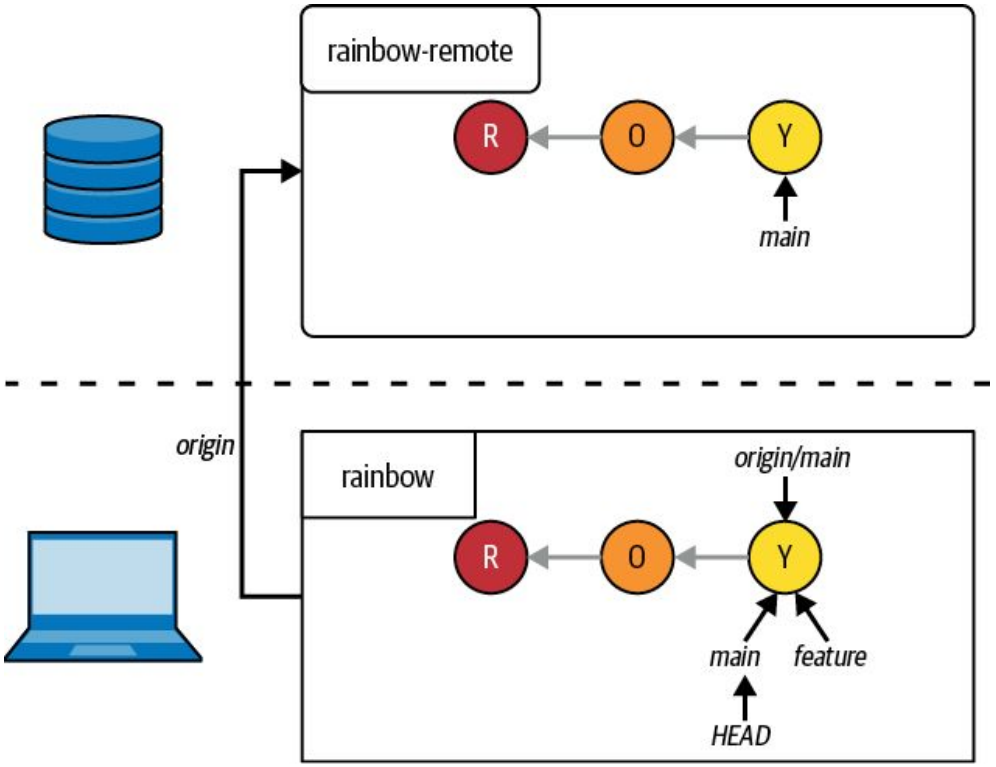
O que deve ser observado:

- Na etapa 2, a saída do comando git push indica que você fez o push de seu branch para o repositório remoto. Lembre-se de que não há problema se os números de sua saída forem um pouco diferentes.

- Na etapa 3, você verá que o diretório refs tem um novo diretório chamado remotes. Dentro do diretório remotes há um diretório chamado origin, e dentro desse diretório existe um arquivo chamado main. Esse é o novo branch de rastreamento remoto origin/main .
- A etapa 4 mostra que no repositório rainbow-remote existe um branch remoto main. Se você acessar seu serviço de hospedagem, encontrará os três commits do branch local main.
- Não existe um branch feature no repositório remoto.

Todas essas observações são mostradas na seção Visualize 7-5.

[VISUALIZE 7-5]



Projeto Rainbow depois que você fez o push de seu branch local main para o repositório remoto.

[NOTA]

Para comandos como git commit, git push e git merge, não importa se suas saídas tiverem números um pouco diferentes dos encontrados nas saídas deste livro. O mesmo pode ser dito para outros comandos que você usará em capítulos posteriores do livro como git clone, git fetch e git pull.

Observe que, quando fazemos o push de um branch específico para um repositório remoto, só os dados desse branch são enviados para o repositório. Você fez o push do branch main para o repositório remoto, mas o branch feature não foi enviado.

Atualmente, em seu repositório rainbow, os branches main e feature têm os mesmos commits. Em outras palavras, seus históricos de desenvolvimento, que podem ser rastreados se seguirmos para trás os commits e os links pais, são compostos dos mesmos commits. Logo, não é tão óbvio que na seção Visualize 7-5 o branch feature não esteja no repositório remoto. Contudo, em projetos do mundo real, geralmente os branches têm diferentes históricos de desenvolvimento e, portanto, são compostos de commits diferentes. Isso significa que, se você não fizer o push de um branch específico para o repositório remoto, alguns dos commits não aparecerão no repositório.

Se você também quiser fazer o push de seu branch feature para o repositório remoto, terá de fazê-lo explicitamente. Consulte a seção Acompanhe 7-4 e faça isso agora.

[ACOMPANHE 7-4]

❶ rainbow $ **git switch feature**

Switched to branch 'feature'

❷ rainbow $ **git push origin feature**

Total 0 (delta 0), reused 0 (delta 0), pack-reused 0

remote:

remote: Create a pull request for 'feature' on GitHub by visiting:

remote: https://github.com/gitlearningjourney/rainbow-remote/pull/new/feature

remote:

To github.com:gitlearningjourney/rainbow-remote.git

* [new branch] feature -> feature

❸ rainbow $ **git branch --all**

* feature

main

remotes/origin/feature

remotes/origin/main

❹ Acesse seu repositório rainbow-remote no serviço de hospedagem e atualize a página. Examine a lista de branches.

❺ rainbow $ **git log**

commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> feature**, **origin/main**, **origin/feature**, **main**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 10:09:59 2022 +0100

yellow

commit 7acb333f08e12020efb5c6b563b285040c9dba93

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 09:42:07 2022 +0100

orange

[NOTA]

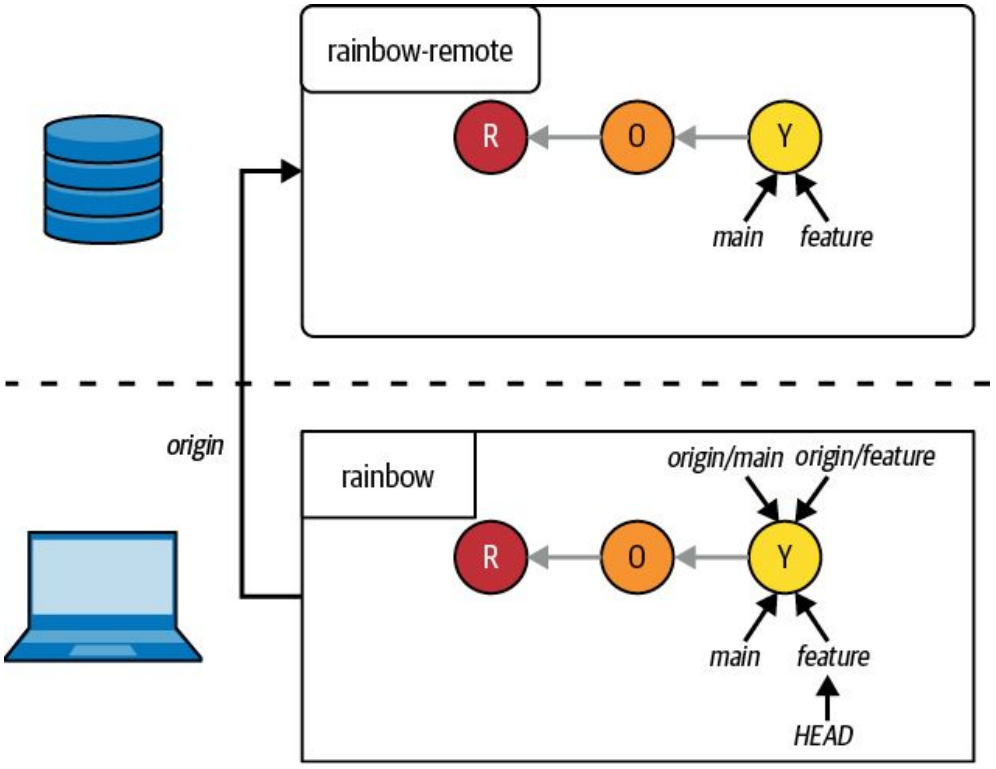
Desse ponto em diante no livro, a saída de git log nas seções Acompanhe só exibirá os últimos commits que foram feitos em um repositório específico.

O que deve ser observado:

- Na etapa 3, podemos ver que no repositório rainbow existem dois branches de rastreamento remoto, origin/main e origin/feature.
- Na etapa 4, você verá que no repositório rainbow-remote existem dois branches remotos, main e feature.

Isso é mostrado na seção Visualize 7-6.

[VISUALIZE 7-6]



Projeto Rainbow depois que você fez o push do branch local feature para o repositório remoto.

Você criou um repositório remoto com dados. No Capítulo 6, mencionei que existem duas maneiras de fazer alterações em um repositório remoto:

1. Fazendo login no serviço de hospedagem por meio do site e executando as alterações diretamente nesse local.
2. Fazendo as alterações no repositório local e enviando-as para o repositório remoto no serviço de hospedagem.

Neste capítulo, vimos um exemplo da segunda situação, mas agora examinaremos resumidamente a primeira.

Trabalhando em um repositório remoto diretamente em um serviço de hospedagem

Até agora, aprendemos a usar o repositório local para fazer commits (Capítulo 3), trabalhar com branches (Capítulo 4) e executar um merge básico (Capítulo 5).

No entanto, também é possível usar a UI do site de um serviço de

hospedagem para executar várias ações (e fazer muitas outras coisas) semelhantes a essas. Por exemplo, em um serviço de hospedagem, você pode fazer commits diretamente em um repositório remoto, criar branches remotos e, como verá no Capítulo 12, fazer o merge de branches usando um recurso chamado pull request.

Já que o objetivo deste livro é ensinar a usar o Git na linha de comando, não veremos como executar todas as ações correspondentes em um serviço de hospedagem. Contudo, esses recursos estarão amplamente registrados na documentação de seu serviço de hospedagem. Eles podem ser úteis para os colaboradores com os quais você trabalhar no futuro que não souberem usar o Git.

Agora que você aprendeu os aspectos básicos do trabalho com repositórios locais e remotos, criou um repositório remoto para o projeto Rainbow e fez o push de alguns branches para ele, a próxima etapa de sua jornada de aprendizado é começar a trabalhar em colaboração no projeto.

Resumo

Este capítulo introduziu o que são os repositórios remotos, por que os usar e as três etapas da criação de um repositório remoto com dados quando você começar a trabalhar a partir dele. Você definiu um repositório remoto em seu serviço de hospedagem, adicionou uma conexão para acessá-lo no repositório local e fez o push de dados do repositório local para o repositório remoto. Durante o processo, aprendeu os conceitos de branches remotos, branches de rastreamento remoto e branches upstream.

A seguir, no Capítulo 8, seu amigo começará a ajudá-lo no trabalho em seu projeto clonando o repositório remoto.

8

Clonando e buscando

No capítulo anterior, você conheceu os repositórios remotos e criou um para o projeto Rainbow.

Neste capítulo, começará a simular como seria trabalhar com um amigo no projeto Rainbow. Para fazê-lo, aprenderá como funciona a clonagem de repositórios remotos e em que isso difere da inicialização de repositórios localmente. Também aprenderá mais detalhes sobre a definição de branches upstream, a exclusão de branches e a busca de dados em um repositório remoto.

[NOTA]

Alguns editores de texto que têm integrações com o Git permitem definir certas configurações para ele. Este capítulo presumirá que você não ativou nenhuma configuração especial que faça o Git se desviar de seu comportamento padrão.

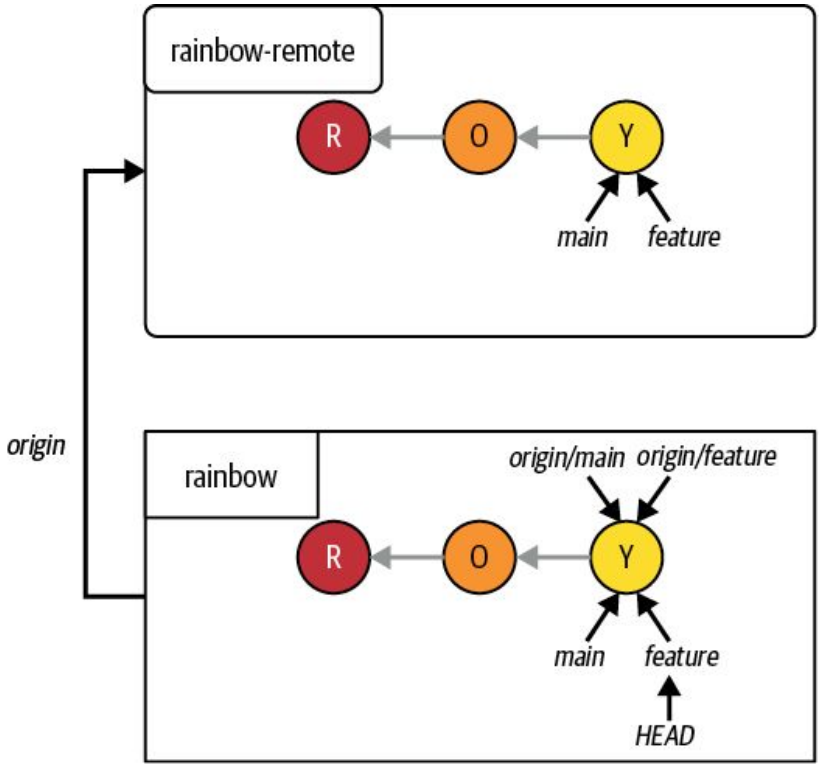
Por exemplo, o Visual Studio Code tem um recurso chamado Autofetch (git.autofetch) que busca alterações periodicamente em um repositório remoto. (Você aprenderá mais sobre a busca posteriormente neste capítulo.) Esse recurso vem desativado por padrão, e para você fazer os exercícios deste capítulo apropriadamente ele deve permanecer desativado. Se o ativou no passado, desative-o antes de continuar.

Se nunca definiu nenhuma configuração especial do Git para seu editor de texto, não precisa se preocupar com isso.

Estado dos repositórios local e remoto

No começo deste capítulo, você deve ter um repositório local chamado rainbow e um repositório remoto chamado rainbow-remote. Esses dois repositórios devem estar em sincronia, isto é, devem conter os mesmos commits e branches. O estado atual esperado dos dois repositórios é mostrado na seção Visualize 8-1.

[VISUALIZE 8-1]



Projeto Rainbow no começo do Capítulo 8, com um repositório local chamado rainbow e um repositório remoto chamado rainbow-remote.

Clonando um repositório remoto

No capítulo anterior, você criou um repositório remoto em um serviço de hospedagem para mostrar ao seu amigo o trabalho que está fazendo no projeto Rainbow. Agora simularemos que ele decidiu ajudá-lo a trabalhar no projeto. Se ele pretende fazer contribuições para o projeto, terá de clonar (ou seja, copiar) seu repositório remoto. Na próxima seção, descreverei como você simulará esse cenário.

No começo do livro, você aprendeu que o Git é uma ferramenta útil para colaboração. Clonar repositórios remotos é uma parte essencial para a colaboração com outras pessoas em um projeto Git, já que permite que elas trabalhem com sua própria cópia do repositório em seu computador. Um número ilimitado de pessoas pode clonar um repositório remoto (contanto que recebam acesso a ele).

Examinaremos o Exemplo 8-1 do Projeto Book como demonstração.

Exemplo 8-1 do Projeto Book

Suponhamos que inicialmente eu tivesse planejado trabalhar em meu projeto Book sozinho e depois mudasse de ideia e decidisse trabalhar com um coautor.

Meu coautor precisará clonar o repositório remoto para ter a própria cópia do projeto Book em seu computador e começar a trabalhar comigo no livro. Para fazer isso, ele precisará de uma conta no serviço de hospedagem que está hospedando o repositório remoto. Em seguida, terei de lhe dar acesso de edição ao repositório para que ele possa comentar e contribuir, independentemente de o repositório ser público ou privado.

Se em algum momento eu quiser que meu editor revise meu trabalho, terei de seguir as mesmas etapas: ele definirá uma conta no serviço de hospedagem, darei acesso ao repositório, direi onde o encontrar e ele clonará o repositório em seu computador.

O Exemplo 8-1 do Projeto Book mostra por que clonar repositórios é uma parte tão importante da colaboração com outras pessoas em um projeto Git. A seguir, veremos como você simulará a experiência de trabalhar com mais alguém no projeto Rainbow.

SIMULAÇÃO DE COLABORAÇÃO

Normalmente, quando duas pessoas estão trabalhando no mesmo projeto, cada uma tem um repositório local em seu computador e as contribuições são feitas em um único repositório remoto. Dado que talvez você não tenha dois computadores ou outra pessoa disponível para ajudá-lo a trabalhar nos exercícios deste e dos próximos capítulos, clonaremos o repositório remoto em seu computador e criaremos um segundo repositório local, que chamaremos de friend-rainbow para diferenciá-lo do repositório rainbow.

Em seguida, você fará de conta que esse segundo repositório local está no computador de seu amigo. De agora em diante, quando mencionarmos que ele está executando alguma ação, você mesmo a executará no repositório friend-rainbow.

Após seu amigo clonar o repositório remoto, você estará trabalhando com dois repositórios locais. Recomendo trabalhar com duas janelas de editor de texto separadas, uma para cada diretório de projeto. Também recomendo trabalhar com duas janelas de linha de comando (ou terminais integrados) separadas, uma para cada repositório, em vez de navegar para dentro e para fora de cada diretório de projeto na mesma janela de linha de comando.

No Git, usamos o termo *clonar* ou *clonagem* para nos referir ao processo de copiar um repositório remoto em um computador a fim de criar um

repositório local, e o comando que utilizamos para fazer isso é `git clone`. Para clonar um repositório remoto, você usará o comando `git clone` e passará a URL do repositório remoto e, opcionalmente, um nome de diretório de projeto.

[SALVE O COMANDO]

git clone <URL> <nome_diretório>
Clona um repositório remoto.

Se você não passar um nome de diretório de projeto, o repositório local receberá o nome de projeto de repositório remoto. Por exemplo, se não passar um nome de diretório de projeto na próxima seção Acompanhe quando clonar o repositório rainbow-remote, o repositório local de seu amigo também se chamará rainbow-remote. Já que isso tornaria esse exercício de aprendizado muito confuso, você passará o nome de diretório de projeto friend-rainbow para o comando git clone como argumento para criar um repositório local com esse nome.

O comando git clone faz o seguinte:

1. Cria um diretório de projeto dentro do diretório atual.
2. Cria (inicializa) o repositório local.
3. Baixa todos os dados do repositório remoto.
4. Adiciona uma conexão com o repositório remoto que foi clonado; por padrão, ela terá o *nome_remoto* origin no novo repositório local.

Para ver explicitamente o repositório sendo criado, você pode cloná-lo em seu diretório desktop. Isso lhe permitirá ver o novo diretório de projeto aparecer na área de trabalho de seu computador.

Consulte a seção Acompanhe 8-1 para clonar o repositório remoto. Na etapa 3, certifique-se de inserir o comando inteiro em uma única linha.

[ACOMPANHE 8-1]

❶ Abra uma nova janela de linha de comando e navegue para o diretório desktop.

❷ Acesse seu serviço de hospedagem e copie a URL de repositório remoto do protocolo que escolheu, SSH ou HTTPS. Você usará a URL na etapa 3 dessa seção Acompanhe.

❸ desktop $ **git clone https://github.com/gitlearningjourney/rainbow-remote.git friend-rainbow**

Cloning into 'friend-rainbow'...

remote: Enumerating objects: 9, done.

remote: Counting objects: 100% (9/9), done.

remote: Compressing objects: 100% (4/4), done.

Receiving objects: 100% (9/9), done.

Resolving deltas: 100% (1/1), done.

remote: Total 9 (delta 1), reused 9 (delta 1), pack-reused 0

❹ Localize o diretório de projeto recém-criado friend-rainbow em seu sistema de arquivos ou na área de trabalho.

O que deve ser observado:

- Você clonou o repositório remoto em seu computador e criou um segundo repositório local chamado friend-rainbow.
- Você não está no diretório friend-rainbow na linha de comando, porque clonar um repositório Git não significa que navegará automaticamente para ele.

Consulte a seção Acompanhe 8-2 para navegar para o novo diretório de projeto friend-rainbow e examinar seu conteúdo e configuração.

[ACOMPANHE 8-2]

❶ desktop $ **cd friend-rainbow**

❷ Abra o diretório de projeto friend-rainbow em uma nova janela do editor de texto.

❸ friend-rainbow $ **git remote -v**

origin https://github.com/gitlearningjourney/rainbow-remote.git (fetch)
origin https://github.com/gitlearningjourney/rainbow-remote.git (push)

❹ friend-rainbow $ **git branch --all**

* main
remotes/origin/HEAD -> origin/main
remotes/origin/feature
remotes/origin/main

❺ friend-rainbow $ **git log**

commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> main**, **origin/main**, **origin/feature**, **origin/HEAD**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
commit 7acb333f08e12020efb5c6b563b285040c9dba93
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange

O que deve ser observado:

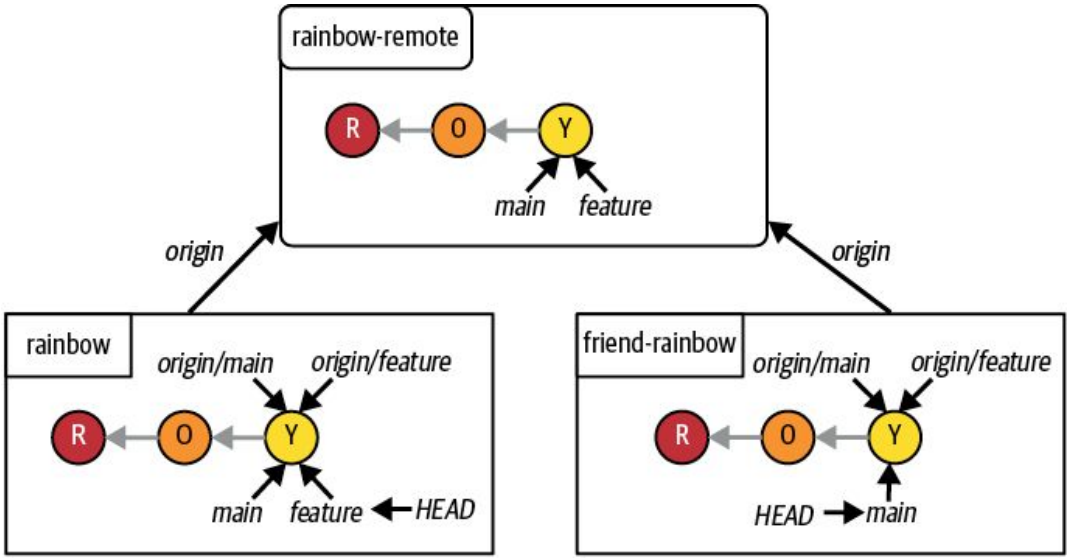
- Na etapa 1, você usou o comando cd para navegar para o repositório friend-rainbow na linha de comando. Isso é necessário porque clonar um repositório não nos faz navegar automaticamente para ele.
- Na etapa 3, na saída de git remote, o *nome_remoto* origin referente ao repositório remoto já está listado.
- Na etapa 4, a saída de git branch mostra um ponteiro chamado origin/HEAD que aponta para o branch de rastreamento remoto origin/main, e também mostra que você tem um branch de rastreamento remoto origin/feature. Podemos ver que não existe um branch local feature.

Todas as observações anteriores são mostradas na seção Visualize 8-2.

[NOTA]

De agora em diante, expandirei os Diagramas Repositório para mostrar o estado dos dois repositórios locais, rainbow e friend-rainbow, e do repositório remoto, rainbow-remote.

[VISUALIZE 8-2]



Projeto Rainbow depois que seu amigo clonou o repositório remoto e criou um repositório local chamado friend-rainbow.

Nas próximas seções, examinaremos com detalhes o que ocorre durante o processo de clonagem e responderei às perguntas a seguir:

- O que é o ponteiro origin/HEAD?
- Por que não existe um branch local feature no novo repositório local friend-rainbow?
- Por que o *nome_remoto* origin foi criado automaticamente para o novo repositório local?

O QUE É ORIGIN/HEAD?

Anteriormente, você deve ter notado que existe um ponteiro chamado origin/HEAD no repositório friend-rainbow. O que é esse ponteiro?

Quando clonamos um repositório, o Git precisa saber em que branch ele deve estar quando a clonagem terminar. O ponteiro origin/HEAD determina qual será esse branch. No projeto Rainbow, origin/HEAD aponta para o branch main, e é por isso que seu amigo que clonou o repositório está no branch main no repositório friend-rainbow.

[NOTA]

Para simplificar, não incluirei o ponteiro origin/HEAD nos diagramas da seção Visualize.

Por outro lado, observe que, no repositório rainbow, atualmente você está no branch feature – mas, no repositório friend-rainbow, nem mesmo existe um branch local feature. Saberemos por que isso ocorre na próxima seção.

A CLONAGEM DE REPOSITÓRIOS E OS DIFERENTES TIPOS DE BRANCHES

Na saída de git log produzida pela etapa 5 da seção Acompanhe 8-2, podemos ver que no repositório friend-rainbow não há referência ao branch local feature. No entanto, existe uma referência ao branch de rastreamento remoto origin/feature. Isso ocorre porque, quando clonamos um repositório, o comando git clone cria branches de rastreamento remoto para todos os branches que estão presentes no repositório remoto que está sendo clonado, mas o único branch local que é criado é aquele para o qual origin/HEAD aponta.

Para seu amigo trabalhar no branch feature, ele terá de passar para esse branch. O Git criará então um branch local feature baseando-se em para onde o branch de rastreamento remoto estava apontando.

Consulte Acompanhe 8-3 para simular seu amigo passando para o branch feature.

[ACOMPANHE 8-3]

❶ friend-rainbow $ **git branch --all**

```
* main
remotes/origin/HEAD -> origin/main
remotes/origin/feature
remotes/origin/main
```

❷ friend-rainbow $ **git switch feature**

```
branch 'feature' set up to track 'origin/feature'.
Switched to a new branch 'feature'
```

❸ friend-rainbow $ **git branch --all**

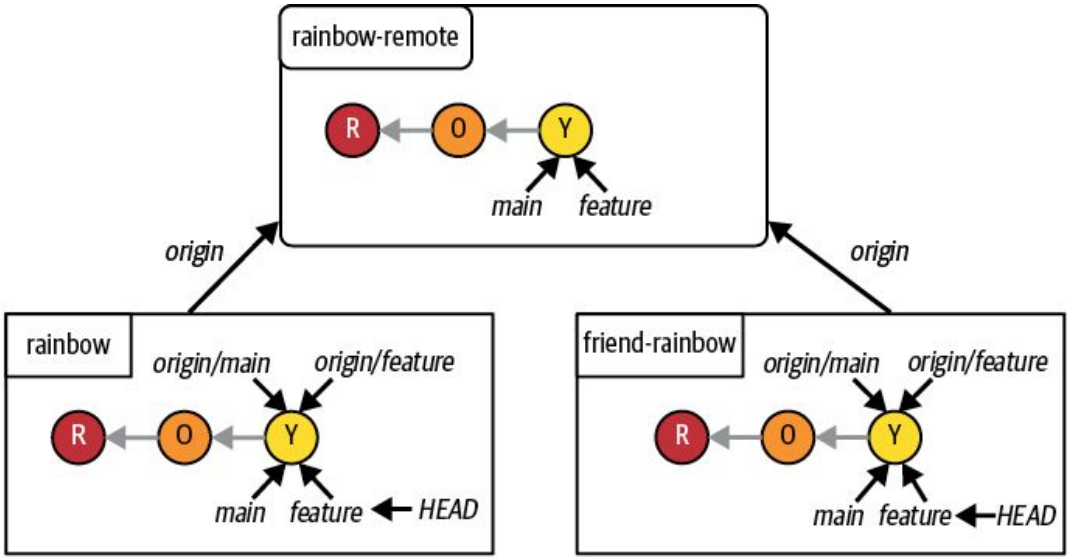
```
* feature
main
remotes/origin/HEAD -> origin/main
remotes/origin/feature
remotes/origin/main
```

O que deve ser observado:

- Na etapa 3, na saída de `git branch --all`, podemos ver que existe um novo branch local `feature` e que seu amigo está nele.

Isso é mostrado na seção Visualize 8-3.

[VISUALIZE 8-3]



Projeto Rainbow depois que seu amigo passou para o branch feature.

Você acabou de aprender a criar e passar para novos branches locais baseados nos branches de rastreamento remoto que baixou de um repositório remoto. Agora discutiremos por que o repositório friend-rainbow já está com o *nome_remoto* origin atribuído à conexão com o repositório remoto.

NOME_REMOTO ORIGIN

Você deve se lembrar de que, no Capítulo 7, aprendeu que, para um repositório local se comunicar com um repositório remoto, aquele precisa ter uma conexão com um *nome_remoto* referente ao repositório remoto armazenada dentro dele. Para trabalhar com o repositório remoto, você teve de associar explicitamente sua URL com um *nome_remoto* usando o comando git remote add <*nome_remoto*> <*URL*>. Isso ocorreu porque criou o repositório rainbow localmente com o comando git init e ele ainda não havia tido nenhuma interação com o repositório remoto.

No entanto, na etapa 3 da seção Acompanhe 8-2, na saída de git remote, você viu que o *nome_remoto* origin já estava sendo listado. Ou seja, o repositório friend-rainbow já está com a URL do repositório remoto associada ao *nome_remoto* origin. Isso ocorreu porque o repositório local de seu amigo não foi originado localmente; foi clonado diretamente de um repositório remoto. No momento da clonagem, a URL do repositório remoto foi

associada a um *nome_remoto* no repositório local, e origin é o *nome_remoto* padrão que o Git associa a um repositório remoto quando o clonamos.

Agora que você sabe um pouco mais sobre o processo de clonagem, voltaremos a falar sobre os branches. No Capítulo 4, você aprendeu a criar branches e a se alternar entre eles. A seguir, aprenderá a excluí-los.

Excluindo branches

A principal razão para a exclusão de branches é para mantermos um projeto Git organizado e limpo. Antes de excluir um branch, você deve sempre verificar se o integrou a outro branch ou se certificar se não deseja usar algum trabalho que se encontre apenas nesse branch.

[NOTA]

Quando você excluir um branch com commits que não façam parte de nenhum outro branch, não deve excluir os commits que fazem parte desse branch. Os commits continuarão existindo no histórico de commits. No entanto, deixarão de ser fáceis de alcançar, porque não haverá uma referência de branch simples que conduza aos commits, e eles não farão parte do histórico de desenvolvimento de nenhum branch existente.

Para demonstrar como a exclusão de branches funciona, suponhamos que seu amigo decidisse que não precisa do branch feature e, portanto, quisesse removê-lo. Para excluir totalmente um branch, temos de excluir o branch remoto, o branch de rastreamento remoto e o branch local. Para excluir um branch remoto e um branch de rastreamento remoto, você usará o comando git push <*nome_remoto*> -d <*nome_branch*> (em que -d é a abreviação de “delete”). Com esse comando, será feito o upload de uma exclusão para o repositório remoto. É preciso passar para ele o nome de um branch remoto.

[SALVE O COMANDO]

git push <nome_remoto> -d <nome_branch>

Exclui um branch remoto e o branch de rastreamento remoto associado.

[NOTA]

Você também pode excluir um branch remoto diretamente no site de um serviço de hospedagem, mas lembre-se de que isso não excluirá o branch de rastreamento remoto. Percorreremos esse processo em “Excluindo branches remotos”, na página 253.

Para excluir um branch local, você usará o comando `git branch` com a opção `-d`, passando o nome do branch que deseja excluir.

[SALVE O COMANDO]

git branch -d <nome_ branch>
Exclui um branch local.

Consulte a seção Acompanhe 8-4 para executar a exclusão do branch para seu amigo. É bom ressaltar que você não pode estar em um branch quando o excluir, logo (simulando ser seu amigo) terá de passar do branch feature para o branch main no repositório friend-rainbow para excluir o branch local feature.

[ACOMPANHE 8-4]

❶ friend-rainbow $ **git branch --all**

```
* feature
main
remotes/origin/HEAD -> origin/main
remotes/origin/feature
remotes/origin/main
```

❷ friend-rainbow $ **git push origin -d feature**

```
To github.com:gitlearningjourney/rainbow-remote.git
- [deleted] feature
```

❸ Acesse o repositório rainbow-remote em seu serviço de hospedagem, atualize a página e examine a lista de branches. O branch feature não deve aparecer.

❹ friend-rainbow $ **git branch --all**

```
* feature
main
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

❺ friend-rainbow $ **git switch main**

```
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

❻ friend-rainbow $ **git branch -d feature**

```
Deleted branch feature (was fc8139c).
```

❼ friend-rainbow $ **git branch --all**

```
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

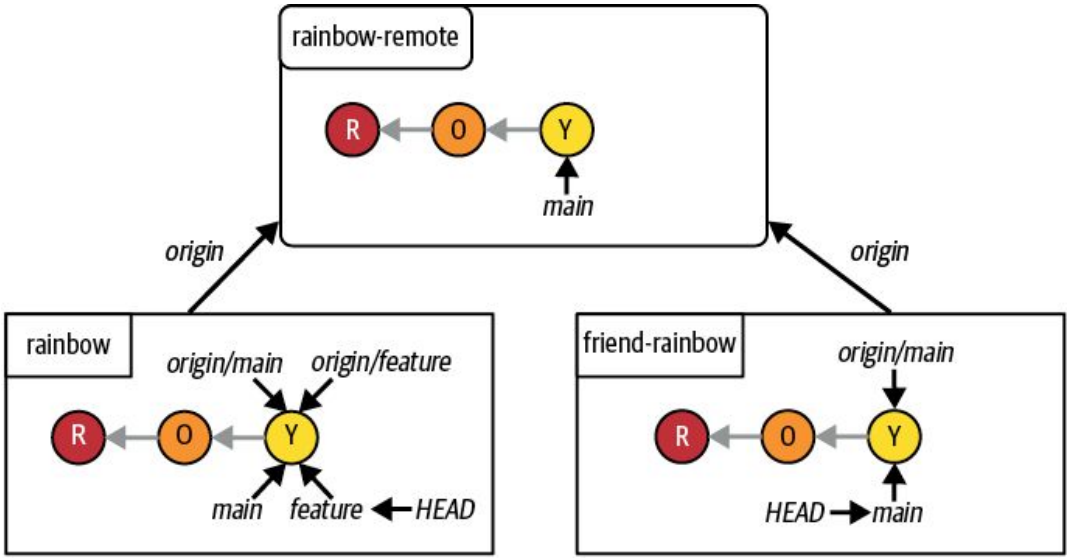
O que deve ser observado:

- Na etapa 2, você excluiu o branch remoto feature e o branch de rastreamento remoto origin/feature.
- Na etapa 3, podemos ver que não existe um branch remoto feature no repositório rainbow-remote.
- Na etapa 6, você excluiu o branch local feature.
- Na etapa 7, a saída de git branch --all indica que não existe um branch local feature no repositório friend-rainbow.

A seção Visualize 8-4 mostra o estado dos repositórios local e remoto após a execução dessas alterações.

Como você pode ver, no repositório rainbow ainda existe um branch feature e um branch de rastreamento remoto origin/feature. O fato de seu amigo ter excluído os branches feature e origin/feature no repositório friend-rainbow não afetou o repositório rainbow. Você excluirá esses branches no repositório rainbow posteriormente neste capítulo, mas, por enquanto, veremos como a colaboração funciona a partir do momento em que seu amigo começar a fazer contribuições para o projeto Rainbow.

[VISUALIZE 8-4]



Projeto Rainbow depois que seu amigo excluiu o branch remoto feature, o branch de rastreamento remoto origin/feature e o branch local feature.

Colaboração no Git e branches

Agora que você vai começar a colaborar com outras pessoas no projeto Rainbow, pode querer discutir quais serão suas convenções no Git. Por exemplo, no Capítulo 3 mencionei que certas equipes têm regras sobre o que incluir em uma mensagem de commit, e no Capítulo 4 você aprendeu que determinadas equipes também podem ter convenções sobre como nomear seus branches.

As equipes também podem ter regras sobre como gerenciar os branches. Por exemplo, pode haver convenções sobre:

- Que branches poderão ser integrados uns aos outros.
- Quando os branches terão de ser criados.
- Qual deve ser o processo de revisão para o trabalho em um branch.

Um exemplo de uma regra que você pode encontrar em um projeto Git é a de que as pessoas só trabalhem em seus próprios branches de tópicos e evitem trabalhar nos branches de tópicos de outros envolvidos. Isso ajuda a evitar conflitos de merge (que abordaremos no Capítulo 10). Não aplicaremos essa regra ao projeto Rainbow neste livro, para simplificar o

processo e permitir que você se concentre nos novos conceitos que está aprendendo em cada capítulo em vez de no gerenciamento de branches. No entanto, lembre-se de que você pode definir ou encontrar diferentes regras para os branches nos projetos Git em que trabalhar no futuro. É importante conversar com seus colaboradores sobre as convenções relacionadas aos branches e ter uma compreensão clara do que são os branches e de como trabalhar em repositórios locais e remotos com outras pessoas pode afetá-los.

Agora examinaremos alguns exemplos para ver como a colaboração no projeto Rainbow funciona.

FAZENDO UM COMMIT NO REPOSITÓRIO LOCAL

Nesta seção, seu amigo adicionará uma nota sobre a cor green (verde) ao arquivo `rainbowcolors.txt` e fará um commit no branch `main` de seu repositório local. Lembre-se de que agora você está simulando ser seu amigo, o que significa que trabalhará no diretório de projeto dele, `friend-rainbow`. Como discutido anteriormente, você precisa estar com esse diretório de projeto aberto em uma janela separada do editor de texto.

Consulte a seção Acompanhe 8-5 para simular que você é seu amigo fazendo um commit.

[NOTA]

Na saída de git log na etapa 4 da seção Acompanhe 8-5, você é o autor tanto do commit yellow (amarelo) quanto do commit green (verde). Isso ocorre porque está simulando ser seu amigo fazendo contribuições para o projeto Rainbow. No entanto, na verdade, se outra pessoa tivesse feito o commit green, ela seria a autora do commit e seu nome e endereço de email seriam exibidos na saída. Lembre-se disso no restante dos capítulos deste livro.

[ACOMPANHE 8-5]

❶ Abra o arquivo rainbowcolors.txt no diretório de projeto friend-rainbow em seu editor de texto, adicione “Green is the fourth color of the rainbow.” na linha 4 e salve o arquivo.

❷ friend-rainbow $ **git add rainbowcolors.txt**

❸ friend-rainbow $ **git commit -m "green"**

[main 6987cd2] green

1 file changed, 2 insertions(+), 1 deletion(-)

❹ friend-rainbow $ **git log**

commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (**HEAD -> main**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 11:49:03 2022 +0100

green

commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**origin/main**, **origin/HEAD**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sat Feb 19 10:09:59 2022 +0100

yellow

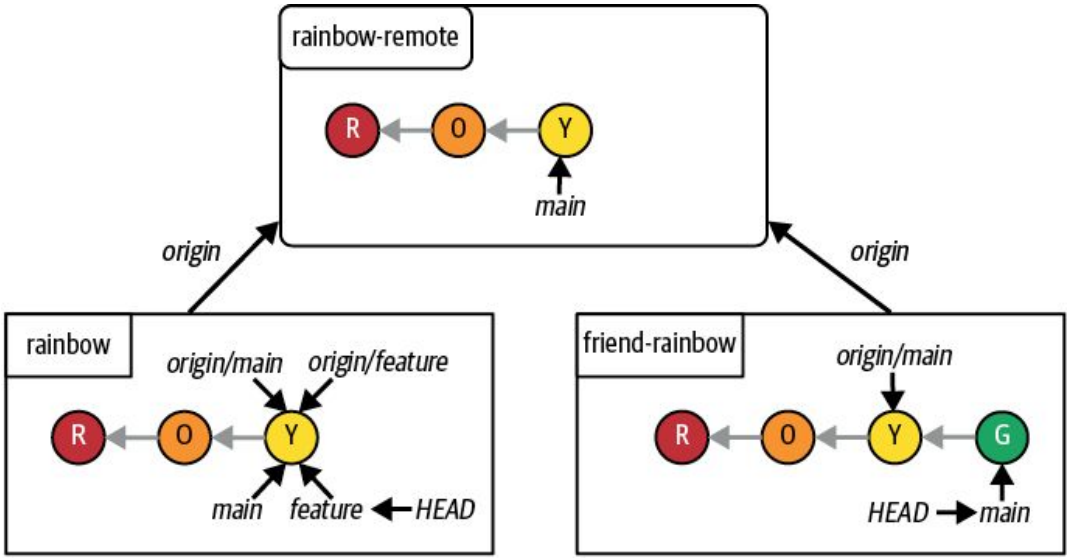
❺ Acesse o repositório rainbow-remote em seu serviço de hospedagem e atualize a página. O commit green ainda não estará lá.

O que deve ser observado:

- Na etapa 4, a saída de git log indica que no repositório friend-rainbow:
  - O branch local main foi atualizado para apontar para o commit green.
  - O branch de rastreamento remoto origin/main ainda aponta para o commit yellow.

A seção Visualize 8-5 mostra as alterações no repositório friend-rainbow.

[VISUALIZE 8-5]



Projeto Rainbow depois que seu amigo fez o commit green no branch main do repositório friend-rainbow.

Como você pode ver, o commit green ainda não apareceu no repositório rainbow-remote. Isso está ocorrendo porque os repositórios remotos não são atualizados automaticamente; o trabalho feito nos repositórios locais tem de ser enviado explicitamente para eles.

Observe também que o branch de rastreamento remoto origin/main ainda aponta para o commit yellow. Esse comportamento ocorre porque, como mencionado no Capítulo 7, os branches de rastreamento remoto representam o estado de branches remotos. Já que seu amigo ainda não fez o push das alterações para o repositório remoto, o branch remoto main não foi atualizado e, portanto, o branch de rastreamento remoto origin/main também não o foi.

Nesse momento não há como você ter o commit green no repositório rainbow, porque a atualização de um repositório local com as alterações feitas em outro repositório local é um processo de duas etapas. Primeiro, o repositório local que tem as alterações precisa fazer um push explícito delas para um repositório remoto. Em seguida, o repositório local que não tem as alterações tem de fazer um fetch explícito e integrar as alterações a partir do repositório remoto.

Agora veremos como seu amigo fará o push do trabalho que ele executou para o repositório remoto.

FAZENDO PUSH PARA O REPOSITÓRIO REMOTO

No Capítulo 7, você aprendeu que, quando é feito o push de um trabalho de um branch local para o branch remoto, o Git precisa de alguma maneira saber para que branch remoto ocorrerá o push. Se o branch local tiver um branch upstream definido para ele, você poderá usar `git push` sem argumentos e o Git fará o push do trabalho automaticamente para esse branch. Contudo, se não houver um branch upstream definido para o branch local em que você está trabalhando, será preciso especificar para qual branch remoto será feito o push quando o comando `git push` for inserido.

Na última vez que usamos o comando `git push` no repositório `rainbow`, passamos para ele o *nome_remoto* e o nome do branch. Isso ocorreu porque você não havia definido um branch upstream para seu branch local. Lembre-se de que um branch upstream é o branch remoto que um branch local específico rastreia. Quando clonamos um repositório, branches upstream são definidos *automaticamente* para os branches que existem no repositório clonado.

O comando que devemos usar para ver se branches upstream foram definidos é `git branch` com a opção `-vv` (que é a abreviação de “very verbose”). Esse comando também informa se um branch local está adiantado ou atrasado em relação a um branch upstream, se este tiver sido definido.

[SALVE O COMANDO]

git branch -vv

Lista os branches locais e seus branches upstream, se houver algum.

Existem dois benefícios principais da definição de branches upstream:

1. Se você tiver feito o fetch (baixado) dos commits do repositório remoto, poderá verificar se seu repositório está adiantado ou atrasado em relação ao repositório remoto usando o comando `git branch -vv` ou o comando `git status`. (Abordaremos o processo de fetching na próxima seção deste capítulo.)
2. Você pode simplificar os comandos que usa com o repositório remoto porque não terá de especificar o nome do branch e o *nome_remoto*. Por exemplo, poderá usar o comando `git push` sem passar nenhum argumento.

Veremos esses dois benefícios nas próximas seções Acompanhe.

Já que seu amigo clonou o repositório, o branch upstream de seu branch local `main` já está definido como sendo o branch remoto `main`. Consulte a seção Acompanhe 8-6 para ver como ele pode verificar o status do repositório remoto.

[ACOMPANHE 8-6]

❶ friend-rainbow $ **git branch -vv**

* main 6987cd2 [origin/main: ahead 1] green

❷ friend-rainbow $ **git status**

On branch main

Your branch is ahead of 'origin/main' by 1 commit.

(use "git push" to publish your local commits)

nothing to commit, working tree clean

O que deve ser observado:

- Na etapa 1, a saída de git branch -vv mostra que o branch upstream definido para o branch local main é o branch remoto main no repositório remoto com *nome_remoto* origin e que o branch local main está um commit adiantado.
- Na etapa 2, a saída de git status também indica que o branch local main está um commit adiantado em relação ao branch upstream origin/main.

Agora, consulte a seção Acompanhe 8-7 para ver como seu amigo pode usar o comando git push sem nenhum argumento para atualizar o repositório remoto.

[ACOMPANHE 8-7]

❶ friend-rainbow $ **git push**

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 295 bytes | 295.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:gitlearningjourney/rainbow-remote.git
fc8139c..6987cd2 main -> main
```

❷ friend-rainbow $ **git status**

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

❸ friend-rainbow $ **git log**

```
commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (HEAD -> main, origin/main, origin/HEAD)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green
commit fc8139cbf8442cdbb5e469285abaac6de919ace6
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
```

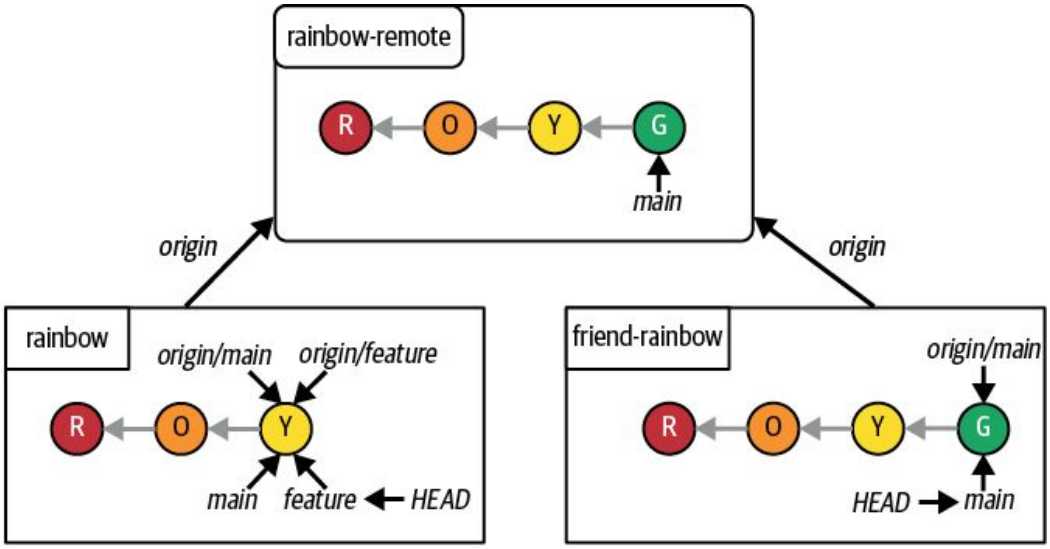
❹ Acesse o repositório rainbow-remote em seu serviço de hospedagem e atualize a página. Agora o commit green deve aparecer.

O que deve ser observado:

- Na etapa 3, a saída de git log indica que, no repositório friend-rainbow, o branch de rastreamento remoto origin/main foi atualizado para apontar para o commit green.
- Na etapa 4, podemos ver que, no repositório rainbow-remote, o branch remoto main foi atualizado para apontar para o commit green.

A seção Visualize 8-6 mostra essas observações.

[VISUALIZE 8-6]



Projeto Rainbow depois que seu amigo fez o push do commit green para o repositório remoto.

Repare que o repositório rainbow ainda não tem o commit green e que o branch local main e o branch de rastreamento remoto origin/main desse repositório ainda apontam para o commit yellow.

Agora queremos verificar se o branch local main do repositório rainbow está em sincronia com o branch main dos repositórios rainbow-remote e friend-rainbow. Para fazer isso, você terá de conhecer o fetching.

Incorporando alterações do repositório remoto

O branch local main e o branch de rastreamento remoto origin/main do repositório rainbow ainda apontam para o commit yellow porque repositórios locais não são atualizados com novos dados de repositórios remotos. Da mesma forma que precisamos executar ações explícitas para atualizar um repositório remoto com alterações de um repositório local, temos de executar ações explícitas para atualizar branches locais e branches de rastreamento remoto com alterações de um repositório remoto.

A incorporação de alterações de um branch remoto em um branch local é um processo de duas etapas: primeiro você faz o fetch das alterações do repositório remoto e depois as integra no branch local do repositório local. Começaremos examinando a primeira etapa.

FAZENDO O FETCH DE ALTERAÇÕES DO REPOSITÓRIO REMOTO

No Git, usamos os termos *fetch* ou *fetching* para nos referir ao processo de download de dados de um repositório remoto para um repositório local, e o comando que usamos para fazer isso é `git fetch`. O comando `git fetch` baixa todos os commits necessários para atualizar todos os branches de rastreamento remoto do repositório local a fim de refletir o estado dos branches remotos do repositório remoto especificado. Quando não é fornecido um *nome_remoto* como argumento para o comando `git fetch`, por padrão o repositório remoto com *nome_remoto* `origin` é usado, a menos que haja um branch upstream definido para o branch atual.

[SALVE O COMANDO]

git fetch <nome_remoto>
Baixa dados do repositório remoto <nome_remoto>.

git fetch
Baixa dados do repositório remoto com *nome_remoto* origin.

O comando git fetch só afeta branches de rastreamento remoto. Ele não afeta branches locais. Em outras palavras, esse comando só faz o fetch (download) dos dados; ele não os integra em nenhum branch local. Logo, nada mudará em seu diretório de trabalho quando você fizer o fetch de dados de um repositório remoto.

Integraremos as alterações na próxima seção; por enquanto, consulte a seção Acompanhe 8-8 para fazer o fetch dos dados do repositório remoto.

[ACOMPANHE 8-8]

❶ Acesse o diretório de projeto rainbow em sua janela de linha de comando.

❷ rainbow $ **git log --all**

commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> feature**, **origin/main**, **origin/feature**, **main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
commit 7acb333f08e12020efb5c6b563b285040c9dba93
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange

❸ rainbow $ **git fetch**

remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0
Unpacking objects: 100% (3/3), 275 bytes | 91.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
fc8139c..6987cd2 main -> origin/main

❹ rainbow $ **git log --all**

commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (**origin/main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green
commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> feature**, **origin/feature**, **main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow

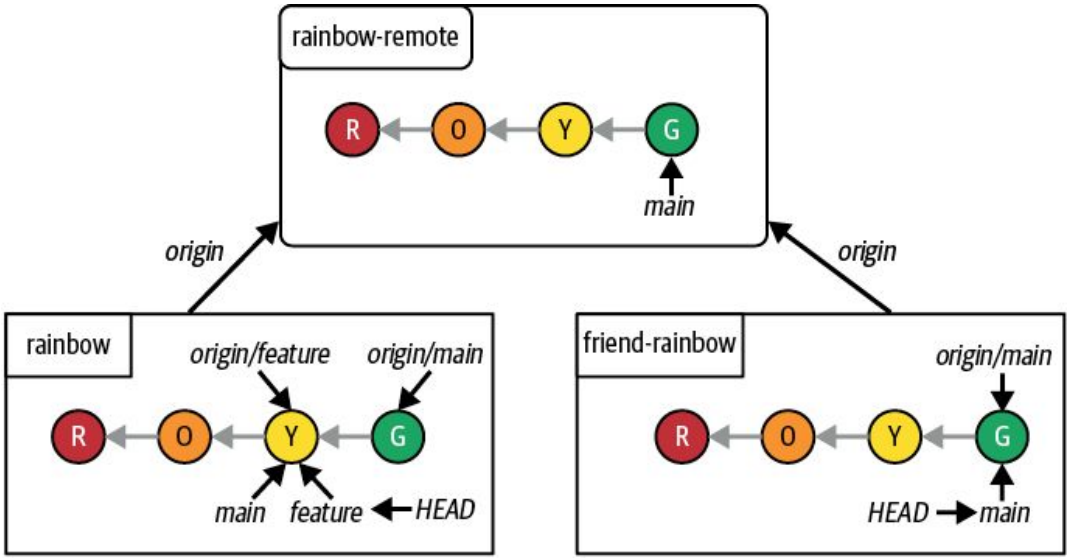
O que deve ser observado:

- Na etapa 2, a saída de git log mostra que:
  - O branch local main e o branch de rastreamento remoto origin/main estão apontando para o commit yellow.
- Na etapa 4, na saída de git log vemos que:
  - O branch de rastreamento remoto origin/main está apontando para o commit green.

- O branch local main ainda está apontando para o commit yellow.

A seção Visualize 8-7 mostra essas observações.

[VISUALIZE 8-7]



Projeto Rainbow depois que você fez o fetch de dados do repositório rainbow-remote para o repositório rainbow.

Você acabou de ver o comando git fetch atualizar os branches de rastreamento remoto do repositório local de acordo com os branches remotos existentes no repositório remoto. Agora está pronto para a segunda etapa do processo: incorporar as alterações do repositório remoto em um branch local.

INTEGRANDO ALTERAÇÕES EM UM BRANCH LOCAL

Após fazer o fetch das alterações de um repositório remoto e atualizar os branches de rastreamento remoto de um repositório local, estamos prontos para atualizar um branch local. Você deve se lembrar de que, no Capítulo 5, mencionei que o Git fornece duas maneiras de integrar alterações, chamadas *merging* e *rebasing*. Abordaremos o rebasing no Capítulo 11. Por enquanto, você continuará usando o merging.

Em “Tipos de Merges”, na página 82, você conheceu dois tipos de merges: fast-forward e three-way. No merge fast-forward que executou no Capítulo 5, fez o merge de um branch chamado feature em main. Tanto feature quanto main eram branches locais do repositório rainbow. Nesta seção, você fará o merge do branch de rastreamento remoto origin/main no branch local main do repositório rainbow. Os tipos de branches envolvidos no merge são diferentes, mas o processo de execução é o mesmo. Novamente esse será um merge fast-forward.

Como você aprendeu no Capítulo 5, ao executar um merge temos de estar no branch que receberá a integração, que nesse caso será o branch main do repositório rainbow. Portanto, na seção Acompanhe 8-9, você passará para o branch main antes de executar o merge.

Para integrar os commits que estavam no branch main do repositório remoto em seu branch local main, você usará o comando git merge. Dessa vez, passará o nome de um branch de rastreamento remoto, origin/main.

Consulte a seção Acompanhe 8-9 para executar o merge.

[ACOMPANHE 8-9]

❶ rainbow $ **git switch main**
Switched to branch 'main'

❷ rainbow $ **git merge origin/main**
Updating fc8139c..6987cd2
Fast-forward
rainbowcolors.txt | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)

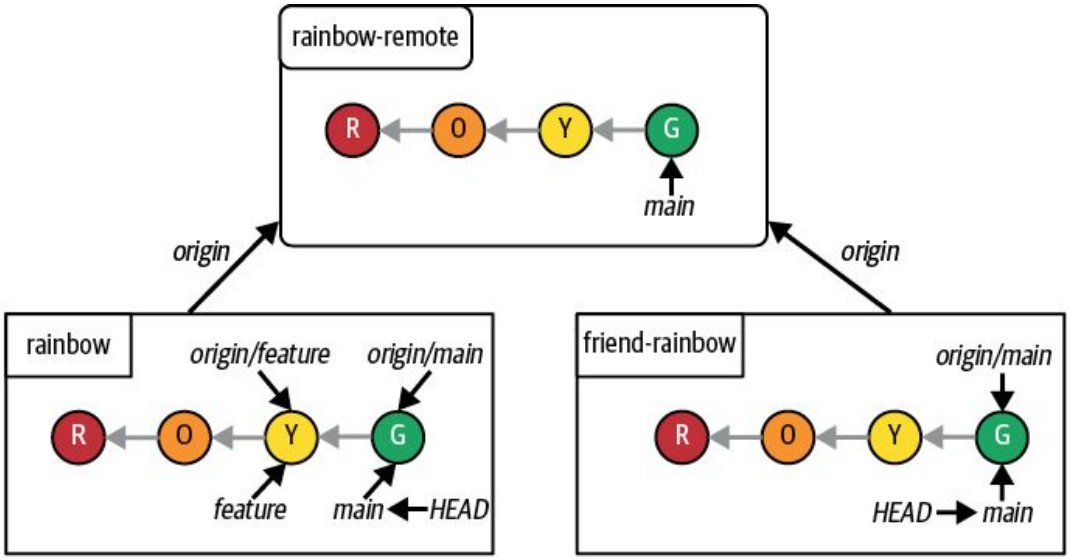
❸ rainbow $ **git log**
commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (**HEAD -> main**, **origin/main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green
commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**origin/feature**, **feature**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow

O que deve ser observado:

- Na etapa 3, a saída de git log indica que o branch local main aponta para o commit green.

Isso é mostrado na seção Visualize 8-8.

[VISUALIZE 8-8]



Projeto Rainbow depois que você fez o merge do branch de rastreamento remoto origin/main no branch local main no repositório rainbow.

Acabamos de ver que o branch local foi atualizado após a conclusão do merge. Na seção Visualize 8-8, também vimos que o branch feature continua aparecendo no repositório rainbow. Executaremos mais algumas exclusões de branches para organizar o repositório rainbow.

Excluindo branches (continuação)

Ainda temos o branch local feature e o branch de rastreamento remoto origin/feature no repositório rainbow. Para tornar tudo mais simples, agora você trabalhará no branch main a fim de excluir esses branches.

Em “Excluindo branches”, na página 135, você usou o comando git push <*nome_remoto*> -d <*nome_branch*> para excluir o branch remoto feature e o branch de rastreamento remoto origin/feature no repositório friend-rainbow de uma só vez. Não existe mais um branch remoto feature; logo, para excluir o branch de rastreamento remoto origin/feature no repositório rainbow, você pode utilizar o comando git fetch com a opção -p (que é a abreviação de “prune”, ou seja, suprimir). Esse comando com essa opção excluirá qualquer branch de rastreamento remoto que corresponda a branches remotos que tiverem sido excluídos no repositório remoto.

[SALVE O COMANDO]

git fetch -p

Remove os branches de rastreamento remoto que correspondam a branches remotos excluídos e baixa dados do repositório remoto.

Para excluir o branch local feature, você usará o mesmo comando que usou anteriormente neste capítulo. Consulte a seção Acompanhe 8-10 para executar as exclusões de branches.

[ACOMPANHE 8-10]

❶ rainbow $ **git branch --all**
feature
* main
remotes/origin/feature
remotes/origin/main

❷ rainbow $ **git fetch -p**
From github.com:gitlearningjourney/rainbow-remote
- [deleted] (none) -> origin/feature

❸ rainbow $ **git branch --all**
feature
* main
remotes/origin/main

❹ rainbow $ **git branch -d feature**
Deleted branch feature (was fc8139c).

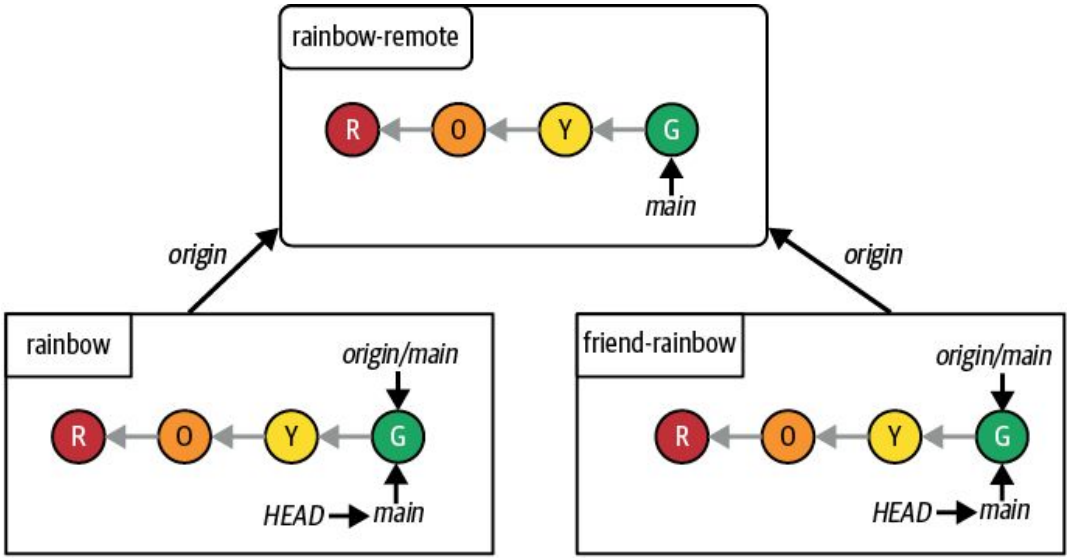
❺ rainbow $ **git branch --all**
* main
remotes/origin/main

O que deve ser observado:

- Na etapa 2, você excluiu o branch de rastreamento remoto origin/feature.
- Na etapa 4, excluiu o branch local feature.
- Na etapa 5, a saída de git branch --all indica que você não tem mais um branch local feature ou um branch de rastreamento remoto origin/feature no repositório rainbow.

Isso é mostrado na seção Visualize 8-9.

[VISUALIZE 8-9]



Projeto Rainbow depois que você excluiu o branch local feature e o branch de rastreamento remoto origin/feature no repositório rainbow.

Resumo

Neste capítulo, você começou a simular como seria trabalhar com outra pessoa em um projeto Git. Seu "amigo" clonou o repositório rainbow-remote e criou um segundo repositório local, que você chamou de friend-rainbow. Durante o processo, vimos que origin é o *nome_remoto* padrão que o Git associa a um repositório remoto quando ele é clonado.

Após seu amigo fazer o commit green no repositório friend-rainbow e fazer o push dele para o repositório remoto, você aprendeu o processo de atualizar um branch local com alterações de um branch remoto em um repositório remoto, que envolve o fetching de alterações do repositório remoto e depois sua integração. No repositório rainbow, você integrou as alterações executando um merge fast-forward. Agora, no Capítulo 9, aprenderá mais sobre o outro tipo de merge mencionado no Capítulo 5: o merge three-way.

9

Merges three-way

No Capítulo 8, você conheceu a clonagem e viu como seria colaborar com outra pessoa em um projeto Git. Percorremos um exemplo de um amigo contribuindo com trabalhos para o projeto Rainbow, fazendo o push do trabalho para o repositório remoto e de você executando o fetch e o merge do trabalho a fim de manter os repositórios em sincronia.

Todos os merges que executamos até agora no projeto Rainbow foram do tipo fast-forward. Neste capítulo, você aprenderá a executar um merge three-way. Durante o processo, também verá um exemplo de definição de branches upstream e aprenderá o que acontece quando editamos arquivos várias vezes no diretório de trabalho entre os commits. Para concluir, aprenderá a executar o pulling a partir de um repositório remoto e discutiremos em que o pulling difere do fetching.

Estado dos repositórios locais e remoto

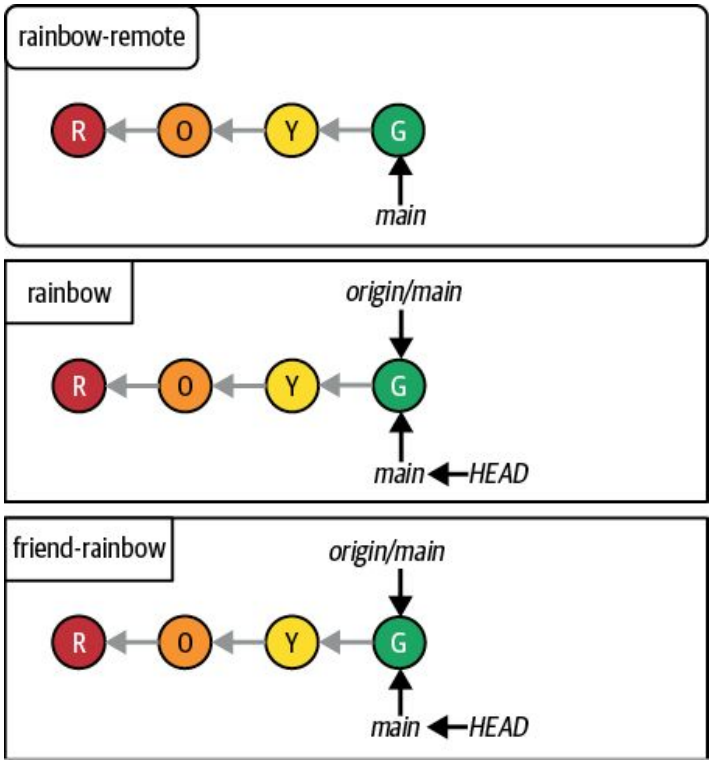
No começo deste capítulo, você deve ter dois repositórios locais chamados rainbow e friend-rainbow e um repositório remoto chamado rainbow-remote. Todos esses três repositórios devem estar em sincronia; em outras palavras, devem conter os mesmos commits e branches. Recomendo que continue usando duas janelas de editor de texto e duas janelas de linha comando separadas para os repositórios rainbow e friend-rainbow ao percorrer os exemplos do capítulo, como discutido em “Simulação de colaboração”, na página 129.

A seção Visualize 9-1 mostra o estado dos repositórios locais e remoto no projeto Rainbow com todos os commits que foram feitos do Capítulo 1 ao Capítulo 8.

Para nos concentrarmos nos commits que você fará neste capítulo, de agora em diante simplificarei os diagramas da seção Visualize e só mostrarei os dois últimos commits que fazem parte do branch main em todos os

repositórios, que são os commits yellow (amarelo) e green (verde). Essa representação é mostrada na seção Visualize 9-2.

[VISUALIZE 9-1]



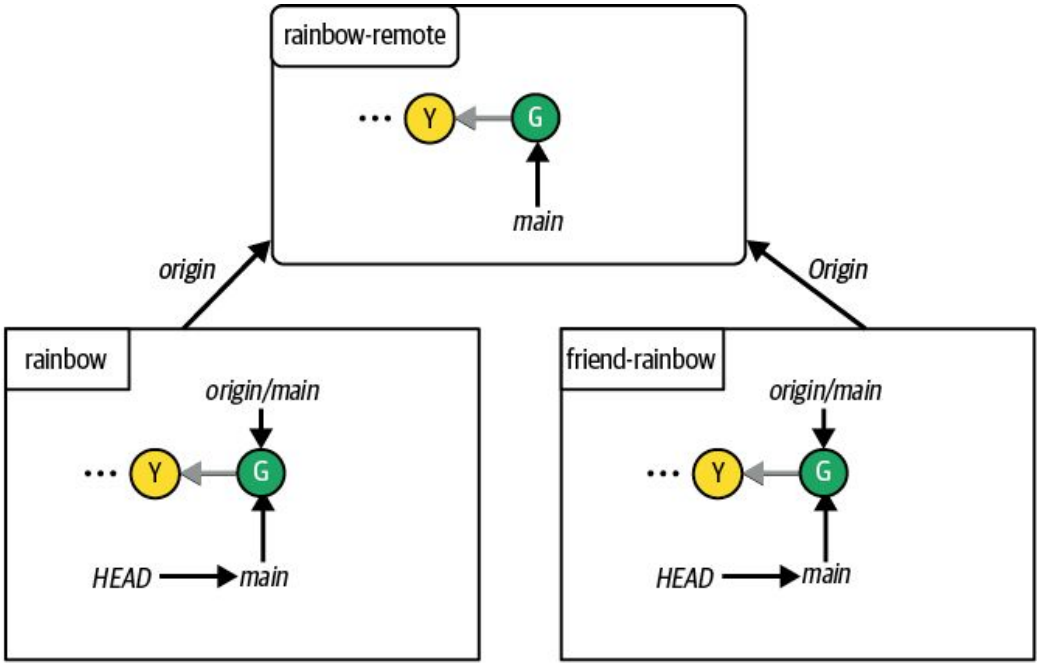
Projeto Rainbow no começo do Capítulo 9 com todos os commits desde o Capítulo 1.

Por que os merges three-way são importantes?

No Capítulo 5, expliquei que o merging envolve integrar as alterações feitas em um branch, chamado de branch de origem, em outro branch, chamado de branch de destino. Também introduzi dois tipos de merges: fast-forward e three-way. Até agora, você só executou merges fast-forward, mas também é importante examinarmos os merges three-way porque eles são comuns nas atividades diárias de um usuário do Git.

Os merges three-way são um pouco mais complicados do que os merges fast-forward porque eles criam commits de merge e podem levar a conflitos. Os *conflitos de merge* surgem quando fazemos o merge de dois branches em que diferentes alterações foram executadas nas mesmas partes do(s) mesmo(s) arquivo(s), ou se um arquivo que foi editado em um branch tiver sido excluído no outro branch.

[VISUALIZE 9-2]



Representação simplificada do projeto Rainbow no começo do Capítulo 9, mostrando apenas os dois últimos commits do branch main em todos os repositórios.

No Capítulo 10, discutirei os conflitos de merge com mais detalhes e veremos um exemplo de um merge three-way com um conflito de merge. No exemplo de merge three-way deste capítulo, seu amigo editará um arquivo diferente do que você editará e, portanto, não haverá conflitos de merge.

Como você aprendeu no Capítulo 5, os merges three-way ocorrem quando os históricos de desenvolvimento dos branches envolvidos no merge divergem – em outras palavras, quando *não* conseguimos alcançar o branch de destino seguindo o histórico de commits (links pais) do branch de origem. Veremos uma demonstração de como essa situação pode surgir no Exemplo 9-1 do Projeto Book.

Exemplo 9-1 do Projeto Book

Suponhamos que, ao trabalhar no projeto Book, tanto eu quanto meu coautor decidíssemos criar ao mesmo tempo um branch fora do branch main para trabalhar em capítulos diferentes. Criei o branch chapter_five e meu coautor criou o branch chapter_six, como visto na Figura 9-1.

Trabalhamos independentemente em nossos capítulos. Meu coautor terminou primeiro seu trabalho no branch chapter_six, executou o merge do trabalho no branch main e fez o push do branch main atualizado para o repositório remoto. Na Figura 9-2, representei o trabalho que meu coautor fez como commit C e estou mostrando o estado de todos os repositórios.

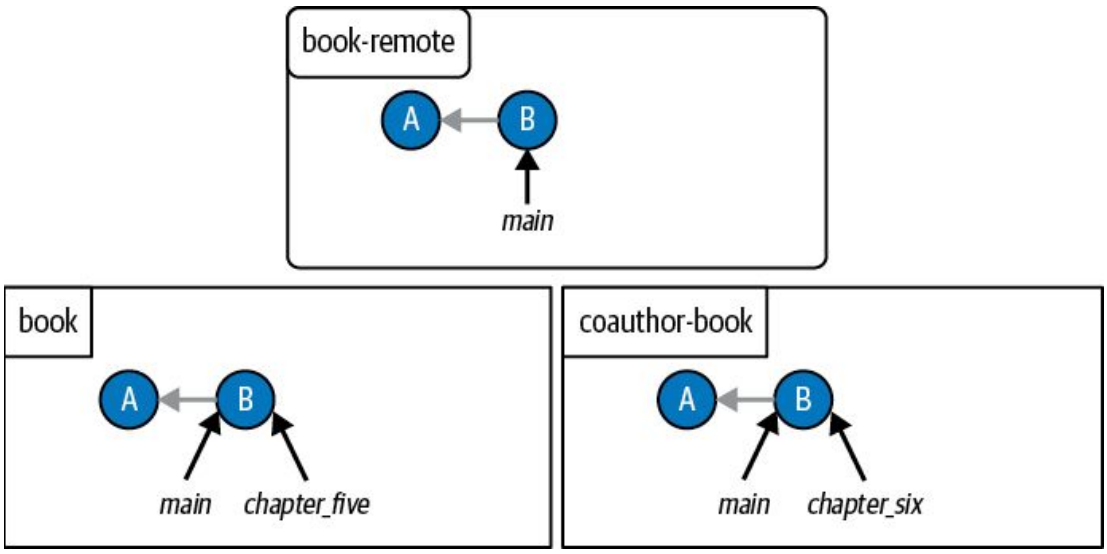


Figura 9-1: Projeto Book depois que meu coautor e eu criamos branches para trabalhar em capítulos diferentes.

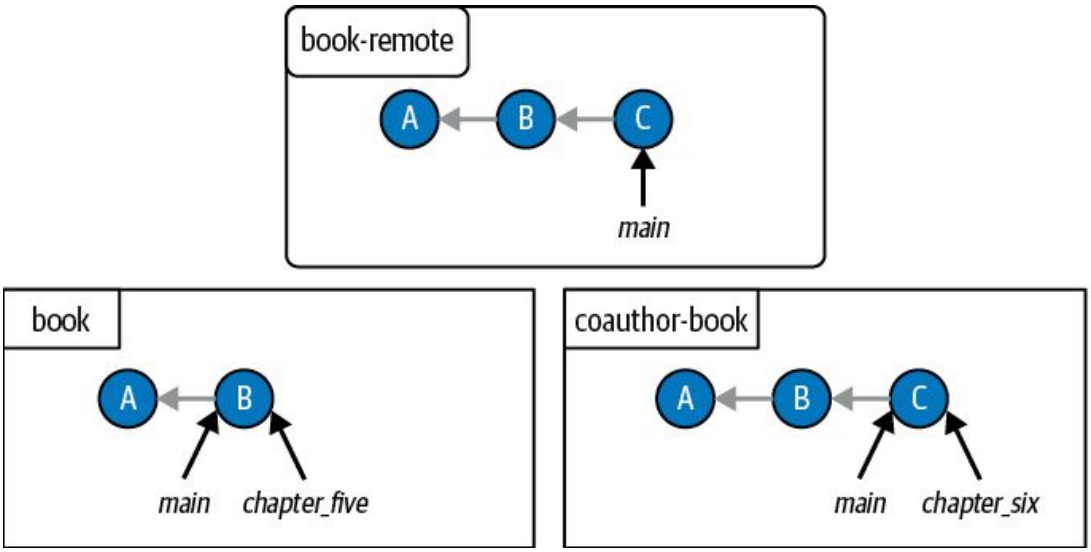


Figura 9-2: Projeto Book depois que meu coautor fez o push de suas alterações para o repositório remoto.

Quando terminei de trabalhar no branch chapter_five, que representarei como commit D, também pretendia fazer o merge do trabalho no branch main. No entanto, meu coautor me informou que ele já tinha adicionado trabalho ao branch remoto main; logo, primeiro tive de atualizar meu branch local main com o trabalho que ele adicionou ao branch remoto main, como visto na Figura 9-3.

Na Figura 9-3, podemos ver que o histórico de desenvolvimento do branch local main no repositório book é composto dos commits A, B e C, enquanto o histórico de desenvolvimento do branch chapter_five é composto dos commits A, B e D. Já que não é possível seguirmos o histórico de desenvolvimento do branch chapter_five para alcançar o branch main, isso significa que os históricos de desenvolvimento desses branches divergiram. Apresentarei a seguir duas opções.

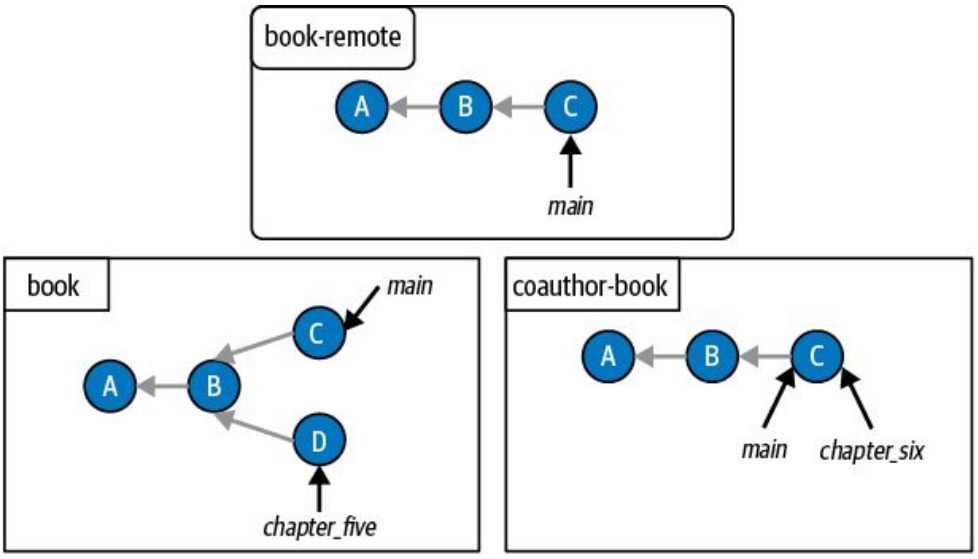


Figura 9-3: Projeto Book depois que contribuí com trabalho para meu branch local chapter_five, fiz o fetch das alterações do branch remoto main e as integrei.

Uma opção é fazer o merge de meu branch local chapter_five no branch local main, o que será um merge three-way, e, em seguida, fazer o push do branch main atualizado para o repositório remoto.

A outra é executar um merge no repositório remoto por meio de um recurso do serviço de hospedagem chamado *pull request*. Você aprenderá o que são os pull requests no Capítulo 12; para os fins deste capítulo, presumiremos que optei pela primeira opção e executei um merge three-way em meu repositório local.

A Figura 9-4 mostra o estado dos repositórios após a integração do branch chapter_five no branch main com um merge three-way. O commit de merge produzido pelo merge three-way está representado como commit M.

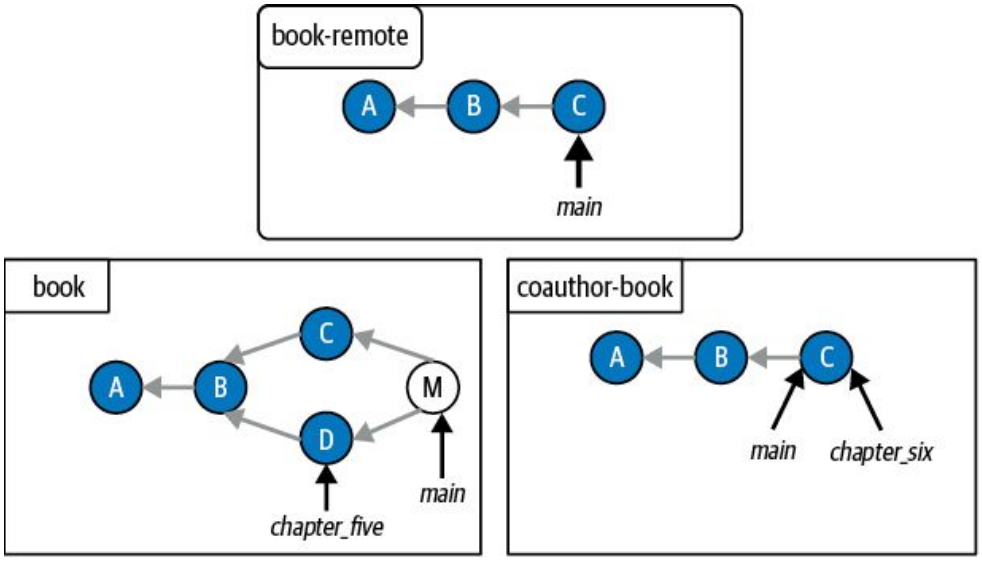


Figura 9-4: Projeto Book depois que fiz o merge do branch chapter_five no branch main usando um merge three-way.

Como você pode ver no Exemplo 9-1 do Projeto Book, os merges three-way produzem commits de merge, que são commits que podem ter mais de um commit pai.

[NOTA]

Algumas pessoas não gostam de merges three-way porque acham que os commits de merge tornam o histórico de commits mais complicado. Para evitar os merges three-way, você pode usar o processo de rebasing, que aprenderá no Capítulo 11.

Agora você criará uma situação no projeto Rainbow em que terá de executar um merge three-way. Durante o processo, aprenderá a definir branches upstream e conhecerá algumas características dos arquivos modificados do diretório de trabalho.

Definindo um cenário de merge three-way

Primeiro consulte a seção Acompanhe 9-1 para começar a listar as cores que *não* fazem parte do arco-íris em um novo arquivo chamado othercolors.txt no diretório de projeto rainbow.

[ACOMPANHE 9-1]

❶ No diretório de projeto rainbow em seu editor de texto, crie um novo arquivo chamado othercolors.txt. Digite “Brown is not a color in the rainbow.” na linha 1 do arquivo e salve-o.

❷ rainbow $ **git add othercolors.txt**

❸ rainbow $ **git commit -m "brown"**

```
[main 7f0a87a] brown
1 file changed, 1 insertion(+)
create mode 100644 othercolors.txt
```

❹ rainbow $ **git log**

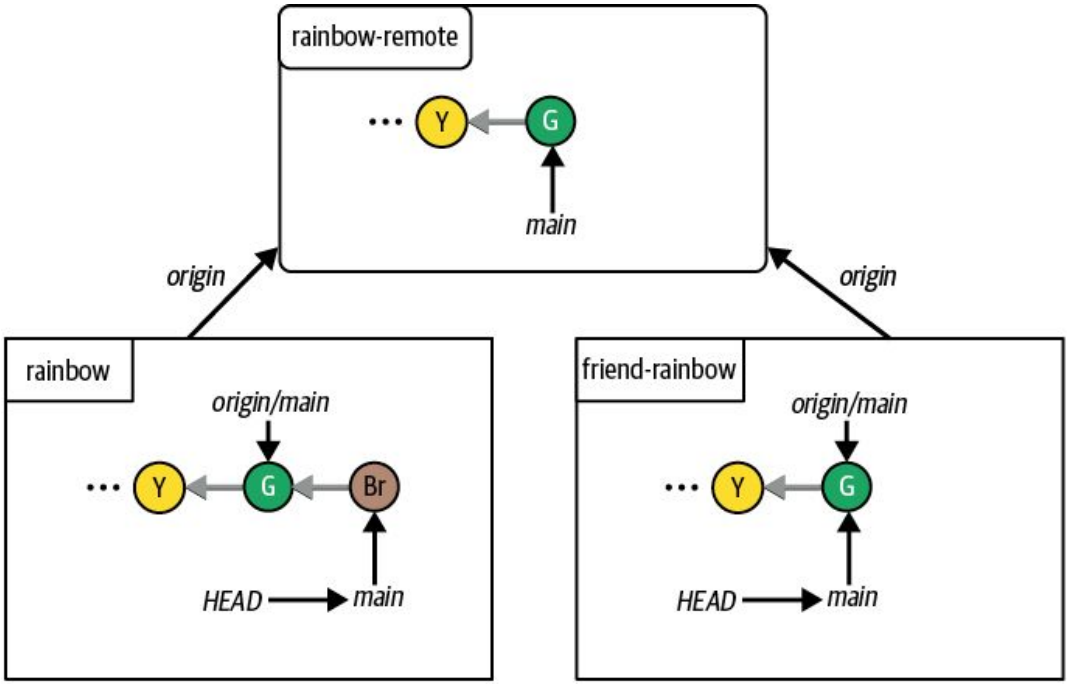
```
commit 7f0a87a318e50638eec50a484bf8dfa76b76d08e (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 12:46:29 2022 +0100
brown
commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (origin/main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green
```

O que deve ser observado:

- Você fez o commit brown (marrom) no repositório rainbow.

Isso é mostrado na seção Visualize 9-3.

[VISUALIZE 9-3]



Projeto Rainbow depois que você fez o commit brown no repositório rainbow.

Antes de você fazer o push de seu commit para o repositório remoto, aprenderemos a definir branches upstream.

Definindo branches upstream

Nos capítulos 7 e 8, mencionei que, quando fazemos o push de algum trabalho de um branch local para um branch remoto, o Git precisa de uma maneira de saber para qual branch remoto queremos fazer o push. Se nenhum branch upstream tiver sido definido para o branch local em que você estiver trabalhando, será preciso especificar para qual branch remoto será feito o push no momento da inserção do comando git push. Se um branch local tiver um branch upstream definido para ele, você poderá usar git push sem argumentos e o Git fará o push do trabalho automaticamente para esse branch.

Você também aprendeu que branches upstream são definidos automaticamente quando clonamos um repositório, mas isso *não* ocorre quando um repositório é inicializado localmente. O repositório rainbow foi inicializado localmente, e você ainda não definiu nenhum branch upstream.

Para evitar ter de especificar o *nome_remoto*, referente ao repositório remoto, e o branch sempre que você usar o comando `git push` no branch `main` em seu repositório `rainbow`, defina um branch upstream para o branch `main` e assim poderá utilizar o comando `git push` sem argumentos.

[NOTA]

Uma vez que um branch upstream tiver sido definido para um branch local, você também poderá usar outros comandos, como git pull, sem argumentos. Conheceremos o comando git pull no fim deste capítulo.

Para definir o branch upstream, você usará o comando git branch com a opção -u, que é a abreviação de --set-upstream-to (definir upstream para). Passe o nome do branch remoto como argumento, especificando o *nome_remoto* que representa o repositório remoto, uma barra e o nome do branch remoto (por exemplo, origin/main). Verá então que pode usar o comando git push sem nenhum argumento adicional.

[SALVE O COMANDO]

git branch -u <nome_remoto>/<nome_branch>
Define um branch upstream para o branch local atual.

Para verificar se um branch upstream foi definido, você usará o comando git branch -vv introduzido no Capítulo 7.

Consulte a seção Acompanhe 9-2 para definir o branch upstream para o branch local main.

[ACOMPANHE 9-2]

❶ rainbow $ **git branch -vv**
* main 7f0a87a brown

❷ rainbow $ **git branch -u origin/main**
branch 'main' set up to track 'origin/main'.

❸ rainbow $ **git branch -vv**
* main 7f0a87a [origin/main: ahead 1] brown

O que deve ser observado:

- Na etapa 1, a saída de git branch -vv mostra que não há nenhum branch upstream definido para o branch local main.
- Na etapa 2, a saída do comando mostra explicitamente que um branch upstream foi definido (branch 'main' set up to track 'origin/main').
- Na etapa 3, a saída de git branch -vv mostra que o branch main do repositório remoto com *nome__remoto* origin foi definido como branch upstream para o branch local main.

Agora que você definiu um branch upstream para seu branch local main, consulte a seção Acompanhe 9-3 para usar o comando git push sem nenhum argumento.

[ACOMPANHE 9-3]

❶ rainbow $ **git push**

```
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 310 bytes | 310.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:gitlearningjourney/rainbow-remote.git
6987cd2..7f0a87a main -> main
```

❷ rainbow $ **git log**

```
commit 7f0a87a318e50638eec50a484bf8dfa76b76d08e (HEAD -> main, origin/main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 12:46:29 2022 +0100
brown
commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green
```

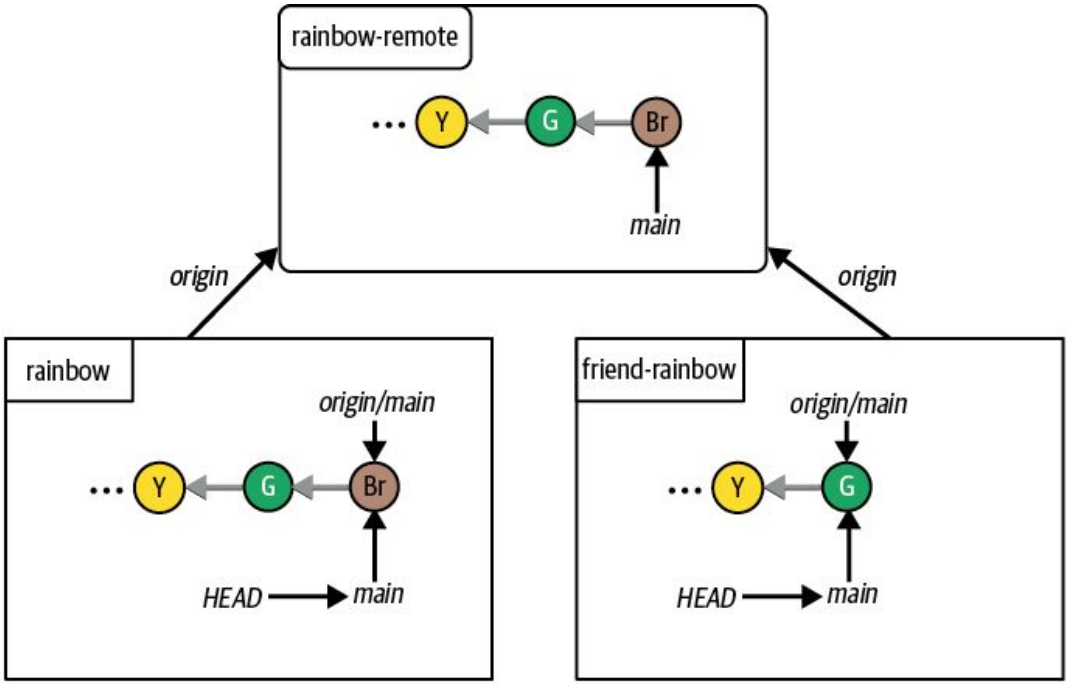
❸ Acesse o repositório rainbow-remote em seu serviço de hospedagem e atualize a página. Você deve ver o commit brown.

O que deve ser observado:

- Você fez o push de seu branch local main para o repositório remoto.

Isso é mostrado na seção Visualize 9-4.

[VISUALIZE 9-4]



Projeto Rainbow depois que você fez o push de seu branch local main para o repositório rainbow-remote.

Você aprendeu a definir branches upstream em um repositório local, fez o commit brown no repositório rainbow e o enviou para o repositório remoto. Para que se encontre em uma situação em que terá de executar um merge three-way, é preciso que existam históricos de desenvolvimento divergentes entre dois branches.

A seguir, seu amigo continuará trabalhando no branch local main em seu repositório local *sem* fazer o fetch das alterações que você enviou para o branch remoto main, o que fará o branch local main do repositório friend-rainbow e o branch main do repositório rainbow-remote divergirem.

Enquanto seu amigo trabalha no branch local main, você também conhecerá algumas características dos arquivos modificados do diretório de trabalho e verá o que acontece quando editamos um arquivo várias vezes entre commits.

Editando o mesmo arquivo várias vezes entre commits

Até agora, você editou arquivos apenas uma vez e os adicionou à área de preparação. No entanto, é importante saber que, se adicionar um arquivo à área de preparação e depois fizer *outra alteração* nele, o Git interpretará isso como uma nova versão do arquivo e o marcará como modificado. Se você quiser então que a versão mais recente do arquivo seja incluída em seu próximo commit, terá de adicionar *novamente* a versão à área de preparação.

Para vermos como funciona, nesta seção seu amigo adicionará a cor blue (azul) à lista de cores em rainbowcolors.txt e incluirá o arquivo editado na área de preparação – mas cometerá um erro de digitação, escrevendo “Bloo is the fifth color of the rainbow.”. Quando perceber, editará o arquivo mais uma vez para corrigir o erro e terá de adicioná-lo novamente à área de preparação para que as alterações mais recentes sejam incluídas no próximo commit.

As próximas seções Visualize usarão os Diagramas Git que criamos no Capítulo 2 e nos concentraremos no repositório friend-rainbow para mostrar tudo que está acontecendo. Primeiro consulte a seção Acompanhe 9-4 para verificar o estado atual do diretório de trabalho friend-rainbow e da área de preparação.

[ACOMPANHE 9-4]

❶ friend-rainbow $ **git status**
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean

O que deve ser observado:

- A saída de git status está exibindo nothing to commit, working tree clean, o que indica que o estado do diretório de trabalho e da área de preparação é o mesmo. Não existem arquivos modificados no diretório de trabalho e não há arquivos recentes na área de preparação.

Essa observação é mostrada na seção Visualize 9-5.

O que deve ser observado:

- A versão do arquivo rainbowcolors.txt no diretório de trabalho e na área de preparação menciona as cores red (vermelho), orange (laranja), yellow (amarelo) e green (verde). Ela está sendo representada como versão A (vA).

[VISUALIZE 9-5]

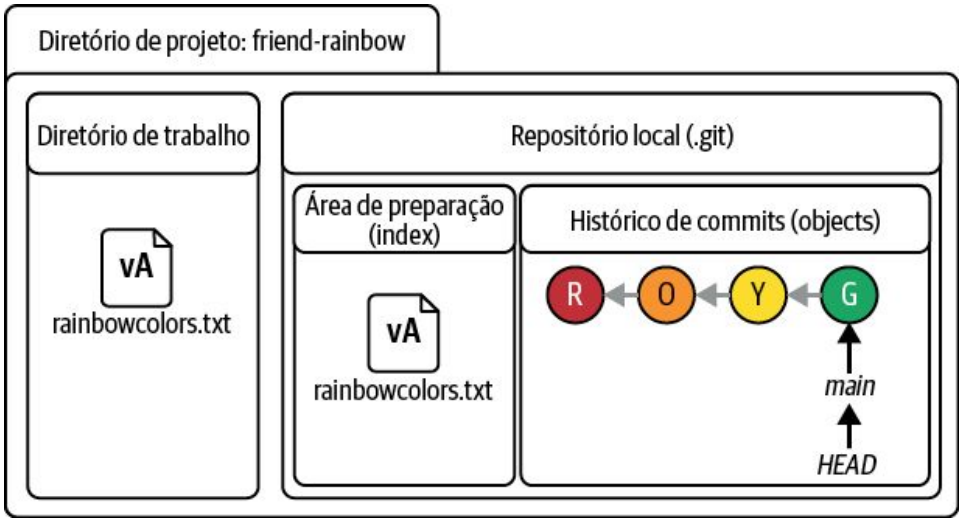


Diagrama Git mostrando o estado atual do diretório de projeto friend-rainbow.

A seguir, na seção Acompanhe 9-5, seu amigo editará o arquivo rainbowcolors.txt e adicionará a cor blue.

[ACOMPANHE 9-5]

❶ Abra o arquivo rainbowcolors.txt do diretório de projeto friend-rainbow em uma janela de editor de texto, adicione “Bloo is the fifth color of the rainbow.” na linha 5 e salve o arquivo. Observe que você está incluindo a frase com um erro de digitação propositalmente para fins de aprendizado.

❷ friend-rainbow $ **git status**

On branch main

Your branch is up to date with 'origin/main'.

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: rainbowcolors.txt

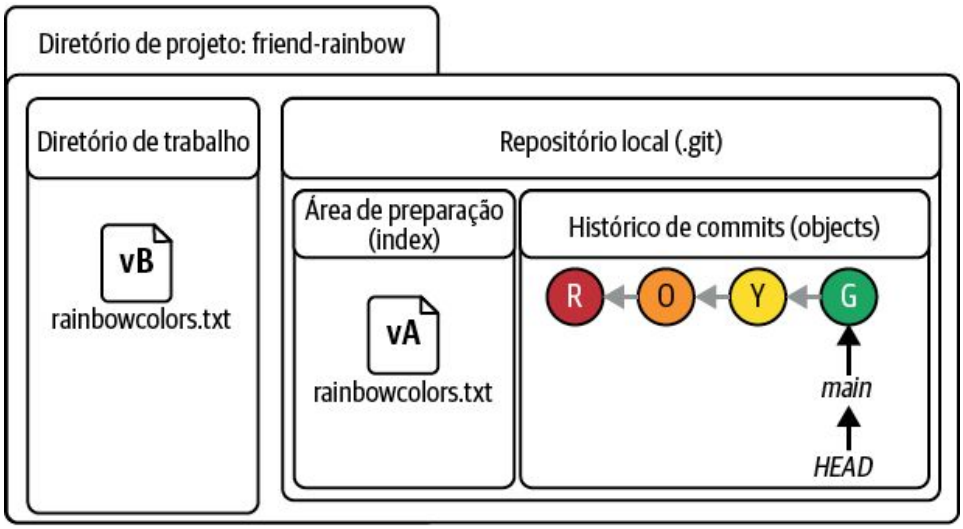
no changes added to commit (use "git add" and/or "git commit -a")

O que deve ser observado:

- Na etapa 2, a saída de git status indica que agora existe um arquivo modificado no diretório de trabalho, que ainda não está preparado para o commit.

Essa observação é mostrada na seção Visualize 9-6.

[VISUALIZE 9-6]



Diretório de projeto friend-rainbow depois que seu amigo editou o arquivo rainbowcolors.txt no diretório de trabalho.

O que deve ser observado:

- A versão do arquivo rainbowcolors.txt na área de preparação ainda é a vA; ela não mudou.
- A versão do arquivo rainbowcolors.txt no diretório de trabalho mudou: ela menciona as cores red, orange, yellow, green e blue. Foi representada como versão B (vB).

A seguir, na seção Acompanhe 9-6, veremos o que acontece quando seu amigo adiciona o arquivo à área de preparação.

[ACOMPANHE 9-6]

❶ friend-rainbow $ **git add rainbowcolors.txt**

❷ friend-rainbow $ **git status**

On branch main

Your branch is up to date with 'origin/main'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

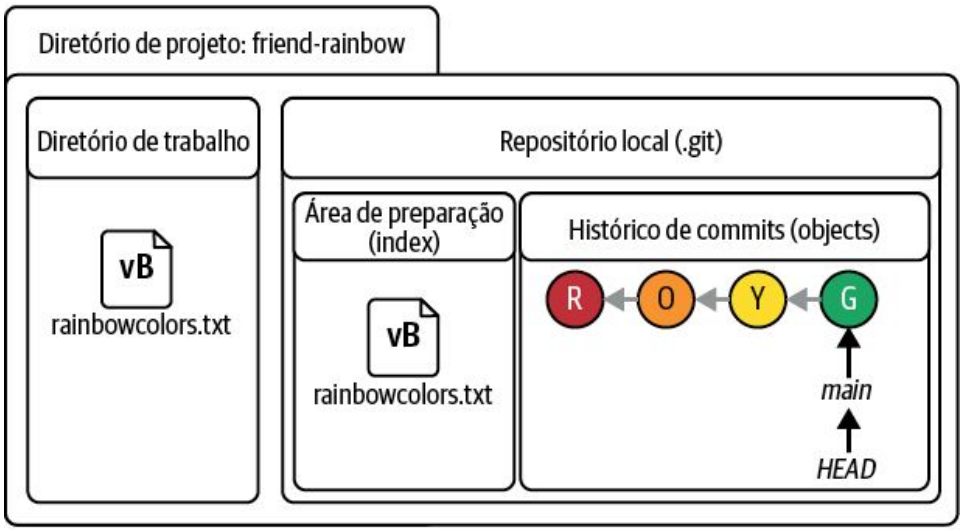
modified: rainbowcolors.txt

O que deve ser observado:

- Na etapa 2, a saída de git status indica que agora existe um arquivo modificado no diretório de trabalho que também está preparado para o commit.

Isso é mostrado na seção Visualize 9-7.

[VISUALIZE 9-7]



Diretório de projeto friend-rainbow depois que seu amigo adicionou o arquivo modificado rainbowcolors.txt à área de preparação.

O que deve ser observado:

- A versão do arquivo rainbowcolors.txt na área de preparação mudou de vA para vB.
- A versão do arquivo rainbowcolors.txt na área de preparação é a mesma do diretório de trabalho.

Seu amigo adicionou a versão atualizada do arquivo rainbowcolors.txt à área de preparação, e ele está pronto para fazer parte de um commit. Consulte a seção Acompanhe 9-7 para ver o que acontecerá quando seu amigo voltar para editar o arquivo a fim de corrigir o erro de digitação, alterando-o de “Bloo” para “Blue”.

[ACOMPANHE 9-7]

❶ No diretório de projeto friend-rainbow em seu editor de texto, corrija o erro de digitação na linha 5 do arquivo rainbowcolors.txt para que a frase exiba “Blue is the fifth color of the rainbow”. Salve o arquivo.

❷ friend-rainbow $ **git status**

On branch main

Your branch is up to date with 'origin/main'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

modified: rainbowcolors.txt

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

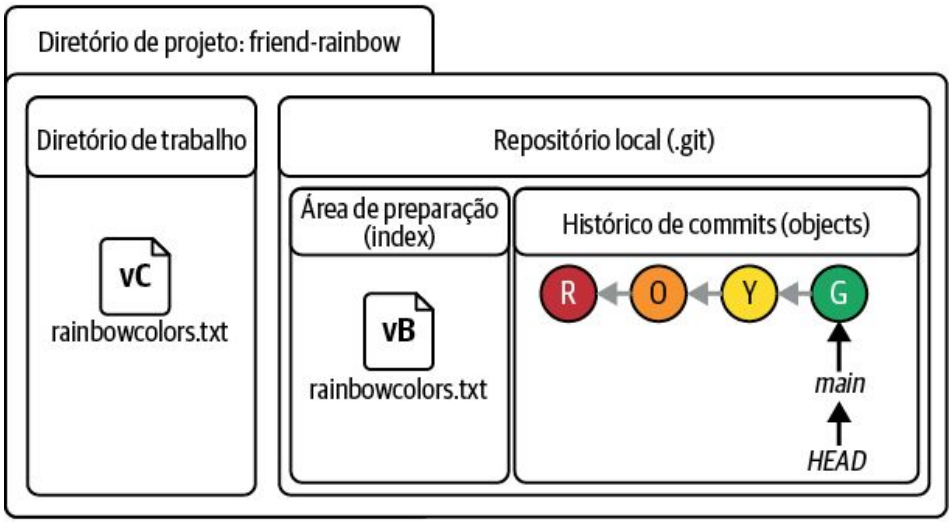
modified: rainbowcolors.txt

O que deve ser observado:

- Na saída de git status na etapa 2, uma versão do arquivo rainbowcolors.txt é listada como um arquivo modificado que está preparado para o commit e outra versão aparece como um arquivo modificado *não* preparado para o commit.

Isso é mostrado na seção Visualize 9-8.

[VISUALIZE 9-8]



Diretório de projeto friend-rainbow depois que seu amigo editou o arquivo rainbowcolors.txt pela segunda vez.

O que deve ser observado:

- A versão do arquivo rainbowcolors.txt na área de preparação é a vB. Essa é a versão com erro de digitação.
- A versão do arquivo rainbowcolors.txt no diretório de trabalho mudou de vB para a versão C (vC). O arquivo vC é a versão sem o erro de digitação.

Por essas observações, podemos deduzir que só porque um arquivo com um nome específico está preparado para o commit isso *não significa* que ele foi atualizado automaticamente com outras alterações que possamos ter feito. É preciso adicionar explicitamente cada versão atualizada de um arquivo à área de preparação para incluir as alterações mais recentes no próximo commit. Consulte a seção Acompanhe 9-8 para ver isso na prática.

[ACOMPANHE 9-8]

❶ friend-rainbow $ **git add rainbowcolors.txt**

❷ friend-rainbow $ **git status**

On branch main

Your branch is up to date with 'origin/main'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

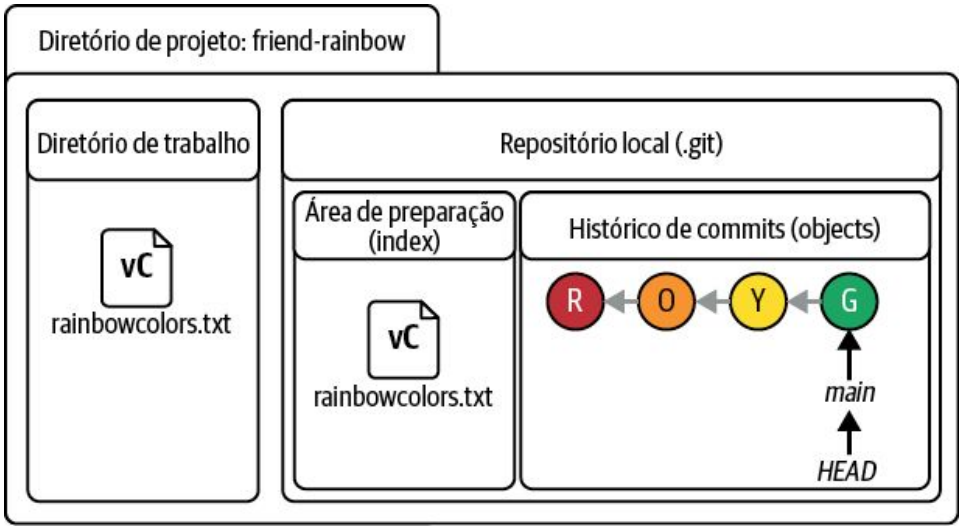
modified: rainbowcolors.txt

O que deve ser observado:

- Na saída de git status na etapa 2, o arquivo rainbowcolors.txt está listado como um arquivo modificado preparado para o commit. Não há mais arquivos modificados que possam ser adicionados à área de preparação.

Podemos ver isso na seção Visualize 9-9.

[VISUALIZE 9-9]



Diretório de projeto friend-rainbow depois que seu amigo adicionou novamente o arquivo rainbowcolors.txt à área de preparação.

O que deve ser observado:

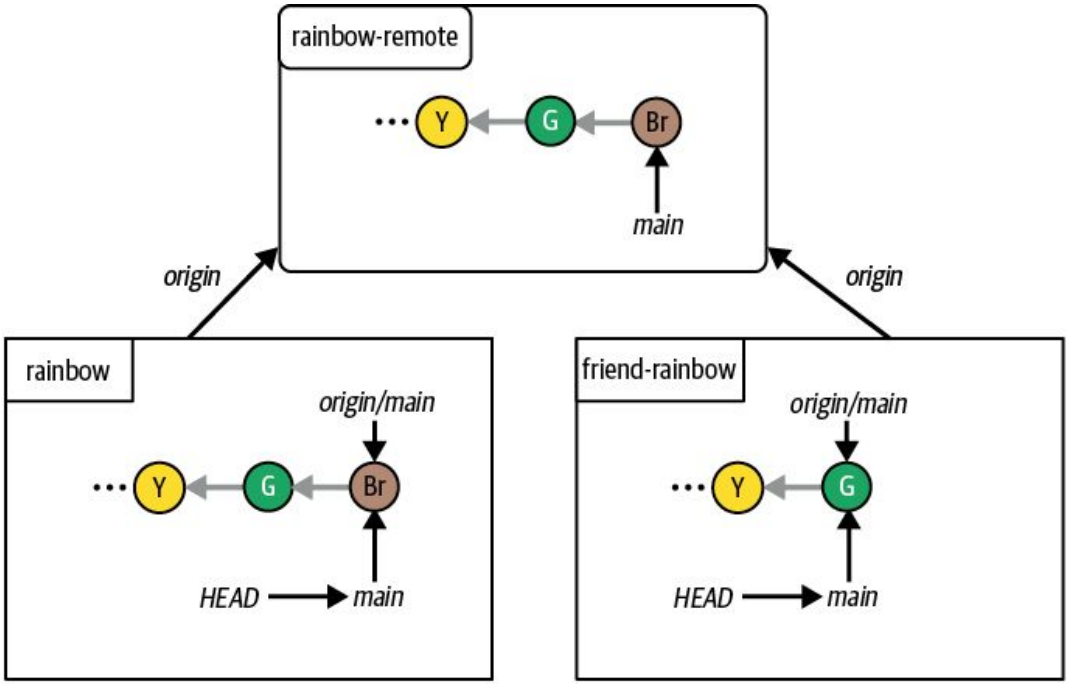
- A versão do arquivo rainbowcolors.txt na área de preparação mudou de vB para vC, o que indica que a versão mais recente do arquivo foi adicionada à área de preparação.

Agora seu amigo terminará de fazer o commit blue e tentará executar o push de seu trabalho para o repositório remoto. No entanto, já que seu branch local main está fora de sincronia com o branch remoto main, ele verá uma mensagem de erro. Isso será mostrado na próxima seção.

Trabalhando ao mesmo tempo com outras pessoas em arquivos diferentes

Na seção Visualize 9-10, podemos ver que seu amigo não executou o fetch do commit brown que você fez no branch main no repositório rainbow e enviou para o repositório remoto.

[VISUALIZE 9-10]



Estado atual dos repositórios locais e remoto.

O branch local main do repositório friend-rainbow está fora de sincronia com o branch remoto main. Seu amigo está pronto para fazer o commit blue e tentar compartilhar seu trabalho, mas quando ele tentar fazer o push do trabalho para o repositório remoto aparecerá uma mensagem de erro. Veremos isso na prática na seção Acompanhe 9-9.

[ACOMPANHE 9-9]

❶ friend-rainbow $ **git commit -m "blue"**
[main 342bbfc] blue
1 file changed, 2 insertions(+), 1 deletion(-)

❷ friend-rainbow $ **git push**
To github.com:gitlearningjourney/rainbow-remote.git
! [rejected] main -> main (fetch first)
error: failed to push some refs to 'github.com:gitlearningjourney/rainbow-remote.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by hint: another repository pushing
hint: to the same ref. You may want hint: to first integrate the remote changes
hint: (e.g., 'git pull hint: ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

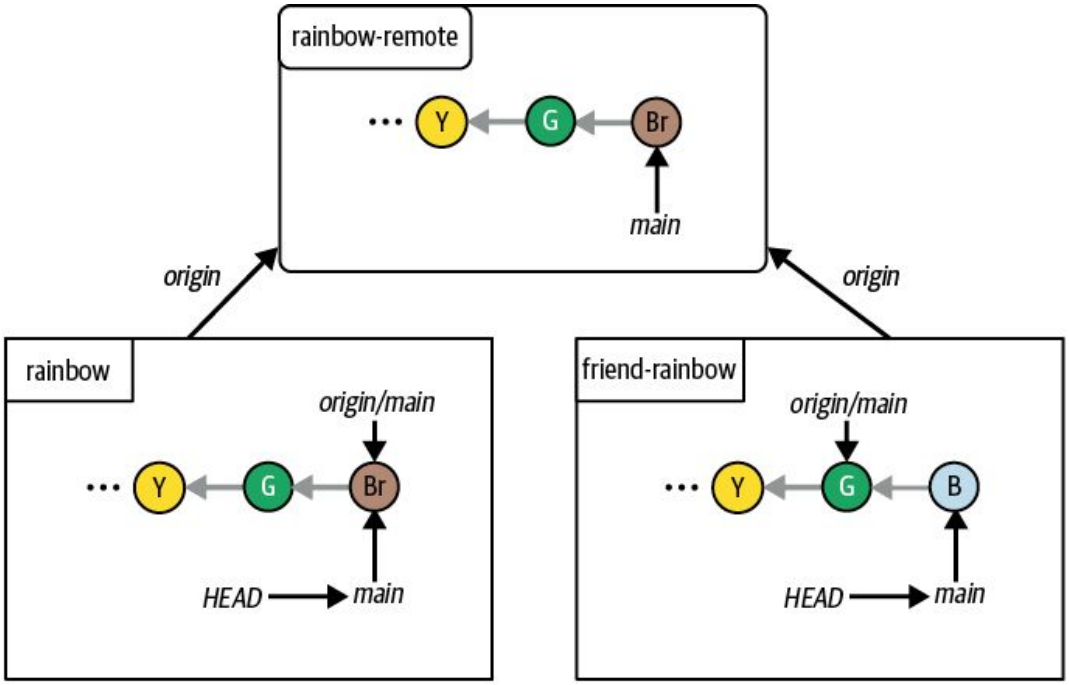
❸ friend-rainbow $ **git log**
commit 342bbfc96bb03053f23ea7f7564ca207c58ceab2 (**HEAD** -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:00:56 2022 +0100
blue
commit 6987cd2996e245ec24ee9c5ea99874f0a01a31cd (**origin/main**, **origin/HEAD**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 11:49:03 2022 +0100
green

O que deve ser observado:

- Na etapa 2, a saída do comando git push mostra que seu amigo recebeu uma mensagem de erro. Ele não pode fazer o push de suas alterações para o repositório remoto.

Os históricos de desenvolvimento do branch local main e do branch remoto main divergiram e o Git não pode fazer o merge das alterações de um no outro com um simples merge fast-forward. A inclusão do commit blue e os históricos de desenvolvimento divergentes são mostrados na seção Visualize 9-11.

[VISUALIZE 9-11]



Projeto Rainbow depois que seu amigo fez o commit blue no repositório friend-rainbow.

O que deve ser observado:

- No repositório friend-rainbow, o branch local main aponta para o commit blue.
- No repositório rainbow-remote, o branch main aponta para o commit brown.

Vejamos com mais detalhes a mensagem de erro que seu amigo recebeu:

> error: failed to push some refs to 'https://github.com/gitlearningjourney/rainbow-remote.git' hint: Updates were rejected because the remote contains work that you do not have locally. This is usually caused by another repository pushing to the same ref. You may want to first integrate the remote changes (e.g., 'git pull ...') before pushing again. See the 'Note about fast-forwards' in 'git push --help' for details.

O Git está dizendo ao seu amigo que existem commits no branch remoto main que ele ainda não baixou (ou extraiu). Está informando que ele tem de fazer o fetch (ou pull) do trabalho remoto e integrá-lo ao seu branch local main antes de poder fazer o push para o repositório remoto. É bom ressaltar que o pulling é semelhante ao fetching, com algumas diferenças; examinaremos o pulling posteriormente neste capítulo. Por enquanto, para integrar as alterações remotas, seu amigo executará um merge three-way.

Merge three-way na prática

No Capítulo 8, você aprendeu que incorporar alterações de um repositório remoto é um processo de duas etapas:

1. Primeiro fazemos o fetch das alterações a partir do repositório remoto.
2. Depois integramos as alterações ao branch local no repositório local.

Seu amigo seguirá as mesmas etapas agora, exceto na etapa 2, em que acabará executando um merge three-way em vez de um merge fast-forward porque os históricos de desenvolvimento dos dois branches envolvidos no merge divergiram.

Na execução de um merge three-way, o Git cria um commit de merge. Para fazê-lo, ele entra em um editor de texto na linha de comando. Aprenderemos com um pouco mais de detalhes como você trabalhará com esse editor de texto de linha de comando.

INTRODUÇÃO AO VIM, O EDITOR DE TEXTO DE LINHA DE COMANDO

Por padrão, o Git usa um editor de texto na linha de comando chamado *Vim* para a criação de mensagens de commit. Se um commit estiver sendo feito (seja um commit comum ou de merge) e uma mensagem de commit não for especificada, o Git entrará no Vim na linha de comando. Ainda não encontramos o Vim neste livro porque sempre que fizemos um commit usando o comando `git commit` no projeto Rainbow passamos a opção `-m` option e especificamos explicitamente uma mensagem de commit.

[NOTA]

O Git permite alterar o editor de texto de linha de comando padrão que ele usa do Vim para outro editor. Por exemplo, você pode defini-lo como o mesmo editor de texto que utiliza para editar arquivos em seu projeto.

O Vim pode parecer complexo porque a linha de comando terá uma aparência diferente; e, a menos que você conheça alguns aspectos básicos, seu uso não será intuitivo. Examinaremos então os aspectos básicos necessários para a navegação no Vim.

Quando o Git entrar no Vim na linha de comando, você terá a opção de inserir texto, editar texto ou aprovar e salvar texto. No caso de um merge three-way, o Git esboçará uma mensagem padrão para o commit de merge e o Vim a apresentará para você, como visto na Figura 9-5.

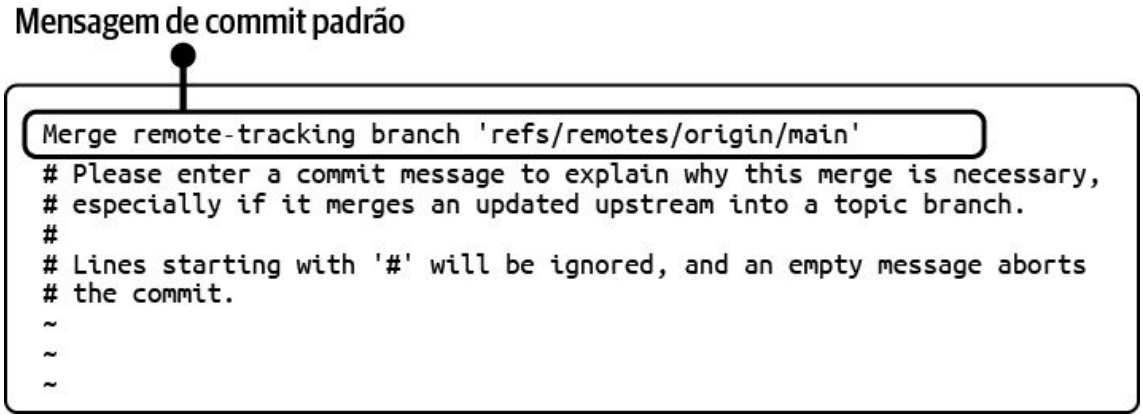


Figura 9-5: Exemplo de uma mensagem de commit padrão esboçada pelo Git e apresentada no Vim na linha de comando.

Neste exemplo, abordaremos apenas como aceitar e salvar a mensagem de commit padrão. Para fazer isso, você deve pressionar a tecla Escape, digitar dois-pontos (:) e pressionar a tecla W, a tecla Q e a tecla Enter uma após a outra em ordem, como visto na Figura 9-6. A letra `w` representa “write” no Vim, que significa salvar, e `q` representa “quit,” que significa sair. (Se algo der errado, você pode pressionar Escape para tentar novamente).

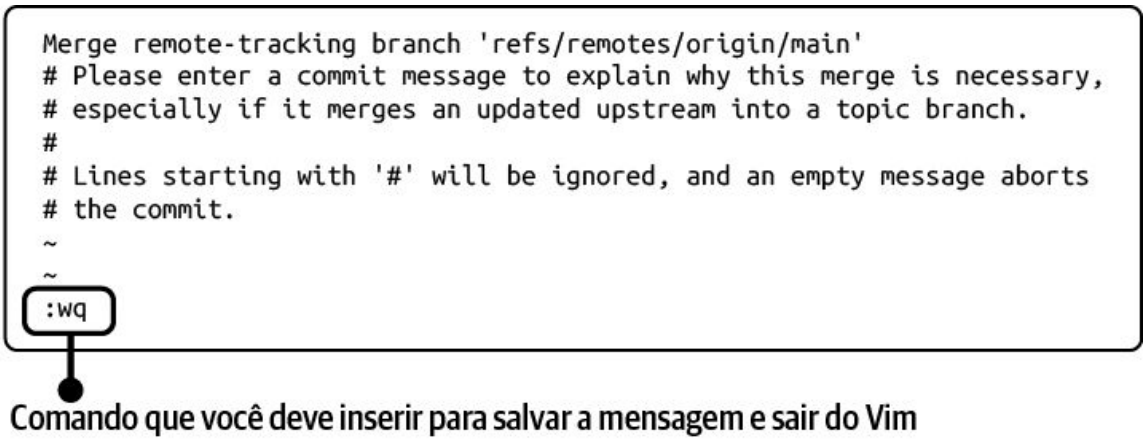


Figura 9-6: Comando que você deve inserir para salvar a mensagem e sair do Vim.

Isso salvará a mensagem de commit e encerrará o editor de texto de linha de comando Vim. Essas instruções serão repetidas em todas as seções Acompanhe que demandarem que você saia do Vim.

Agora que vimos alguns aspectos básicos de como trabalhar com o Vim, seu amigo está pronto para executar as duas etapas de incorporação das alterações remotas em seu branch local.

EXECUTANDO O MERGE THREE-WAY

Após ler a mensagem de erro que o Git apresentou, seu amigo fará o fetch das alterações do branch remoto `main` para seu repositório local. Isso atualizará o branch de rastreamento remoto `origin/main`. Em seguida, ele fará o merge do branch de rastreamento remoto `origin/main` com seu branch local `main`.

Consulte a seção Acompanhe 9-10 para executar as duas etapas de incorporação das alterações remotas no branch local `main` do repositório `friend-rainbow`.

[ACOMPANHE 9-10]

❶ friend-rainbow $ **git fetch**

remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 290 bytes | 145.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
6987cd2..7f0a87a main -> origin/main

❷ friend-rainbow $ **git merge origin/main**

❸ A linha de comando entrará no Vim após o comando git merge ser executado. A mensagem de commit padrão que o Git esboçará para você será Merge remote-tracking branch 'refs/remotes/origin/main' ou Merge remote-tracking branch 'origin/main'. Para aceitar a mensagem padrão no Vim e sair do editor, pressione a tecla Escape, digite **:wq** e pressione a tecla Enter. Você verá a saída a seguir:

Merge made by the 'ort' strategy.
othercolors.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 othercolors.txt

❹ friend-rainbow $ **git log**

commit 225839938563c7458af81daca7beb782dfcbfb27 (**HEAD -> main**)
Merge: 342bbfc 7f0a87a
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:05:46 2022 +0100
Merge remote-tracking branch 'refs/remotes/origin/main'
commit 342bbfc96bb03053f23ea7f7564ca207c58ceab2
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:00:56 2022 +0100
blue
commit 7f0a87a318e50638eec50a484bf8dfa76b76d08e (**origin/main**, **origin/HEAD**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 12:46:29 2022 +0100
brown

O que deve ser observado:

- Na etapa 3, a saída de git merge está exibindo Merge made by the 'ort' strategy. Isso indica que esse foi um merge three-way em vez de um merge fast-forward. (Se você tiver uma versão mais antiga do Git, sua saída pode informar que o merge foi feito pela estratégia “recursiva”, o que também está correto.)

- O Git esboçou uma mensagem de commit padrão que pode ter sido `Merge remote-tracking branch 'refs/remotes/origin/main'` ou `Merge remote-tracking branch 'origin/main'`. As duas estão corretas.
- Na etapa 4, a saída de `git log` está exibindo o commit de merge `2258399` e listando os dois commits pais no campo `Merge`: `342bbfc` (o commit blue) e `7f0a87a` (o commit brown). Os hashes de seus commits serão diferentes dos deste livro, porque os hashes de commit são únicos.

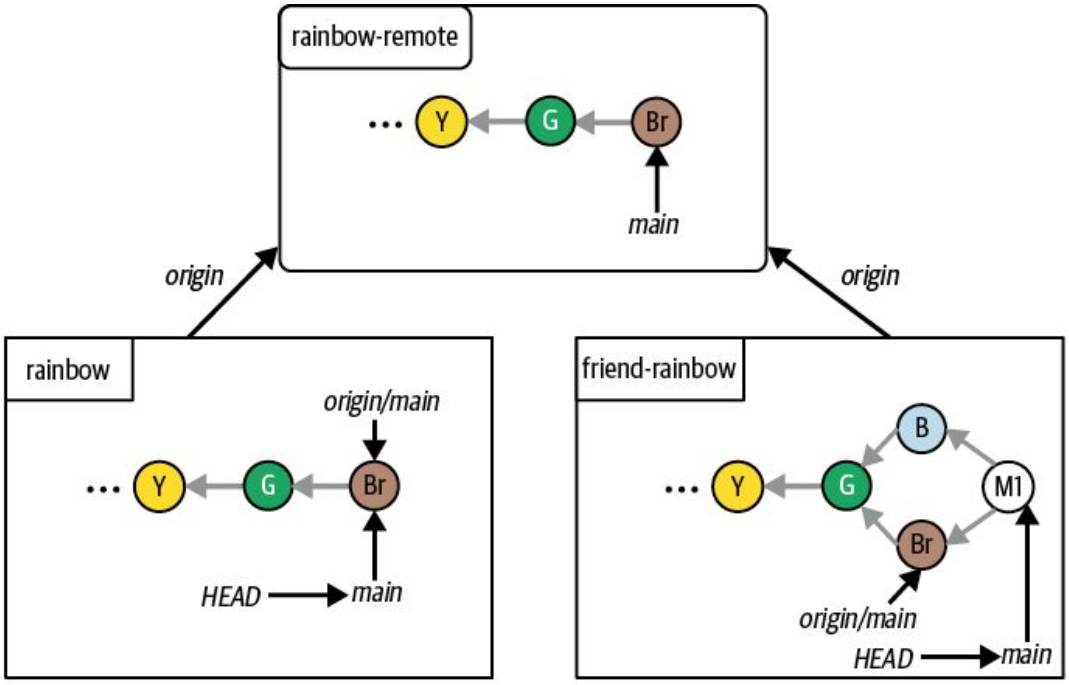
A seção Visualize 9-12 mostra o estado do projeto Rainbow após a execução das etapas da seção Acompanhe 9-10.

O que deve ser observado:

- No repositório `friend-rainbow`, estou exibindo o commit de merge como M1 e mostrando que ele associa os dois históricos de desenvolvimento.

O commit de merge M1 tem dois commits pais. Você deve se lembrar de que, no Capítulo 4, usou o comando `git cat-file -p <commit_hash>` para visualizar os commits pais de um commit. Na próxima seção Acompanhe, seu amigo utilizará o mesmo comando, passando o hash do commit de merge M1, para ver seus commits pais (representados por seus hashes). Para recuperar o hash do commit de merge M1, você pode usar a saída do comando `git log`. Não se esqueça de que deve utilizar o hash do *seu* commit de merge M1, e não o hash de commit deste livro (porque os hashes são únicos).

[VISUALIZE 9-12]



Projeto Rainbow depois que seu amigo fez o fetch do commit brown e executou um merge three-way.

Em seguida, seu amigo fará o push de suas alterações para o repositório remoto a fim de assegurar que ele esteja atualizado. Consulte a seção Acompanhe 9-11 para executar essas ações agora.

[ACOMPANHE 9-11]

❶ Recupere o hash do commit de merge M1 (você pode copiá-lo da saída de git log na seção Acompanhe 9-10). É preciso passar esse hash de commit como argumento para o comando git cat-file -p na etapa 2 dessa seção Acompanhe. Você pode copiar e colar o hash de commit inteiro ou inserir apenas os sete primeiros caracteres, como mostrado aqui.

❷ friend-rainbow $ **git cat-file -p 2258399**

tree 45330906e6041a0cd07849617a25443a9a5b08bd
parent 342bbfc96bb03053f23ea7f7564ca207c58ceab2
parent 7f0a87a318e50638eec50a484bf8dfa76b76d08e
author annaskoulikari <gitlearningjourney@gmail.com> 1645272346 +0100
committer annaskoulikari <gitlearningjourney@gmail.com> 1645272346 +0100
Merge remote-tracking branch 'refs/remotes/origin/main'

❸ friend-rainbow $ **git push**

Enumerating objects: 9, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 586 bytes | 586.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:gitlearningjourney/rainbow-remote.git
7f0a87a..2258399 main -> main

❹ friend-rainbow $ **git log**

commit 225839938563c7458af81daca7beb782dfcbfb27 (**HEAD -> main**, **origin/main**, **origin/HEAD**)
Merge: 342bbfc 7f0a87a
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:05:46 2022 +0100
Merge remote-tracking branch 'refs/remotes/origin/main'
commit 342bbfc96bb03053f23ea7f7564ca207c58ceab2
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:00:56 2022 +0100
blue

❺ Acesse o repositório rainbow-remote em seu serviço de hospedagem e atualize a página. O commit de merge deve estar aí.

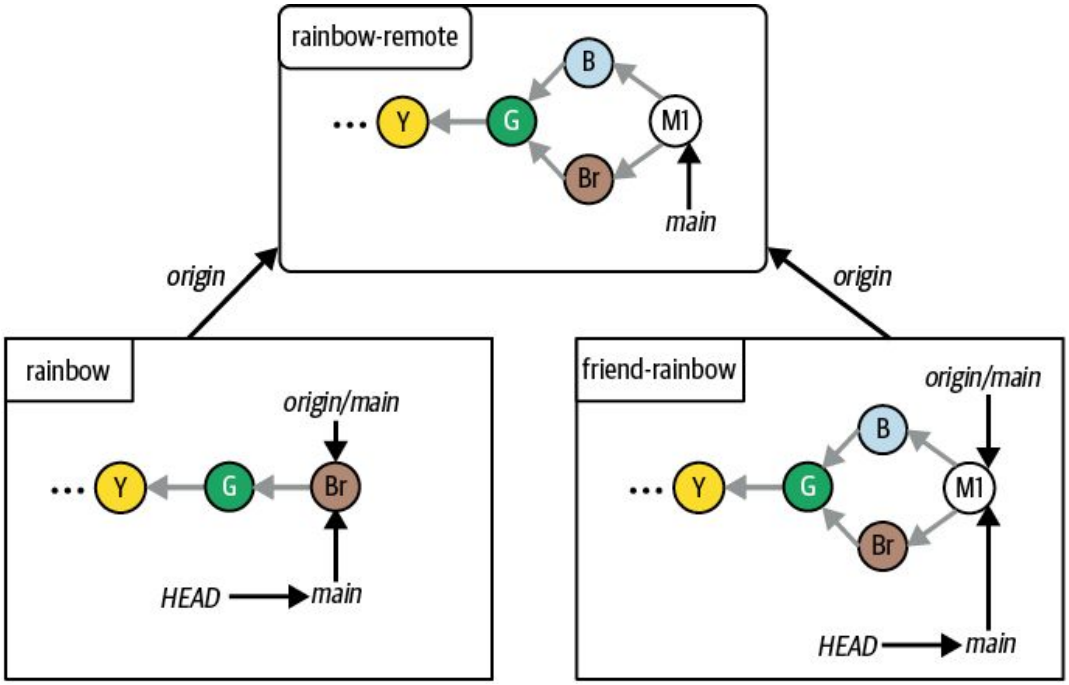
O que deve ser observado:

- Na etapa 2, a saída de git-cat file -p exibe os dois commits listados como pais: 342bbfc96bb03053f23ea7f7564ca207c58ceab2 (o commit blue) e 7f0a87a318e50638eec50a484bf8dfa76b76d08e (o commit brown).

- Na etapa 5, você verá que o commit de merge está no repositório remoto.

Essas observações são mostradas na seção Visualize 9-13.

[VISUALIZE 9-13]



Projeto Rainbow depois que seu amigo fez o push de suas alterações para o repositório remoto.

Agora o commit de merge M1 está no repositório friend-rainbow e no repositório rainbow-remote. Na próxima etapa você sincronizará seu repositório rainbow com o repositório remoto e também atualizará seu branch local com o commit de merge. Para isso, aprenderá a fazer um pull no Git.

Fazendo o pull de alterações de um repositório remoto

Até agora, no projeto Rainbow, quando você quis atualizar seu repositório local com alterações do repositório remoto o fez em duas etapas: primeiro executou o fetch dos dados do repositório remoto (usando o comando git fetch) e depois fez o merge (com o comando git merge) no branch local. Fazer o pull de dados nos permite executar as duas ações ao mesmo tempo.

No Git, usamos o termo *pull* ou *pulling* para nos referir ao processo de buscar dados em um repositório remoto e integrá-los a um branch em um repositório local ao mesmo tempo, e o comando que utilizamos para fazer isso é git pull. Se você não tiver um branch upstream definido para seu branch local, deve especificar o *nome_remoto* referente ao repositório

remoto e o nome do branch que deseja atualizar. Se tiver um branch upstream definido, pode simplesmente usar `git pull` sem nenhum argumento.

[SALVE O COMANDO]

***git pull* <*nome_remoto*> <*nome_branch*>**
Faz o fetch de alterações do repositório remoto <nome_remoto> para o <nome_branch> especificado e as integra.

git pull
Se um branch upstream estiver definido para o branch atual, faz o fetch de alterações do branch upstream e as integra.

Há mais uma coisa que você precisa saber sobre o comando git pull. Como aprendemos, no Git existem duas maneiras de integrar alterações: com o merging e com o rebasing. O método que o comando git pull usará vai depender de se os históricos de desenvolvimento dos branches divergiram e, se isso tiver ocorrido, da opção que você escolher ao inserir o comando:

- Se os históricos de desenvolvimento do branch local e do branch remoto em um git pull *não* divergirem, por padrão ocorrerá um merge fast-forward.
- Se os históricos de desenvolvimento do branch local e do branch remoto em um git pull *divergirem*, você deve informar ao Git se deseja integrar as alterações com merging ou rebasing (caso contrário, verá uma mensagem de erro). Para solicitar ao Git que integre as alterações com merging, deve passar a opção --no-rebase. Para pedir a ele que integre as alterações com rebasing, deve passar a opção --rebase.

Como mostrado na Figura 9-7, o comando git pull combina o comando git fetch e o comando git merge ou git rebase.



Figura 9-7: O comando git pull faz o fetch das alterações e as integra ao mesmo tempo.

A pergunta então é quando fazer o fetch de alterações e integrá-las em duas etapas usando o comando git fetch e o comando git merge ou git rebase e quando executar essas duas etapas ao mesmo tempo utilizando apenas o comando git pull?

É comum os usuários do Git usarem o comando git pull quando os históricos de desenvolvimento dos branches local e remoto não divergem e, portanto, ocorre um simples merge fast-forward. Quando os históricos de

desenvolvimento dos branches local e remoto divergem, geralmente os usuários do Git preferem usar o comando `git fetch` e escolhem se farão um rebase ou um merge em uma etapa separada. Ao executar o processo em duas etapas, eles ganham mais tempo para investigar o que mudará em seu branch local e se preparar para a integração.

Neste livro, você seguirá essa prática comum, portanto só usará o comando `git pull` quando um merge fast-forward atualizar seu branch local.

Consulte a seção Acompanhe 9-12 para treinar o uso do comando `git pull` fazendo o pull das alterações do branch remoto `main` para o branch local `main` no repositório `rainbow`.

[ACOMPANHE 9-12]

❶ rainbow $ **git pull**

```
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 5 (delta 1), reused 5 (delta 1), pack-reused 0
Unpacking objects: 100% (5/5), 566 bytes | 141.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
7f0a87a..2258399 main -> origin/main
Updating 7f0a87a..2258399
Fast-forward
rainbowcolors.txt | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)
```

❷ rainbow $ **git log**

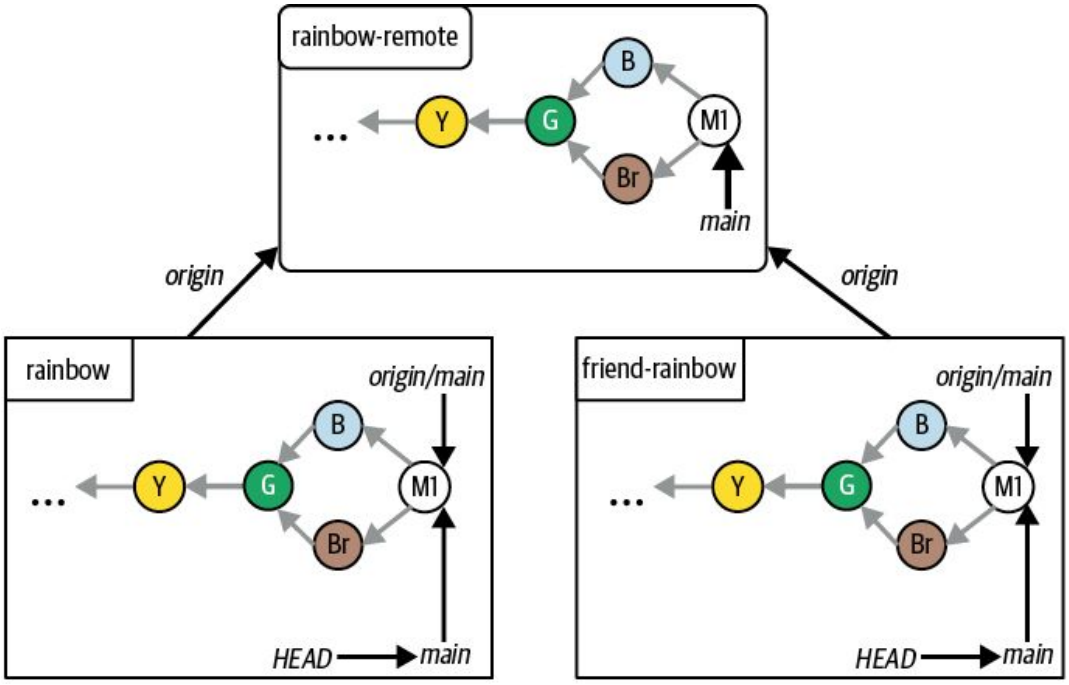
```
commit 225839938563c7458af81daca7beb782dfcbfb27 (HEAD -> main, origin/main)
Merge: 342bbfc 7f0a87a
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:05:46 2022 +0100
Merge remote-tracking branch 'refs/remotes/origin/main'
commit 342bbfc96bb03053f23ea7f7564ca207c58ceab2
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:00:56 2022 +0100
blue
```

O que deve ser observado:

- A primeira metade da saída de `git pull` na etapa 1 exibe os dados que estão sendo buscados ou baixados. A segunda metade da saída indica que um merge `Fast-forward` foi executado.
- A saída de `git log` na etapa 2 mostra que agora o commit blue e o commit de merge estão no repositório `rainbow` e o branch de rastreamento remoto `origin/main` e o branch `main` foram atualizados para apontar para o commit de merge.

Essas observações podem ser vistas na seção Visualize 9-14.

[VISUALIZE 9-14]



Projeto Rainbow depois que você fez o pull de alterações do repositório remoto para o branch main no repositório rainbow.

Agora todos os três repositórios do projeto Rainbow estão em sincronia.

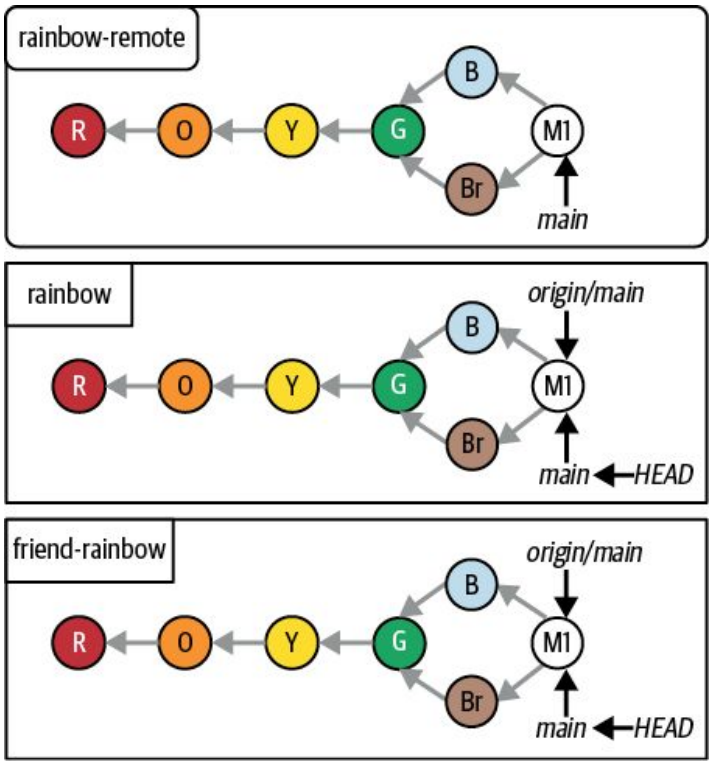
Estado dos repositórios locais e remoto

A seção Visualize 9-15 mostra o estado dos repositórios locais e remoto do projeto Rainbow com todos os commits que foram feitos do Capítulo 1 até o fim deste capítulo.

Resumo

Neste capítulo, você conheceu os merges three-way. Para ver como eles funcionam, executou alterações no projeto Rainbow que fizeram os históricos do branch local main do repositório friend-rainbow e do branch remoto main divergirem.

[VISUALIZE 9-15]



Projeto Rainbow no fim do Capítulo 9 com todos os commits desde o Capítulo 1.

No processo de criar esses históricos divergentes, você aprendeu a definir branches upstream e viu que após defini-los podia usar comandos como git push sem especificar nenhum argumento adicional.

Também aprendemos um detalhe importante sobre editar arquivos no diretório de trabalho várias vezes entre commits: sempre que fazemos uma alteração em um arquivo e o salvamos, o Git interpreta isso como uma nova versão do arquivo e o marca como modificado. Para incluir a versão mais recente do arquivo no próximo commit, é preciso adicionar essa versão à área de preparação.

Quando chegou a hora de executar o merge three-way, você conheceu o editor de texto de linha de comando Vim e aprendeu os aspectos básicos de como salvar as mensagens de commit fornecidas pelo Git e sair do Vim. No fim do processo de merge, vimos que o commit de merge associou os dois históricos de desenvolvimento convergentes.

Para concluir, você aprendeu o que é o processo de pulling e viu que ele nos permite executar ao mesmo tempo as duas etapas de incorporação das alterações de um repositório remoto em um repositório local.

O merge three-way executado neste capítulo não teve nenhum conflito de merge. No Capítulo 10, você conhecerá os conflitos de merge e aprenderá a resolvê-los.

10

Conflitos de merge

No Capítulo 9, você executou um merge three-way em que não houve conflitos de merge.

Neste capítulo, conheceremos os conflitos de merge e veremos como eles surgem e como resolvê-los percorrendo um exemplo prático de um merge three-way com um conflito de merge.

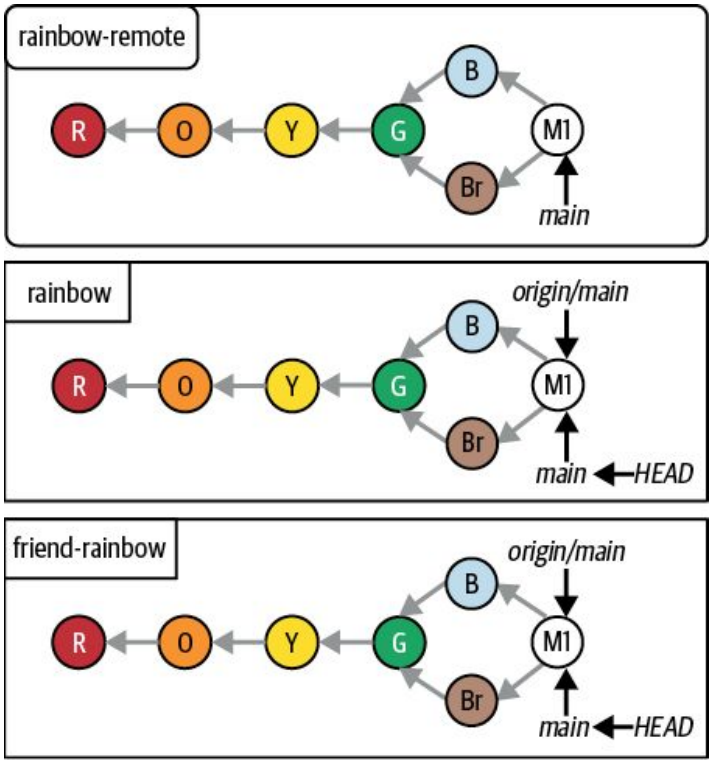
Estado dos repositórios locais e remoto

No começo deste capítulo, você deve ter dois repositórios locais chamados rainbow e friend-rainbow e um repositório remoto chamado rainbow-remote. Todos esses três repositórios devem estar em sincronia, com os mesmos commits e branches. Como sempre, recomendo que você use duas janelas de editor de texto e duas janelas de linha de comando separadas para trabalhar com o repositório rainbow e o repositório friend-rainbow.

A seção Visualize 10-1 mostra o estado dos repositórios locais e remoto do projeto Rainbow com todos os commits que foram feitos do Capítulo 1 ao Capítulo 9.

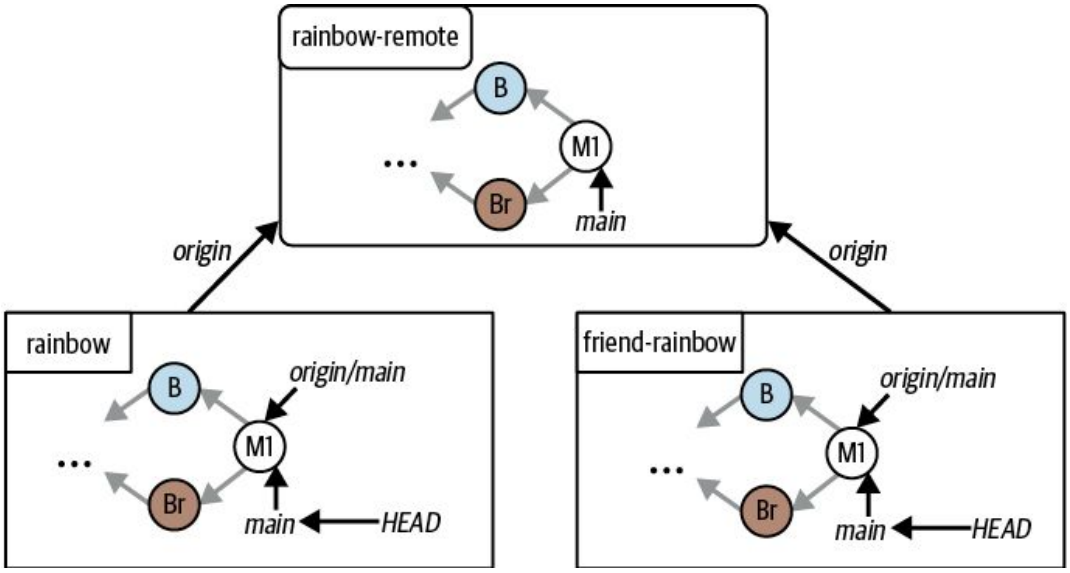
Para nos concentrarmos nos commits que você fará neste capítulo, de agora em diante simplificarei os diagramas da seção Visualize e só mostrarei os três últimos commits que fazem parte do branch main em todos os repositórios: o commit blue (azul), o commit brown (marrom) e o commit de merge M1. Essa representação é mostrada na seção Visualize 10-2.

[VISUALIZE 10-1]



Projeto Rainbow no começo do Capítulo 10 com todos os commits desde o Capítulo 1.

[VISUALIZE 10-2]



Representação simplificada do projeto Rainbow no começo do Capítulo 10, mostrando apenas os três últimos commits do branch main em todos os repositórios.

Introdução aos conflitos de merge

No Capítulo 9, examinamos o que acontece quando trabalhamos em um projeto ao mesmo tempo que outra pessoa e ambos fazem alterações em arquivos diferentes: você editou o arquivo othercolors.txt, enquanto seu amigo editou o arquivo rainbowcolors.txt. Isso permitiu que você fizesse um merge three-way sem nenhum conflito de merge.

Agora veremos o que acontece se quisermos integrar um trabalho em que haverá um conflito de merge. Como você aprendeu no capítulo anterior, os conflitos de merge surgem quando executamos o merge de dois branches em que diferentes alterações foram feitas nas mesmas partes do(s) mesmo(s) arquivo(s) ou se um arquivo que foi editado em um branch tiver sido excluído no outro branch. Nesses casos, o Git não consegue fazer o merge dos arquivos automaticamente e, portanto, temos de fazê-lo manualmente.

Os conflitos de merge podem surgir tanto durante o processo de merging quanto no processo de rebasing. Neste capítulo veremos um exemplo de conflito de merge no projeto Rainbow quando da execução de um merge, e no Capítulo 11 abordaremos um exemplo também no projeto Rainbow na

execução de um rebase.

É bom ressaltar que é comum surgirem conflitos de merge, e, se eles ocorrerem, isso não significa que alguém fez algo errado ao trabalhar em um projeto. A resolução de conflitos de merge é uma etapa corriqueira do trabalho em projetos Git. Examinaremos o Exemplo 10-1 do Projeto Book para ver uma demonstração de como uma situação assim pode ocorrer.

Exemplo 10-1 do Projeto Book

Imaginaremos dois cenários em que eu teria conflitos de merge em meu projeto Book. Como você sabe, o projeto é composto de 10 arquivos de texto, um para cada capítulo do livro, chamados chapter_one.txt, chapter_two.txt e assim por diante. Minha linha de desenvolvimento primária é o branch main, e decidi com meu editor e meu coautor que sempre que eu trabalhar em um capítulo criarei um branch fora do branch main; então, após eles aprovarem o trabalho que executei nesse branch, poderei fazer o merge dele com o branch main. Também combinei com meu coautor que só um de nós deve trabalhar em um capítulo em determinado momento porque, dessa forma, geralmente conseguimos evitar conflitos de merge.

No entanto, agora veremos o que aconteceria se ocorresse uma falha de comunicação acidental entre mim e meu coautor. Ambos decidimos editar o Capítulo 3 e criamos branches fora do branch main para trabalhar nesse capítulo ao mesmo tempo. Criei o branch chapter_three e meu coautor criou o branch chapter_three_coauthor. Nesse caso, editaremos o mesmo arquivo, chapter_three.txt.

Suponhamos que meu coautor integrasse primeiro o trabalho que ele fez no branch chapter_three_coauthor ao branch main. Quando eu integrar meu branch chapter_three à versão mais atualizada do branch main, descobrirei que, além de ter de executar um merge three-way, terei de resolver conflitos de merge.

Isso está ocorrendo porque tanto a versão do arquivo chapter_three.txt de meu branch chapter_three quanto a versão do arquivo no branch main foram editadas de maneiras diferentes, já que criei meu branch fora do branch main. O Git não pode fazer o merge do trabalho automaticamente. É preciso que eu decida qual será a aparência da versão final do arquivo chapter_three.txt que será incluída no commit.

Esse é um dos cenários em que posso ter conflitos de merge. Ele ocorre quando o mesmo arquivo é editado de maneiras diferentes nos dois branches envolvidos no merge.

Agora examinaremos outro cenário em que um conflito de merge pode surgir. Suponhamos que meu coautor e eu concordássemos que o último capítulo do livro, o Capítulo 10, não está bom. Contudo, houve novamente uma falha de comunicação sobre o que fazer com relação a isso.

Ambos criamos acidentalmente e ao mesmo tempo branches fora do branch main: criei o branch chapter_ten e meu coautor criou o chapter_ten_coauthor. Em meu branch chapter_ten, decidi excluir o arquivo chapter_ten.txt, simplesmente porque achei que deveríamos remover esse capítulo de nosso livro. No entanto, ao trabalhar em seu branch chapter_ten_coauthor, meu coautor decidiu editar o arquivo chapter_ten.txt para melhorá-lo.

Mais uma vez, meu coautor fez o merge de seu branch no branch main primeiro. E, quando integrei meu branch à versão mais atualizada do branch main por meio de um merge three-way, surgiu um novo conflito de merge. Esse é outro cenário em que o Git não pode fazer o merge do

trabalho automaticamente; ele não consegue determinar se deve excluir o arquivo chapter_ten.txt ou reter a versão editada. Recapitulando, essa situação ocorre na integração de dois branches quando em um branch um arquivo específico foi editado e no outro branch esse mesmo arquivo foi excluído.

O Exemplo 10-1 do Projeto Book descreveu dois cenários em que conflitos de merge podem surgir. Agora veremos como você pode resolvê-los.

Como resolver conflitos de merge

Quando ocorrerem conflitos de merge, você verá um conjunto de marcadores especiais em cada um dos arquivos envolvidos indicando onde o conflito ocorreu. Esses marcadores, chamados de *marcadores de conflito*, são compostos de sete colchetes angulares esquerdos (<<<<<<<), sete sinais de igualdade (=======) e sete colchetes angulares direitos (>>>>>>>), assim como de referências aos branches envolvidos no merge. A Figura 10-1 mostra um exemplo. Acima da linha de sinais de igualdade você verá o conteúdo do branch de destino e, abaixo dela, verá o conteúdo do branch de origem.

```
<<<<<<< HEAD
{Conteúdo do branch de destino}
=======
{Conteúdo do branch de origem}
>>>>>>>refs/remote/origin/main
```

Figura 10-1: Exemplo dos marcadores de conflito que aparecem quando há um conflito de merge.

Existem duas etapas para a resolução de conflitos de merge:

1. Decidir o que manter, editar o conteúdo e remover os marcadores de conflito.
2. Adicionar o(s) arquivo(s) que você editou à área de preparação e fazer o commit de suas alterações.

Abordaremos cada uma dessas etapas com detalhes quando você tentar resolver conflitos de merge no projeto Rainbow posteriormente neste capítulo. Para se preparar, é preciso criar uma situação com históricos de desenvolvimento divergentes em seus repositórios locais. Faremos isso a seguir.

Definindo um cenário de conflito de merge

Para definirmos um merge three-way com conflitos de merge, você e seu amigo têm de fazer alterações diferentes na mesma parte do mesmo arquivo. Primeiro consulte a seção Acompanhe 10-1 para adicionar indigo (anil) à lista de cores do arquivo rainbowcolors.txt em seu repositório rainbow e fazer o push do branch main atualizado para o repositório remoto.

[ACOMPANHE 10-1]

❶ No diretório de projeto rainbow em seu editor de texto, no arquivo rainbowcolors.txt, adicione “Indigo is the sixth color of the rainbow.” na linha 6. Em seguida, salve o arquivo.

❷ rainbow $ **git add rainbowcolors.txt**

❸ rainbow $ **git commit -m "indigo"**

```
[main 9b0a614] indigo
1 file changed, 2 insertions(+), 1 deletion(-)
```

❹ rainbow $ **git push**

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 326 bytes | 326.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:gitlearningjourney/rainbow-remote.git
2258399..9b0a614 main -> main
```

❺ rainbow $ **git log**

```
commit 9b0a61461c8e8d74ed358e65b2662e3697b94de6 (HEAD -> main, origin/main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:36:11 2022 +0100
indigo
commit 225839938563c7458af81daca7beb782dfcbfb27
Merge: 342bbfc 7f0a87a
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:05:46 2022 +0100
Merge remote-tracking branch 'refs/remotes/origin/main'
```

❻ Acesse o repositório rainbow-remote em seu serviço de hospedagem e atualize a página. Você deve ver o commit indigo.

O que deve ser observado:

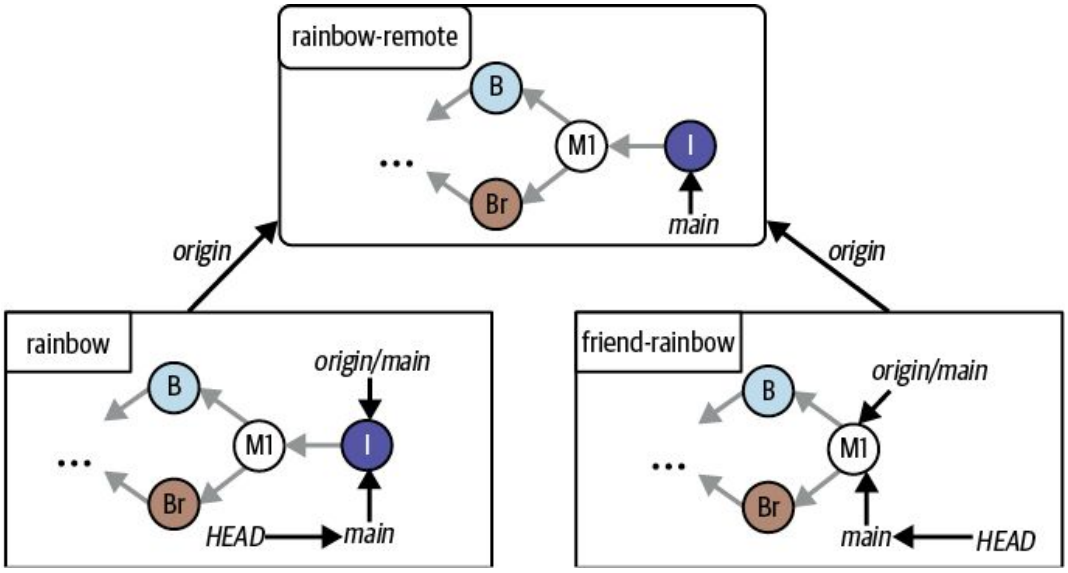
- Existe um commit indigo no repositório rainbow e no repositório rainbow-remote.

Essas alterações são mostradas na seção Visualize 10-3.

Para criar uma situação em que teremos um conflito de merge, na seção Acompanhe 10-2 seu amigo adicionará a cor violet (violeta) à mesma linha do arquivo rainbowcolors.txt e fará um commit sem primeiro executar o push das alterações que você fez do repositório rainbow-remote para seu branch local

main.

[VISUALIZE 10-3]



Projeto Rainbow depois que você fez o commit indigo em seu repositório rainbow e fez o push do branch main atualizado para o repositório remoto.

[ACOMPANHE 10-2]

❶ Em seu editor de arquivos, no arquivo rainbowcolors.txt do diretório de projeto friend-rainbow, adicione na linha 6 “Violet is the seventh color of the rainbow.”. Salve o arquivo.

❷ friend-rainbow $ **git add rainbowcolors.txt**

❸ friend-rainbow $ **git commit -m "violet"**

```
[main 6ad5c15] violet
1 file changed, 2 insertions(+), 1 deletion(-)
```

❹ friend-rainbow $ **git log**

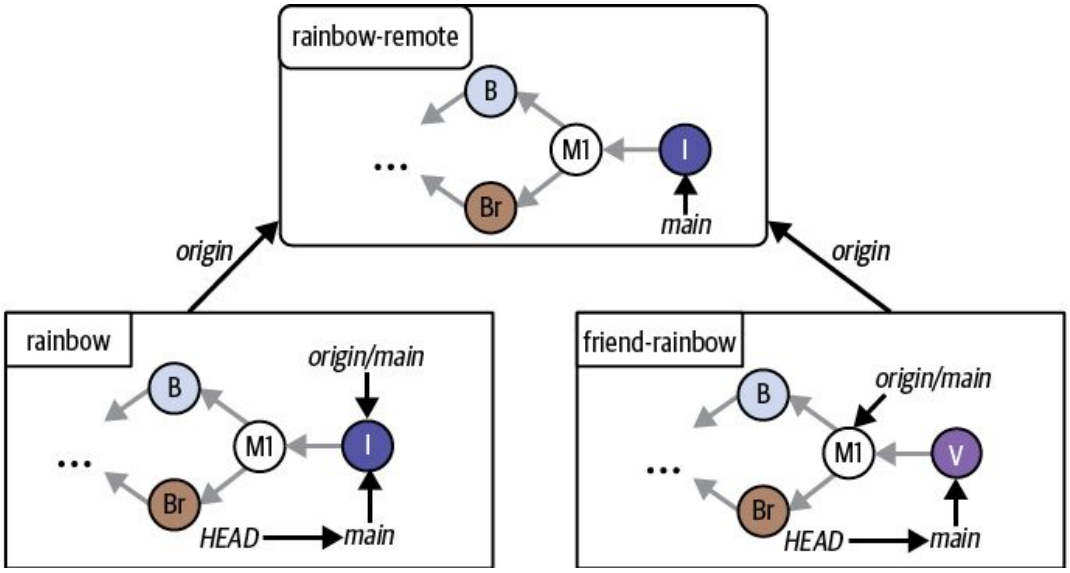
```
commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:41:25 2022 +0100
violet
commit 225839938563c7458af81daca7beb782dfcbfb27 (origin/main, origin/HEAD)
Merge: 342bbfc 7f0a87a
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 13:05:46 2022 +0100
Merge remote-tracking branch 'refs/remotes/origin/main'
```

O que deve ser observado:

- Seu amigo fez o commit violet.

Isso é mostrado na seção Visualize 10-4.

[VISUALIZE 10-4]



Projeto Rainbow depois que seu amigo fez o commit violet no repositório friend-rainbow sem antes fazer o pull do trabalho mais recente do branch main do repositório remoto.

O que deve ser observado:

- No repositório friend-rainbow, o branch local main aponta para o commit violet.
- No repositório rainbow, o branch local main aponta para o commit indigo.

Agora seu amigo terá de executar o fetch das alterações que você fez a partir do repositório remoto e integrá-las antes de fazer o push de suas alterações para o repositório remoto. Como você aprendeu em “Fazendo push para o repositório remoto”, na página 141, ele pode usar o comando git status após fazer o fetch de suas alterações para verificar se seu branch local main divergiu do branch remoto main. Consulte a seção Acompanhe 10-3 para ver isso na prática.

[ACOMPANHE 10-3]

❶ friend-rainbow $ **git fetch**

```
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0
Unpacking objects: 100% (3/3), 322 bytes | 53.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
c5941f8..6f2ea44 main -> origin/main
```

❷ friend-rainbow $ **git status**

```
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
(use "git pull" to merge the remote branch into yours)
nothing to commit, working tree clean
```

❸ friend-rainbow $ **git log --all**

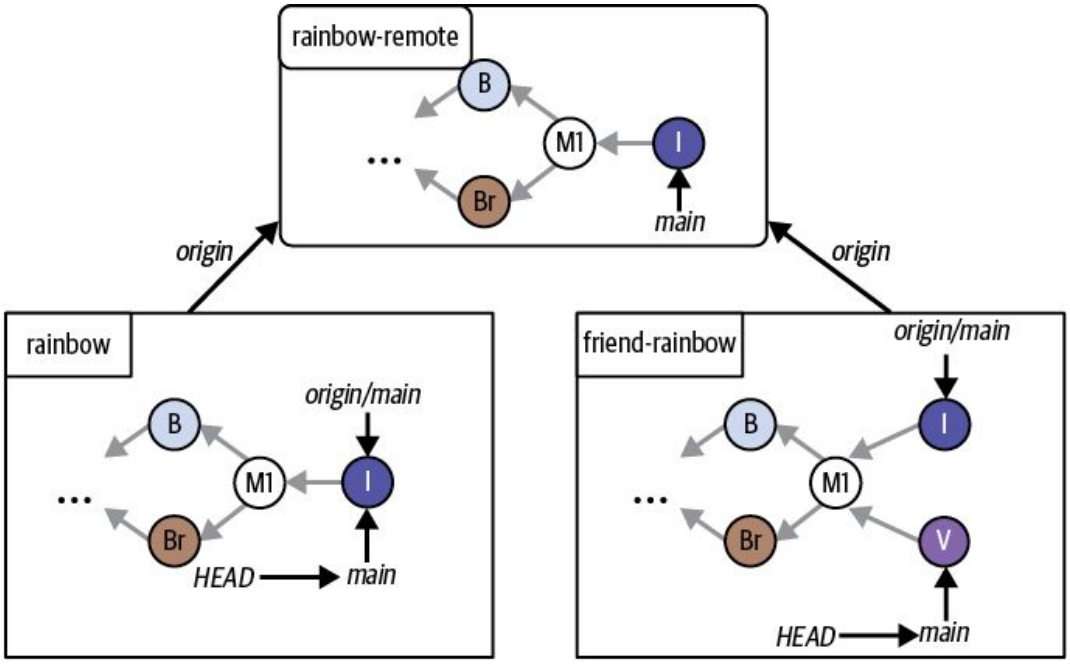
```
commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:41:25 2022 +0100
violet
commit 9b0a61461c8e8d74ed358e65b2662e3697b94de6 (origin/main, origin/HEAD)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:36:11 2022 +0100
indigo
```

O que deve ser observado:

- Na etapa 2, a saída do comando `git status` está informando que `Your branch and 'origin/main' have diverged, and have 1 and 1 different commits each, respectively` (Seu branch e ‘origin/main’ divergiram e cada um tem 1 commit diferente).
- No repositório `friend-rainbow`, o branch de rastreamento remoto `origin/main` aponta para o commit indigo; no entanto, as alterações remotas ainda não foram integradas ao branch local `main`.

Essas observações são mostradas na seção Visualize 10-5.

[VISUALIZE 10-5]



Projeto Rainbow depois que seu amigo fez o fetch das alterações do repositório remoto para o repositório friend-rainbow.

Agora seu amigo executará um merge three-way para integrar as alterações mais recentes do branch remoto main ao seu branch local main e poder fazer o push do branch main atualizado para o repositório remoto. Antes de executarmos o merge e resolvermos os conflitos no repositório friend-rainbow, veremos mais detalhadamente o que esse processo envolve.

Processo de solução de conflitos de merge

Como mencionado anteriormente neste capítulo, existem duas etapas para a solução de conflitos de merge. Nesta seção, vamos examiná-las com mais detalhes. Como lembrete, a primeira etapa é decidir o que manter, editar o conteúdo e remover os marcadores de conflito e a segunda é adicionar todas as alterações à área de preparação e fazer o commit. Veremos cada uma individualmente.

ETAPA 1

Após executarmos o comando git merge em uma situação em que surgirá um conflito de merge, o Git identificará o conflito e inserirá marcadores

indicando os locais do conteúdo conflitante. A primeira etapa da solução de conflitos de merge é decidir o que manter, editar o conteúdo do(s) arquivo(s) em que os conflitos ocorreram e remover os marcadores de conflito. Uma vez que você tiver feito as alterações necessárias, também deve se certificar de salvar o arquivo ou os arquivos em seu editor de texto. No projeto Rainbow, haverá um conflito de merge somente em um arquivo, mas em outros projetos Git em que você trabalhar poderão ocorrer conflitos em vários arquivos.

Para resolver o conflito de merge no Projeto Rainbow, seu amigo manterá tanto a frase sobre a cor indigo quanto a frase sobre a cor violet. Ele deseja se certificar de que as frases estejam na ordem correta; logo, inserirá a frase sobre a cor indigo primeiro (o que a fará ocupar a linha 6) e a frase sobre a cor violet depois (ou seja, na linha 7). Consulte a Figura 10-2 para ver um exemplo da aparência do arquivo rainbowcolors.txt antes e depois de seu amigo fazer essa alteração.

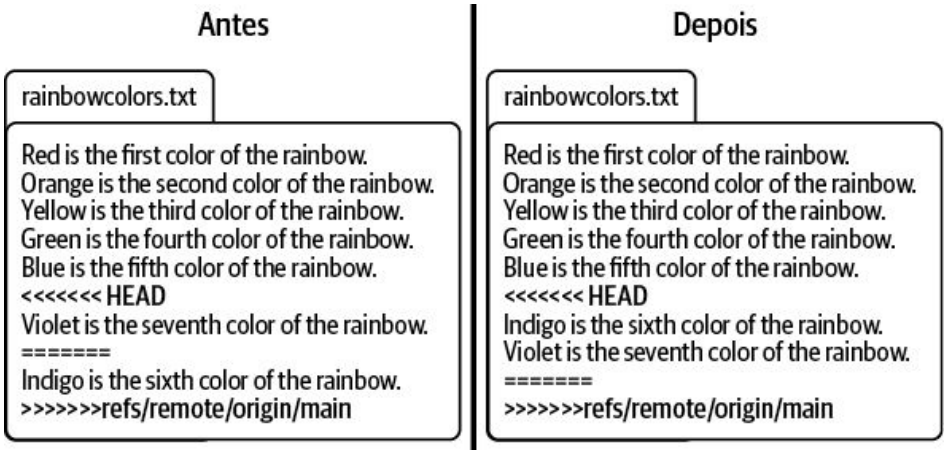


Figura 10-2: Arquivo rainbowcolors.txt antes e depois de seu amigo decidir o que deseja manter e editar o conteúdo.

Agora seu amigo terá de remover os marcadores de conflito do arquivo rainbowcolors.txt, como visto na Figura 10-3, e salvar o arquivo.

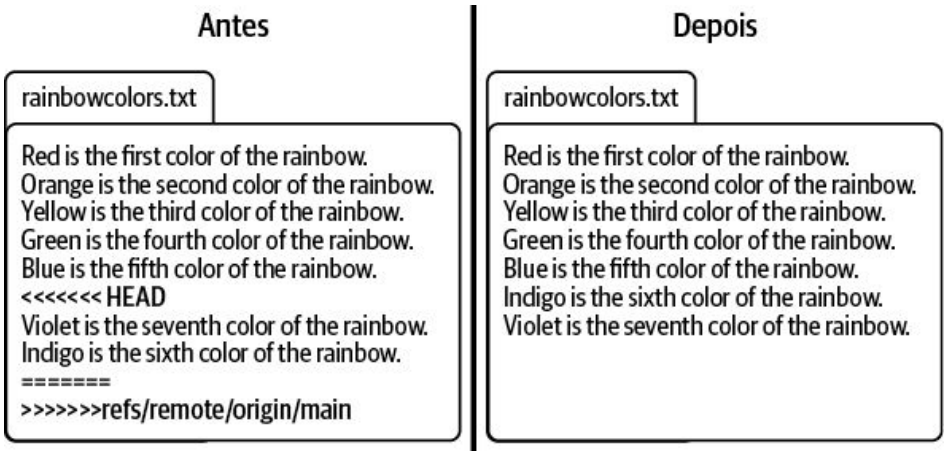


Figura 10-3: Arquivo rainbowcolors.txt antes e depois de seu amigo remover os marcadores de conflito.

ETAPA 2

Após seu amigo terminar de editar o conteúdo, ele estará pronto para a segunda etapa: adicionar o(s) arquivo(s) atualizado(s) à área de preparação e fazer um commit. No projeto Rainbow, existe somente um arquivo com conflito de merge; logo, ele terá de adicionar apenas um arquivo à área de preparação. No entanto, como já mencionado, é comum ocorrerem conflitos de merge em vários arquivos, caso em que você terá de adicionar todos os arquivos modificados à área de preparação.

Na Figura 10-4, podemos ver um exemplo da aparência do histórico de commits no repositório friend-rainbow antes e depois de seu amigo concluir o merge. Esse merge resultará no commit de merge M2, porque se trata de um merge three-way.

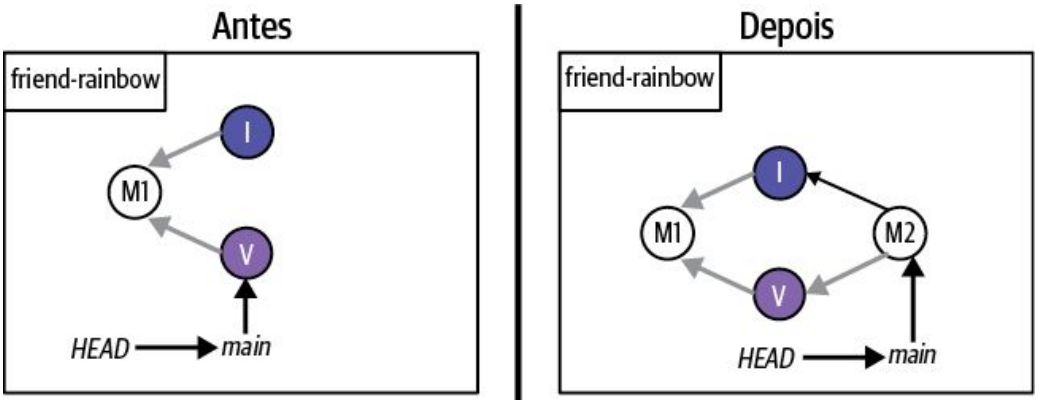


Figura 10-4: Histórico de commits antes e depois de seu amigo concluir o merge three-way.

Agora que você está familiarizado com as duas etapas envolvidas na

solução de conflitos de merge, examinaremos brevemente o que deve fazer se não quiser dar continuidade a um merge.

ABORTANDO UM MERGE

Se em algum momento durante um merge com conflitos você decidir que não deseja continuar integrando dois branches, pode optar por interromper ou "abortar" o merge usando o comando `git merge` com a opção `--abort`. Isso retornará todos os arquivos ao estado em que se encontravam antes do merge.

[SALVE O COMANDO]

git merge --abort

Interrompe o processo de merge e volta ao estado anterior.

Já que agora você sabe como resolver conflitos de merge e o que fazer se quiser abortar um merge e o processo de solução de conflito, veremos um exemplo de solução de conflitos de merge na prática.

Resolvendo conflitos de merge na prática

Na seção Acompanhe 10-4, seu amigo executará um merge three-way e resolverá os conflitos de merge. No decorrer da operação de merge, ele usará o comando git status para ver informações sobre o processo de solução de conflitos.

[ACOMPANHE 10-4]

❶ friend-rainbow $ **git merge origin/main**
Auto-merging rainbowcolors.txt
CONFLICT (content): Merge conflict in rainbowcolors.txt
Automatic merge failed; fix conflicts and then commit the result.

❷ friend-rainbow $ **git status**
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
(use "git pull" to merge the remote branch into yours)
You have unmerged paths.
(fix conflicts and run "git commit")
(use "git merge --abort" to abort the merge)
Unmerged paths:
(use "git add <file>..." to mark resolution)
both modified: rainbowcolors.txt
no changes added to commit (use "git add" and/or "git commit -a")

❸ Acesse o diretório de projeto friend-rainbow em seu editor de texto e examine o arquivo rainbowcolors.txt. Encontre os marcadores de conflito e o conteúdo conflitante.

❹ Execute a etapa 1 da solução de conflitos de merge, que é escolher o que manter, editar o conteúdo e remover os marcadores de conflito. Em nosso caso, você manterá todo o conteúdo, isto é, as alterações do commit violet e as do commit indigo. O comentário sobre a cor indigo entrará na linha 6 e o da cor violet entrará na linha 7. Certifique-se de salvar o arquivo rainbowcolors.txt quando terminar de fazer suas edições.

❺ friend-rainbow $ **git add rainbowcolors.txt**

❻ friend-rainbow $ **git status**
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
(use "git pull" to merge the remote branch into yours)
All conflicts fixed but you are still merging.
(use "git commit" to conclude merge)
Changes to be committed:
modified: rainbowcolors.txt

❼ friend-rainbow $ git commit -m “merge commit 2”
[main f10f972] merge commit 2

❽ friend-rainbow $ **git log**
commit f10f9725e3319af840a3d891ca8950436a219eb0 (**HEAD -> main**)
Merge: 6ad5c15 9b0a614
Author: annaskoulikari <gitlearningjourney@gmail.com>

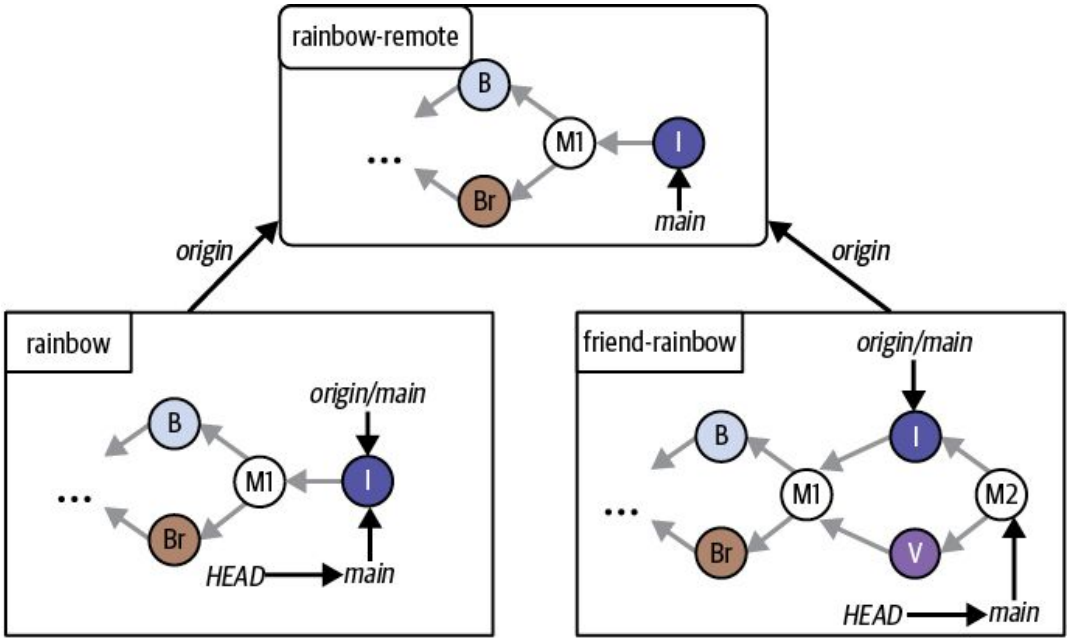
Date: Sun Feb 20 09:11:06 2022 +0100
merge commit 2
commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:41:25 2022 +0100
violet
commit 9b0a61461c8e8d74ed358e65b2662e3697b94de6 (**origin/main**, **origin/HEAD**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:36:11 2022 +0100
indigo

O que deve ser observado:

- Na etapa 8, a saída de git log mostra que:
  - O branch local main aponta para o commit de merge 2.
  - Os dois commits pais do commit de merge 2 são 6ad5c15 (o commit violet) e 9b0a614 (o commit indigo). Lembre-se de que os hashes de commit em seu repositório friend-rainbow serão diferentes dos deste livro porque os hashes são únicos.

Essas observações são mostradas na seção Visualize 10-6, em que o novo commit de merge é representado como M2.

[VISUALIZE 10-6]



Projeto Rainbow depois que seu amigo resolveu os conflitos de merge no repositório friend-rainbow e fez outro commit de merge.

Seu amigo executou o merge three-way com sucesso e resolveu os conflitos de merge. Agora discutiremos a importância de se manter atualizado com um repositório remoto.

Ficando atualizado com um repositório remoto

É mais demorado integrar alterações de um branch em outro quando existem conflitos de merge. No exemplo do projeto Rainbow, só havia um arquivo com um pequeno conflito de merge. Contudo, em projetos do mundo real pode haver um número bem maior de arquivos com conflitos de merge e estes podem ser muito mais difíceis de resolver. Ficar atualizado com as alterações feitas em branches relevantes em um repositório remoto é importante porque pode ajudar a limitar o número de possíveis conflitos de merge que você terá de resolver em um momento posterior.

Sempre que você criar um novo branch, é recomendável que o baseie na versão mais atualizada do branch remoto relevante no repositório remoto. Geralmente esse branch é o branch main (ou qualquer branch que sua equipe estiver usando como linha primária de desenvolvimento). No entanto,

dependendo do fluxo de trabalho da equipe no Git, pode ser outro branch.

Se você estiver trabalhando sozinho em um branch existente, é recomendável que atualize seu branch com as alterações feitas no branch remoto relevante do repositório remoto executando um merge. E caso se encontre em uma situação em que esteja trabalhando no mesmo branch com outra pessoa, deve sempre se certificar de fazer o fetch e o merge de qualquer alteração que tiver sido executada nesse branch no repositório remoto antes de continuar a trabalhar nele.

Após memorizar esse lembrete sobre permanecer atualizado, nos asseguraremos de que seu amigo faça o push do trabalho que ele executou em seu branch local `main` para o repositório remoto e que você sincronize seu branch local `main` no repositório `rainbow` com o branch remoto `main`.

Sincronizando os repositórios

Para todos os repositórios estarem em sincronia, seu amigo terá de fazer o push dos novos commits de seu branch local `main` para o repositório remoto e você terá de fazer o pull de todas as alterações para seu branch local `main` no repositório `rainbow`. Consulte a seção Acompanhe 10-5 para executar essas etapas agora.

[ACOMPANHE 10-5]

❶ friend-rainbow $ **git push**

Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 614 bytes | 614.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To github.com:gitlearningjourney/rainbow-remote.git
9b0a614..f10f972 main -> main

❷ friend-rainbow $ **git log**

commit f10f9725e3319af840a3d891ca8950436a219eb0 (**HEAD -> main**, **origin/main**, **origin/HEAD**)
Merge: 6ad5c15 9b0a614
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:11:06 2022 +0100
merge commit 2
commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:41:25 2022 +0100
violet

❸ Vá até a janela de linha de comando em que você está no repositório rainbow e execute o comando a seguir.

❹ rainbow $ **git pull**

remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (4/4), done.
Unpacking objects: 100% (6/6), 594 bytes | 99.00 KiB/s, done.
remote: Total 6 (delta 2), reused 6 (delta 2), pack-reused 0
From github.com:gitlearningjourney/rainbow-remote
9b0a614..f10f972 main -> origin/main
Updating 9b0a614..f10f972
Fast-forward
rainbowcolors.txt | 5 ++++-
1 file changed, 4 insertions(+), 1 deletion(-)

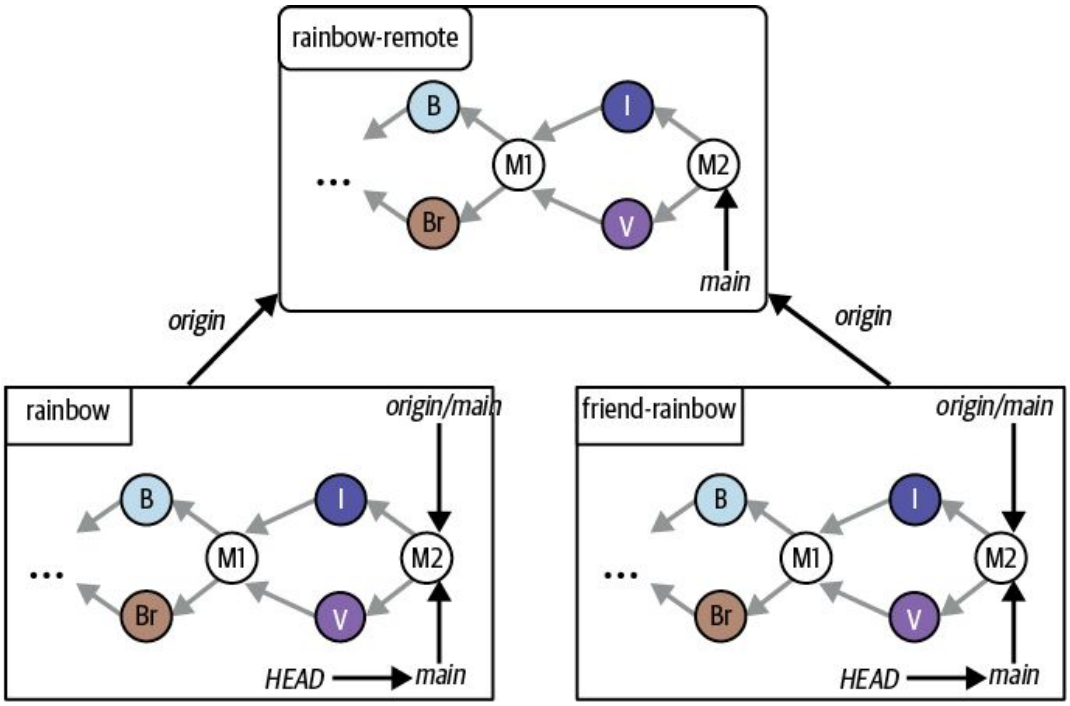
❺ rainbow $ **git log**

commit f10f9725e3319af840a3d891ca8950436a219eb0 (**HEAD -> main**, **origin/main**)
Merge: 6ad5c15 9b0a614
Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sun Feb 20 09:11:06 2022 +0100
merge commit 2
commit 6ad5c15f033b68ad27f2c9bce8bfa93329b3c23e
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 08:41:25 2022 +0100
violet

Agora todos os três repositórios do projeto Rainbow estão em sincronia, como mostrado na seção Visualize 10-7.

[VISUALIZE 10-7]



Projeto Rainbow depois que seu amigo fez o push de suas alterações para o repositório remoto e você fez o pull das alterações para seu repositório rainbow.

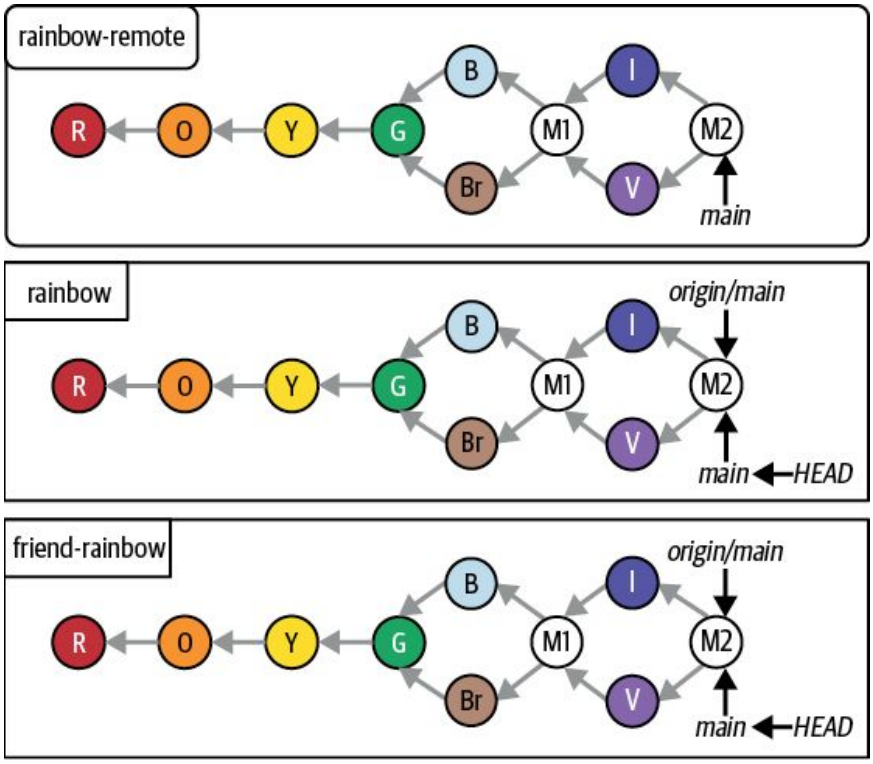
O que deve ser observado:

- No repositório rainbow-remote, o branch remoto main aponta para o commit de merge M2.
- No repositório rainbow, o branch local main e o branch de rastreamento remoto origin/main apontam para o commit de merge M2.
- No repositório friend-rainbow, o branch de rastreamento remoto origin/main foi atualizado para apontar para o commit de merge M2.

Estado dos repositórios locais e remoto

A seção Visualize 10-8 mostra o estado dos repositórios locais e remoto do projeto Rainbow com todos os que foram feitos do Capítulo 1 até o fim deste capítulo.

[VISUALIZE 10-8]



Projeto Rainbow no fim do Capítulo 10 com todos os commits desde o Capítulo 1.

Resumo

Neste capítulo, você conheceu os conflitos de merge, que surgem quando fazemos o merge de dois branches nos quais diferentes alterações foram executadas nas mesmas partes do(s) mesmo(s) arquivo(s) ou quando um arquivo que foi editado em um branch é excluído no outro. Também conheceu as duas etapas envolvidas na solução dos conflitos: decidir que conteúdo manter, remover os marcadores de conflito e salvar os arquivos e, em seguida, adicionar os arquivos editados à área de preparação e fazer um commit.

Conflitos de merge podem surgir durante um merge three-way ou durante o processo de rebasing. Até agora nos concentramos no processo de merging; no Capítulo 11, você conhecerá o rebasing e percorrerá um exemplo envolvendo o projeto Rainbow.

11

Rebasing

Como você sabe, o Git oferece duas maneiras principais para integrarmos as alterações de um branch em outro: merging e rebasing. Nos capítulos 5 e 9, conhecemos os merges fast-forward e three-way, respectivamente. No Capítulo 10, examinamos os conflitos de merge e vimos que eles podem surgir tanto durante um merge three-way quanto durante um rebase.

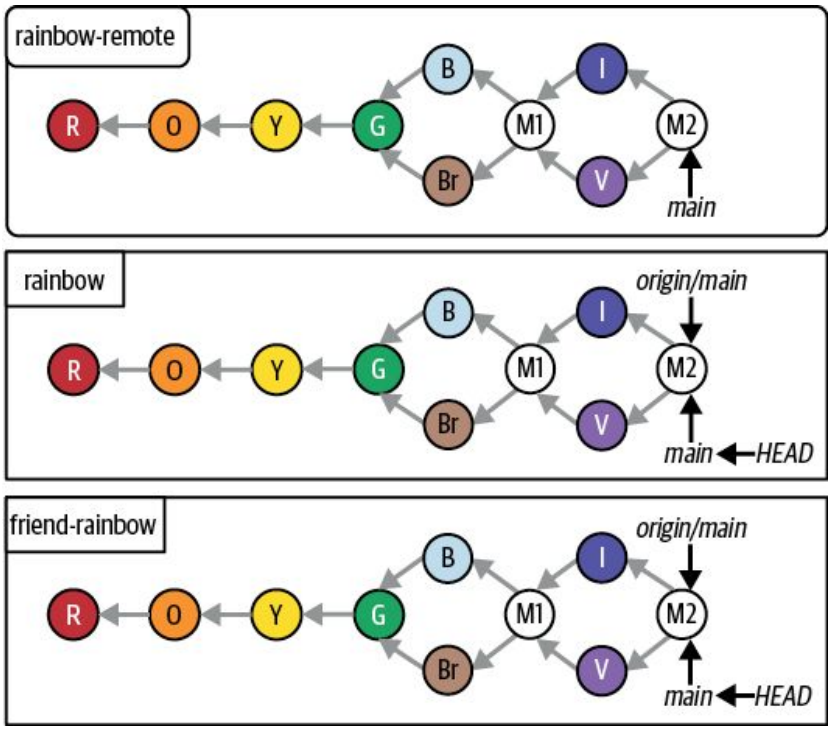
Neste capítulo, você conhecerá o rebasing e acompanhará um exemplo. Introduzirei os cinco estágios do processo de rebasing e mostrarei como resolver conflitos de merge durante o processo. Veremos em que o merging e o rebasing diferem e em quais casos pode ser preferível executar o rebase em vez do merge. Além disso, introduzirei a regra de ouro do rebasing, que o ajudará a determinar quando não o executar. Você também conhecerá mais alguns detalhes sobre a área de preparação e aprenderá a usá-la como espaço para organizar o que incluirá em seu próximo commit.

Estado dos repositórios locais e remoto

No começo deste capítulo, você deve ter dois repositórios locais chamados `rainbow` e `friend-rainbow` e um repositório remoto chamado `rainbow-remote`. Todos esses três repositórios devem conter os mesmos commits e branches. Como sempre, ao trabalhar nos exemplos do capítulo, recomendo que você use duas janelas de editor de texto e duas janelas de linha de comando separadas para o repositório `rainbow` e o repositório `friend-rainbow`.

A seção Visualize 11-1 mostra o estado dos repositórios locais e remoto no projeto Rainbow com todos os commits que foram feitos do Capítulo 1 ao Capítulo 10.

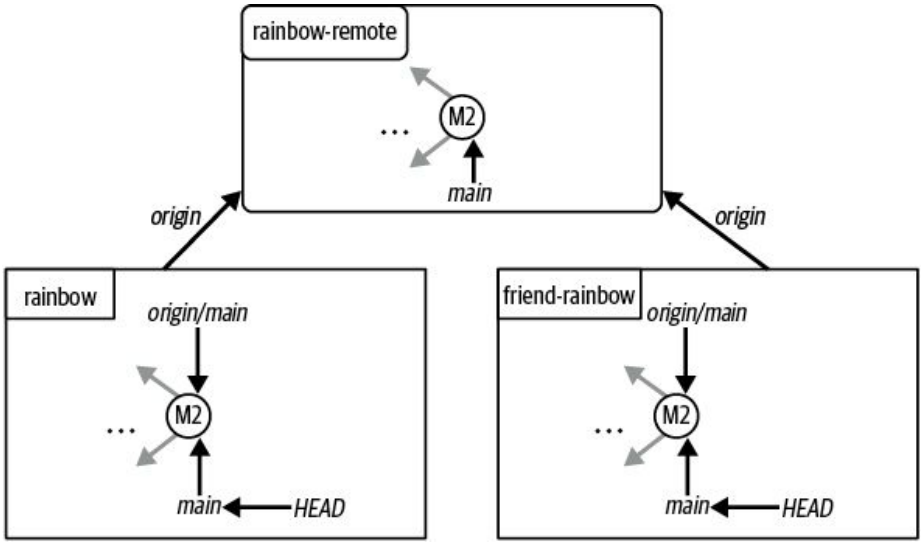
[VISUALIZE 11-1]



Projeto Rainbow no começo do Capítulo 11 com todos os commits desde o Capítulo 1.

Para nos concentrarmos nos commits que você fará neste capítulo, de agora em diante simplificarei os diagramas da seção Visualize e só mostrarei o último commit que foi feito no branch main em todos os repositórios, que é o commit de merge M2. Essa representação é mostrada na seção Visualize 11-2.

[VISUALIZE 11-2]



Representação simplificada do projeto Rainbow no começo do Capítulo 11, mostrando apenas o último commit do branch main em todos os repositórios.

Integrando alterações no Git

Até agora, nos concentramos no merging como uma maneira de integrar alterações no Git. Você viu na prática que um merge fast-forward apenas move o ponteiro do branch de destino para apontar para o commit mais recente, enquanto um merge three-way cria um commit de merge que associa os históricos de desenvolvimento do branch de origem e do branch de destino (e às vezes também leva a conflitos de merge).

Se olharmos o estado dos diversos repositórios na seção Visualize 11-1, veremos que até o commit green (verde) tínhamos um histórico de projeto linear, mas depois do commit green o histórico de commits é não linear devido aos commits de merge criados pelos merges three-way. Algumas equipes e pessoas preferem manter um histórico de projeto linear porque o acham mais organizado e simples. Você pode usar o processo de rebasing para evitar merges three-way e commits de merge e manter um histórico de projeto linear.

O rebasing pega todo o trabalho que fizemos nos commits de um branch e o reaplica a outro branch, criando commits totalmente novos. Isso faz parecer que criamos um branch a partir de um commit totalmente diferente do commit original a partir do qual o criamos.

Para executar um rebase, você precisa estar no branch para o qual deseja executar o processo. Use o comando `git rebase` e passe o nome do branch em que deseja fazer o rebase.

[SALVE O COMANDO]

git rebase <nome_branch>
Reaplica os commits a outro branch.

Já que o rebasing cria commits totalmente novos, isso significa que ele altera o histórico de commits. É importante tomar cuidado ao usar operações do Git que façam isso. Discutiremos situações em que não é recomendável fazer o rebase de um branch em “Regra de ouro do Rebasing”, na página 230. Primeiro, no entanto, mostrarei como o rebasing pode ser usado em um projeto Git e veremos um exemplo na prática envolvendo o projeto Rainbow. Começaremos examinando por que você pode querer adotar essa abordagem.

Por que o rebasing é útil?

Para mostrar como o rebasing pode ajudá-lo a evitar merges three-way e a manter um histórico de projeto linear, revisitaremos a demonstração introduzida no Exemplo 9-1 do Projeto Book. Vejamos então o Exemplo 11-1 do Projeto Book.

Exemplo 11-1 do Projeto Book

Usando o exemplo de cenário do Capítulo 9, suponhamos que tanto meu coautor quanto eu criássemos um branch diferente baseado no branch main quando ele está apontando para o commit B, a fim de trabalhar em capítulos separados. Criei o branch chapter_five e meu coautor criou o branch chapter_six, como mostrado na Figura 11-1.

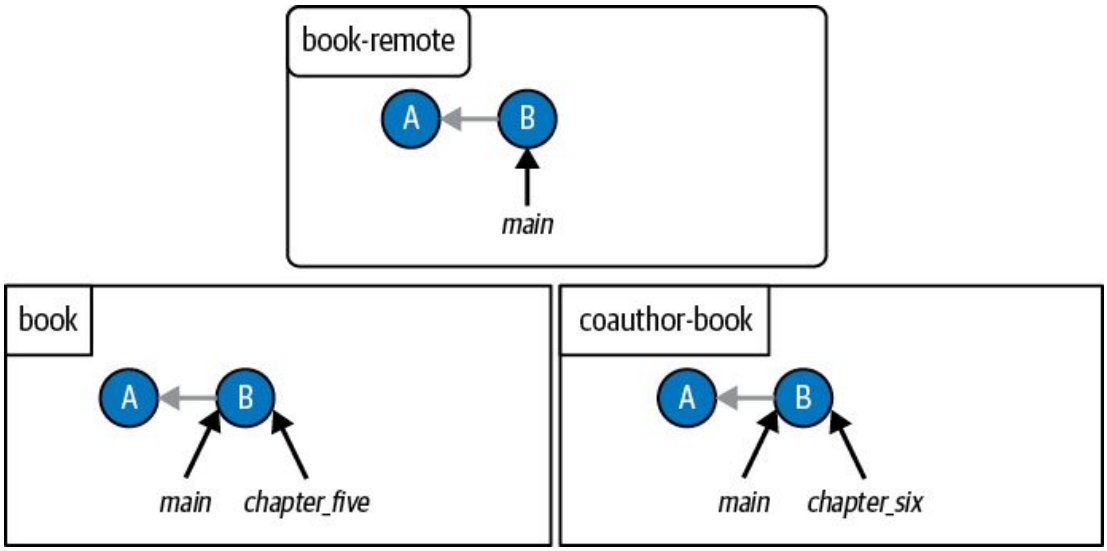


Figura 11-1: Projeto Book quando tanto meu coautor quanto eu criamos branches para trabalhar

em capítulos diferentes.

Meu coautor trabalhou no Capítulo 6 do livro e fez o commit C. Ele fez o merge de seu trabalho no branch main de seu repositório local e fez o push do branch main atualizado para o repositório remoto. Ao mesmo tempo, trabalhei no Capítulo 5 e fiz o commit D em meu repositório local. Contudo, ainda não fiz o merge de meu trabalho ou o compartilhei. Esses desenvolvimentos são mostrados na Figura 11-2.

Quando eu decidir que as alterações que fiz no Capítulo 5 estão prontas para serem integradas ao meu branch local main, terei duas opções. Uma é fazer o fetch das alterações do branch remoto main e executar um merge three-way para integrar chapter_five a main, o que acabará produzindo um commit de merge (representado como commit M na Figura 11-3). Essa é a opção apresentada no Capítulo 9.

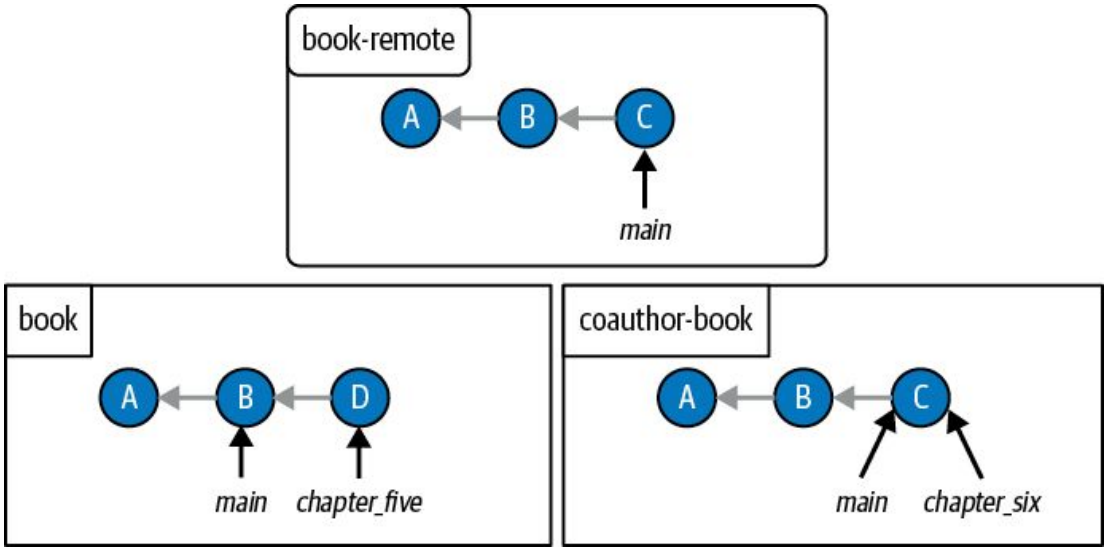


Figura 11-2: Projeto book depois que meu coautor fez o push de suas alterações para o repositório remoto.

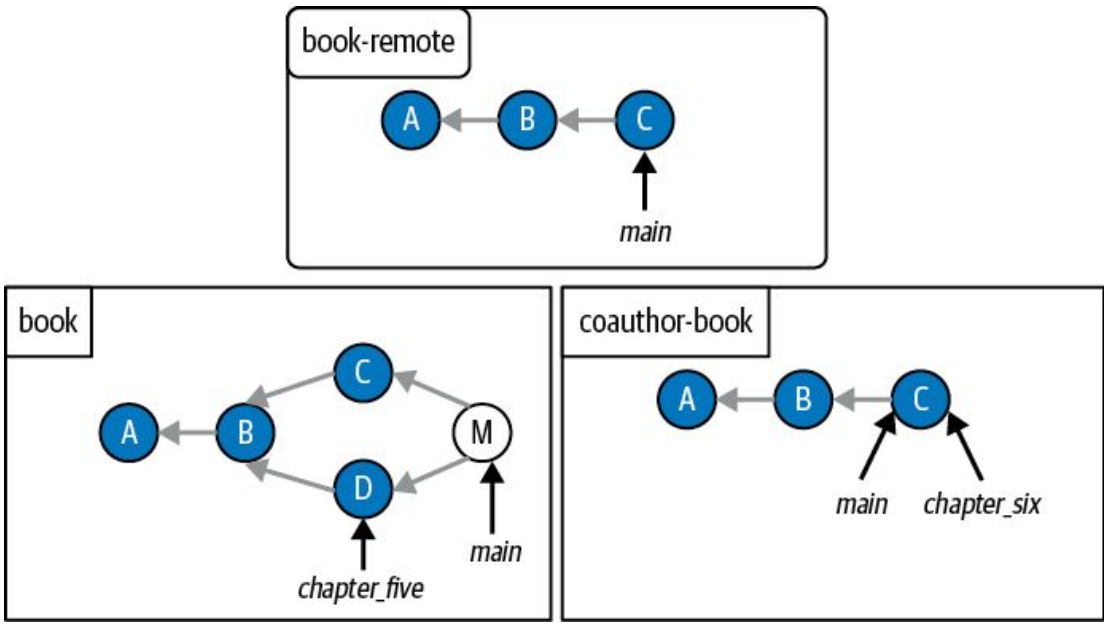


Figura 11-3: Projeto Book se eu integrasse as alterações do repositório remoto usando um merge three-way.

A outra opção é fazer o pull das alterações do repositório remoto, assim meu branch main será atualizado para apontar para o commit C e poderei fazer o rebase em um momento posterior. A Figura 11-4 mostra o estado do projeto Book depois que fiz o pull das alterações do branch remoto main.

Agora posso fazer o rebase de meu branch chapter_five acima do branch main atualizado. Como mencionado anteriormente, o rebasing pegará todos os commits que fiz no branch chapter_five e os reaplicará ao branch main, criando commits totalmente novos. Nesse exemplo, o único commit que fiz no branch chapter_five desde que ele divergiu do branch main foi o commit D. Quando for feito o rebase no branch, um novo commit D' será criado, como visto na Figura 11-5. O apóstrofo (') no identificador indica que esse é um commit de rebase (rebased commit).

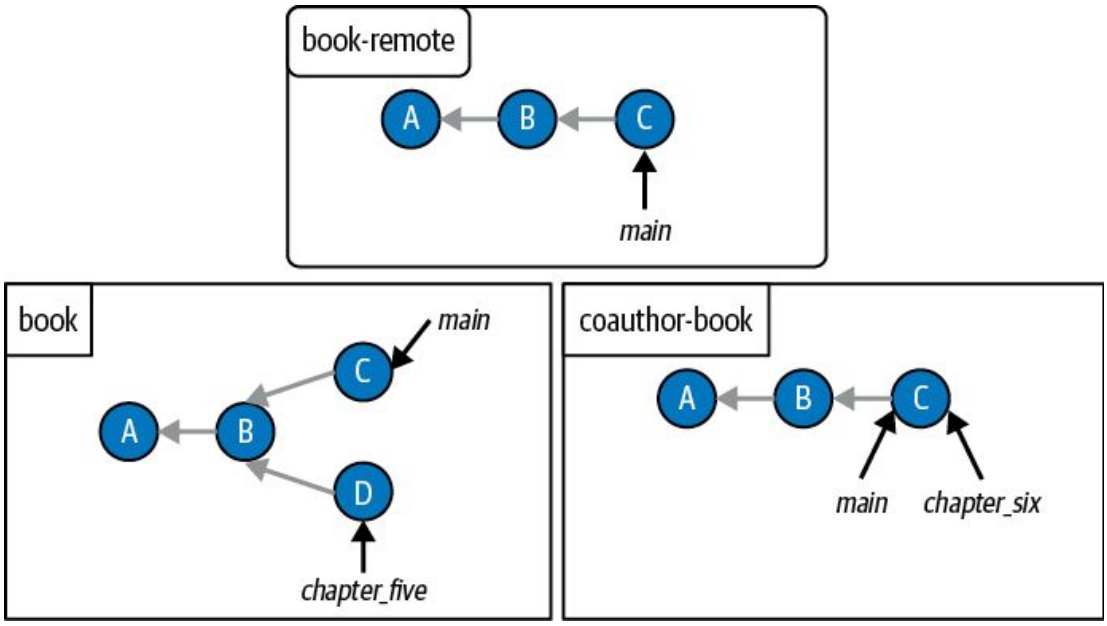


Figura 11-4: Projeto Book depois que fiz o pull das alterações do branch remoto main.

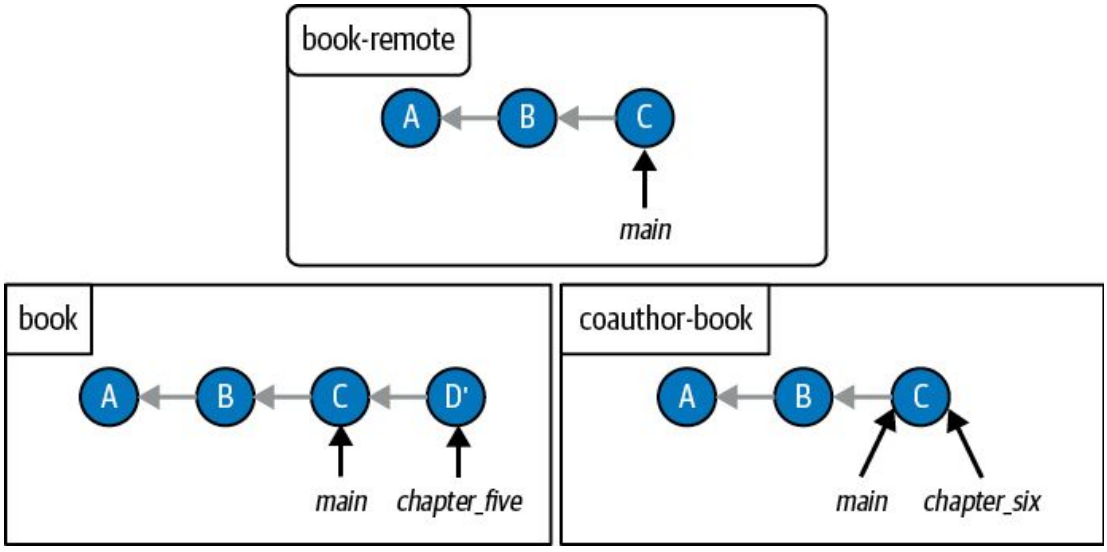


Figura 11-5: Projeto Book depois que integrei as alterações do repositório remoto fazendo o rebase de meu branch chapter_five no branch main.

O commit D' representa a nova versão do commit D que foi criada durante o processo de rebasing. Ela inclui todas as alterações que fiz no commit D original, mas é um commit totalmente novo com outro hash de commit.

Após fazer o rebase de meu branch chapter_five, posso integrá-lo a main com um simples merge fast-forward, como visto na Figura 11-6.

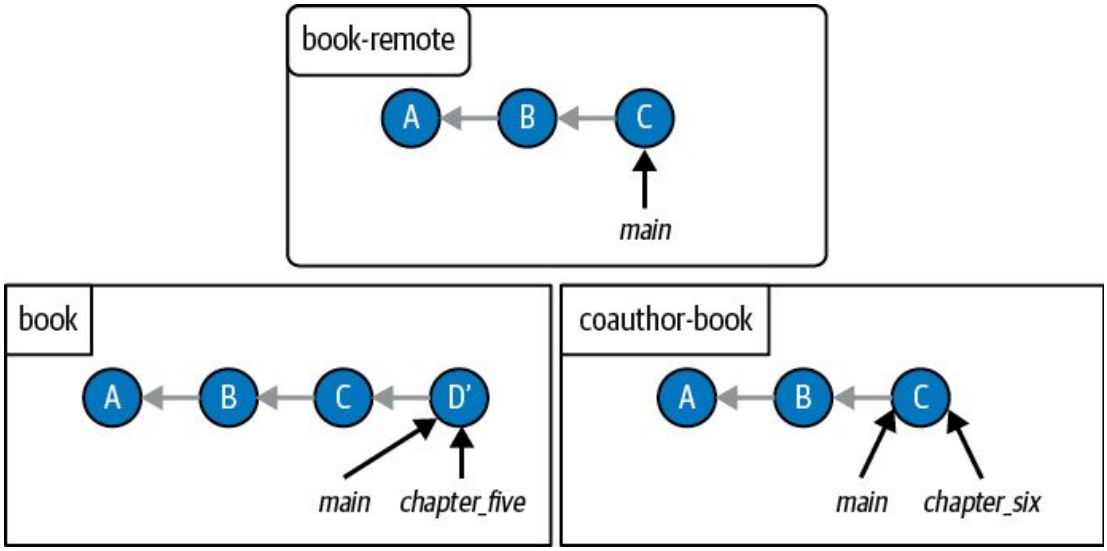


Figura 11-6: Projeto Book depois que integrei o branch chapter_five ao branch main com um simples merge fast-forward.

Executando o rebase de meu branch posso manter um histórico de commits linear e evitar fazer commits de merge adicionais.

É bom ressaltar que o commit D original ainda existe no histórico de commits; no entanto, ele não faz mais parte de nenhum dos branches do repositório book e é por isso que não é mostrado na Figura 11-6. O rebasing de um branch não exclui os commits desse branch, apenas os recria. Os commits antigos ainda existem no histórico de commits.

Agora que você viu um exemplo hipotético de como o rebasing pode ser usado para mantermos um histórico de projeto linear, criaremos uma situação no projeto Rainbow em que você terá históricos de desenvolvimento divergentes entre dois branches para poder percorrer um caso na prática.

Definindo o exemplo de rebasing

Para treinar a execução de um rebase, você terá de criar históricos divergentes. Faremos um commit no repositório rainbow e o enviaremos para o repositório remoto. Em seguida, sem fazer o fetch das alterações do

repositório remoto, seu amigo fará dois commits em seu repositório friend-rainbow. Após fazer os commits, ele fará o fetch das alterações do repositório remoto e executará o rebase de seu branch.

Consulte a seção Acompanhe 11-1 para fazer um commit no repositório rainbow.

[ACOMPANHE 11-1]

❶ Abra o arquivo othercolors.txt do diretório de projeto rainbow em uma janela de editor de texto. Adicione “Gray is not a color in the rainbow.” na linha 2 e salve o arquivo.

❷ rainbow $ **git add othercolors.txt**

❸ rainbow $ **git commit -m "gray"**

```
[main 6f2cf36] gray
1 file changed, 2 insertions(+), 1 deletion(-)
```

❹ rainbow $ **git push**

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 327 bytes | 327.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:gitlearningjourney/rainbow-remote.git
f10f972..6f2cf36 main -> main
```

❺ rainbow $ **git log**

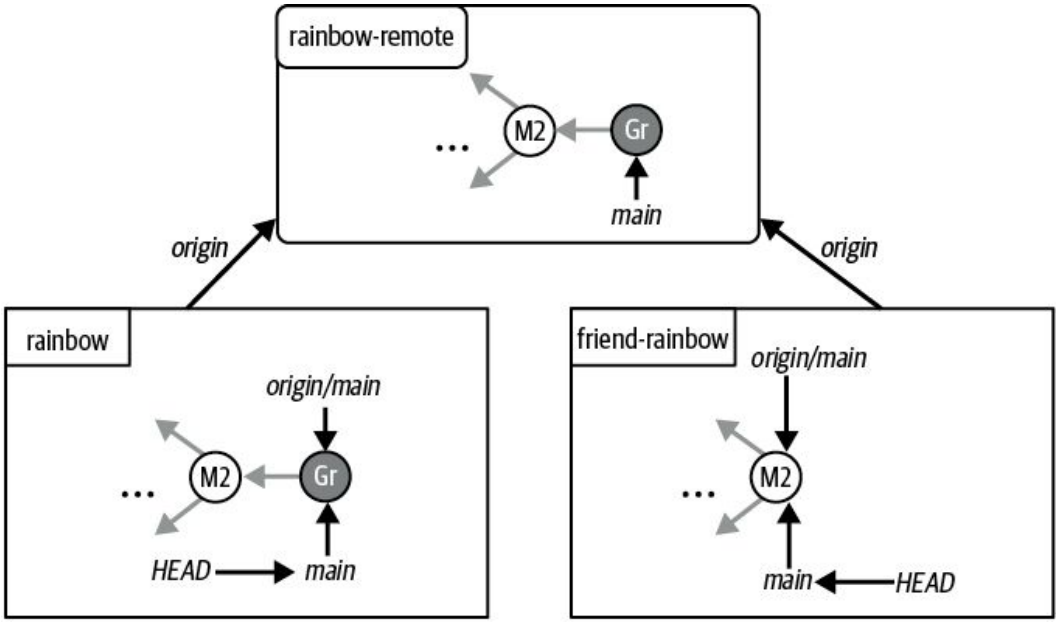
```
commit 6f2cf3698e6bf9078e8e0340ec9948f590405091 (HEAD -> main, origin/main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:27:58 2022 +0100
gray
commit f10f9725e3319af840a3d891ca8950436a219eb0
Merge: 6ad5c15 9b0a614
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:11:06 2022 +0100
merge commit 2
```

O que deve ser observado:

- Você fez o commit gray (cinza) e o enviou para o repositório remoto.

Isso é mostrado na seção Visualize 11-3.

[VISUALIZE 11-3]



Projeto Rainbow depois que você fez o commit gray no branch local main do repositório rainbow e o enviou para o repositório remoto.

Agora seu amigo adicionará alterações ao mesmo branch do repositório friend-rainbow, o que levará a históricos divergentes entre o branch main dele e o seu. Essa também será uma boa oportunidade para a introdução de alguns recursos úteis da área de preparação.

Preparando arquivos e removendo-os da preparação

Nesta seção, seu amigo fará alterações nos dois arquivos do repositório friend-rainbow e os adicionará à área de preparação. No entanto, em seguida ele decidirá que deseja fazer dois commits separados para os diferentes tipos de trabalho. Logo, terá de *remover um arquivo da área de preparação* (unstage a file).

Examinaremos o Exemplo 11-2 do Projeto Book, em que revisitarei o cenário visto no Exemplo 3-2, para saber por que podemos ter de remover um arquivo da preparação.

Exemplo 11-2 do Projeto Book

Suponhamos que eu estivesse trabalhando nos capítulos 1, 2 e 3 de meu livro, editando os

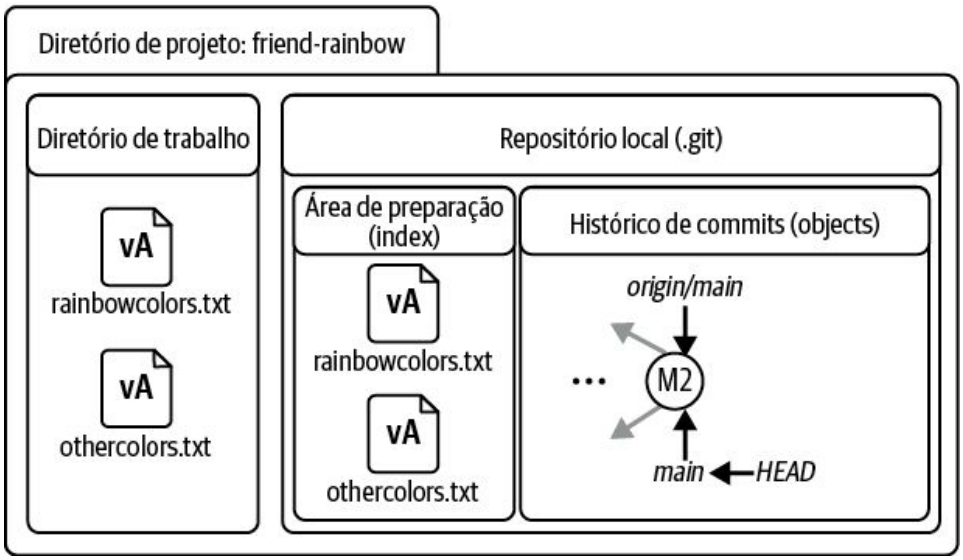
arquivos correspondentes a esses capítulos: chapter_one.txt, chapter_two.txt e chapter_three.txt. Adicionei todos os arquivos de capítulos à área de preparação, o que significa que as alterações que fiz nos capítulos 1, 2 e 3 serão salvas em meu próximo commit. É esperado que o editor revise todas as alterações.

Antes de fazer o commit, entretanto, percebi que só as alterações do Capítulo 2 estão realmente prontas para ser revisadas. Em outras palavras, quero que apenas as alterações que fiz em chapter_two.txt sejam incluídas em meu próximo commit. Já que a área de preparação é um espaço prévio em que tenho liberdade para adicionar e remover arquivos modificados e decidir o que será incluído no próximo commit, posso remover os arquivos modificados chapter_two.txt e chapter_three.txt dessa área para que só as alterações do arquivo modificado chapter_two.txt sejam incluídas no próximo commit. Isso significa que meu editor só revisará essas alterações.

Agora que você entendeu por que é importante saber como remover um arquivo da área de preparação, está pronto para remover um arquivo dessa área no projeto Rainbow.

Para examinar o que está ocorrendo no próximo exemplo, nos concentraremos no repositório friend-rainbow nos diagramas da seção Visualize, usando o Diagrama Git introduzido no Capítulo 2. O estado atual do repositório friend-rainbow é mostrado na seção Visualize 11-4.

[VISUALIZE 11-4]



Diretório de projeto friend-rainbow antes de seu amigo editar arquivos.

O que deve ser observado:

- As versões atuais dos arquivos rainbowcolors.txt e othercolors.txt que se encontram no diretório de trabalho e na área de preparação estão sendo representadas como versão A (vA).

Na seção Acompanhe 11-2, seu amigo fará algumas alterações nos arquivos.

[ACOMPANHE 11-2]

❶ No diretório de projeto friend-rainbow em seu editor de texto:

No arquivo othercolors.txt, adicione “Black is not a color in the rainbow.” na linha 2 e salve o arquivo.

No arquivo rainbowcolors.txt, adicione “These are the colors of the rainbow.” na linha 8 e salve o arquivo.

❷ friend-rainbow $ **git status**

On branch main

Your branch is up to date with 'origin/main'.

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: othercolors.txt

modified: rainbowcolors.txt

no changes added to commit (use "git add" and/or "git commit -a")

O que deve ser observado:

- Seu amigo editou tanto o arquivo rainbowcolors.txt quanto o arquivo othercolors.txt e eles estão sendo listados como arquivos modificados que não foram adicionados à área de preparação.

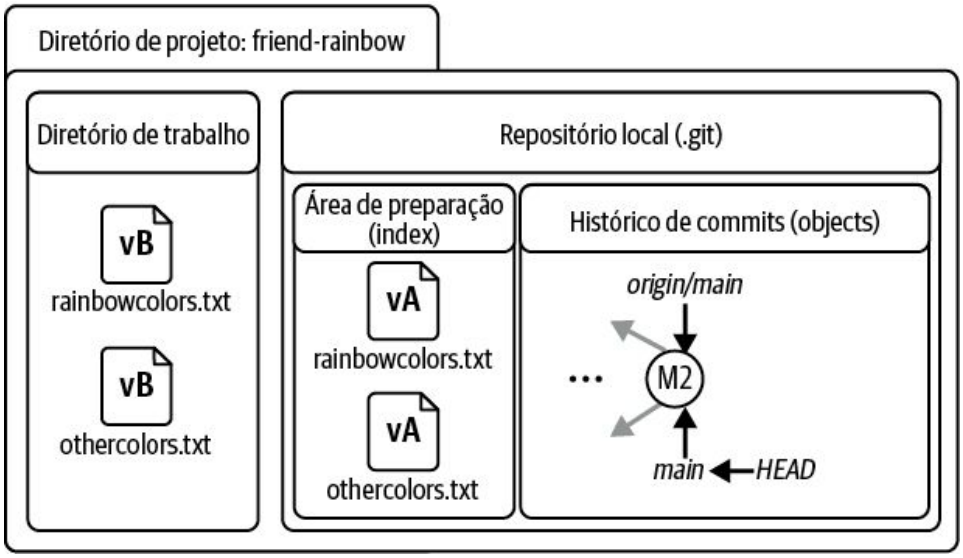
Isso é mostrado na seção Visualize 11-5.

O que deve ser observado:

- As versões modificadas dos arquivos rainbowcolors.txt e othercolors.txt que seu amigo editou estão representadas como versão B (vB).
- As versões dos arquivos othercolors.txt e rainbowcolors.txt na área de preparação (vA) são diferentes das versões do diretório de trabalho (vB).

Na seção Acompanhe 11-3, seu amigo adicionará os arquivos atualizados à área de preparação para incluí-los no próximo commit.

[VISUALIZE 11-5]



Diretório de projeto friend-rainbow depois que seu amigo editou os arquivos rainbowcolors.txt e othercolors.txt no diretório de trabalho.

[ACOMPANHE 11-3]

❶ friend-rainbow $ **git add rainbowcolors.txt othercolors.txt**

❷ friend-rainbow $ **git status**

On branch main

Your branch is up to date with 'origin/main'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

modified: othercolors.txt

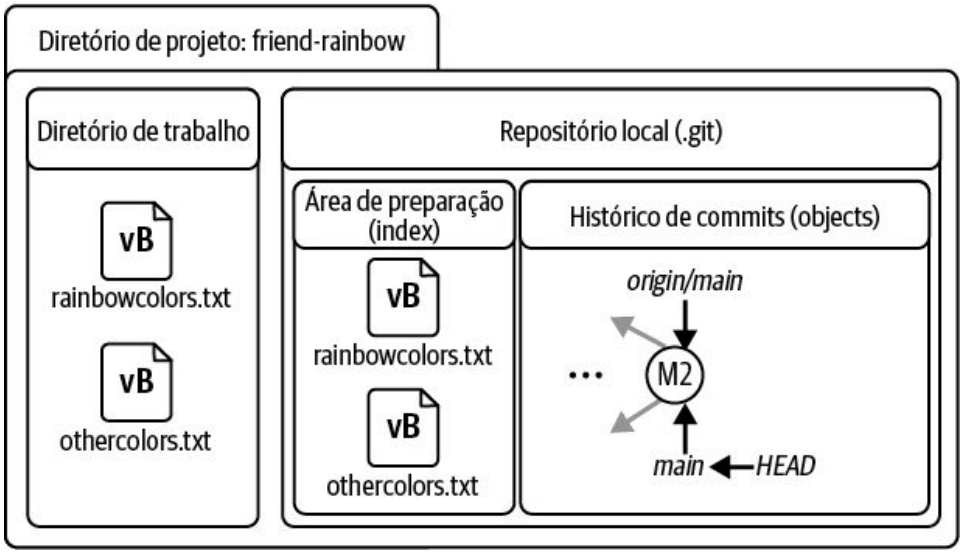
modified: rainbowcolors.txt

O que deve ser observado:

- Seu amigo adicionou os arquivos modificados rainbowcolors.txt e othercolors.txt à área de preparação.

Isso é mostrado na seção Visualize 11-6.

[VISUALIZE 11-6]



Diretório de projeto friend-rainbow depois que seu amigo adicionou as versões atualizadas dos arquivos rainbowcolors.txt e othercolors.txt à área de preparação.

O que deve ser observado:

- As versões dos arquivos othercolors.txt e rainbowcolors.txt na área de preparação mudaram de vA para vB.

Agora suponhamos que seu amigo decidisse fazer commits separados para o trabalho de adicionar a cor black (preta) à lista de cores que não fazem parte do arco-íris e o de fazer comentários sobre o arco-íris.

Em “Introdução à área de preparação”, na página 45, mencionei que a área de preparação é como um espaço de rascunho em que podemos adicionar e remover arquivos para definir o que será incluído no próximo commit. Até agora, usamos o comando git add para adicionar arquivos à área de preparação. Veremos então como remover um arquivo modificado da área de preparação ou, em outras palavras, como alterar a versão do arquivo na área de preparação. Na saída do comando git status na seção Acompanhe 11-3, o Git forneceu as instruções de como remover um arquivo da área de preparação: (use "git restore --staged <*file*>..." to unstage). Como essa saída indica, para remover arquivos da área de preparação você pode usar o comando git restore com a opção --staged, passando os nomes dos arquivos que deseja remover, separados por espaços.

[SALVE O COMANDO]

git restore --staged <nome_arquivo>

Restaura um arquivo com outra versão na área de preparação.

[NOTA]

Se você tiver uma versão do Git mais antiga que a versão 2.23, não terá acesso ao comando git restore. Sua saída pode sugerir que use o comando git reset, que é outro comando que pode remover um arquivo da área de preparação. Na próxima seção Acompanhe, na etapa 1, você terá de inserir git reset HEAD rainbowcolors.txt em vez de git restore --staged rainbowcolors.txt para remover o arquivo rainbowcolors.txt da área de preparação. Lembre-se de que no Git geralmente existem muitas maneiras de obter o mesmo resultado.

Na seção Acompanhe 11-4, seu amigo usará o comando git restore com a opção --staged, como instruído na saída de git status na seção Acompanhe 11-3, passando o nome do arquivo rainbowcolors.txt para removê-lo da área de preparação. Isso significa que seu próximo commit só incluirá as alterações que ele fez no arquivo othercolors.txt.

[ACOMPANHE 11-4]

❶ friend-rainbow $ **git restore --staged rainbowcolors.txt**

❷ friend-rainbow $ **git status**

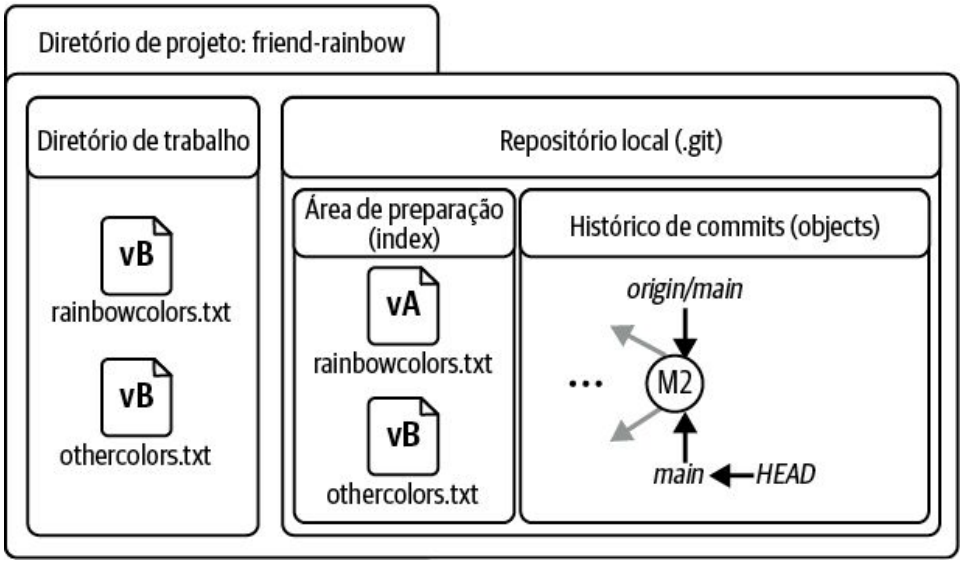
On branch main
Your branch is up to date with 'origin/main'.
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
modified: othercolors.txt
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: rainbowcolors.txt

O que deve ser observado:

- A saída de git status mostra que a versão atualizada do arquivo othercolors.txt ainda está na área de preparação, enquanto a versão atualizada do arquivo rainbowcolors.txt foi removida e não está mais na área de preparação.

Isso pode ser visto na seção Visualize 11-7.

[VISUALIZE 11-7]



Diretório de projeto friend-rainbow depois que seu amigo removeu o arquivo rainbowcolors.txt da área de preparação.

O que deve ser observado:

- Seu amigo removeu o arquivo rainbowcolors.txt da área de preparação; logo, a versão da área de preparação voltou a ser a vA.
- A versão atualizada de othercolors.txt (vB) ainda está na área de preparação.
- A versão de rainbowcolors.txt com as alterações mais recentes de seu amigo (vB) ainda está no diretório de trabalho.

Na seção Acompanhe 11-5, seu amigo fará um commit que só incluirá a alteração feita no arquivo othercolors.txt.

[ACOMPANHE 11-5]

❶ friend-rainbow $ **git commit -m "black"**
[main 29bdadd] black
1 file changed, 2 insertions(+), 1 deletion(-)

❷ friend-rainbow $ **git status**
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: rainbowcolors.txt
no changes added to commit (use "git add" and/or "git commit -a")

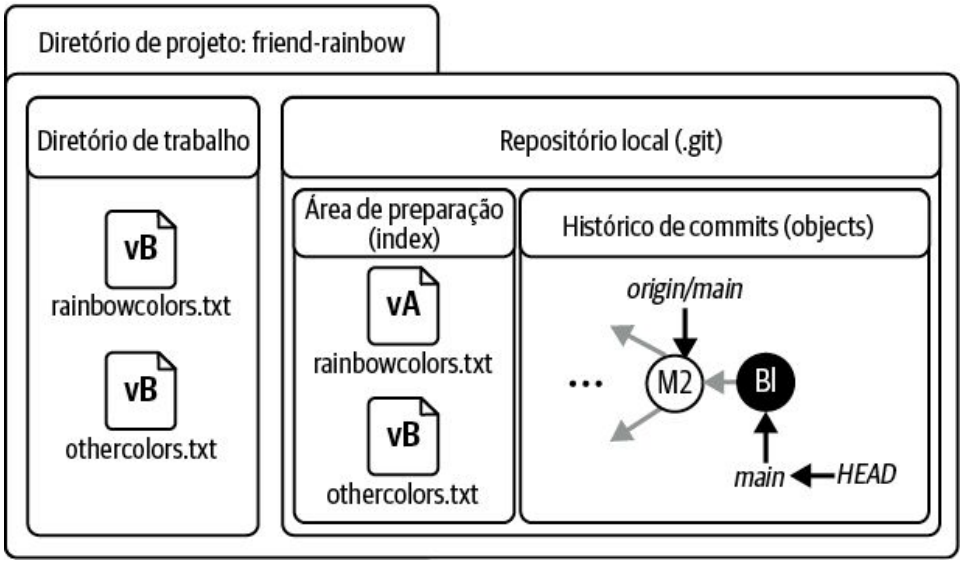
❸ friend-rainbow $ **git log**
commit 29bdadd50ddea41c75b476e776b6204a555b3d54 (**HEAD -> main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:07:38 2022 +0100
black
commit f10f9725e3319af840a3d891ca8950436a219eb0 (**origin/main**, **origin/HEAD**)
Merge: 6ad5c15 9b0a614
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:11:06 2022 +0100
merge commit 2

O que deve ser observado:

- Seu amigo fez o commit black (preto).
- Na etapa 2, a saída de git status menciona que o arquivo rainbowcolors.txt é um arquivo modificado do diretório de trabalho.

A seção Visualize 11-8 mostra o estado do diretório de projeto friend-rainbow após a seção Acompanhe 11-5.

[VISUALIZE 11-8]



Diretório de projeto friend-rainbow depois que seu amigo fez o commit black.

Na seção Acompanhe 11-6, seu amigo adicionará as alterações que fez no arquivo rainbowcolors.txt à área de preparação e fará outro commit.

[ACOMPANHE 11-6]

❶ friend-rainbow $ **git add rainbowcolors.txt**

❷ friend-rainbow $ **git commit -m "rainbow"**
[main 51dc6ec] rainbow
1 file changed, 1 insertion(+), 1 deletion(-)

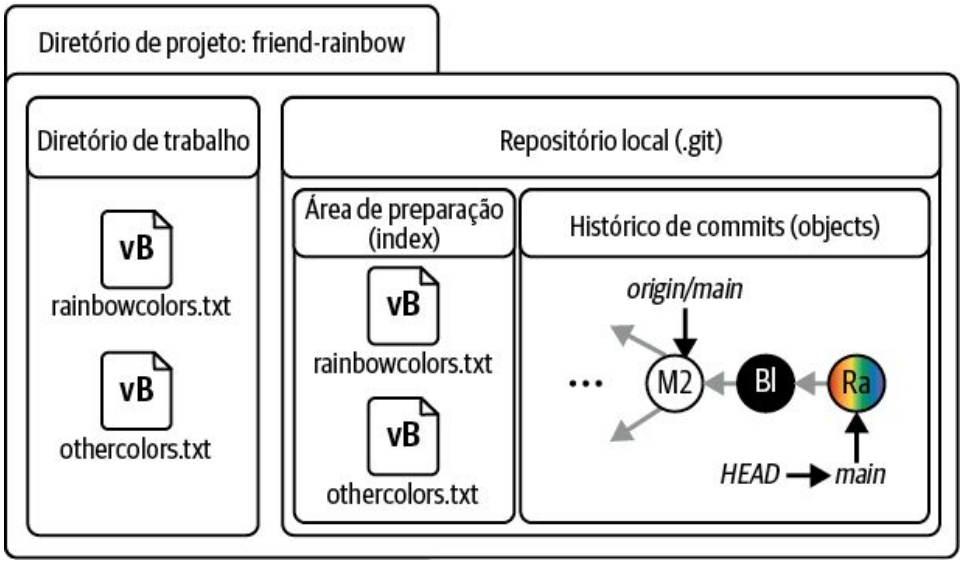
❸ friend-rainbow $ **git log**
commit 51dc6ecb327578cca503abba4a56e8c18f3835e1 (**HEAD -> main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:10:11 2022 +0100
rainbow
commit 29bdadd50ddea41c75b476e776b6204a555b3d54
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:07:38 2022 +0100
black
commit f10f9725e3319af840a3d891ca8950436a219eb0 (**origin/main**, **origin/HEAD**)
Merge: 6ad5c15 9b0a614
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:11:06 2022 +0100
merge commit 2

O que deve ser observado:

- Seu amigo fez o commit rainbow (arco-íris).

Isso é mostrado na seção Visualize 11-9.

[VISUALIZE 11-9]



Diretório de projeto friend-rainbow depois que seu amigo adicionou o arquivo rainbowcolors.txt à área de preparação e fez um commit.

O que deve ser observado:

- Agora a versão do arquivo rainbowcolors.txt na área de preparação é a vB, que é a versão que faz parte do commit rainbow.

Você acabou de ver como seu amigo adicionou e removeu arquivos na área de preparação para definir que commits queria fazer. O branch local main do repositório rainbow e o branch local main do repositório friend-rainbow agora têm históricos de desenvolvimento divergentes. Antes de seu amigo dar prosseguimento ao exemplo de rebasing, ele tem de se certificar de fazer o fetch de todo o trabalho que você enviou para o branch remoto main do repositório remoto.

Preparando-se para o rebasing

Como vimos no Exemplo 11-1 do Projeto Book anteriormente neste capítulo, para executar o rebase de um branch primeiro você tem de fazer o fetch de todo o trabalho executado no branch no qual será feito o rebase. Logo, a fim de se preparar para fazer o rebase de seu branch, na seção Acompanhe 11-7 seu amigo fará o fetch das atualizações mais recentes do repositório remoto.

[ACOMPANHE 11-7]

❶ friend-rainbow $ **git fetch**

```
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 307 bytes | 153.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
f10f972..6f2cf36 main -> origin/main
```

❷ friend-rainbow $ **git log --all**

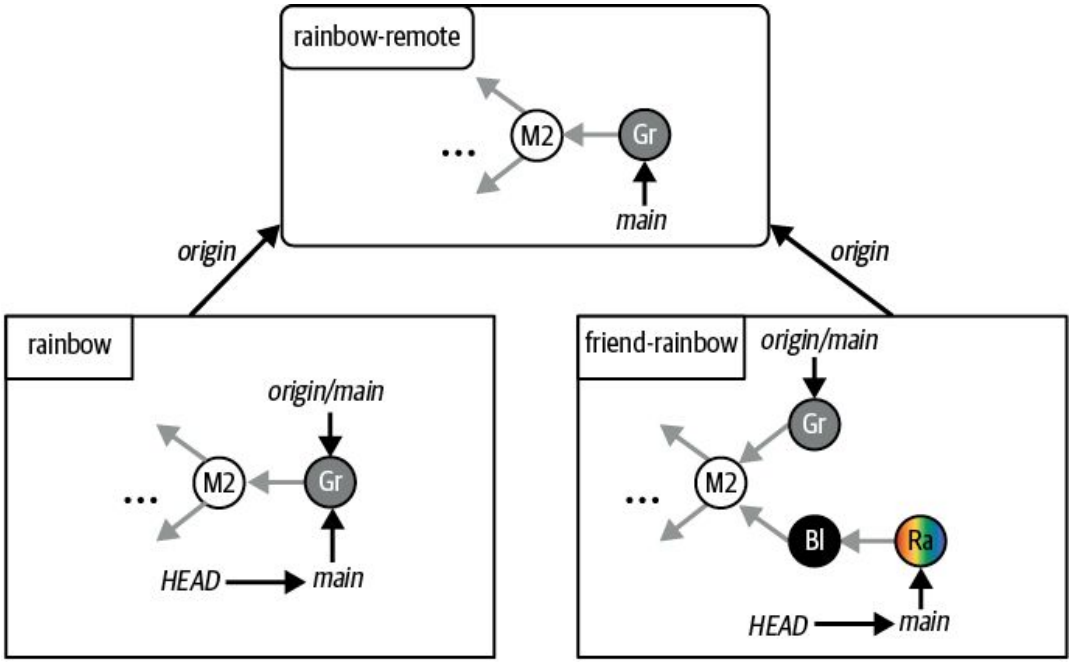
```
commit 51dc6ecb327578cca503abba4a56e8c18f3835e1 (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:10:11 2022 +0100
rainbow
commit 29bdadd50ddea41c75b476e776b6204a555b3d54
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:07:38 2022 +0100
black
commit 6f2cf3698e6bf9078e8e0340ec9948f590405091 (origin/main, origin/HEAD)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:27:58 2022 +0100
gray
```

O que deve ser observado:

- Seu amigo fez o fetch do commit gray (cinza) do repositório remoto e o branch de rastreamento remoto origin/main foi atualizado para apontar para ele.

Isso é mostrado na seção Visualize 11-10.

[VISUALIZE 11-10]



Projeto Rainbow depois que seu amigo atualizou o repositório friend-rainbow fazendo o fetch de seu trabalho a partir do repositório remoto.

O branch de rastreamento remoto origin/main do repositório friend-rainbow representa a versão mais recente do branch remoto main. Agora seu amigo está pronto para fazer o rebase de seu branch local main no branch de rastreamento remoto origin/main. A seguir, examinaremos os estágios do processo de rebasing.

Os cinco estágios do processo de rebasing

Esta seção o conduzirá pelos cinco estágios do processo de rebasing. Para fornecer uma demonstração visual dos estágios, incluirei diagramas da seção Visualize exibindo o exemplo que você executará posteriormente neste capítulo, quando seu amigo fizer o rebase de seu branch local main do repositório friend-rainbow no branch de rastreamento remoto origin/main.

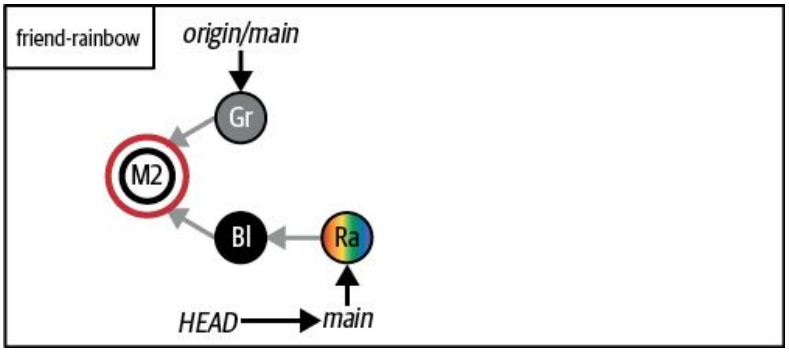
Para iniciar o processo de rebasing, você usará o comando git rebase. O Git executará então os cinco estágios do processo; o único momento em que você terá de se envolver ativamente será se houver algum conflito de merge. Discutiremos essa situação com mais detalhes em “Rebasing e conflitos de merge”, na página 224.

ESTÁGIO 1: ENCONTRAR O ANCESTRAL COMUM

No primeiro estágio do processo, o Git identificará o ancestral comum dos dois branches envolvidos no rebasing: o branch em que você está e aquele em que será feito o rebase.

No exemplo do projeto Rainbow, o branch em que seu amigo está é o branch local `main` do repositório `friend-rainbow` e o branch no qual ele fará o rebase será o branch de rastreamento remoto `origin/main`. O ancestral comum será o commit de merge M2, como mostrado na seção Visualize 11-11.

[VISUALIZE 11-11]



Estágio 1: Localizar o ancestral comum dos branches envolvidos no rebasing (aqui, o commit de merge M2).

ESTÁGIO 2: ARMAZENAR INFORMAÇÕES SOBRE OS BRANCHES ENVOLVIDOS NO REBASING

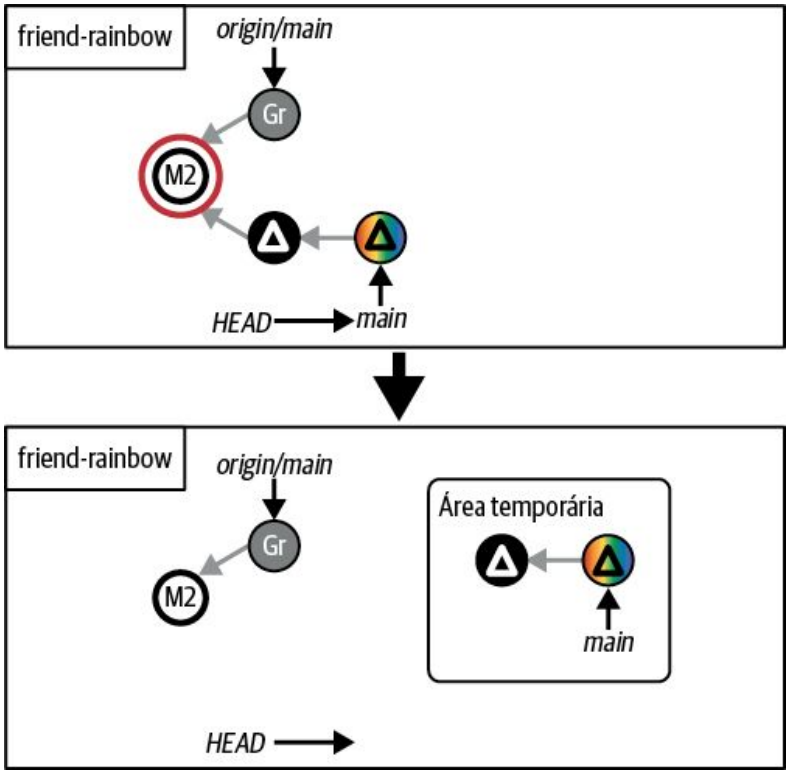
No estágio 2, o Git salvará em uma área temporária as alterações introduzidas por cada commit do branch em que você está. Também serão salvas informações adicionais nessa área temporária, como em qual branch você está fazendo o rebase e para onde ele estava apontando quando o rebase foi iniciado.

No exemplo do projeto Rainbow, as alterações introduzidas pelo commit black e pelo commit rainbow serão salvas na área temporária com informações sobre o branch remoto main, como mostrado na seção Visualize 11-12.

[NOTA]

Nos próximos diagramas da seção Visualize, triângulos (△) serão usados para representar as alterações introduzidas pelos commits.

[VISUALIZE 11-12]



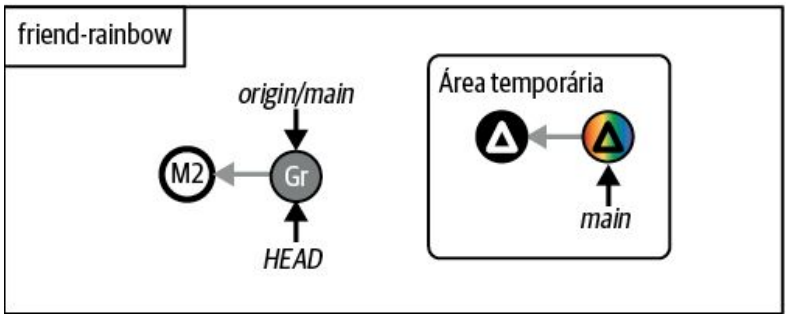
Estágio 2: Salvar informações sobre o rebasing em uma área temporária.

ESTÁGIO 3: REDEFINIR HEAD

No estágio 3, o Git redefinirá HEAD para que aponte para o mesmo commit do branch no qual você está fazendo o rebase.

No exemplo do projeto Rainbow, ele redefinirá HEAD para apontar para o mesmo commit para o qual o branch de rastreamento remoto origin/main está apontando, que é o commit gray (cinza), como mostrado na seção Visualize 11-13.

[VISUALIZE 11-13]



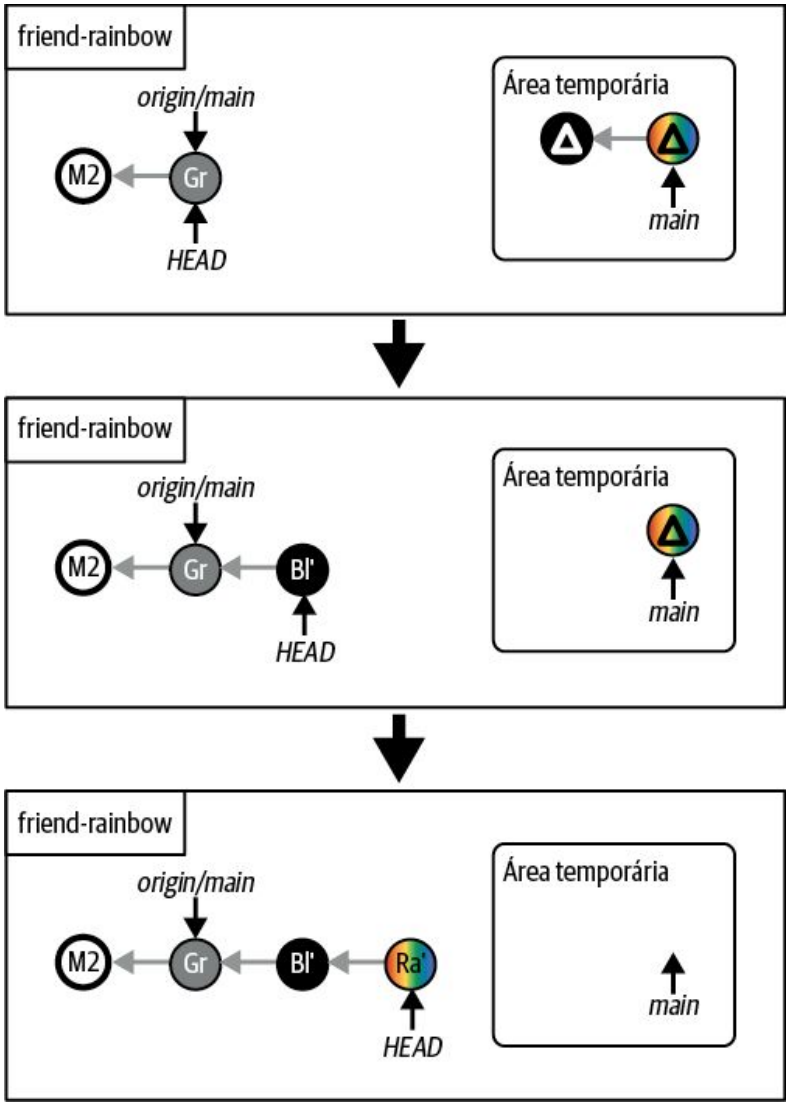
Estágio 3: Redefinir HEAD para apontar para o mesmo commit do branch no qual você está fazendo o rebase (aqui, o commit gray).

ESTÁGIO 4: APLICAR AS ALTERAÇÕES E FAZER O COMMIT

No estágio 4, o Git aplicará o conjunto de alterações de cada commit, fazendo um commit após aplicar cada conjunto.

No exemplo do projeto Rainbow, primeiro ele aplicará as alterações introduzidas pelo commit black (preto), criará um novo commit e, em seguida, aplicará as alterações introduzidas pelo commit rainbow e fará um novo commit, como mostra a seção Visualize 11-14.

[VISUALIZE 11-14]



Estágio 4: Aplicar as alterações e fazer cada um dos commits (aqui, os commits black e rainbow).

Na seção Visualize 11-14, o novo commit black e o novo commit rainbow estão representados como Bl’ e Ra’ (com um apóstrofo) porque são commits totalmente novos.

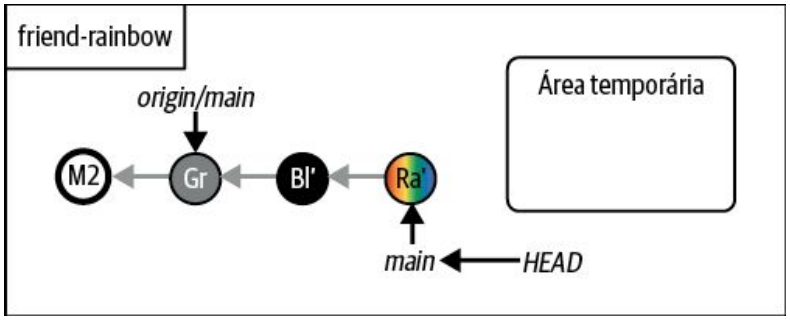
ESTÁGIO 5: MUDAR PARA O BRANCH EM QUE FOI FEITO O REBASE

No estágio 5, o Git fará o branch em que você fez o rebase apontar para o último commit que foi reaplicado e fará o checkout desse branch para que HEAD aponte para ele.

No exemplo do projeto Rainbow, o branch main apontará para o novo

commit rainbow, como mostrado na seção Visualize 11-15.

[VISUALIZE 11-15]



Estágio 5: Passar para o branch em que você fez o rebase.

Isso conclui nossa verificação dos cinco estágios do processo de rebasing. Como mencionado no início desta seção, o Git executa todo o processo automaticamente; a única coisa que você precisa fazer é iniciá-lo com o comando git rebase. Também mencionei que a única situação que demandará que você se envolva no processo de rebasing será se o Git encontrar conflitos de merge. Examinaremos brevemente esse cenário na próxima seção.

Rebasing e conflitos de merge

No Capítulo 10, você conheceu os conflitos de merge, que surgem quando integramos dois branches em que diferentes alterações foram feitas nas mesmas partes do(s) mesmo(s) arquivo(s) ou se um arquivo editado em um branch tiver sido excluído no outro branch.

O Git executa o processo de rebasing inteiro independentemente, a menos que encontre conflitos de merge. Nesse caso, você terá de interferir e resolvê-los. O processo para a solução de conflitos de merge durante o rebasing é semelhante ao usado quando fazemos um merge three-way, com algumas pequenas diferenças.

Na resolução de conflitos em um merge three-way, todos os conflitos de merge são apresentados ao mesmo tempo; após resolver todos os conflitos e adicionar todos os arquivos atualizados à área de preparação, fazemos o commit de merge final. No processo de rebasing, por outro lado, já que o Git aplica as alterações de cada commit uma a uma, ele pausa o processo quando encontra conflitos de merge em algum commit reaplicado. Isso significa que você pode ter de resolver conflitos de merge várias vezes ao

fazer o rebase, dependendo de quantos commits tiverem conflitos.

Após você terminar de resolver os conflitos de merge em um commit específico, terá de adicionar os arquivos atualizados à área de preparação e instruir o Git a retomar o processo de rebasing inserindo o comando `git rebase` com a opção `--continue`. O Git continuará então fazendo o rebase do restante dos commits. Não é preciso fazer nenhum commit explicitamente, como fazemos em um merge three-way com conflitos.

Como ocorre com um merge, se em algum momento durante o processo de solução de conflitos de merge em um rebasing você decidir que não quer continuar o processo de rebasing, pode optar por interromper ou abortar o processo usando o comando `git rebase` com a opção `--abort`. Isso retornará todos os seus arquivos ao estado em que se encontravam antes do rebasing.

[SALVE O COMANDO]

git rebase --continue
Continua o processo de rebasing após a resolução de conflitos de merge.

git rebase --abort
Interrompe o processo e retorna ao estado anterior ao rebasing.

Agora que abordamos o que você pode esperar do rebasing, é hora de praticar com o projeto Rainbow.

Rebasing de um branch na prática

Consulte a seção Acompanhe 11-8, em que seu amigo fará o rebase do branch main de seu repositório local no branch de rastreamento remoto origin/main.

[ACOMPANHE 11-8]

❶ A partir do diretório de projeto friend-rainbow, anote os hashes do commit black e do commit rainbow. Você pode usar o comando git log para isso, já que ele inclui os hashes de commit em sua saída. No exemplo deste livro, os hashes de commit são:

Commit black: 29bdadd50ddea41c75b476e776b6204a555b3d54

Commit rainbow: 51dc6ecb327578cca503abba4a56e8c18f3835e1

❷ friend-rainbow $ **git rebase origin/main**

```
Auto-merging othercolors.txt
CONFLICT (content): Merge conflict in othercolors.txt
error: could not apply 29bdadd... black
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
Could not apply 29bdadd... black
```

❸ friend-rainbow $ **git status**

```
interactive rebase in progress; onto 6f2cf36
Last command done (1 command done):
pick 29bdadd black
Next command to do (1 remaining command):
pick 51dc6ec rainbow
(use "git rebase --edit-todo" to view and edit)
You are currently rebasing branch 'main' on 'bcb1dc0'.
(fix conflicts and then run "git rebase --continue")
(use "git rebase --skip" to skip this patch)
(use "git rebase --abort" to check out the original branch)
Unmerged paths:
(use "git restore --staged <file>..." to unstage)
(use "git add <file>..." to mark resolution)
both modified: othercolors.txt
no changes added to commit (use "git add" and/or "git commit -a")
```

O que deve ser observado:

- A operação de rebasing foi interrompida porque, quando o Git estava aplicando as alterações que foram incluídas no commit black, ele encontrou um conflito de merge.
- Na etapa 2, o comando git rebase está informando que você deve Resolve all conflicts manually, mark them as resolved with "git add/rm <conflicted_files>", then run "git rebase --continue" (resolver todos os conflitos manualmente, marcá-los

como resolvidos com "git add/rm <*arquivos_conflitantes*>" e executar "git rebase --continue").

- Na etapa 3, o comando git status também está exibindo informações sobre o rebasing e mostrando em quais arquivos você tem de resolver conflitos de merge.

Na seção Acompanhe 11-9, seu amigo aplicará as etapas que aprendemos no Capítulo 10 para resolver conflitos de merge. Ao terminar, ele executará o comando git rebase --continue.

[ACOMPANHE 11-9]

❶ Execute a etapa 1 da resolução de conflitos de merge, que é selecionar o que manter, editar o conteúdo e remover os marcadores de conflito. Você manterá todas as alterações dos dois branches. Mantenha a frase “Gray is not a color in the rainbow.” acima da frase “Black is not a color in the rainbow.”. Certifique-se de salvar o arquivo othercolors.txt quando terminar de fazer suas edições.

❷ friend-rainbow $ **git add othercolors.txt**

❸ friend-rainbow $ **git status**

interactive rebase in progress; onto 6f2cf36
Last command done (1 command done):
pick 29bdadd black
Next command to do (1 remaining command):
pick 51dc6ec rainbow
(use "git rebase --edit-todo" to view and edit)
You are currently rebasing branch 'main' on 'bcb1dc0'.
(all conflicts fixed: run "git rebase --continue")
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
modified: othercolors.txt

❹ friend-rainbow $ **git rebase --continue**

❺ Aceite a mensagem de commit padrão do editor Vim e saia do editor. Como aprendemos no Capítulo 9, para fazer isso você deve pressionar a tecla Escape, digitar **:wq** e pressionar Enter. Verá então a saída a seguir:

[detached HEAD e055f2b] black
1 file changed, 3 insertions(+), 1 deletion(-)
Successfully rebased and updated refs/heads/main.

❻ friend-rainbow $ **git log**

commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (**HEAD -> main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:10:11 2022 +0100
rainbow
commit e055f2bc66aed1f3627041900a8c825c7a875206
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:07:38 2022 +0100
black
commit 6f2cf3698e6bf9078e8e0340ec9948f590405091 (**origin/main**, **origin/HEAD**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:27:58 2022 +0100
gray

O que deve ser observado:

- Na etapa 3, a saída de `git status` está exibindo `all conflicts fixed: run "git rebase --continue"` (todos os conflitos corrigidos: execute “git rebase -- continue”).
- Na etapa 6, a saída de `git log` indica que o processo de rebasing criou um novo commit rainbow e um novo commit black. Os hashes de commit são novos, assim como o são os timestamps (a data e a hora de cada commit).

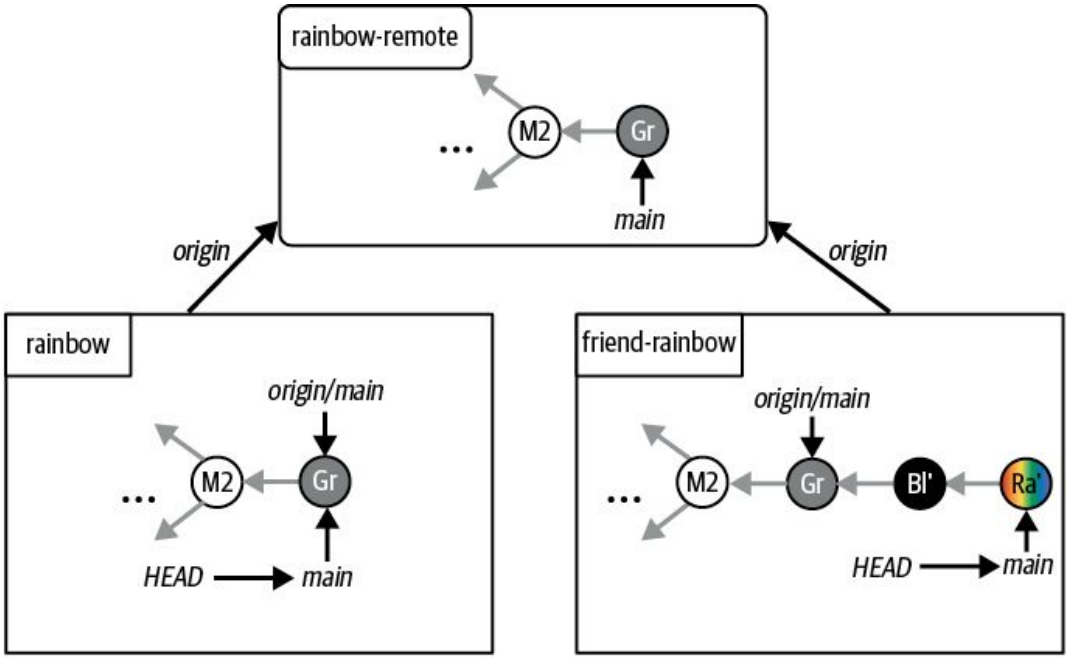
O resultado desse processo é mostrado na seção Visualize 11-16.

O que deve ser observado:

- Representamos o novo commit black e o novo commit rainbow como Bl’ e Ra’ (com um apóstrofo).
- No repositório `friend-rainbow`, o commit gray, o novo commit black e o novo commit rainbow formam um histórico de projeto linear. Em outras palavras, não há commit de merge.

A Figura 11-7 compara os hashes dos antigos commits black e rainbow com os dos novos commits black e rainbow do livro. Os hashes de commit de seus repositórios serão diferentes dos deste livro porque os hashes de commit são únicos.

[VISUALIZE 11-16]



Projeto Rainbow depois que seu amigo fez o rebase de seu branch local main no branch de rastreamento remoto origin/main.

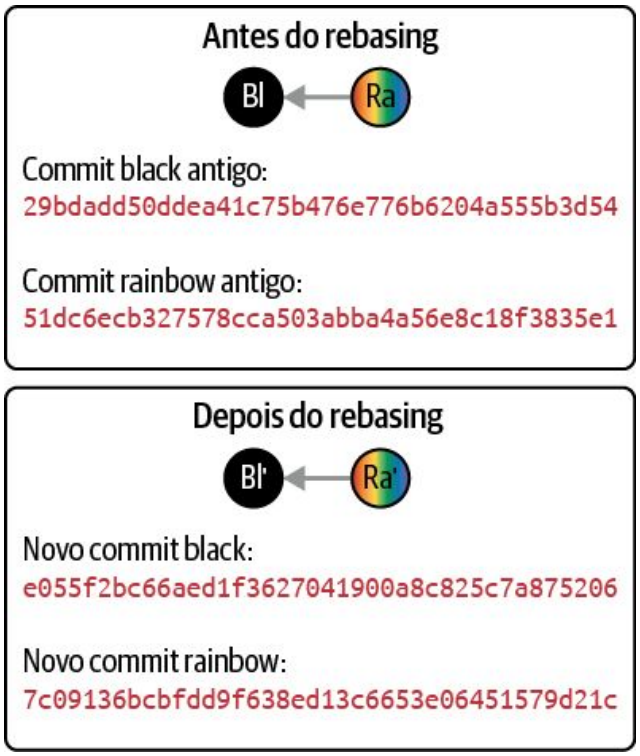


Figura 11-7: Os hashes dos commits black e rainbow antigos são diferentes dos hashes dos commits black e rainbow novos.

Nesse exercício, vimos que o rebasing recria o histórico — e isso nos leva à regra de ouro do rebasing.

Regra de ouro do rebasing

Como você viu no exemplo do projeto Rainbow, o rebasing cria commits totalmente novos. Isso significa que ele altera o histórico de commits. Você deve sempre tomar cuidado quando alterar o histórico de commits porque isso pode trazer complicações para o projeto, principalmente quando estiver trabalhando com outras pessoas.

A regra de ouro do rebasing diz que *você não deve fazer o rebase de um branch no qual outras pessoas possam ter baseado seu trabalho*. Por exemplo, se você fizer o push de um branch para o repositório remoto, ele será considerado um branch público. Ou seja, outros colaboradores também podem estar trabalhando nesse branch em seus repositórios locais ou podem fazer o push de algum trabalho para ele no repositório remoto.

Nesse caso, você deve evitar fazer o rebase do branch. Para ver a importância da regra de ouro do rebasing, examinaremos o Exemplo 11-3 do Projeto Book, em que revisitaremos a situação discutida no Exemplo 11-1.

Exemplo 11-3 do Projeto Book

No Exemplo 11-1 do Projeto Book, introduzi uma situação em que existiam dois commits no branch main (commits A e B) quando criei o branch chapter_five e meu coautor criou o branch chapter_six. Subsequentemente, meu coautor fez o merge do commit C do branch chapter_six em seu branch main e fez o push do branch main atualizado para o repositório remoto. Ao mesmo tempo, adicionei o commit D ao branch chapter_five em meu repositório local. Esse cenário inicial é mostrado na Figura 11-8.

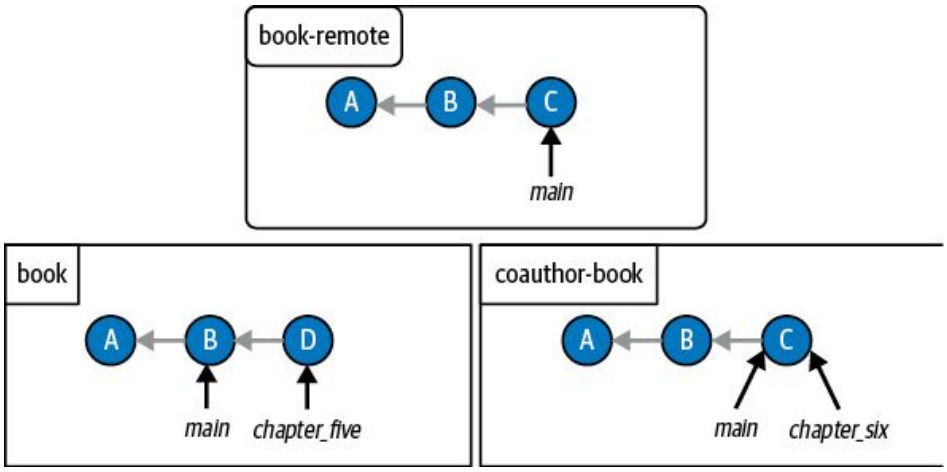


Figura 11-8: Projeto Book na situação em que o branch local chapter_five e o branch remoto main divergiram.

Suponhamos que eu fizesse o push do branch chapter_five para o repositório remoto e, violando a regra de ouro do rebasing, também fizesse o pull do commit C a partir do repositório remoto e atualizasse meu branch local main para poder fazer o rebase do branch chapter_five na versão mais recente do branch main. Essa situação pode ser vista na Figura 11-9.

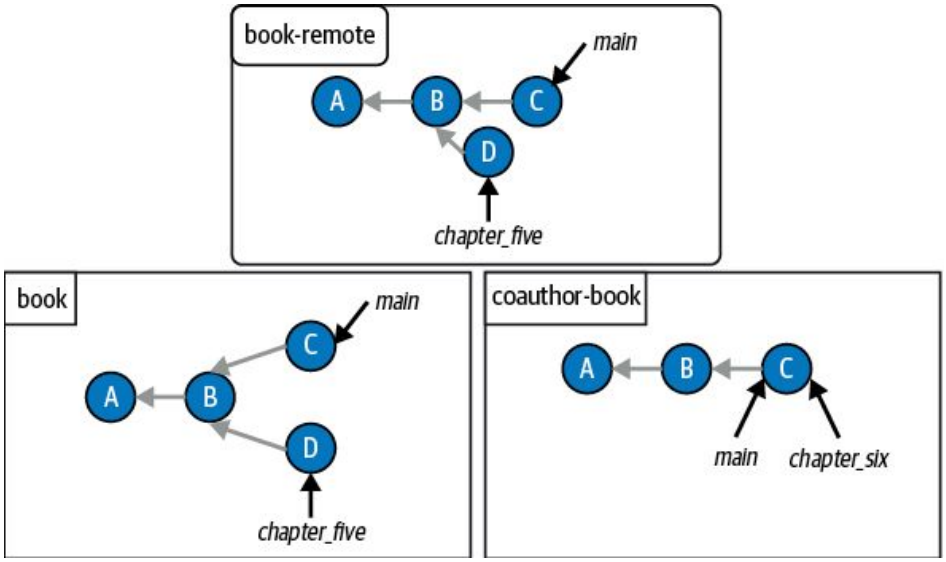


Figura 11-9: Projeto Book depois que fiz o push do branch chapter_five para o repositório remoto e fiz o pull dos commits no branch main.

Não notei o erro que cometi e dei prosseguimento ao plano de fazer o rebase do branch chapter_five no branch main. A Figura 11-10 mostra o estado dos repositórios remoto e locais após o rebasing. D' no repositório book representa o commit D recriado depois do rebasing.

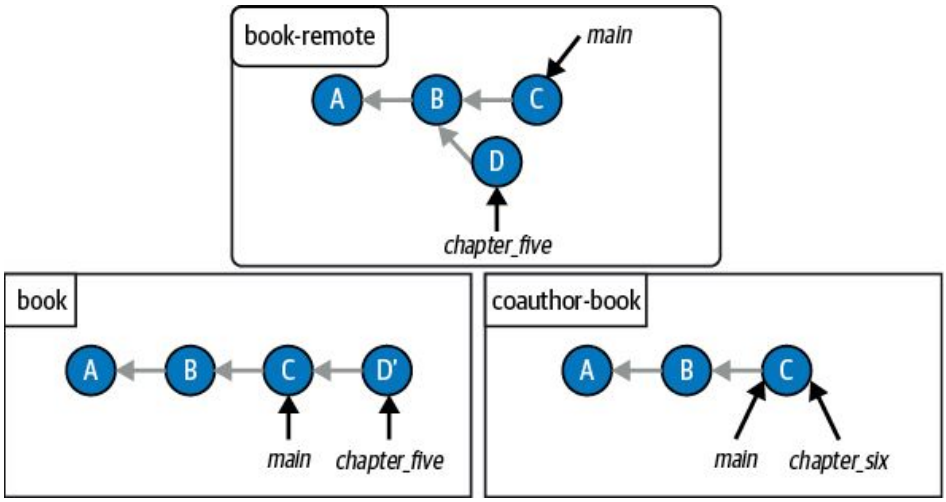


Figura 11-10: Projeto Book depois que fiz o rebase do branch chapter_five no branch main.

Lembre-se de que ainda não fiz o push de meu branch chapter_five atualizado para o repositório remoto. Suponhamos que meu coautor visse o branch remoto chapter_five no repositório remoto e decidisse fazer alguma contribuição. Ele fez o pull do branch chapter_five em seu repositório coauthor-book, adicionou os commits E e F e então os enviou para o repositório remoto. Essa situação é mostrada na Figura 11-11.

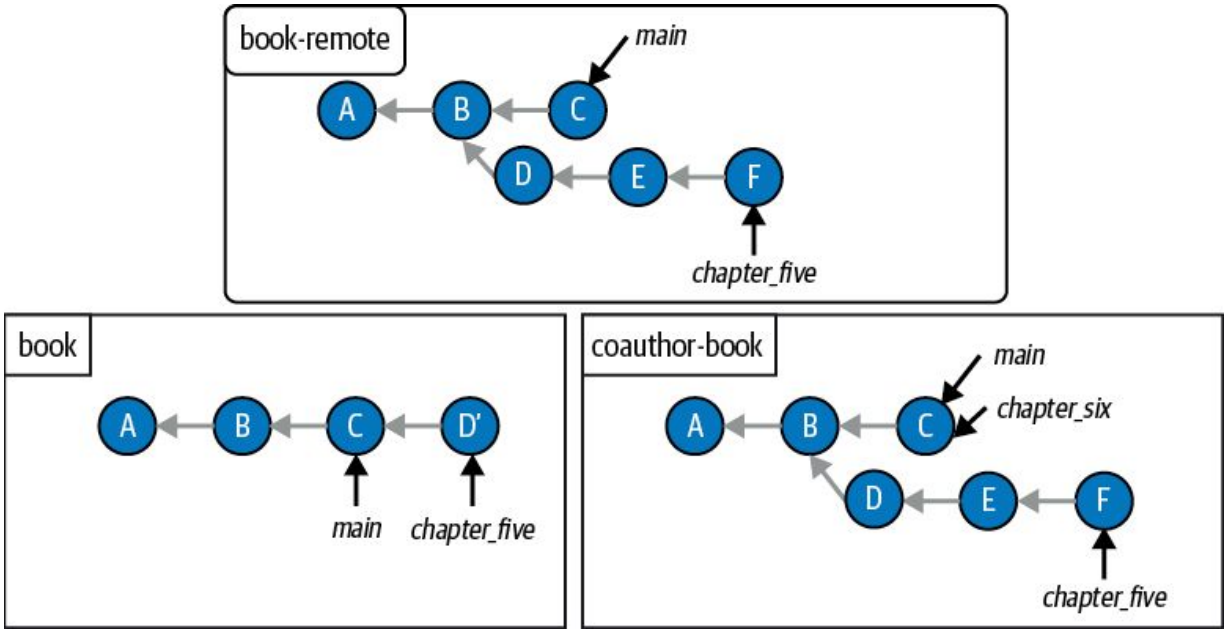


Figura 11-11: Projeto Book depois que meu coautor fez contribuições para o branch remoto chapter_five.

Quando eu tentar fazer o push de meu branch chapter_five para o repositório remoto, verei a mensagem de erro a seguir:

error: failed to push some refs to 'github.com:gitlearningjourney/rainbow-remote.git'

hint: Updates were rejected because the tip of your current branch is behind its remote counterpart. Integrate the remote changes (e.g. 'git pull ...') before pushing again. See the 'Note about fast-forwards' in 'git push --help' for details.

Essa mensagem de erro está informando que não posso fazer o push de minhas alterações para o repositório remoto. Como você pode ver na Figura 11-11, os históricos de commit do branch remoto chapter_five e do branch local chapter_five não contêm mais os mesmos commits: o branch remoto chapter_five é composto dos commits A, B, D, E e F, enquanto o branch local chapter_five é composto dos commits A, B, C e D'.

Essa é uma situação complicada, porque agora meu coautor e eu não temos uma maneira fácil de resolver o problema. Teremos de conversar para resolver a questão e ver como podemos solucionar as disparidades entre nossos branches.

O Exemplo 11-3 do Projeto Book mostra por que você deve seguir a regra de ouro quando usar a operação de rebasing. Para concluir, você pode fazer o rebase de um branch seguramente se:

- Tiver um branch local que nunca enviou para o repositório remoto.
- Tiver um branch local que enviou para um repositório remoto, mas estiver certo de que ninguém baseou seu trabalho nele ou enviou contribuições.

Isso é tudo. Se houver alguma possibilidade de alguém ter trabalhado no branch, é recomendável evitar o rebasing.

Sincronizando os repositórios

Para termos certeza de que os repositórios do projeto Rainbow estão em sincronia, seu amigo terá de fazer o push de suas alterações para o repositório remoto e você terá de fazer o pull das alterações para o repositório rainbow. Consulte a seção Acompanhe 11-10 para fazer isso agora.

[ACOMPANHE 11-10]

❶ friend-rainbow $ **git push**

Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 652 bytes | 652.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:gitlearningjourney/rainbow-remote.git
6f2cf36..7c09136 main -> main

❷ friend-rainbow $ **git log**

commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (**HEAD -> main**, **origin/main**, **origin/HEAD**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:10:11 2022 +0100
rainbow
commit e055f2bc66aed1f3627041900a8c825c7a875206
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:07:38 2022 +0100
black

❸ Vá até a janela de linha de comando em que você está no repositório rainbow para executar o comando na etapa 4.

❹ rainbow $ **git pull**

remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 1), reused 6 (delta 1), pack-reused 0
Unpacking objects: 100% (6/6), 632 bytes | 105.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
6f2cf36..7c09136 main -> origin/main
Updating 6f2cf36..7c09136
Fast-forward
othercolors.txt | 4 +++-
rainbowcolors.txt | 2 +-
2 files changed, 4 insertions(+), 2 deletions(-)

❺ rainbow $ **git log**

commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (**HEAD -> main**, **origin/main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:10:11 2022 +0100

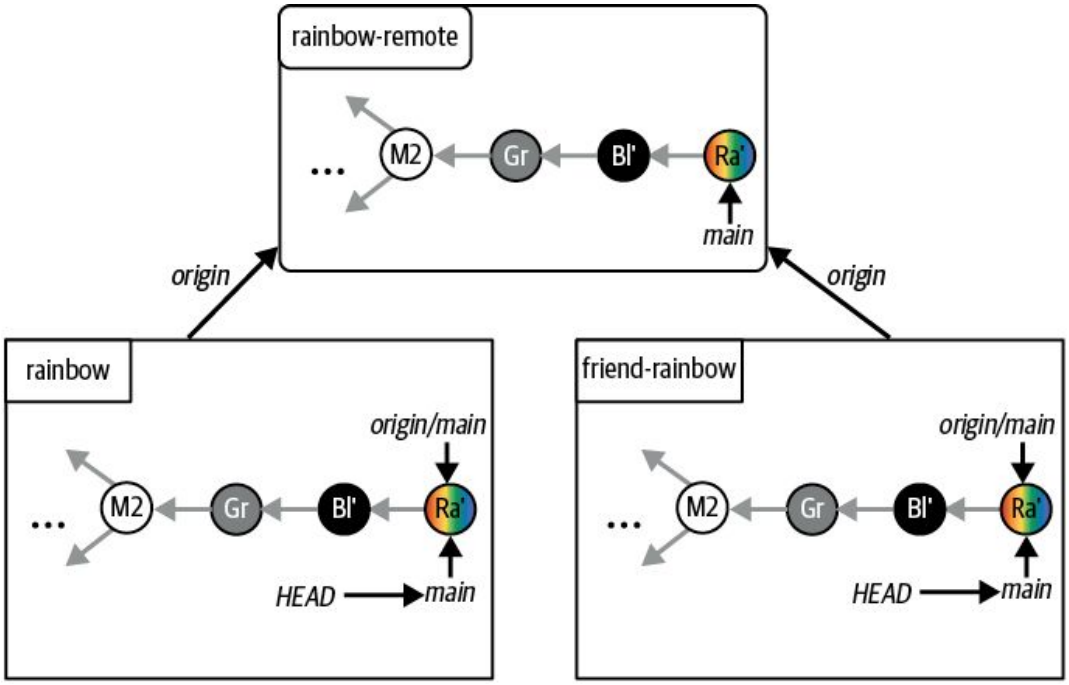
rainbow
commit e055f2bc66aed1f3627041900a8c825c7a875206
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:07:38 2022 +0100
black

Agora todos os três repositórios do projeto Rainbow estão em sincronia, como mostrado na seção Visualize 11-17.

Estado dos repositórios locais e remoto

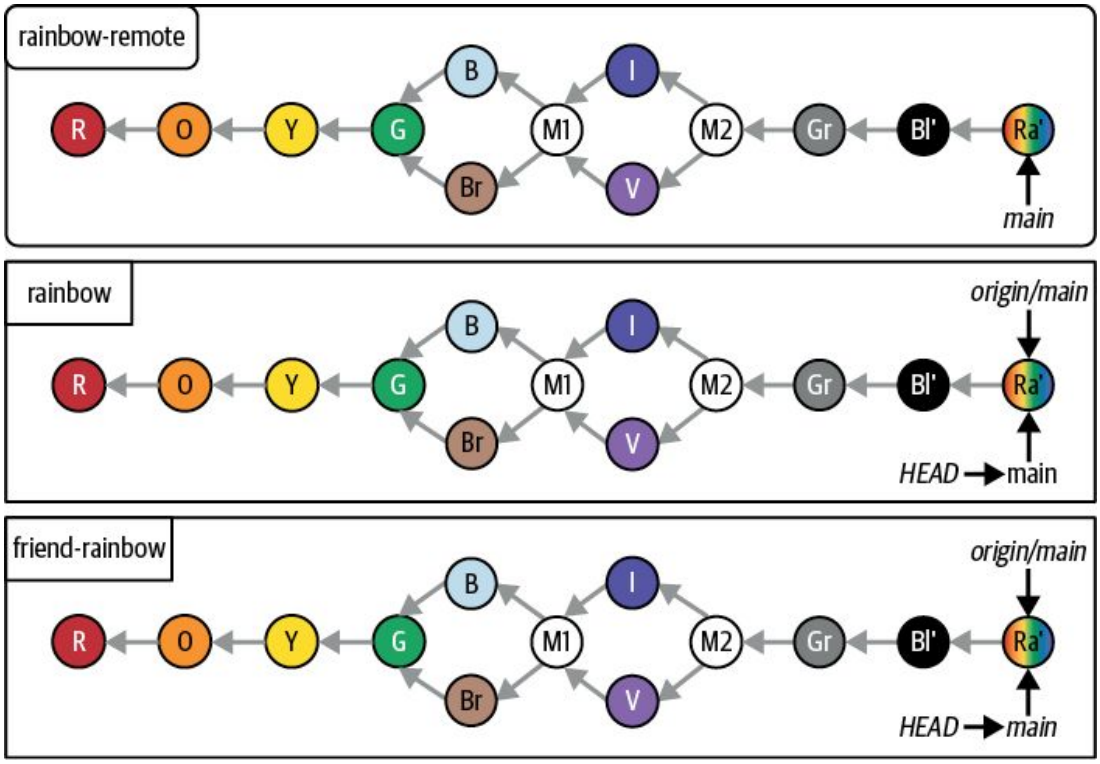
A seção Visualize 11-18 mostra o estado dos repositórios locais e remoto do projeto Rainbow com todos os commits que foram feitos do Capítulo 1 até o fim deste capítulo.

[VISUALIZE 11-17]



Projeto Rainbow depois que seu amigo fez o push de seu branch local main para o repositório remoto e você fez um pull a partir do repositório remoto para atualizar seu branch local main.

[VISUALIZE 11-18]



Projeto Rainbow no fim do Capítulo 11 com todos os commits desde o Capítulo 1.

Resumo

Neste capítulo, você conheceu o rebasing – a segunda maneira de integrar alterações de um branch em outro no Git – e aprendeu como ele pode ser usado para evitarmos merges three-way e commits de merge. Vimos os cinco estágios do processo de rebasing que o Git executa e, em seguida, você executou um rebase com um exemplo no projeto Rainbow. Durante o processo, também praticou resolver conflitos de merge.

Já que o rebasing recria o histórico de commits, introduzi a regra de ouro, segundo a qual não devemos fazer o rebase de branches em que outros colaboradores tiverem baseado seu trabalho. Você também aprendeu a remover arquivos, ou retirar arquivos modificados, da área de preparação, para personalizar com exatidão o que incluirá em um commit.

Até agora, vimos como você pode integrar alterações de um branch em outro trabalhando com um repositório local por meio do merging ou do rebasing. A seguir, no Capítulo 12, examinaremos uma ferramenta de

colaboração útil para projetos Git que lhe permitirá integrar alterações remotamente, chamada pull request.

12

Pull Requests (Merge Requests)

Até agora, só examinamos como integrar as alterações de um branch em outro trabalhando com um repositório local.

Neste capítulo, conheceremos os pull requests, uma ferramenta útil para a colaboração que permite integrar as alterações de um branch em outro em um repositório remoto. Durante o processo, você também aprenderá um truque conveniente para definir com mais facilidade branches upstream para novos branches locais.

Recursos adicionais que podem ajudá-lo enquanto você lê o capítulo estão disponíveis no repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*).

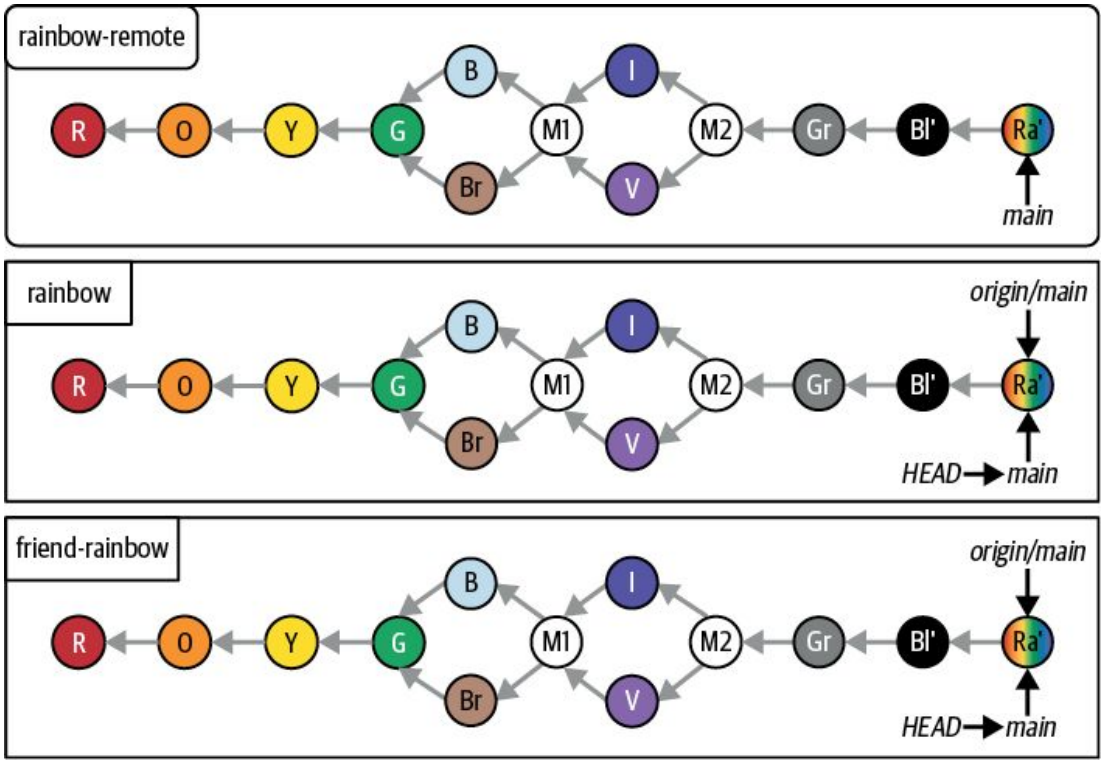
Estado dos repositórios locais e remoto

No começo deste capítulo, você deve ter dois repositórios locais chamados rainbow e friend-rainbow e um repositório remoto chamado rainbow-remote. Todos esses três repositórios devem estar em sincronia, com os mesmos commits e branches.

A seção Visualize 12-1 mostra o estado dos repositórios locais e remoto no projeto Rainbow com todos os commits que foram feitos do Capítulo 1 ao Capítulo 11.

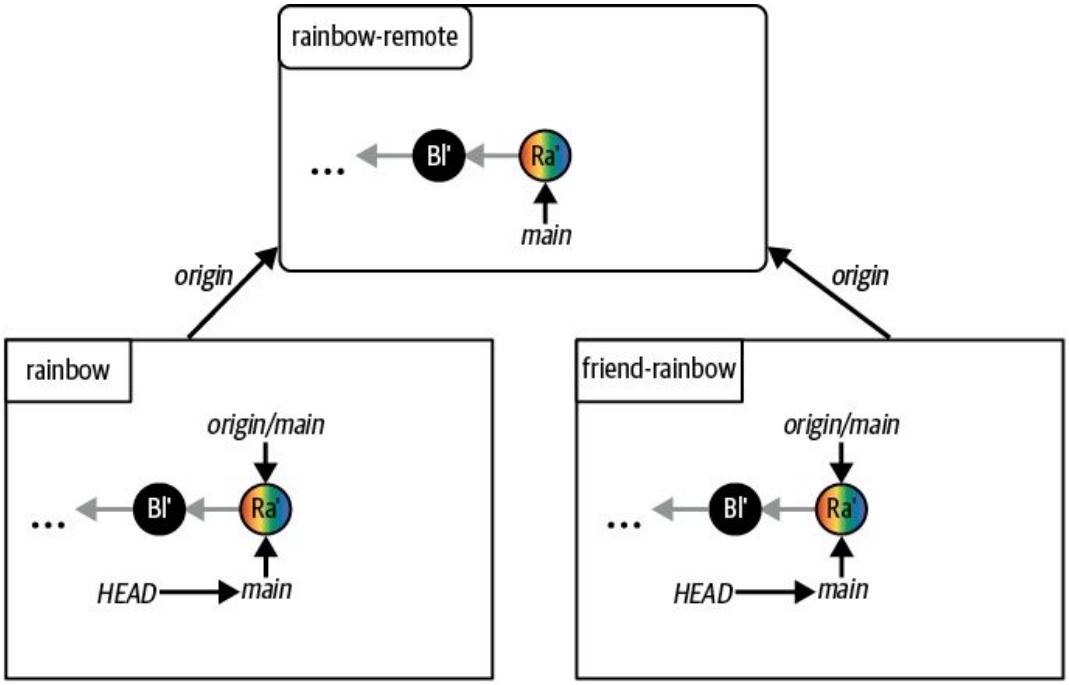
Para nos concentrarmos nos commits que você fará neste capítulo, de agora em diante simplificarei os diagramas da seção Visualize e só mostrarei os dois últimos commits que fazem parte do branch main em todos os repositórios, que são os commits black (preto) e rainbow (arco-íris). Essa representação é mostrada na seção Visualize 12-2.

[VISUALIZE 12-1]



Projeto Rainbow no começo do Capítulo 12 com todos os commits desde o Capítulo 1.

[VISUALIZE 12-2]



Representação simplificada do projeto Rainbow no começo do Capítulo 12, mostrando apenas os dois últimos commits do branch main em todos os repositórios.

Introdução aos pull requests

Um *pull request* (também chamado de merge request) é um recurso oferecido pelos serviços de hospedagem que nos permite compartilhar com os colaboradores o trabalho que fizemos em um branch, coletar feedbacks sobre esse trabalho e integrá-lo ao projeto remotamente no serviço de hospedagem. Embora os pull requests não sejam um recurso do Git, e sim dos serviços que hospedam projetos usando o Git, eles são tão úteis no trabalho diário com o Git que achei conveniente apresentá-los.

Os pull requests podem ser integrados por merging ou rebasing, mas a opção padrão (e mais comum) é o merging; logo, é ela que usaremos no exemplo deste capítulo. No restante do capítulo, chamarei o processo de integrar um pull request de “fazer o merge de um pull request”.

Quando criamos um pull request, dizemos que “abrimos” um pull request. Após o pull request ser revisado, aprovado e integrado, nós o “fechamos”. Você também pode fechar um pull request se decidir não o integrar e quiser removê-lo da lista de pull requests abertos (por exemplo, se o pull request

não for aprovado).

O processo de execução de um pull request pode ser dividido em nove etapas:

1. Crie um branch no repositório local.
2. Adicione trabalho fazendo commits no branch.
3. Faça o push do branch para o repositório remoto.
4. Crie (ou abra) um pull request no serviço de hospedagem.
5. Submeta o pull request à revisão e incorpore a ele os feedbacks das outras pessoas.
6. Obtenha a aprovação do pull request.
7. Faça o merge do pull request.
8. Se estiver lidando com um branch de tópico (branch de recurso), exclua o branch remoto.
9. Faça o pull das alterações para sincronizar seu repositório local com o repositório remoto e execute uma limpeza excluindo o branch local e o branch de rastreamento remoto.

[NOTA]

Se existirem conflitos de merge entre os branches no pull request, você não poderá integrá-lo. Primeiro deve resolver os conflitos de merge. Alguns serviços de hospedagem dão suporte à solução de conflitos de merge em seu site; no entanto, é mais comum que a solução seja aplicada no repositório local com o uso do processo descrito no Capítulo 10.

Antes de ver um exemplo, examinaremos brevemente alguns itens dos quais você terá de se lembrar quando trabalhar com um serviço de hospedagem.

Particularidades dos serviços de hospedagem

As etapas específicas para a criação e o gerenciamento de pull requests com o GitHub, o GitLab e o Bitbucket são diferentes e você pode ter de consultar a documentação de seu serviço de hospedagem para fazer alguns dos exercícios das seções Acompanhe deste capítulo. Para obter recursos adicionais, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*).

A terminologia também pode diferir entre os serviços de hospedagem: enquanto o GitHub e o Bitbucket usam o termo “pull request”, o GitLab se refere ao mesmo recurso como um “merge request”. Lembre-se dessas diferenças quando fizer os exercícios das seções Acompanhe do capítulo.

[NOTA]

Ainda que o termo “pull” seja usado no nome, os pull requests não têm nenhuma relação com o comando git pull.

Para concluir, no Capítulo 6, quando examinamos a seleção de um serviço de hospedagem e a definição de acesso HTTPS ou SSH, recomendei que você use uma conta de serviço de hospedagem pessoal em vez da conta da empresa. Fiz isso porque é possível definir configurações adicionais em um serviço de hospedagem para o processo de criação e aprovação de pull requests. Por exemplo, você (ou a empresa para a qual trabalha) poderia definir certos requisitos para a criação de um pull request ou criar restrições para quem poderá aprovar pull requests ou para quantas pessoas de uma equipe terão de aprovar um pull request antes de ele poder ser integrado.

Os exemplos deste livro presumem que nenhuma configuração adicional foi definida. Contudo, lembre-se disso se estiver usando uma conta da empresa, já que podem existir requisitos e restrições adicionais que afetarão sua possibilidade de criar e gerenciar pull requests.

Agora discutiremos por que você pode querer usar pull requests.

Por que usar Pull Requests?

Os pull requests facilitam a comunicação e a colaboração em projetos Git fornecendo um mecanismo simples para a revisão de trabalho. Eles têm um recurso de comentário útil que permitirá que você e seus colaboradores adicionem comentários a linhas específicas dos arquivos de um projeto, respondam a esses comentários e iniciem threads de discussão. Isso ajuda a organizar o processo de revisão.

Já que os pull requests são totalmente gerenciados na UI do serviço de hospedagem, eles também permitem que não usuários do Git forneçam feedback em projetos Git – você não precisa saber como usar o Git para fazer comentários em um pull request! Examinaremos o Exemplo 12-1 do Projeto Book para ver uma demonstração de como pode ser útil usar pull requests.

Exemplo 12-1 do Projeto Book

Suponhamos que eu criasse um branch fora do branch main chamado chapter_nine para trabalhar no Capítulo 9 de meu livro. Fiz dois commits nesse branch, os commits W e X, e fiz o

push de meu branch chapter_nine para o repositório remoto, criando um branch remoto chapter_nine. O estado dos repositórios locais e remoto é mostrado na Figura 12-1.

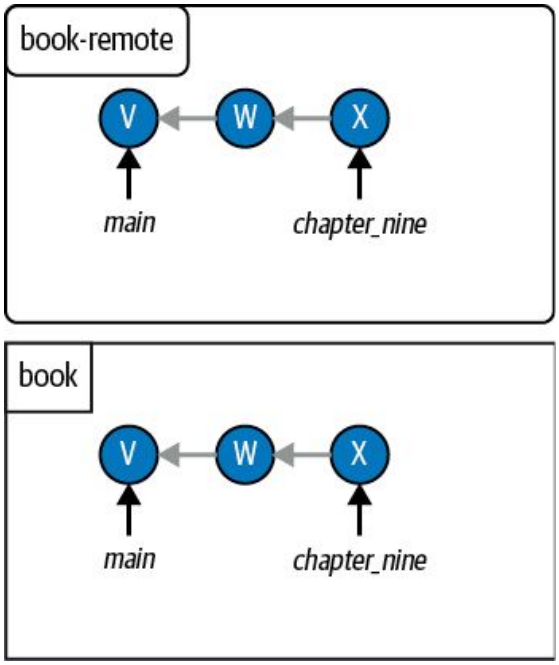


Figure 12-1: Projeto Book depois que fiz o push do branch chapter_nine para o repositório remoto.

Lembre-se de que combinei com meu editor que ele deve revisar meu trabalho antes de eu integrá-lo ao branch main. Isso significa que terá de revisar o novo branch chapter_nine. Uma opção seria ele clonar o repositório remoto em seu computador local e fazer o checkout do branch para visualizar meu arquivo atualizado. No entanto, essa não é uma maneira fácil para ele fornecer feedback e comentários. Suponhamos também que meu editor ainda não tivesse aprendido a usar o Git e, portanto, tivesse dificuldades para clonar e trabalhar em um repositório local.

Em vez disso, me certificarei de que meu editor tenha acesso ao repositório remoto e, então, farei um pull request no repositório para executar o merge do branch remoto chapter_nine no branch remoto main. Posso enviar para meu editor a URL do pull request ou simplesmente dizer a ele que vá até o repositório remoto e encontre o pull request chamado “Chapter 9 updates”.

Assim meu editor poderá usar o recurso de comentário para fazer perguntas e fornecer feedback. Suponhamos que ele detectasse uma inconsistência no capítulo que precisasse de correção e deixasse um comentário no pull request para eu saber que tenho de ver seu feedback. Após resolver o problema no capítulo em meu repositório local, farei outro commit no branch local chapter_nine (commit Y) e o enviarei para o repositório remoto. Isso atualizará o branch remoto chapter_nine e, portanto, também atualizará automaticamente o pull request. A Figura 12-2 mostra o estado atualizado dos repositórios locais e remoto.

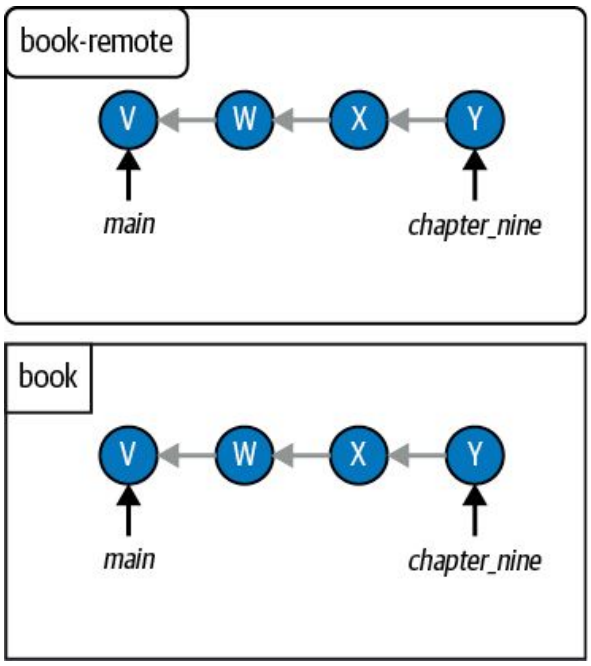


Figura 12-2: Projeto Book depois que fiz o commit Y no branch chapter_nine e o enviei para o repositório remoto.

O fato de eu ter feito o pull request tornará mais fácil para meu editor fornecer feedback em meu projeto Book e para eu compartilhar minhas atualizações após incorporar o feedback.

O Exemplo 12-1 do Projeto Book mostrou por que os pull requests são muito úteis. Agora veremos como eles integram o trabalho.

Entendendo como os Pull Requests são integrados

Como você sabe, existem dois tipos de merges no Git: um *merge fast-forward* ocorre quando os históricos de desenvolvimento do branch de origem e do branch de destino não divergem, enquanto, se eles divergirem, ocorrerá um *merge three-way* e um commit de merge será feito. Você viu exemplos de execução dos dois tipos de merges nos capítulos 5 e 9, respectivamente.

Por padrão, fazer um merge remotamente é diferente de fazê-lo localmente. A configuração padrão da maioria dos serviços de hospedagem é a de que um merge remoto com um pull request ocorrerá com uma opção de merge chamada *non-fast-forward*. Com essa opção, mesmo se os históricos de desenvolvimento do branch de origem e do branch de destino *não* divergirem, um commit de merge *também* será feito.

Os merges feitos com a opção non-fast-forward também são chamados de

merges explícitos. Isso ocorre porque eles sempre mostram explicitamente onde aconteceu um merge com um commit de merge.

É possível alterar a configuração no serviço de hospedagem para que o merge seja feito com uma opção ou abordagem diferente. No entanto, já que a opção non-fast-forward é a mais comum, é ela que examinaremos neste livro. Revisitaremos o cenário do Exemplo 12-1 do Projeto Book no Exemplo 12-2 para ver como geralmente os pull requests integram trabalho.

Exemplo 12-2 do Projeto Book

No Exemplo 12-1 do Projeto Book, descrevi como fiz um pull request chamado “Chapter 9 updates” para meu editor poder revisar o trabalho feito no branch chapter_nine que eu queria integrar ao branch main. A Figura 12-3 mostra o estado atual do projeto Book.

Após meu editor revisar o trabalho mais recente que enviei para o branch remoto chapter_nine, que é representado pelo commit Y, ele não terá mais nenhum feedback para me dar. Logo, selecionará o botão no site do serviço de hospedagem para aprovação do pull request.

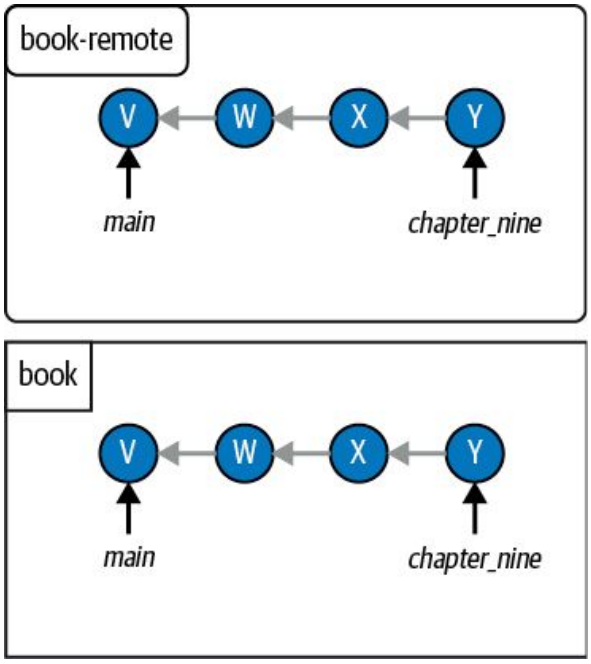


Figura 12-3: Estado dos repositórios local e remoto no projeto Book.

Isso significa que posso fazer o merge do branch remoto chapter_nine no branch remoto main selecionando o botão no site para integração do pull request. Já que estou usando as configurações padrão, esse será um merge non-fast-forward. Mesmo se os históricos de desenvolvimento do branch remoto chapter_nine e do branch remoto main não divergirem, um commit de merge será feito. Ele é representado como commit M na Figura 12-4.

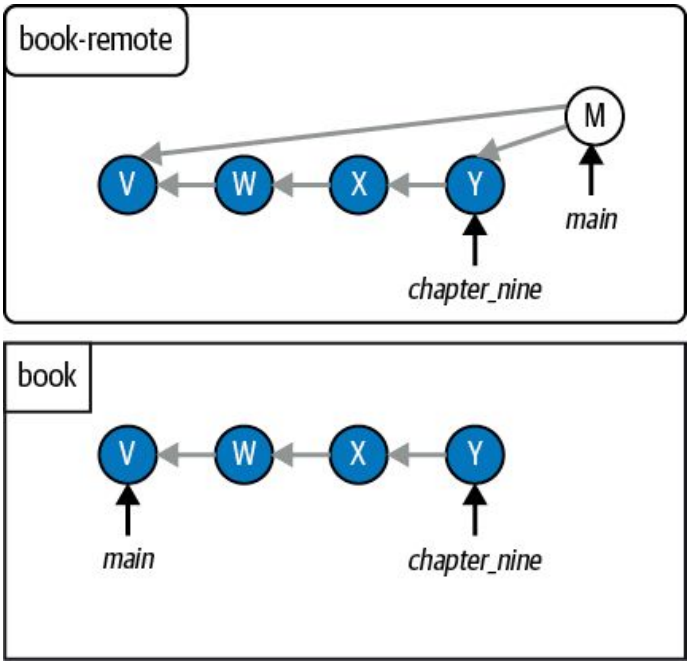


FigurA 12-4: Projeto Book depois que fiz o merge do pull request para integrar o branch chapter_nine ao branch main.

Os commits pais do commit de merge M são o commit V, para onde o branch main estava apontando antes do merge, e o commit Y, o commit mais recente no branch chapter_nine.

As últimas ações que precisam ser executadas são excluir o branch chapter_nine e fazer o pull da versão mais atualizada do branch main do repositório remoto para meu repositório local, de modo que meu branch local main fique em sincronia com o repositório remoto. Isso é mostrado na Figura 12-5.

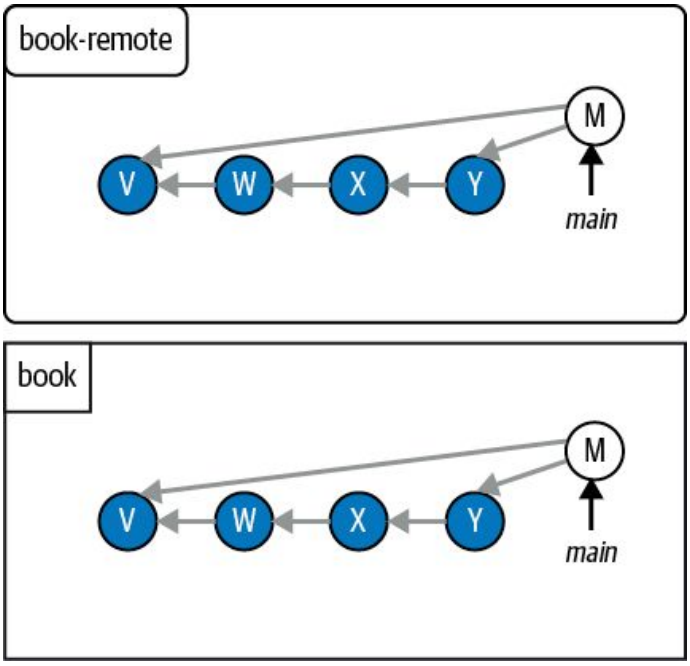


FigurA 12-5: Projeto Book depois que excluí o branch chapter_nine e fiz o pull das alterações do

branch remoto main para meu branch local main.

Agora que vimos uma demonstração de um pull request no Exemplo 12-2 do Projeto Book, passaremos para o projeto Rainbow para que você possa executar um processo de pull request.

Preparando-se para fazer um pull request

Para percorrer um exemplo de pull request, você começará executando as etapas 1 e 2 do processo (descritas em “Introdução aos Pull Requests”, na página 239) no repositório rainbow.

Na seção Acompanhe 12-1, usaremos o comando git switch com a opção -c, introduzido em “Criando um branch e mudando para ele ao mesmo tempo”, na página 101, para podermos criar e mudar imediatamente para um novo branch chamado topic. Em seguida, adicionaremos algum trabalho a ele.

[NOTA]

Você usará o nome genérico topic para o novo branch porque, como mencionado em “Por que usamos branches?”, na página 62, é comum nos referirmos a branches que são criados para o trabalho em uma parte específica de um projeto no Git de *branches de tópico* (ou *branches de recurso*). Em um projeto Git real, geralmente o nome do branch contém uma breve descrição do recurso ou tópico em que estamos trabalhando.

[ACOMPANHE 12-1]

❶ rainbow $ **git switch -c topic**
Switched to a new branch 'topic'

❷ Abra o diretório de projeto rainbow em seu editor de texto. No arquivo othercolors.txt, na linha 4, adicione “Pink is not a color in the rainbow.” e salve o arquivo.

❸ rainbow $ **git add othercolors.txt**

❹ rainbow $ **git commit -m "pink"**
[topic 4c35a5c] pink
1 file changed, 1 insertion(+), 1 deletion(-)

❺ rainbow $ **git log**
commit 4c35a5c02c3dc03f044cbdfdbb0ae55161af6a86 (**HEAD -> topic**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Jul 3 14:16:01 2022 +0200
pink
commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (**origin/main**, **main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:10:11 2022 +0100
rainbow

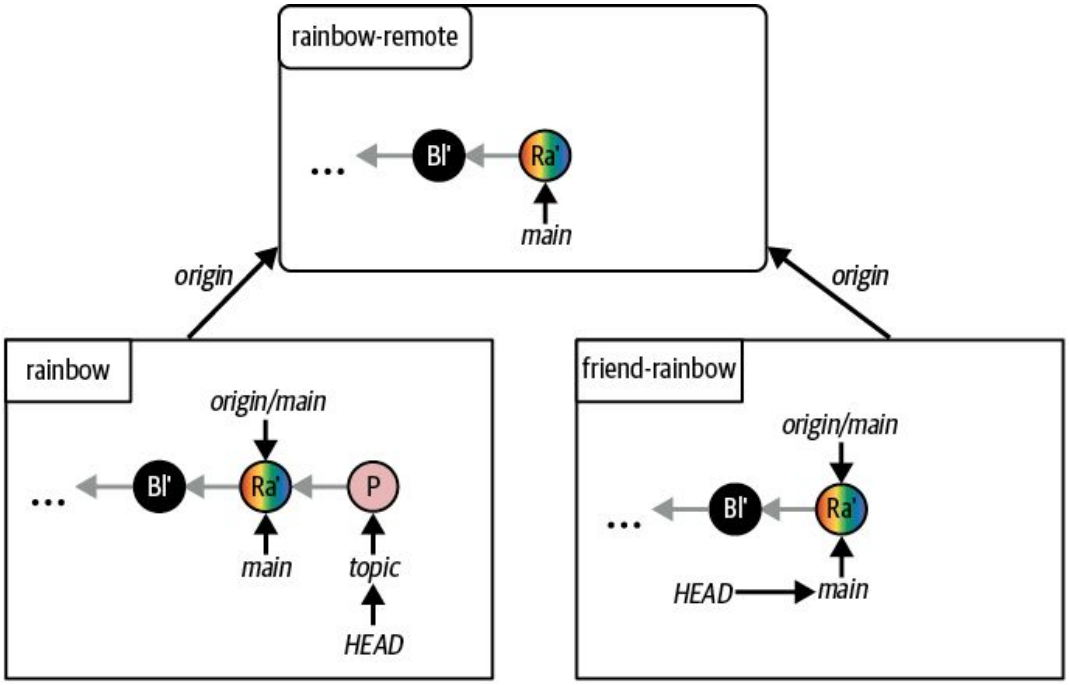
O que deve ser observado:

- Você criou o commit pink (rosa) no branch topic do repositório rainbow.

Isso é mostrado na seção Visualize 12-3.

O branch topic é um novo branch local. Nenhum branch upstream foi definido para ele. (Lembre-se de que um *branch upstream* é o branch remoto que um branch local específico rastreia; você conheceu esse conceito no Capítulo 7 e aprendeu a definir um branch upstream no Capítulo 9).

[VISUALIZE 12-3]



Projeto Rainbow depois que você fez o commit pink no branch topic do repositório rainbow.

Agora você executará a etapa 3 do processo de pull request, que é fazer o push de seu trabalho para o repositório remoto. No decorrer da tarefa, introduzirei uma maneira mais fácil de definir branches upstream.

Uma maneira mais fácil de definir branches upstream

No Capítulo 9, você aprendeu a usar o comando git branch -u <nome_remoto>/<nome_branch> para definir um branch upstream. No entanto, como estamos nos aproximamos do fim deste livro, contarei um pequeno segredo. É comum os usuários do Git se esquecerem de definir branches upstream. Ou não os definirem por pura preguiça.

Contudo, existe um truque útil que os usuários do Git costumam usar para definir branches upstream quando fazem o push de um novo branch para um repositório remoto. Se você usar o comando git push sem nenhum argumento em um branch que *não* tenha um branch upstream definido, o Git emitirá um aviso na saída do comando. O aviso fornecerá o comando a ser usado na definição de um branch upstream (git push --set-upstream

<nome_remoto> <nome_branch>) e sugerirá um *nome_remoto* e um nome de branch (o Git presumirá que você deseja que o branch remoto tenha o mesmo nome do branch local). Se você copiar e colar esse comando na linha de comando e executá-lo, poderá realizar duas tarefas de uma só vez: fará o push do branch local para o repositório remoto e definirá um branch upstream para ele.

Consulte a seção Acompanhe 12-2 para treinar isso em um exemplo.

[ACOMPANHE 12-2]

❶ rainbow $ **git branch -vv**

main 7c09136 [origin/main] rainbow

* topic 4c35a5c pink

❷ rainbow $ **git push**

fatal: The current branch topic has no upstream branch.

To push the current branch and set the remote as upstream, use

git push --set-upstream origin topic

❸ Copie o comando git push --set-upstream origin topic incluído na saída do comando git push e cole-o na linha de comando.

❹ rainbow $ **git push --set-upstream origin topic**

Enumerating objects: 5, done.

Counting objects: 100% (5/5), done.

Delta compression using up to 4 threads

Compressing objects: 100% (3/3), done.

Writing objects: 100% (3/3), 314 bytes | 314.00 KiB/s, done.

Total 3 (delta 1), reused 0 (delta 0), pack-reused 0

remote: Resolving deltas: 100% (1/1), completed with 1 local object.

remote:

remote: Create a pull request for 'topic' on GitHub by visiting:

remote: https://github.com/gitlearningjourney/rainbow-remote/pull/new/topic

remote:

To github.com:gitlearningjourney/rainbow-remote.git

* [new branch] topic -> topic

branch 'topic' set up to track 'origin/topic'.

❺ rainbow $ **git branch -vv**

main 7c09136 [origin/main] rainbow

* topic 4c35a5c [origin/topic] pink

❻ rainbow $ **git log**

commit 4c35a5c02c3dc03f044cbdfdbb0ae55161af6a86 (**HEAD -> topic**, **origin/topic**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sun Jul 3 14:16:01 2022 +0200

pink

commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (**origin/main**, **main**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sun Feb 20 10:10:11 2022 +0100

rainbow

❼ Acesse o repositório rainbow-remote em seu serviço de hospedagem e atualize a página. Você deve ver o novo branch topic que criou, e, se selecionar visualizar os commits no branch topic, também verá o commit pink.

O que deve ser observado:

- Na etapa 2, a saída do comando `git push` está exibindo um aviso: `The current branch topic has no upstream branch` (O branch topic atual não tem um branch upstream). Ela também está fornecendo as instruções a seguir: `To push the current branch and set the remote as upstream, use git push --set-upstream origin topic` (Para fazer o push do branch atual e definir o branch remoto como branch upstream, use git push --set-upstream origin topic).
- Na etapa 4, a saída de `git push` recomenda que você faça um pull request para o novo branch: `Create a pull request for 'topic' on GitHub by visiting: https://github.com/gitlearningjourney/rainbow-remote/pull/new/topic`.

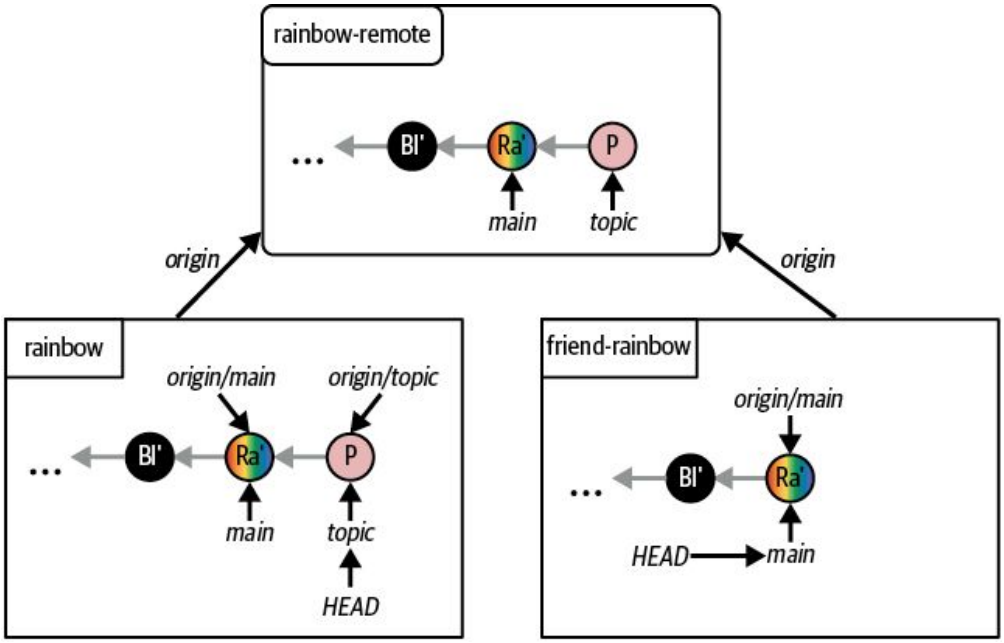
O estado atual dos repositórios depois da seção Acompanhe 12-2 é mostrado na seção Visualize 12-4.

Você acabou de ver como podemos usar o comando `git push` para definir facilmente um branch upstream fazendo ao mesmo tempo o push de um branch para o repositório remoto. Agora executará a etapa 4 do processo de pull request.

[NOTA]

Esse truque presume que você deseja que o branch upstream de seu branch local tenha o mesmo nome no repositório remoto. Se não for esse o caso, você pode ter de editar o comando git push que o Git forneceu para indicar o branch remoto alternativo que quer definir como branch upstream.

[VISUALIZE 12-4]



Projeto Rainbow depois que você fez o push do branch topic para o repositório rainbow-remote e definiu ao mesmo tempo o branch upstream para o branch topic local.

Criando um pull request em um serviço de hospedagem

Quando você criar um pull request, precisará definir o branch de origem e o branch de destino. No exemplo do projeto Rainbow, topic é o branch de origem e main é o branch de destino. Em outras palavras, você está fazendo o merge de topic em main. Para criar o pull request, será preciso usar a UI de seu serviço de hospedagem. Se quiser acessar recursos adicionais, vá até o repositório de *Aprendendo Git* (https://github.com/gitlearningjourney/learning-git).

Na página web de criação do pull request, você poderá inserir informações sobre ele. O único campo obrigatório na maioria dos serviços de hospedagem é um título. Como ocorre com as mensagens de commit e os nomes de branches, se estiver trabalhando com uma equipe em um projeto Git, você deve verificar com os membros se eles têm convenções estabelecidas para os títulos de pull requests. Para o exemplo do projeto Rainbow deste livro usaremos “Adding the color pink” e deixaremos todos os outros campos vazios.

No GitLab e no Bitbucket, ao fazer um pull request também podemos selecionar uma opção que excluirá automaticamente o branch que estamos integrando após o merge do pull request. Você deve optar se deseja selecionar essa opção (se ela estiver disponível em seu serviço de hospedagem). Seja como for, após fazer o merge do pull request nos certificaremos de que o branch remoto seja excluído (se ele ainda estiver presente), já que essa é a etapa 8 do processo.

Consulte a seção Acompanhe 12-3 para criar seu pull request.

[ACOMPANHE 12-3]

❶ Acesse seu repositório remoto no serviço de hospedagem. Siga as etapas de criação de um pull request para fazer o merge de topic (o branch de origem) em main (o branch de destino). O serviço de hospedagem pode preencher automaticamente o campo de título do pull request com a mensagem do último commit, que no seu caso é “pink”. Você deve alterar o título para “Adding the color pink”. Para acessar recursos adicionais, vá até o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*).

Agora que você criou seu primeiro pull request, é hora de passar para as etapas 5 e 6 do processo e submetê-lo à revisão e aprovação.

Revisando e aprovando um pull request

Os pull requests dão a oportunidade de os colaboradores de um projeto revisarem nosso trabalho, fazerem contribuições ou simplesmente o aprovarem. Geralmente a UI do serviço de hospedagem fornece uma maneira de visualizar as linhas que foram alteradas em cada arquivo que foi modificado e como elas foram alteradas, usando cores e símbolos para exibir essas informações. A Figura 12-6 mostra como um serviço de hospedagem poderia indicar que você adicionou uma frase na linha 4 do arquivo othercolors.txt sobre a cor pink (rosa).



Figura 12-6: Exemplo de como um serviço de hospedagem exibe os arquivos alterados em um pull request.

Quando os colaboradores revisarem seu trabalho, eles poderão optar por deixar comentários no pull request. Se seu pull request tiver comentários, você poderá revisá-los, fazer alterações e executar o push de um ou mais commits adicionais para o branch que está integrando. Isso atualizará automaticamente o pull request com os novos commits.

Um colaborador também pode decidir executar o pull do branch para seu repositório local e fazer outras alterações no branch adicionando commits e enviando-os para o repositório remoto. Mais uma vez, o pull request será

atualizado automaticamente com os commits adicionais.

No exemplo do projeto Rainbow, suponhamos que você dissesse ao seu amigo que fez o pull request e pedisse a ele que o revisasse. Seu amigo irá até o pull request e o examinará, decidirá se concorda com as alterações que você fez da maneira como estão e aprovará o pull request.

Normalmente, quando um colaborador revisa nosso pull request, ele faz login em sua conta no serviço de hospedagem, executa a revisão, e aprova selecionando o botão “aprovar” na UI. No entanto, já que você está simulando ser duas pessoas, esse botão “aprovar” pode não aparecer. Nesse caso, será preciso simular que seu amigo aprovou o pull request.

Consulte a seção Acompanhe 12-4 agora para simular que você é seu amigo revisando e aprovando o pull request.

[ACOMPANHE 12-4]

❶ Simule que você é seu amigo e vá até o pull request em seu serviço de hospedagem.

❷ No pull request, visualize os arquivos que foram alterados. Se necessário, consulte a documentação do serviço de hospedagem para ver detalhes de como fazer isso.

❸ “Aprove” o pull request.

Após o pull request ser aprovado (se não literalmente, pelo menos no sentido figurado), você poderá passar para a etapa 7 do processo.

Fazendo o merge de um pull request

Agindo como você mesmo novamente, volte ao seu serviço de hospedagem e execute as etapas para fazer o merge do pull request. Geralmente isso demanda apenas ir até o pull request na UI e selecionar o botão relevante. Consulte a seção 12-5 para fazer o merge de seu pull request agora.

[ACOMPANHE 12-5]

❶ Siga as etapas de seu serviço de hospedagem para fazer o merge do pull request.

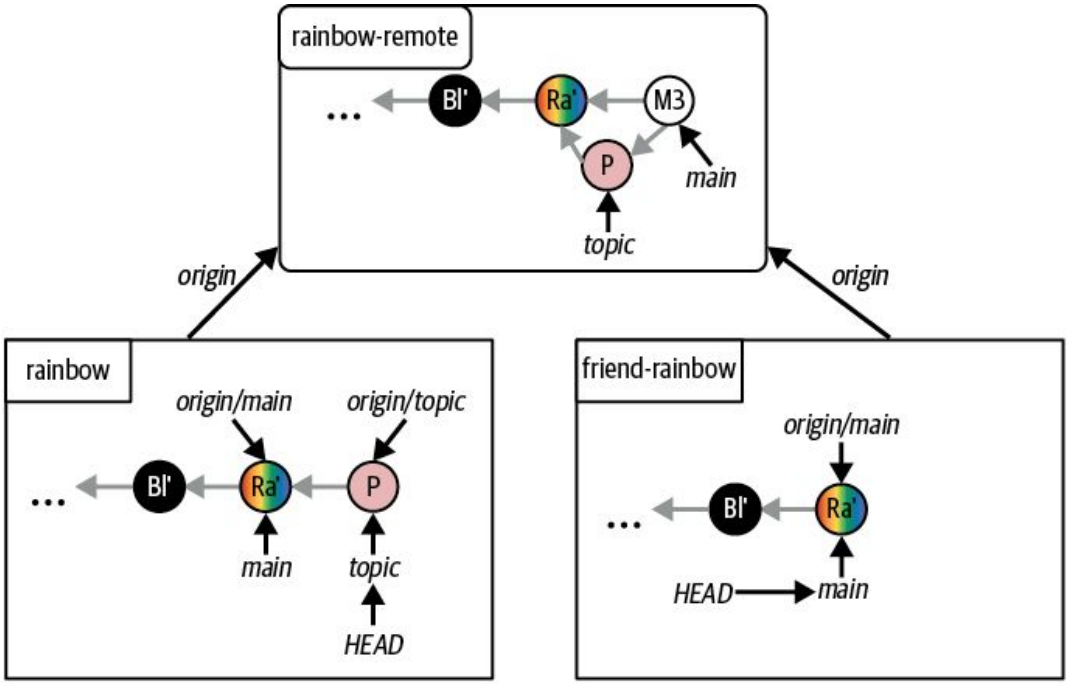
❷ Revise a lista de commits no repositório remoto. Encontre o commit de merge mais recente e selecione-o. Anote seu hash e os hashes de seus dois commits pais.

O que deve ser observado:

- Você fez o merge do branch remoto `topic` no branch remoto `main` e criou um commit de merge.

Isso pode ser visto na seção Visualize 12-5; o novo commit de merge chama-se M3.

[VISUALIZE 12-5]



Projeto Rainbow depois que você fez o merge do pull request no repositório remoto.

Como mencionado anteriormente neste capítulo, esse merge foi executado com a opção padrão non-fast-forward. Isso significa que, mesmo se os históricos de desenvolvimento do branch remoto topic e do branch remoto main não divergirem, o merge criará um commit de merge.

Na seção Visualize 12-5, podemos ver que o branch remoto topic ainda está no repositório rainbow-remote e o branch local topic e o branch de rastreamento remoto origin/topic continuam no repositório rainbow. Na oitava etapa do processo de pull request, excluiremos o branch remoto topic.

Excluindo branches remotos

Se o branch que você integrou em um pull request for um branch de tópico, é comum que ele seja excluído após o pull request ser integrado porque presumiremos que o trabalho nesse branch foi concluído. Para trabalhos novos, você criará novos branches e executará o processo de pull request novamente. Isso manterá o repositório remoto organizado e sem branches antigos.

Se no momento da criação do pull request o serviço de hospedagem lhe

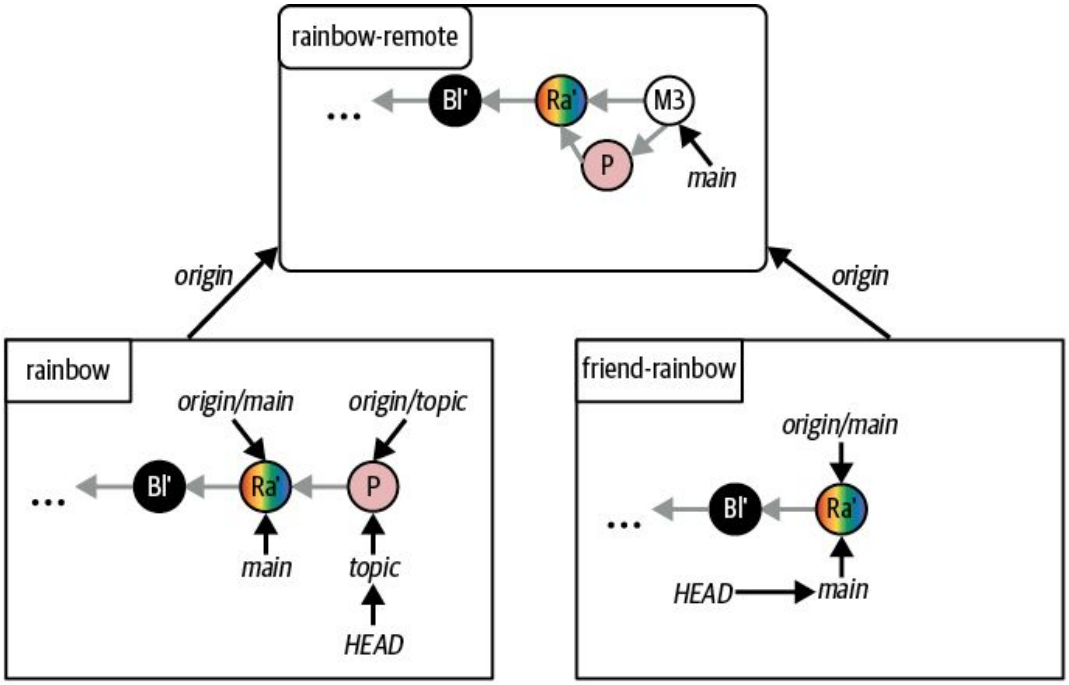
ofereceu a opção de exclusão automática do branch uma vez que o pull request tiver sido integrado, você pode não ter mais um branch remoto topic. Se não for esse o caso, ou se você preferiu não selecionar essa opção, consulte a seção Acompanhe 12-6 para excluir o branch remoto topic.

[ACOMPANHE 12-6]

❶ Acesse seu repositório remoto no serviço de hospedagem e exclua o branch remoto topic. Se necessário, consulte a documentação do serviço de hospedagem para ver detalhes de como fazer isso.

A seção Visualize 12-6 mostra o estado dos repositórios após a exclusão do branch remoto topic.

[VISUALIZE 12-6]



Projeto Rainbow depois que você excluiu o branch remoto topic.

Como você pode ver, agora os branches locais main dos dois repositórios locais estão fora de sincronia com o branch main do repositório remoto. Além disso, o branch local topic e o branch de rastreamento remoto origin/topic ainda estão no repositório rainbow. A seguir, você se certificará de que os envolvidos façam o pull das alterações mais recentes do repositório remoto para que a linha primária de desenvolvimento do projeto Rainbow, o branch main, fique atualizado nos dois repositórios locais. Também limpará o repositório rainbow excluindo o branch local topic e o branch de rastreamento remoto origin/topic.

Sincronizando os repositórios locais e fazendo a limpeza

A última etapa do processo de pull request é você e seu amigo sincronizarem seus repositórios locais com o repositório remoto. Primeiro você sincronizará o repositório rainbow, e já que o branch topic foi integrado, também excluirá o branch local topic e o branch de rastreamento remoto origin/topic.

Você deve se lembrar de que no Capítulo 8 usou o comando git fetch com a opção -p (que é a abreviação de “prune”, ou seja, suprimir) para excluir qualquer branch de rastreamento remoto que correspondesse a branches remotos que foram excluídos do repositório remoto. Dessa vez usará a opção -p com o comando git pull, e ela terá o mesmo efeito. Consulte a seção Acompanhe 12-7 para sincronizar o repositório rainbow.

[ACOMPANHE 12-7]

❶ rainbow $ **git switch main**
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

❷ rainbow $ **git pull -p**
From github.com:gitlearningjourney/rainbow-remote
- [deleted] (none) -> origin/topic
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
Unpacking objects: 100% (1/1), 626 bytes | 626.00 KiB/s, done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
7c09136..2f833d6 main -> origin/main
Updating 7c09136..2f833d6
Fast-forward
othercolors.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)

❸ rainbow $ **git branch -d topic**
Deleted branch topic (was 4c35a5c).

❹ rainbow $ **git log**
commit 2f833d6fa783882c5f832da9e1eafe6d405d3468 (**HEAD -> main**, **origin/main**)
Merge: 7c09136 4c35a5c
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Mon Jul 4 05:50:08 2022 +0200
Merge pull request #1 from gitlearningjourney/topic
Adding the color pink
commit 4c35a5c02c3dc03f044cbdfdbb0ae55161af6a86
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Jul 3 14:16:01 2022 +0200
pink

O que deve ser observado:

- Você excluiu o branch local topic e o branch de rastreamento remoto origin/topic.
- Você atualizou seu branch local main.
- O commit de merge (M3) tem uma mensagem e uma descrição que foram geradas automaticamente pelo serviço de hospedagem. No exemplo deste livro, a mensagem de commit é Merge pull request #1 from gitlearningjourney/topic e a descrição é Adding the color pink, que foi o título do

pull request que você criou. Cada serviço de hospedagem pode ter um template um pouco diferente no qual eles baseiam essa mensagem de commit padrão.

- Os pais do commit de merge M3 são `7c09136` (o commit rainbow) e `4c35a5c` (o commit pink). Como sempre, os hashes de seu repositório serão diferentes dos deste livro porque os hashes de commit são únicos.

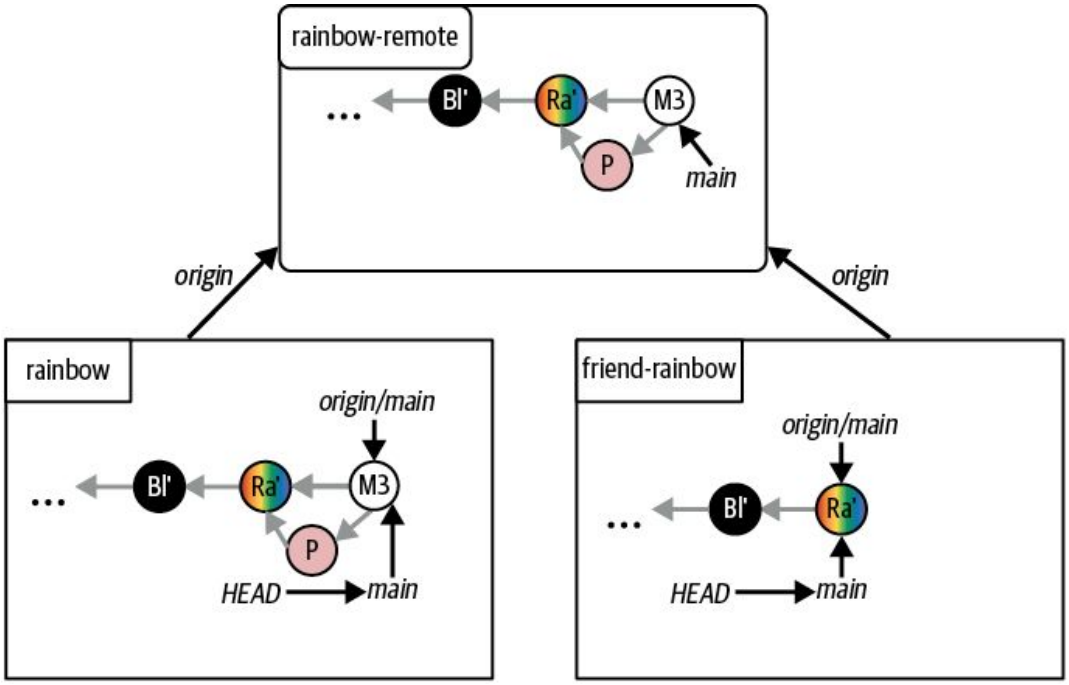
O estado do projeto Rainbow após a seção Acompanhe 12-7 é mostrado na seção Visualize 12-7.

O que deve ser observado:

- No repositório `rainbow`, o branch `main` aponta para o commit de merge mais recente, M3.

Agora, na seção Acompanhe 12-8, seu amigo sincronizará seu branch local `main` com as alterações do repositório remoto.

[VISUALIZE 12-7]



Projeto Rainbow depois que você fez o pull das alterações do branch remoto main para o branch local main no repositório rainbow e excluiu o branch local topic e o branch de rastreamento remoto origin/topic.

[ACOMPANHE 12-8]

❶ friend-rainbow $ **git pull**

```
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 1), reused 2 (delta 0), pack-reused 0
Unpacking objects: 100% (4/4), 831 bytes | 166.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
7c09136..2f833d6 main -> origin/main
Updating 7c09136..2f833d6
Fast-forward
othercolors.txt | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)
```

❷ friend-rainbow $ **git log**

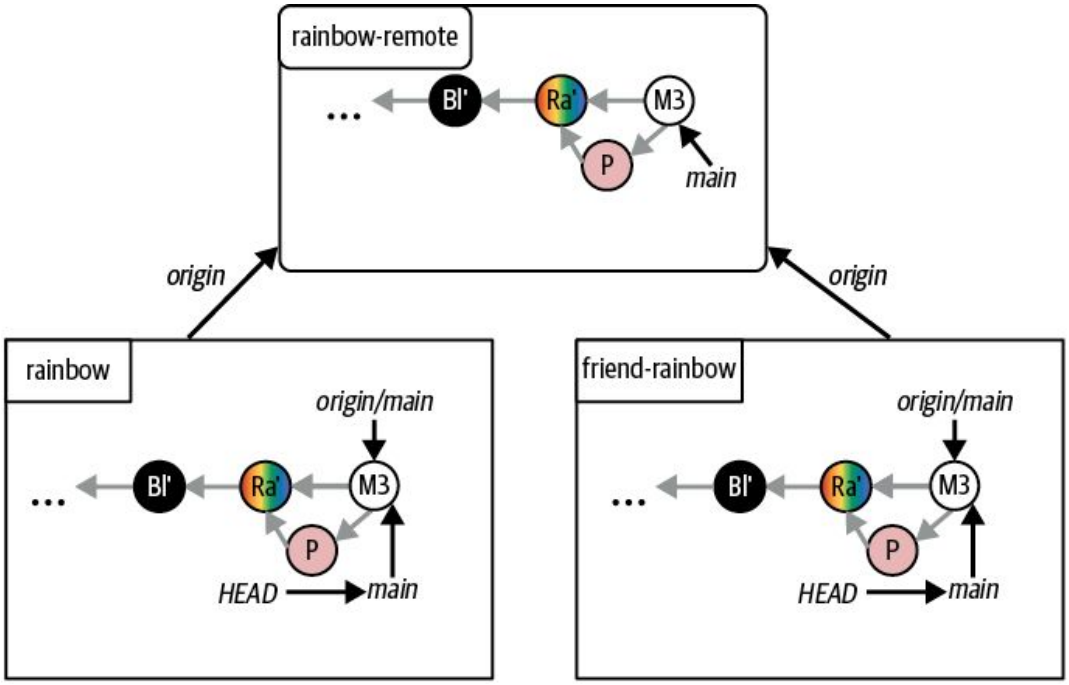
```
commit 2f833d6fa783882c5f832da9e1eafe6d405d3468 (HEAD -> main, origin/main, origin/HEAD)
Merge: 7c09136 4c35a5c
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Mon Jul 4 05:50:08 2022 +0200
Merge pull request #1 from gitlearningjourney/topic
Adding the color pink
commit 4c35a5c02c3dc03f044cbdfdbb0ae55161af6a86
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Jul 3 14:16:01 2022 +0200
pink
```

O que deve ser observado:

- Seu amigo atualizou seu branch local main.

Isso é mostrado na seção Visualize 12-8.

[VISUALIZE 12-8]

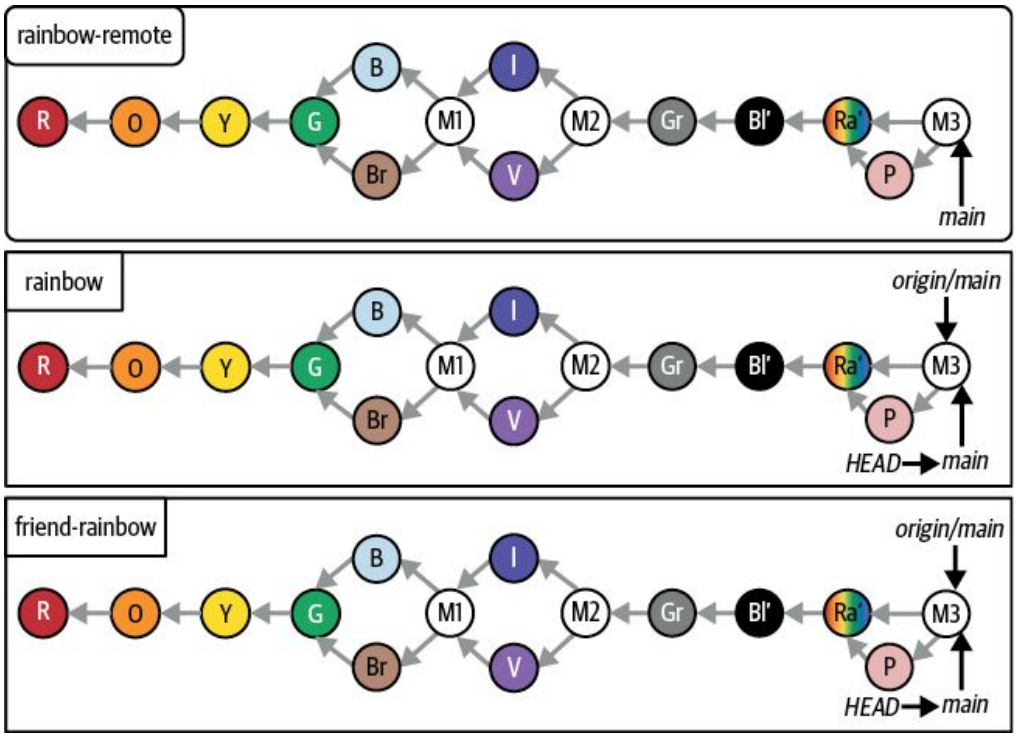


Projeto Rainbow depois que seu amigo fez o pull das alterações do branch remoto main para o branch local main no repositório friend-rainbow.

Estado dos repositórios locais e remoto

Você chegou ao fim da experiência oferecida por *Aprendendo Git*. A seção Visualize 12-9 mostra o estado dos repositórios locais e remoto do projeto Rainbow com todos os commits que foram feitos do Capítulo 1 até o fim do Capítulo 12.

[VISUALIZE 12-9]



Projeto Rainbow no fim do Capítulo 12 com todos os commits desde o Capítulo 1.

Resumo

Neste capítulo, você conheceu os pull requests. Discutimos por que eles são uma ferramenta útil para a colaboração em projetos Git com várias pessoas, já que facilitam a comunicação e o processo de revisão. Você executou todas as nove etapas do processo de pull request em um exemplo no projeto Rainbow. Enquanto trabalhava nesse exemplo, viu que, por padrão, os pull requests são integrados com a opção non-fast-forward, que produz um commit de merge até mesmo no caso de um merge fast-forward em que os branches envolvidos não divergiram. Também conheceu um truque simples em que o Git gerou um comando com o qual você fez o push de um branch para um repositório remoto e definiu ao mesmo tempo um branch upstream para ele.

Leia o Epílogo para ver algumas considerações finais e certifique-se de examinar os apêndices, que fornecem materiais de consulta úteis.

Epílogo

Parabéns – você chegou ao fim de *Aprendendo Git*! Espero que este livro o tenha ajudado em sua jornada de aprendizado do Git e que agora você tenha certeza de que detém um modelo mental sólido dos aspectos básicos de como o Git funciona.

Embora estejamos no fim do livro, esse é apenas o início de sua aventura com o Git. A próxima etapa é começar a usar o Git para controlar seus projetos com versões. Durante o processo, você aprenderá a utilizar vários outros recursos que existem no universo do Git. Também terá de descobrir que fluxo de trabalho do Git funcionará para você ou sua equipe, ou, se já estiver em uma equipe, aprender como o fluxo de trabalho que a equipe usa no Git funciona.

Agora que você fez toda a jornada de aprendizado trabalhando no projeto Rainbow dos capítulos 1 ao 12, talvez queira revisitar o conteúdo de um capítulo específico ou fazer algum dos exercícios novamente. Nesse caso, consulte o Apêndice A para aprender a criar a configuração mínima necessária do projeto Rainbow a fim de poder examinar mais uma vez qualquer um dos capítulos.

Para concluir, se este livro o ajudou, sinta-se à vontade para compartilhá-lo com outras pessoas para que ele possa ajudá-las na jornada de aprendizado do Git.

A

Pré-requisitos dos capítulos

Aprendendo Git é uma experiência de aprendizado prática projetada para ser lida do Capítulo 1 ao Capítulo 12 de maneira linear. No decorrer do livro, você trabalhará no projeto Rainbow para aprender como o Git funciona.

No entanto, podem ocorrer situações em que você queira ou precise começar em um capítulo específico. Por exemplo:

- Você fez os exercícios do livro inteiro uma vez e deseja revisá-los de um capítulo específico em diante.
- Algo deu errado no projeto Rainbow em um capítulo anterior e você não conseguiu resolver; portanto, deseja continuar a partir de um novo capítulo.

Nesse caso, poderá usar as instruções da seção relevante deste apêndice para recriar o projeto Rainbow com a aparência que ele teria no começo do capítulo a partir do qual deseja começar. Isso fornecerá a configuração mínima necessária para você continuar trabalhando no projeto Rainbow a partir do capítulo que escolheu. Para obter mais informações e explicações detalhadas sobre as etapas de cada seção Acompanhe, consulte o conteúdo do capítulo em questão.

[NOTA]

Para os capítulos 9 a 11, você não recriará todos os commits que foram feitos nos capítulos anteriores. Só recriará um commit semelhante ao último commit que foi feito antes do capítulo a partir do qual deseja começar. Isso significa que os repositórios do seu projeto Rainbow terão menos commits que os do projeto Rainbow mostrados nos diagramas da seção Visualize do conteúdo dos capítulos. Lembre-se disso ao trabalhar nesses capítulos.

Todas as instruções presumem que você executou pelo menos uma vez as ações do Capítulo 1 no computador que está usando para os exercícios do projeto Rainbow. Elas consistem em instalar o Git, selecionar um editor de texto e definir as variáveis user.name e user.email. Se não as tiver executado, você terá de começar com a seção Acompanhe de “Pré-requisitos de configuração para todos os capítulos”, na página 262.

As instruções usam os mesmos nomes que foram usados para os repositórios no restante do livro: rainbow, friend-rainbow e rainbow-remote. Se você ainda tiver esses repositórios em seu computador local e no serviço de hospedagem, terá de utilizar nomes um pouco diferentes, por exemplo, rainbow1, friend-rainbow1 e rainbow-remote1. Recomendo que empregue nomes semelhantes, porque esses nomes são mencionados em todo o conteúdo de cada capítulo.

Pré-requisitos de configuração para todos os capítulos

Se você não está com o Git instalado ou não tem um editor de texto pronto para ser usado, execute as etapas da seção Acompanhe A-1. Caso contrário, pode saltá-la.

[ACOMPANHE A-1]

❶ Acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) e siga as etapas para baixar o Git para seu sistema operacional.

❷ Selecione seu editor de texto preferido. Se ainda não tiver um editor de texto em seu computador, baixe um. Para obter mais informações, consulte “Preparando um editor de texto”, na página 36.

Agora você deve definir as variáveis de configuração user.name e user.email com os valores apropriados. Se ainda não fez isso, consulte a seção Acompanhe A-2. Para obter mais informações sobre as variáveis de configuração do Git, leia “Definindo configurações do Git”, na página 34. Certifique-se de fornecer seu nome e endereço de email em substituição aos placeholders da seção Acompanhe.

[ACOMPANHE A-2]

❶ **$ git config --global user.name** "<nome>"

❷ **$ git config --global user.email** "<email>"

Pré-requisitos de configuração do Capítulo 2

Na seção Acompanhe A-3, você preparará a configuração básica para começar a trabalhar a partir do Capítulo 2.

[ACOMPANHE A-3]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop **$ mkdir rainbow**

❹ desktop **$ cd rainbow**

❺ Abra o diretório de projeto rainbow em uma janela de editor de texto.

Pré-requisitos de configuração do Capítulo 3

Na seção Acompanhe A-4, você preparará a configuração básica do repositório rainbow para começar a trabalhar a partir do Capítulo 3.

[ACOMPANHE A-4]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop $ mkdir rainbow

❹ desktop $ cd rainbow

❺ rainbow $ git init -b main

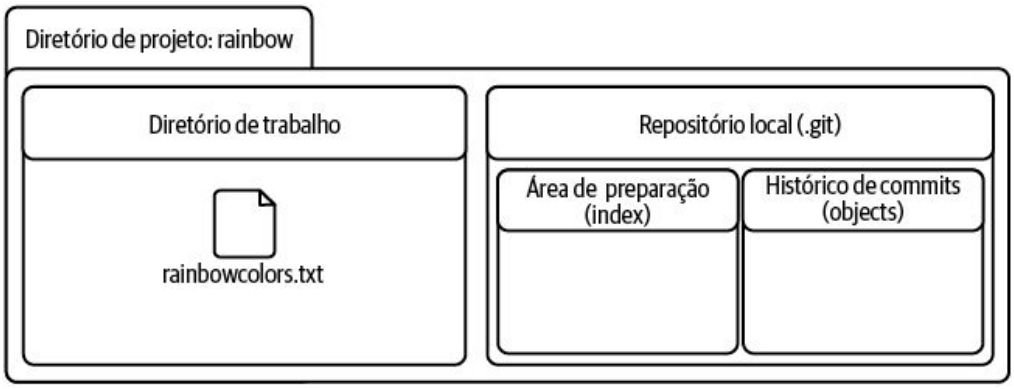
Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❻ Abra o diretório de projeto rainbow em uma janela do editor de texto e crie um arquivo chamado rainbowcolors.txt dentro desse diretório.

❼ No arquivo rainbowcolors.txt em seu editor de texto, adicione “Red is the first color of the rainbow.” na linha 1 e salve o arquivo.

No fim desse processo, o repositório rainbow que você criou conterá o arquivo rainbowcolors.txt no diretório de trabalho, como mostrado na seção Visualize A-1.

[VISUALIZE A-1]



Repositório rainbow recriado para o trabalho começar no Capítulo 3, sem você ter passado pelo Capítulo 2.

Pré-requisitos de configuração do Capítulo 4

Na seção Acompanhe A-5, você preparará a configuração básica do repositório rainbow para começar a trabalhar a partir do Capítulo 4.

[ACOMPANHE A-5]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop **$ mkdir rainbow**

❹ desktop **$ cd rainbow**

❺ rainbow **$ git init -b main**

Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❻ Abra o diretório de projeto rainbow em uma janela do editor de texto e crie um arquivo chamado rainbowcolors.txt dentro desse diretório.

❼ No arquivo rainbowcolors.txt em seu editor de texto, adicione “Red is the first color of the rainbow.” na linha 1 e salve o arquivo.

❽ rainbow **$ git add rainbowcolors.txt**

❾ rainbow **$ git commit -m "red"**

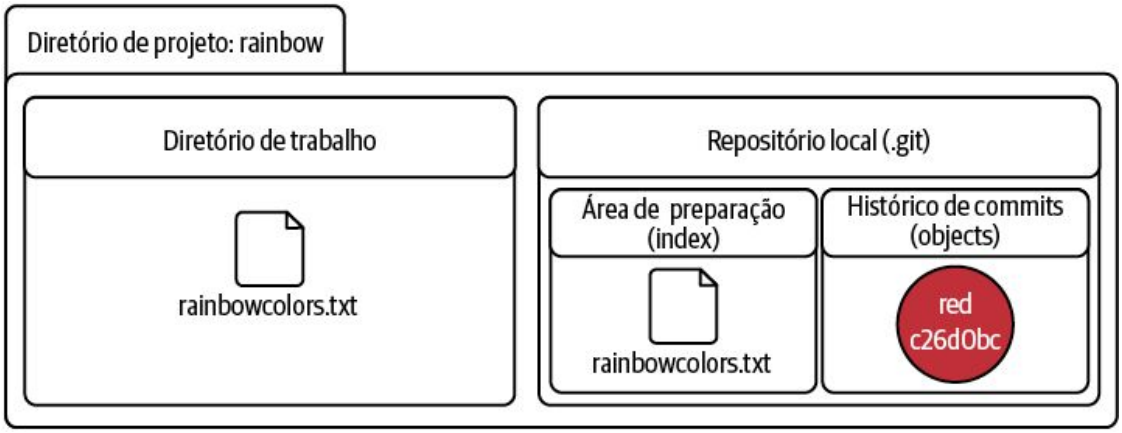
[main (root-commit) c26d0bc] red

1 file changed, 1 insertion(+)

create mode 100644 rainbowcolors.txt

No fim desse processo, o repositório rainbow que você criou conterá o commit red (vermelho), como mostrado na seção Visualize A-2.

[VISUALIZE A-2]



Repositório rainbow recriado para o trabalho começar no Capítulo 4, sem você ter passado pelo Capítulo 3.

Pré-requisitos de configuração do Capítulo 5

Na seção Acompanhe A-6, você preparará a configuração básica do repositório rainbow para começar a trabalhar a partir do Capítulo 5.

[ACOMPANHE A-6]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop **$ mkdir rainbow**

❹ desktop **$ cd rainbow**

❺ rainbow **$ git init -b main**

Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❻ Abra o diretório de projeto rainbow em uma janela do editor de texto e crie um arquivo chamado rainbowcolors.txt dentro desse diretório.

❼ No arquivo rainbowcolors.txt em seu editor de texto, adicione “Red is the first color of the rainbow.” na linha 1 e salve o arquivo.

❽ rainbow **$ git add rainbowcolors.txt**

❾ rainbow **$ git commit -m "red"**

[main (root-commit) c26d0bc] red

1 file changed, 1 insertion(+)

create mode 100644 rainbowcolors.txt

❿ No arquivo rainbowcolors.txt em seu editor de texto, adicione “Orange is the second color of the rainbow.” na linha 2 e salve o arquivo.

 rainbow **$ git add rainbowcolors.txt**

 rainbow **$ git commit -m "orange"**

[main 7acb333] orange

1 file changed, 2 insertions(+), 1 deletion(-)

 rainbow **$ git branch feature**

 rainbow **$ git switch feature**

Switched to branch 'feature'

 No arquivo rainbowcolors.txt em seu editor de texto, adicione “Yellow is the third color of the rainbow.” na linha 3 e salve o arquivo.

 rainbow **$ git add rainbowcolors.txt**

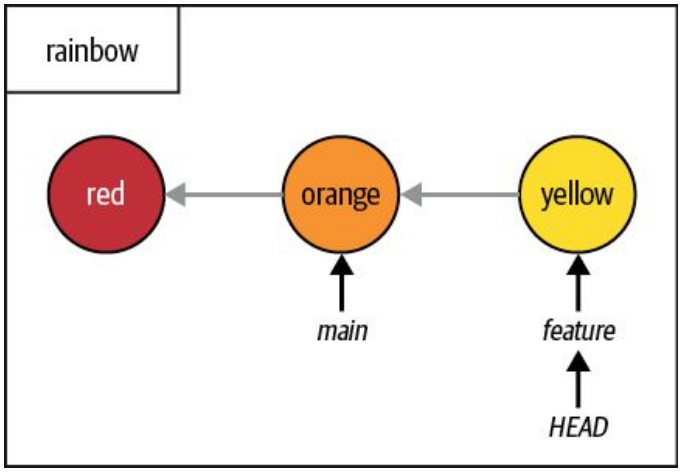
 rainbow **$ git commit -m "yellow"**

[feature fc8139c] yellow

1 file changed, 2 insertions(+), 1 deletion(-)

No fim desse processo, o repositório rainbow que você criou conterá os commits red (vermelho), orange (laranja) e yellow (amarelo), como mostrado na seção Visualize A-3.

[VISUALIZE A-3]



Repositório rainbow recriado para o trabalho começar no Capítulo 5, sem você ter passado pelo Capítulo 4.

Pré-requisitos de configuração dos capítulos 6 e 7

Na seção Acompanhe A-7, você preparará a configuração básica do repositório rainbow para começar a trabalhar a partir do Capítulo 6 ou 7.

[ACOMPANHE A-7]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop **$ mkdir rainbow**

❹ **desktop $ cd rainbow**

❺ rainbow **$ git init -b main**

Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❻ Abra o diretório de projeto rainbow em uma janela do editor de texto e crie um arquivo chamado rainbowcolors.txt dentro desse diretório.

❼ No arquivo rainbowcolors.txt em seu editor de texto, adicione “Red is the first color of the rainbow.” na linha 1 e salve o arquivo.

❽ rainbow **$ git add rainbowcolors.txt**

❾ rainbow **$ git commit -m "red"**

[main (root-commit) c26d0bc] red

1 file changed, 1 insertion(+)

create mode 100644 rainbowcolors.txt

❿ No arquivo rainbowcolors.txt em seu editor de texto, adicione “Orange is the second color of the rainbow.” na linha 2 e salve o arquivo.

 rainbow **$ git add rainbowcolors.txt**

 rainbow **$ git commit -m "orange"**

[main 7acb333] orange

1 file changed, 2 insertions(+), 1 deletion(-)

 rainbow **$ git branch feature**

 rainbow **$ git switch feature**

Switched to branch 'feature'

 No arquivo rainbowcolors.txt em seu editor de texto, adicione “Yellow is the third color of the rainbow.” na linha 3 e salve o arquivo.

 rainbow **$ git add rainbowcolors.txt**

 rainbow **$ git commit -m "yellow"**

[feature fc8139c] yellow

1 file changed, 2 insertions(+), 1 deletion(-)

 rainbow **$ git switch main**

Switched to branch 'main'

 rainbow **$ git merge feature**

Updating 7acb333..fc8139c

Fast-forward

rainbowcolors.txt | 3 ++-

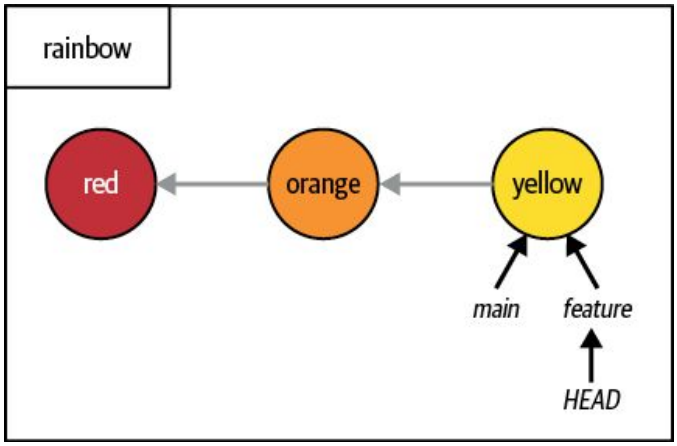
1 file changed, 2 insertions(+), 1 deletion(-)

[NOTA]

Se você estiver usando esta seção para começar a trabalhar no Capítulo 7 e ainda não tiver uma conta de serviço de hospedagem e/ou detalhes de autenticação definidos para o acesso HTTPS ou SSH, deve ir até o Capítulo 6 e fazer os exercícios desse capítulo antes de continuar. Seguir as instruções desta seção não será suficiente para fazer os exercícios do Capítulo 7.

No fim desse processo, o repositório rainbow que você criou conterá os commits red, orange e yellow, como mostrado na seção Visualize A-4.

[VISUALIZE A-4]



Repositório rainbow recriado para o trabalho começar no Capítulo 6 ou 7, sem você ter passado pelo Capítulo 5.

Pré-requisitos de configuração do Capítulo 8

Na seção Acompanhe A-8, você preparará a configuração básica do projeto Rainbow para começar a trabalhar a partir do Capítulo 8. Na etapa 24, certifique-se de inserir o comando inteiro em uma única linha.

[ACOMPANHE A-8]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop **$ mkdir rainbow**

❹ desktop **$ cd rainbow**

❺ rainbow **$ git init -b main**

Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❻ Abra o diretório de projeto rainbow em uma janela do editor de texto e crie um arquivo chamado rainbowcolors.txt dentro desse diretório.

❼ No arquivo rainbowcolors.txt em seu editor de texto, adicione “Red is the first color of the rainbow.” na linha 1 e salve o arquivo.

❽ rainbow **$ git add rainbowcolors.txt**

❾ rainbow **$ git commit -m "red"**

[main (root-commit) c26d0bc] red

1 file changed, 1 insertion(+)

create mode 100644 rainbowcolors.txt

❿ No arquivo rainbowcolors.txt em seu editor de texto, adicione “Orange is the second color of the rainbow.” na linha 2 e salve o arquivo.

 rainbow $ git add rainbowcolors.txt

 rainbow **$ git commit -m "orange"**

[main 7acb333] orange

1 file changed, 2 insertions(+), 1 deletion(-)

 rainbow **$ git branch feature**

 rainbow **$ git switch feature**

Switched to branch 'feature'

 No arquivo rainbowcolors.txt em seu editor de texto, adicione “Yellow is the third color of the rainbow.” na linha 3 e salve o arquivo.

 rainbow **$ git add rainbowcolors.txt**

 rainbow **$ git commit -m "yellow"**

[feature fc8139c] yellow

1 file changed, 2 insertions(+), 1 deletion(-)

 rainbow **$ git switch main**

Switched to branch 'main’

 rainbow **$ git merge feature**

Updating 7acb333..fc8139c

Fast-forward

rainbowcolors.txt | 3 ++-

1 file changed, 2 insertions(+), 1 deletion(-)

 Se você ainda não tiver uma conta de serviço de hospedagem e/ou detalhes de autenticação definidos para o acesso HTTPS ou SSH, deve ir até o Capítulo 6 e concluir essa parte do processo agora. Volte a esta seção Acompanhe ao terminar.

 Faça login na conta de seu serviço de hospedagem.

 Crie um repositório remoto. Para obter mais informações sobre como fazer isso, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) ou vá diretamente para a documentação de seu serviço de hospedagem.

Quando criar o repositório para esse exercício:

- Para o nome do repositório, use rainbow-remote.
- Você pode optar por tornar o repositório público ou privado. Recomento que o torne privado.
- Não inclua nenhum arquivo. Por exemplo, não inclua um arquivo README ou um arquivo .gitignore.
- Se for solicitado a fornecer um nome de branch padrão, deixe o campo em branco ou configure-o com main.

 Quando terminar as etapas de criação do repositório remoto, localize a URL do repositório. Se não souber onde encontrá-la, consulte a documentação de seu serviço de hospedagem.

Existirão duas versões da URL, uma para o acesso HTTPS e a outra para o acesso SSH. Nos exemplos deste livro, as duas URLs do repositório remoto são:

- HTTPS: https://github.com/gitlearningjourney/rainbow-remote.git
- SSH: git@github.com:gitlearningjourney/rainbow-remote.git

Copie a URL do protocolo que escolheu usar. Nas próximas etapas, onde você encontrar a minha URL, em vez dela deve usar a sua.

 rainbow **$ git remote add origin https://github.com/gitlearningjourney/rainbow-remote.git**

 rainbow **$ git push origin main**

```
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 747 bytes | 373.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To github.com:gitlearningjourney/rainbow-remote.git
* [new branch] main -> main
```

 rainbow **$ git switch feature**

```
Switched to branch 'feature'
```

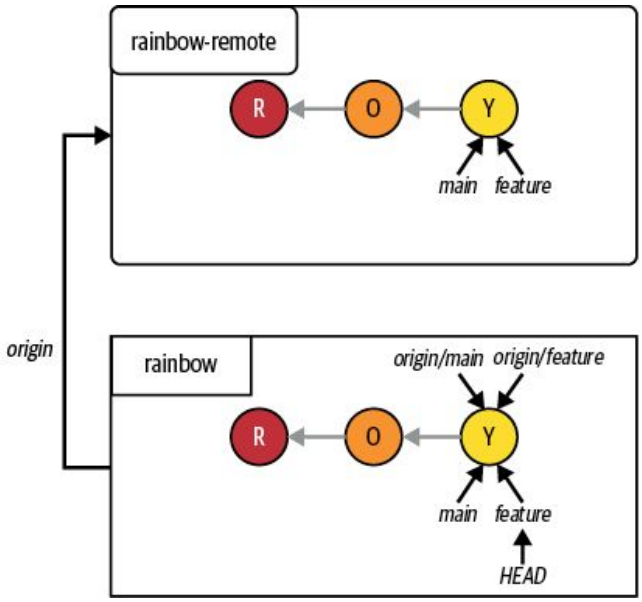
 rainbow **$ git push origin feature**

```
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'feature' on GitHub by visiting:
remote: https://github.com/gitlearningjourney/rainbow-remote/pull/new/feature
remote:
```

```
To github.com:gitlearningjourney/rainbow-remote.git
* [new branch] feature -> feature
```

No fim desse processo, o projeto Rainbow que você criou conterá os commits red, orange e yellow em todos os repositórios, como mostrado na seção Visualize A-5.

[VISUALIZE A-5]



Projeto Rainbow recriado para o trabalho começar no Capítulo 8, sem você ter passado pelo Capítulo 7.

Pré-requisitos de configuração do Capítulo 9

Na seção Acompanhe A-9, você preparará a configuração básica do projeto Rainbow para começar a trabalhar a partir do Capítulo 9. Nas etapas 14 e 17, certifique-se de inserir o comando inteiro em uma única linha.

[ACOMPANHE A-9]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop **$ mkdir rainbow**

❹ desktop **$ cd rainbow**

❺ rainbow **$ git init -b main**

Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❻ Abra o diretório de projeto rainbow em uma janela do editor de texto e crie um arquivo chamado rainbowcolors.txt dentro desse diretório.

❼ Adicione o texto a seguir ao arquivo rainbowcolors.txt em seu editor de texto e salve o arquivo:

Red is the first color of the rainbow.

Orange is the second color of the rainbow.

Yellow is the third color of the rainbow.

Green is the fourth color of the rainbow.

❽ rainbow **$ git add rainbowcolors.txt**

❾ rainbow **$ git commit -m "green"**

[main (root-commit) 4e59074] "green"

1 file changed, 4 insertions(+)

create mode 100644 rainbowcolors.txt

❿ Se você ainda não tiver uma conta de serviço de hospedagem e/ou detalhes de autenticação definidos para o acesso HTTPS ou SSH, deve ir até o Capítulo 6 e concluir essa parte do processo agora. Volte a esta seção Acompanhe ao terminar.

 Faça login na conta de seu serviço de hospedagem.

 Crie um repositório remoto. Para obter mais informações sobre como fazer isso, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) ou vá diretamente para a documentação de seu serviço de hospedagem.

Quando criar o repositório para esse exercício:

- Para o nome do repositório, use rainbow-remote.
- Você pode optar por tornar o repositório público ou privado. Recomento que o torne privado.
- Não inclua nenhum arquivo. Por exemplo, não inclua um arquivo README ou um arquivo .gitignore.
- Se for solicitado a fornecer um nome de branch padrão, deixe o campo em branco ou configure-o com main.

 Quando terminar as etapas de criação do repositório remoto, localize a URL do repositório. Se não souber onde encontrá-la, consulte a documentação de seu serviço de hospedagem.

Existirão duas versões da URL, uma para o acesso HTTPS e a outra para o acesso SSH. Nos exemplos deste livro, as duas URLs do repositório remoto são:

- HTTPS: https://github.com/gitlearningjourney/rainbow-remote.git
- SSH: git@github.com:gitlearningjourney/rainbow-remote.git

Copie a URL do protocolo que escolheu usar. Nas próximas etapas, onde você encontrar a minha URL, em vez dela deve usar a sua.

- rainbow **$ git remote add origin https://github.com/gitlearningjourney/rainbow-remote.git**
- rainbow **$ git push origin main**

  Enumerating objects: 3, done.

  Counting objects: 100% (3/3), done.

  Delta compression using up to 4 threads

  Compressing objects: 100% (2/2), done.

  Writing objects: 100% (3/3), 311 bytes | 311.00 KiB/s, done.

  Total 3 (delta 0), reused 0 (delta 0), pack-reused 0

  To github.com:gitlearningjourney/rainbow-remote.git

  * [new branch] main -> main
- Abra uma nova janela de linha de comando para navegar para o diretório desktop a fim de simular que você é seu amigo agora.
- desktop **$ git clone https://github.com/gitlearningjourney/rainbow-remote.git friend-rainbow**

  Cloning into 'friend-rainbow'...

  remote: Enumerating objects: 3, done.

  remote: Counting objects: 100% (3/3), done.

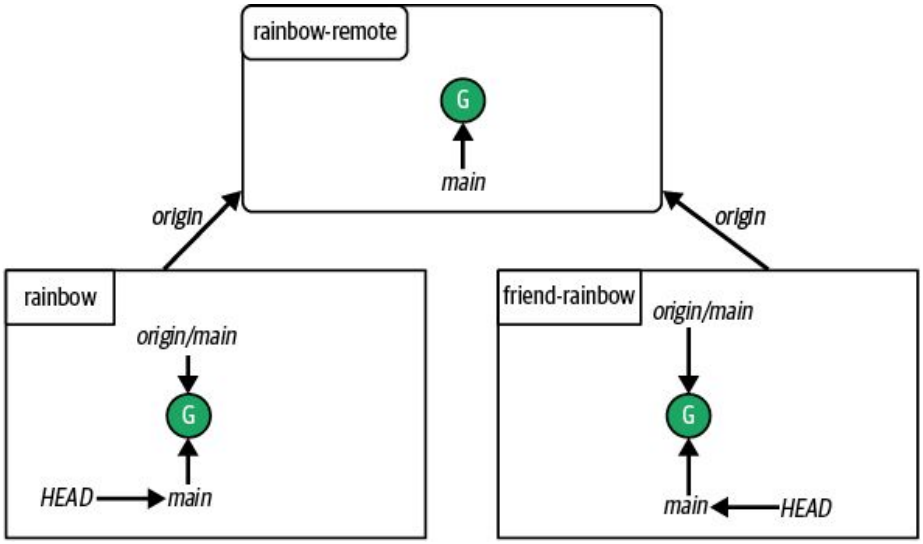
  remote: Compressing objects: 100% (2/2), done.

  remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0

  Receiving objects: 100% (3/3), done.
- desktop **$ cd friend-rainbow**
- Abra o diretório de projeto friend-rainbow em uma nova janela do editor de texto.

No fim desse processo, a configuração mínima do projeto Rainbow que você criou conterá apenas um commit, o commit green (verde), em todos os repositórios, como mostrado na seção Visualize A-6.

[VISUALIZE A-6]



Projeto Rainbow recriado para o trabalho começar no Capítulo 9, sem você ter passado pelo Capítulo 8.

Pré-requisitos de configuração do Capítulo 10

Na seção Acompanhe A-10, você preparará a configuração básica do projeto Rainbow para começar a trabalhar a partir do Capítulo 10. Nas etapas 15 e 19, certifique-se de inserir o comando inteiro em uma única linha.

[ACOMPANHE A-10]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop **$ mkdir rainbow**

❹ desktop **$ cd rainbow**

❺ rainbow **$ git init -b main**

Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❻ Abra o diretório de projeto rainbow em uma janela do editor de texto e crie um arquivo chamado rainbowcolors.txt e outro arquivo chamado othercolors.txt dentro desse diretório.

❼ Adicione o texto a seguir ao arquivo rainbowcolors.txt em seu editor de texto e salve o arquivo:

Red is the first color of the rainbow.

Orange is the second color of the rainbow.

Yellow is the third color of the rainbow.

Green is the fourth color of the rainbow.

Blue is the fifth color of the rainbow.

❽ Adicione o texto a seguir ao arquivo othercolors.txt em seu editor de texto e salve o arquivo:

Brown is not a color in the rainbow.

❾ rainbow **$ git add rainbowcolors.txt othercolors.txt**

❿ rainbow **$ git commit -m "fake merge commit 1"**

2 files changed, 6 insertions(+)

create mode 100644 othercolors.txt

create mode 100644 rainbowcolors.txt

☐ Se você ainda não tiver uma conta de serviço de hospedagem e/ou detalhes de autenticação definidos para o acesso HTTPS ou SSH, deve ir até o Capítulo 6 e concluir essa parte do processo agora. Volte a esta seção Acompanhe ao terminar.

☐ Faça login na conta de seu serviço de hospedagem.

☐ Crie um repositório remoto. Para obter mais informações sobre como fazer isso, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) ou vá diretamente para a documentação de seu serviço de hospedagem.

Quando criar o repositório para esse exercício:

- Para o nome do repositório, use rainbow-remote.
- Você pode optar por tornar o repositório público ou privado. Recomento que o torne privado.
- Não inclua nenhum arquivo. Por exemplo, não inclua um arquivo README ou um arquivo .gitignore.
- Se for solicitado a fornecer um nome de branch padrão, deixe o campo em branco ou configure-o com main.

☐ Quando terminar as etapas de criação do repositório remoto, localize a URL do repositório. Se não souber onde encontrá-la, consulte a documentação de seu serviço de hospedagem.

Existirão duas versões da URL, uma para o acesso HTTPS e a outra para o acesso SSH. Nos exemplos deste livro, as duas URLs do repositório remoto são:

• HTTPS: https://github.com/gitlearningjourney/rainbow-remote.git

• SSH: git@github.com:gitlearningjourney/rainbow-remote.git

Copie a URL do protocolo que escolheu usar. Nas próximas etapas, onde você encontrar a minha URL, em vez dela deve usar a *sua*.

- rainbow **$ git remote add origin https://github.com/gitlearningjourney/rainbow-remote.git**
- rainbow **$ git push origin main**

  Enumerating objects: 4, done.

  Counting objects: 100% (4/4), done.

  Delta compression using up to 4 threads

  Compressing objects: 100% (3/3), done.

  Writing objects: 100% (4/4), 394 bytes | 394.00 KiB/s, done.

  Total 4 (delta 0), reused 0 (delta 0), pack-reused 0

  To github.com:gitlearningjourney/rainbow-remote.git

  * [new branch] main -> main
- rainbow **$ git branch -u origin/main**

  branch 'main' set up to track 'origin/main'.
- Abra uma nova janela de linha de comando para navegar para o diretório desktop a fim de simular que você é seu amigo agora.
- desktop **$ git clone https://github.com/gitlearningjourney/rainbow-remote.git friend-rainbow**

  Cloning into 'friend-rainbow'...

  remote: Enumerating objects: 4, done.

  remote: Counting objects: 100% (4/4), done.

  remote: Compressing objects: 100% (3/3), done.

  remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0

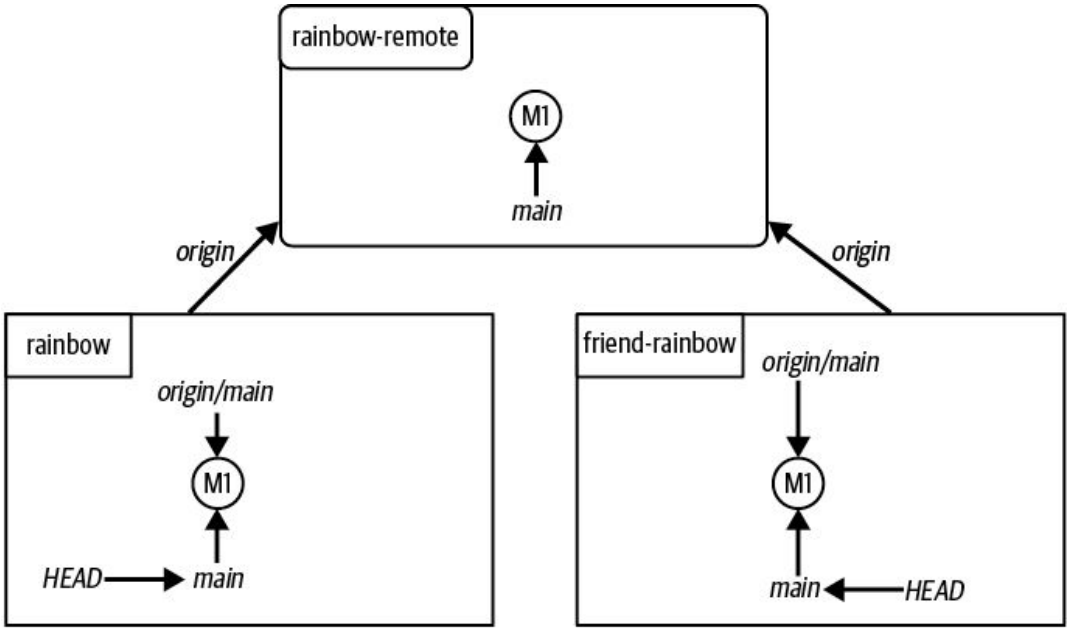
  Receiving objects: 100% (4/4), done.
- desktop **$ cd friend-rainbow**
- Abra o diretório de projeto friend-rainbow em uma nova janela do editor de texto.

No fim desse processo, a configuração mínima do projeto Rainbow que você criou conterá apenas um commit, o commit de merge fictício 1 (M1), em todos os repositórios, como mostrado na seção Visualize A-7.

[NOTA]

O último commit criado no Capítulo 9 foi o commit de merge 1 (M1), que tem dois pais. Seu commit será o único existente nos repositórios; logo, não terá nenhum commit pai. Portanto, é um commit de merge fictício.

[VISUALIZE A-7]



Projeto Rainbow recriado para o trabalho começar no Capítulo 10, sem você ter passado pelo Capítulo 9.

Pré-requisitos de configuração do Capítulo 11

Na seção Acompanhe A-11, você preparará a configuração básica do projeto Rainbow para começar a trabalhar a partir do Capítulo 11. Nas etapas 15 e 19, certifique-se de inserir o comando inteiro em uma única linha.

[ACOMPANHE A-11]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop **$ mkdir rainbow**

❹ desktop **$ cd rainbow**

❺ rainbow **$ git init -b main**

Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❻ Abra o diretório de projeto rainbow em uma janela do editor de texto e crie um arquivo chamado rainbowcolors.txt e outro arquivo chamado othercolors.txt dentro desse diretório.

❼ Adicione o texto a seguir ao arquivo rainbowcolors.txt em seu editor de texto e salve o arquivo:

Red is the first color of the rainbow.

Orange is the second color of the rainbow.

Yellow is the third color of the rainbow.

Green is the fourth color of the rainbow.

Blue is the fifth color of the rainbow.

Indigo is the sixth color of the rainbow.

Violet is the seventh color of the rainbow.

❽ Adicione o texto a seguir ao arquivo othercolors.txt em seu editor de texto e salve o arquivo:

Brown is not a color in the rainbow.

❾ rainbow **$ git add rainbowcolors.txt othercolors.txt**

❿ rainbow **$ git commit -m "fake merge commit 2"**

[main (root-commit) 32fa0b7] fake merge commit 2

2 files changed, 8 insertions(+)

create mode 100644 othercolors.txt

create mode 100644 rainbowcolors.txt

 Se você ainda não tiver uma conta de serviço de hospedagem e/ou detalhes de autenticação definidos para o acesso HTTPS ou SSH, deve ir até o Capítulo 6 e concluir essa parte do processo agora. Volte a esta seção Acompanhe ao terminar.

 Faça login na conta de seu serviço de hospedagem.

 Crie um repositório remoto. Para obter mais informações sobre como fazer isso, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) ou vá diretamente para a documentação de seu serviço de hospedagem.

Quando criar o repositório para esse exercício:

- Para o nome do repositório, use rainbow-remote.
- Você pode optar por tornar o repositório público ou privado. Recomento que o torne privado.
- Não inclua nenhum arquivo. Por exemplo, não inclua um arquivo README ou um arquivo .gitignore.
- Se for solicitado a fornecer um nome de branch padrão, deixe o campo em branco ou

configure-o com main.

- Quando terminar as etapas de criação do repositório remoto, localize a URL do repositório. Se não souber onde encontrá-la, consulte a documentação de seu serviço de hospedagem.

  Existirão duas versões da URL, uma para o acesso HTTPS e a outra para o acesso SSH. Nos exemplos deste livro, as duas URLs do repositório remoto são:

  • HTTPS: https://github.com/gitlearningjourney/rainbow-remote.git

  • SSH: git@github.com:gitlearningjourney/rainbow-remote.git

  Copie a URL do protocolo que escolheu usar. Nas próximas etapas, onde você encontrar a minha URL, em vez dela deve usar a *sua*.

- rainbow **$ git remote add origin https://github.com/gitlearningjourney/rainbow-remote.git**

- rainbow **$ git push origin main**

  Enumerating objects: 4, done.

  Counting objects: 100% (4/4), done.

  Delta compression using up to 4 threads

  Compressing objects: 100% (3/3), done.

  Writing objects: 100% (4/4), 413 bytes | 413.00 KiB/s, done.

  Total 4 (delta 0), reused 0 (delta 0), pack-reused 0

  To github.com:gitlearningjourney/rainbow-remote.git

  * [new branch] main -> main

- rainbow **$ git branch -u origin/main**

  branch 'main' set up to track 'origin/main'.

- Abra uma nova janela de linha de comando para navegar para o diretório desktop a fim de simular que você é seu amigo agora.

- desktop **$ git clone https://github.com/gitlearningjourney/rainbow-remote.git friend-rainbow**

  Cloning into 'friend-rainbow'...

  remote: Enumerating objects: 4, done.

  remote: Counting objects: 100% (4/4), done.

  remote: Compressing objects: 100% (3/3), done.

  remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0

  Receiving objects: 100% (4/4), done.

- desktop **$ cd friend-rainbow**

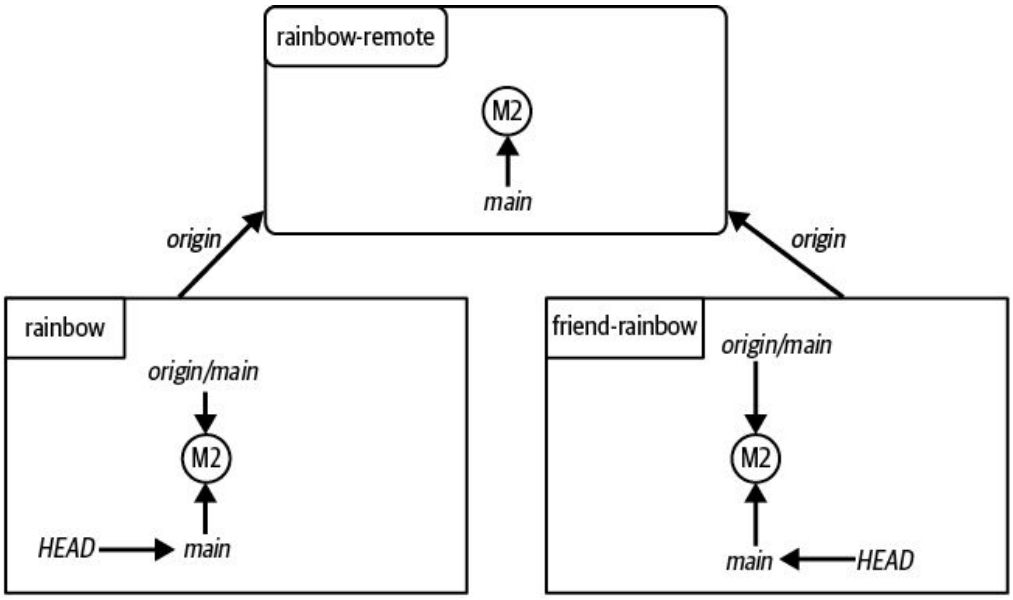
- Abra o diretório de projeto friend-rainbow em uma nova janela do editor de texto.

No fim desse processo, a configuração mínima do projeto Rainbow que você criou conterá apenas um commit, o commit de merge fictício 2 (M2), em todos os repositórios, como mostrado na seção Visualize A-8.

[NOTA]

O último commit criado no Capítulo 10 foi o commit de merge 2 (M2), que tem dois pais. Seu commit será o único existente nos repositórios; logo, não terá nenhum commit pai. Portanto, é um commit de merge fictício.

[VISUALIZE A-8]



Projeto Rainbow recriado para o trabalho começar no Capítulo 11, sem você ter passado pelo Capítulo 10.

Pré-requisitos de configuração do Capítulo 12

Na seção Acompanhe A-12, você preparará a configuração básica do projeto Rainbow para começar a trabalhar a partir do Capítulo 12. Nas etapas 15 e 19, certifique-se de inserir o comando inteiro em uma única linha.

[ACOMPANHE A-12]

❶ Use sua aplicação de linha de comando para abrir uma janela de linha de comando.

❷ **$ cd desktop**

❸ desktop **$ mkdir rainbow**

❹ desktop **$ cd rainbow**

❺ rainbow **$ git init -b main**

Initialized empty Git repository in /Users/annaskoulikari/desktop/rainbow/.git/

❻ Abra o diretório de projeto rainbow em uma janela do editor de texto e crie um arquivo chamado rainbowcolors.txt e outro arquivo chamado othercolors.txt dentro desse diretório.

❼ Adicione o texto a seguir ao arquivo rainbowcolors.txt em seu editor de texto e salve o arquivo:

Red is the first color of the rainbow.

Orange is the second color of the rainbow.

Yellow is the third color of the rainbow.

Green is the fourth color of the rainbow.

Blue is the fifth color of the rainbow.

Indigo is the sixth color of the rainbow.

Violet is the seventh color of the rainbow.

These are the colors of the rainbow.

❽ Adicione o texto a seguir ao arquivo othercolors.txt em seu editor de texto e salve o arquivo:

Brown is not a color in the rainbow.

Gray is not a color in the rainbow.

Black is not a color in the rainbow.

❾ rainbow **$ git add rainbowcolors.txt othercolors.txt**

❿ rainbow **$ git commit -m "rainbow"**

[main (root-commit) 56b92dc] "rainbow"

2 files changed, 11 insertions(+)

create mode 100644 othercolors.txt

create mode 100644 rainbowcolors.txt

☐ Se você ainda não tiver uma conta de serviço de hospedagem e/ou detalhes de autenticação definidos para o acesso HTTPS ou SSH, deve ir até o Capítulo 6 e concluir essa parte do processo agora. Volte a esta seção Acompanhe ao terminar.

☐ Faça login na conta de seu serviço de hospedagem.

☐ Crie um repositório remoto. Para obter mais informações sobre como fazer isso, acesse o repositório de *Aprendendo Git* (*https://github.com/gitlearningjourney/learning-git*) ou vá diretamente para a documentação de seu serviço de hospedagem.

Quando criar o repositório para esse exercício:

• Para o nome do repositório, use rainbow-remote.

• Você pode optar por tornar o repositório público ou privado. Recomento que o torne privado.

- Não inclua nenhum arquivo. Por exemplo, não inclua um arquivo README ou um arquivo .gitignore.
- Se for solicitado a fornecer um nome de branch padrão, deixe o campo em branco ou configure-o com main.

Quando terminar as etapas de criação do repositório remoto, localize a URL do repositório. Se não souber onde encontrá-la, consulte a documentação de seu serviço de hospedagem.

Existirão duas versões da URL, uma para o acesso HTTPS e a outra para o acesso SSH. Nos exemplos deste livro, as duas URLs do repositório remoto são:

- HTTPS: https://github.com/gitlearningjourney/rainbow-remote.git
- SSH: git@github.com:gitlearningjourney/rainbow-remote.git

Copie a URL do protocolo que escolheu usar. Nas próximas etapas, onde você encontrar a minha URL, em vez dela deve usar a *sua*.

rainbow **$ git remote add origin https://github.com/gitlearningjourney/rainbow-remote.git**

rainbow **$ git push origin main**

```
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 445 bytes | 445.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:gitlearningjourney/rainbow-remote.git
* [new branch] main -> main
```

rainbow **$ git branch -u origin/main**

```
branch 'main' set up to track 'origin/main'.
```

Abra uma nova janela de linha de comando para navegar para o diretório desktop a fim de simular que você é seu amigo agora.

desktop **$ git clone https://github.com/gitlearningjourney/rainbow-remote.git friend-rainbow**

```
Cloning into 'friend-rainbow'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0
Receiving objects: 100% (4/4), done.
```

desktop **$ cd friend-rainbow**

Abra o diretório de projeto friend-rainbow em uma nova janela do editor de texto.

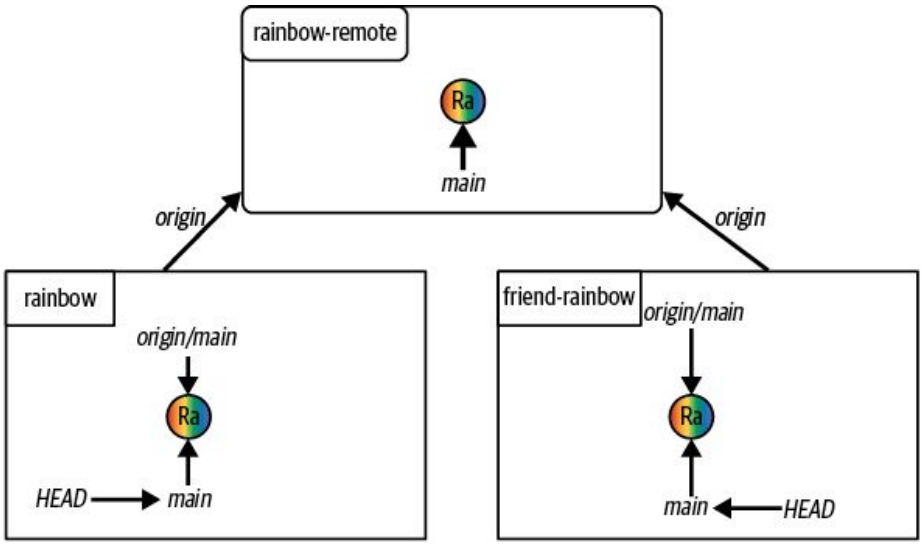
No fim desse processo, a configuração mínima do projeto Rainbow que você criou conterá apenas um commit, o commit rainbow (arco-íris), em todos os repositórios, como mostrado na seção Visualize A-9.

[NOTA]

O commit rainbow do começo do Capítulo 12 é o que foi produzido pelo rebasing, e é por isso que ele se chama Ra’ nos diagramas da seção Visualize daquele capítulo.

Seu commit rainbow no fim desse processo será um commit comum. Lembre-se disso quando trabalhar no capítulo. Consulte o Apêndice C para ver um resumo da linguagem visual do livro, se precisar recapitular.

[VISUALIZE A-9]



Projeto Rainbow recriado para o trabalho começar no Capítulo 12, sem você ter passado pelo Capítulo 11.

B

Consulta rápida de comandos

CAPÍTULO 1	
clear	Limpa a tela da linha de comando
pwd	Exibe o caminho do diretório de trabalho atual
ls	Lista os arquivos e diretórios visíveis
ls -a	Lista os arquivos e diretórios ocultos e visíveis
cd <*caminho_diretório*>	Muda de diretório
mkdir <*nome_diretório*>	Cria um diretório
git config --global --list	Lista as variáveis do arquivo de configuração global do Git e seus valores
git config --global user.name "<*nome*>"	Define seu nome no arquivo de configuração global do Git
git config --global user.email "<*email*>"	Define seu endereço de email no arquivo de configuração global do Git

CAPÍTULO 2	
git init	Inicializa um repositório Git
git init -b <*nome_branch*>	Inicializa um repositório Git e configura o nome do branch inicial com <*nome_branch*>

CAPÍTULO 3	
git status	Exibe o estado do diretório de trabalho e da área de preparação
git add <*nome_arquivo*>	Adiciona um arquivo à área de preparação
git add <*nome_arquivo*> <*nome_arquivo*> ...	Adiciona vários arquivos à área de preparação
git add -A	Adiciona à área de preparação todos os arquivos do diretório de trabalho que foram editados ou alterados
git commit -m "<*mensagem*>"	Cria um novo commit com uma mensagem
git log	Exibe uma lista de commits na ordem cronológica inversa

CAPÍTULO 4

CAPÍTULO 4	
git branch	Lista os branches locais
git branch <*nome_novo_branch*>	Cria um branch
git switch <*nome_branch*>	Muda de branch
git checkout <*nome_branch*>	Muda de branch

CAPÍTULO 5	
git merge <*nome_branch*>	Integra alterações de um branch em outro branch
git log --all	Exibe uma lista de commits na ordem cronológica inversa para todos os branches de um repositório local
git checkout <*hash_commit*>	Faz o checkout de um commit
git switch -c <*nome_novo_branch*>	Cria um novo branch e muda para ele
git checkout -b <*nome_novo_branch*>	Cria um novo branch e muda para ele

CAPÍTULO 7	
git push	Faz o upload de dados para um repositório remoto
git remote add <*nome_remoto*> <URL>	Adiciona uma conexão com um repositório remoto chamado <*nome_remoto*> em <*URL*>
git remote	Lista pelo *nome_remoto* as conexões com repositórios remotos armazenadas no repositório local
git remote -v	Lista as conexões com repositórios remotos existentes no repositório local com *nomes_remoto* e URLs
git push <*nome_remoto*> <*nome_branch*>	Faz o upload de conteúdo de <*nome_branch*> para o repositório remoto <*nome_remoto*>
git branch --all	Lista os branches locais e os branches de rastreamento remoto

CAPÍTULO 8	
git clone <URL> <*nome_diretório*>	Clona um repositório remoto
git push <*nome_remoto*> -d <*nome_branch*>	Exclui um branch remoto e o branch de rastreamento remoto associado
git branch -d <*nome_branch*>	Exclui um branch local
git branch -vv	Lista os branches locais e seus branches upstream, se houver algum

CAPÍTULO 8	
git fetch *<nome_remoto>*	Baixa dados do repositório remoto *<nome_remoto>*
git fetch	Baixa dados do repositório remoto com *nome_remoto* origin
git fetch -p	Remove os branches de rastreamento remoto correspondentes aos branches remotos excluídos e baixa dados do repositório remoto

CAPÍTULO 9	
git branch -u *<nome_remoto>*/*<nome_branch>*	Define um branch upstream para o branch local atual
git pull *<nome_remoto>* *<nome_branch>*	Faz o fetch de alterações do repositório remoto *<nome_remoto>* e as integra ao *<nome_branch>* especificado
git pull	Se um branch upstream tiver sido definido para o branch atual, faz o fetch das alterações do branch upstream e as integra

CAPÍTULO 10	
git merge --abort	Interrompe o processo de merging e retorna ao estado anterior ao processo

CAPÍTULO 11	
git rebase *<nome_branch>*	Reaplica commits a outro branch
git restore --staged *<nome_arquivo>*	Restaura um arquivo com outra versão na área de preparação
git rebase --continue	Continua com o processo de rebasing após a resolução de conflitos de merge
git rebase --abort	Interrompe o processo de rebasing e volta ao estado anterior ao processo

C

Referência sobre a linguagem visual

Commits

Nos diagramas deste livro, os commits são representados como círculos. Os commits comuns do projeto Rainbow têm cor sólida e contêm o nome completo ou uma abreviação do nome (a única exceção é o commit rainbow, que contém todas as cores do arco-íris). Consulte a Figura C-1 para ver um exemplo.



Figura C-1: Exemplo de commits comuns do projeto Rainbow usando cor e abreviações dos nomes dos commits para diferenciá-los uns dos outros.

Se um commit comum for recriado devido a uma operação de rebasing (discutido no Capítulo 11), adicionaremos um apóstrofo ao nome do commit recriado para diferenciá-lo do commit original. Consulte a Figura C-2 para ver um exemplo.



Figura C-2: Exemplo de commits produzidos por rebasing no projeto Rainbow, com apóstrofos nas abreviações para diferenciá-los dos commits originais.

No decorrer do livro, os commits de merge são representados por um círculo branco com borda preta grossa e a letra M (também podem incluir um número). Um exemplo pode ser visto na Figura C-3.



Figura C-3: Exemplo de um commit de merge.

Os commits comuns do projeto Book são representados por círculos azuis, com letras do alfabeto sendo usadas para diferenciá-los uns dos outros. Consulte a Figura C-4 para ver um exemplo.



Figura C-4: Exemplo de commits comuns do projeto Book usando letras do alfabeto para diferenciá-los uns dos outros.

Diagrama Git

O Diagrama Git, introduzido no Capítulo 2, representa um diretório de projeto do Git. Ele é composto das quatro áreas do Git: o diretório de trabalho, a área de preparação, o histórico de commits e o repositório local. O diretório de trabalho e a área de preparação podem conter arquivos, e o histórico de commits pode conter commits e branches. Os branches e o ponteiro (referência) HEAD são representados por setas pretas, e os links pais entre os commits são representados por setas cinza. Um exemplo de Diagrama Git é mostrado na Figura C-5.

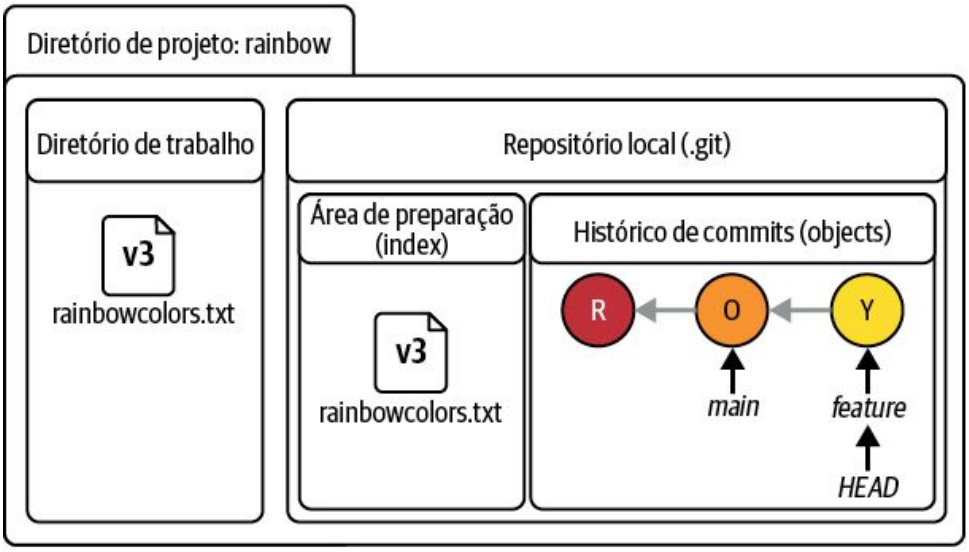


Figura C-5: Exemplo de um Diagrama Git com representações do diretório de trabalho, da área de preparação, do histórico de commits e do repositório local, mostrando o estado do projeto Rainbow no começo do Capítulo 5.

Diagrama Repositório

O Diagrama Repositório representa um ou vários repositórios. Os repositórios locais são sempre representados por retângulos comuns. Os repositórios remotos são representados por retângulos com cantos arredondados. Os repositórios podem conter commits e branches. O

Diagrama Repositório é usado do Capítulo 4 em diante.

Na Figura C-6, podemos ver um exemplo de um Diagrama Repositório com um repositório local.

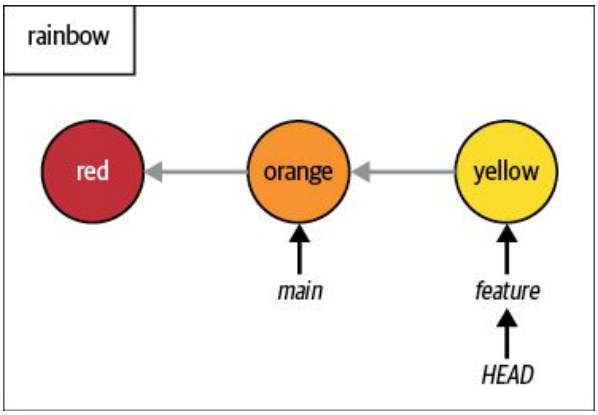


Figura C-6: Exemplo de um Diagrama Repositório com um repositório local, representando o estado do projeto Rainbow no fim do Capítulo 4.

Na Figura C-7, podemos ver um exemplo de um Diagrama Repositório com dois repositórios locais e um repositório remoto.

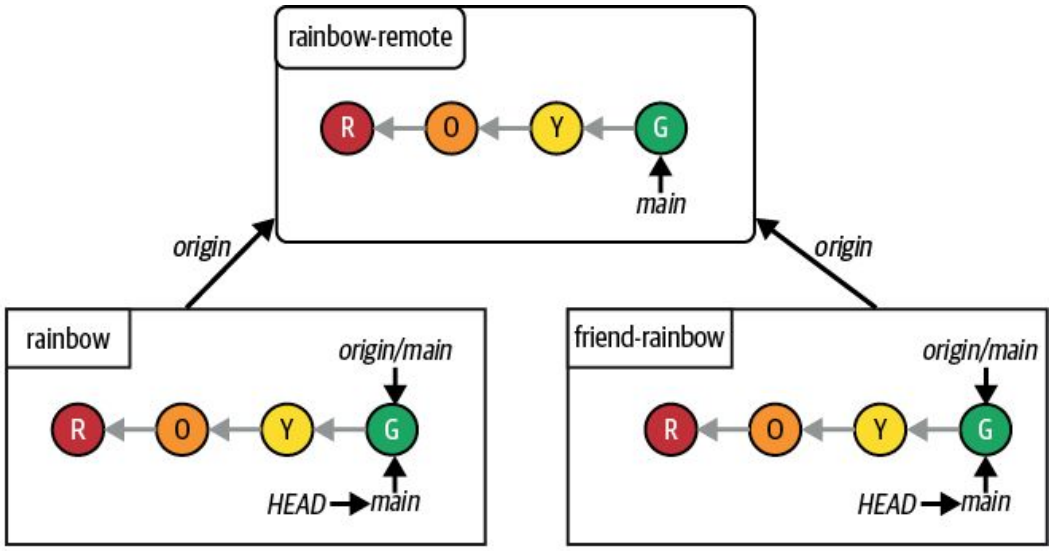


Figura C-7: Exemplo de um Diagrama Repositório com dois repositórios locais e um repositório remoto, representando o estado do projeto Rainbow no fim do Capítulo 8.

Sobre a autora

Anna Skoulikari é uma inovadora que tem usado suas habilidades em comunicação para ensinar o Git de maneira simples, prática e visual. Ela ensina a usar o Git utilizando vários tipos de mídia, o que inclui seu muito

bem avaliado curso online. No decorrer de sua jornada pela área de tecnologia, trabalhou como UX designer, desenvolvedora front-end e redatora técnica.

Colofão

O animal da capa de *Aprendendo Git* é um verdim-de-testa-dourada (*Chloropsis aurifrons*), pássaro que vive no subcontinente indiano, no sudoeste da China e no sudeste da Ásia.

Essa espécie de verdim é um pássaro pequeno e verde brilhante com a face e a garganta pretas e uma mancha laranja e dourada na testa. Bicos longos e afiados e línguas com pontas como um pincel os ajudam a se alimentar de insetos das cascas e folhas das árvores. Esse pássaro é comum em jardins e florestas tropicais, onde comem insetos, frutas e até mesmo – quando flutuam de forma estacionária como um beija-flor – néctar.

Trata-se de um pássaro muito territorial e agressivo ao lidar com outros pássaros, e é comum que ataque outros animais, incluindo humanos, caso se sinta ameaçado. O *Chloropsis aurifrons* se reproduz na primavera, construindo um pequeno ninho em forma de xícara com gravetos e folhas. A fêmea põe dois ou três ovos, que o macho choca por duas semanas. Os filhotes permanecem no ninho por mais duas semanas antes de ganhar penas.

Muitos dos animais das capas da O’Reilly estão em perigo; todos eles são importantes para o mundo. A ilustração da capa é de Karen Montgomery, baseada em uma gravura em preto e branco da obra *Royal Natural History* de Lydekker.


Entendendo
algoritmos
Um guia *ilustrado* para programadores e outros curiosos
Aditya Y. Bhargava
novatec
MANNING

Entendendo Algoritmos

Bhargava, Aditya Y.
9788575226629
264 páginas

Compre agora e leia

Um guia ilustrado para programadores e outros curiosos. Um algoritmo nada mais é do que um procedimento passo a passo para a resolução de um problema. Os algoritmos que você mais utilizará como um programador já foram descobertos, testados e provados. Se você quer entendê-los, mas se recusa a estudar páginas e mais páginas de provas, este é o livro certo. Este guia cativante e completamente ilustrado torna simples aprender como utilizar os principais algoritmos nos seus programas. O livro Entendendo Algoritmos apresenta uma abordagem agradável para esse tópico essencial da ciência da computação. Nele, você aprenderá como aplicar algoritmos comuns nos problemas de programação enfrentados diariamente. Você começará com tarefas básicas como a ordenação e a pesquisa. Com a prática, você enfrentará problemas mais complexos, como a compressão de dados e a inteligência artificial. Cada exemplo é apresentado em detalhes e inclui diagramas e códigos completos em Python. Ao final deste livro, você terá dominado algoritmos amplamente aplicáveis e saberá quando e onde utilizá-los. O que este livro inclui A abordagem de algoritmos de pesquisa, ordenação e algoritmos gráficos Mais de 400 imagens com descrições detalhadas Comparações de desempenho entre algoritmos Exemplos de código em Python Este livro de fácil leitura e repleto de imagens é destinado a programadores autodidatas, engenheiros ou pessoas que gostariam de recordar o assunto.

Compre agora e leia


O'REILLY
Introdução à linguagem
SQL
ABORDAGEM PRÁTICA
PARA INICIANTES
novatec
Thomas Nield

Introdução à Linguagem SQL

Nield, Thomas
9788575227466
144 páginas

Compre agora e leia

Atualmente as empresas estão coletando dados a taxas exponenciais e mesmo assim poucas pessoas sabem como acessá-los de maneira relevante. Se você trabalha em uma empresa ou é profissional de TI, este curto guia prático lhe ensinará como obter e transformar dados com o SQL de maneira significativa. Você dominará rapidamente os aspectos básicos do SQL e aprenderá como criar seus próprios bancos de dados. O autor Thomas Nield fornece exercícios no decorrer de todo o livro para ajudá-lo a praticar em casa suas recém descobertas aptidões no uso do SQL, sem precisar empregar um ambiente de servidor de banco de dados. Além de aprender a usar instruções-chave do SQL para encontrar e manipular seus dados, você descobrirá como projetar e gerenciar eficientemente bancos de dados que atendam às suas necessidades. Também veremos como: •Explorar bancos de dados relacionais, usando modelos leves e centralizados •Usar o SQLite e o SQLiteStudio para criar bancos de dados leves em minutos •Consultar e transformar dados de maneira significativa usando SELECT, WHERE, GROUP BY e ORDER BY •Associar tabelas para obter uma visualização mais completa dos dados da empresa •Construir nossas próprias tabelas e bancos de dados centralizados usando princípios de design normalizado •Gerenciar dados aprendendo como inserir, excluir e atualizar registros

Compre agora e leia


Loiane Groner
Estruturas de dados e algoritmos com JavaScript
2ª Edição
Escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript
novatec
Packt>

Estruturas de dados e algoritmos com JavaScript

Groner, Loiane
9788575227282
408 páginas

Compre agora e leia

Uma estrutura de dados é uma maneira particular de organizar dados em um computador com o intuito de usar os recursos de modo eficaz. As estruturas de dados e os algoritmos são a base de todas as soluções para qualquer problema de programação. Com este livro, você aprenderá a escrever códigos complexos e eficazes usando os recursos mais recentes da ES 2017. O livro Estruturas de dados e algoritmos com JavaScript começa abordando o básico sobre JavaScript e apresenta a ECMAScript 2017, antes de passar gradualmente para as estruturas de dados mais importantes, como arrays, filas, pilhas e listas ligadas. Você adquirirá um conhecimento profundo sobre como as tabelas hash e as estruturas de dados para conjuntos funcionam, assim como de que modo as árvores e os mapas hash podem ser usados para buscar arquivos em um disco rígido ou para representar um banco de dados. Este livro serve como um caminho para você mergulhar mais fundo no JavaScript. Você também terá uma melhor compreensão de como e por que os grafos – uma das estruturas de dados mais complexas que há – são amplamente usados em sistemas de navegação por GPS e em redes sociais. Próximo ao final do livro, você descobrirá como todas as teorias apresentadas podem ser aplicadas para solucionar problemas do mundo real, trabalhando com as próprias redes de computador e com pesquisas no Facebook. Você aprenderá a: • declarar, inicializar, adicionar e remover itens de arrays, pilhas e filas; • criar e usar listas ligadas, duplamente ligadas e ligadas circulares; • armazenar elementos únicos em tabelas hash, dicionários e conjuntos; • explorar o uso de árvores binárias e árvores binárias de busca; • ordenar estruturas de dados usando algoritmos como bubble sort, selection sort, insertion sort, merge sort e quick sort; •

pesquisar elementos em estruturas de dados usando ordenação sequencial e busca binária

Compre agora e leia


TERCEIRA EDIÇÃO
CURSO INTENSIVO DE PYTHON
UMA INTRODUÇÃO PRÁTICA E BASEADA EM PROJETOS À PROGRAMAÇÃO
ERIC MATTHES
novatec
no starch press

Curso Intensivo de Python - 3ª edição

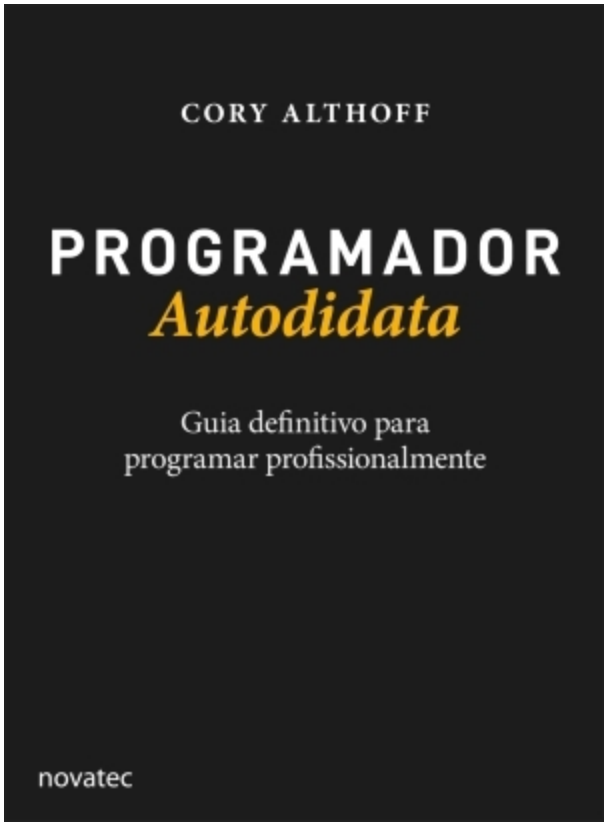
Matthes, Eric
9788575228449
624 páginas

Compre agora e leia

O Curso Intensivo de Python é o guia de programação Python mais vendido do mundo. Com essa introdução rápida e abrangente, você será capaz de escrever, programar, solucionar problemas e desenvolver aplicações que funcionam num piscar de olhos. Comece aprendendo os conceitos básicos de programação, como variáveis, listas, classes e loops, e pratique a escrita de código limpo com exercícios para cada tópico. Você também aprenderá como tornar seus programas interativos e testar seu código com segurança antes de adicioná-lo a um projeto. Assim, é possível colocar seus conhecimentos em prática, desenvolvendo um jogo arcade inspirado no Space Invaders, criando um conjunto de visualizações de dados com bibliotecas úteis do Python e implementando uma aplicação simples online. Com este livro, você aprenderá como: • Usar bibliotecas e ferramentas efetivas do Python, como pytest, Pygame, Matplotlib, Plotly e Django. • Criar jogos 2D cada vez mais complexos que respondam às teclas pressionadas e cliques do mouse. • Gerar visualizações de dados interativas usando uma variedade de conjuntos de dados. • Desenvolver aplicações que permitam aos usuários criar contas e gerenciar seus próprios dados e fazer deploy de aplicações online. • Solucionar erros de código e resolver problemas comuns de programação. Novidades desta edição: Esta terceira edição foi totalmente revisada para retratar o que há de mais recente em programação Python. A cobertura nova e atualizada inclui o VS Code para edição de texto, o módulo pathlib para manipulação de arquivos, pytest para testar seu código, bem como os recursos mais recentes da Matplotlib, do Plotly e do Django. Se você está pensando em se aprofundar na programação, com o Curso Intensivo de Python você terá as habilidades

para escrever programas rapidamente. Por que esperar mais? Apertem os cintos e vamos programar!

Compre agora e leia


CORY ALTHOFF
PROGRAMADOR
Autodidata
Guia definitivo para
programar profissionalmente
novatec

Programador Autodidata

Althoff, Cory
9788575228364
264 páginas

Compre agora e leia

Este livro não é apenas sobre aprender a programar. É sobre todas as outras coisas que você precisa saber que as aulas e os livros não ensinam. Trata-se de um roteiro, um guia que o levará de escrever seu primeiro programa a passar em sua primeira entrevista técnica. O livro foi dividido em cinco seções: 1. COMECE a programar em Python. 2. APRENDA a programação orientada a objetos e crie um programa poderoso para se sentir motivado. 3. APRENDA a usar ferramentas como o Git, o Bash e as expressões regulares. Em seguida, use suas novas habilidades de codificação para criar um web scraper. 4. ESTUDE os fundamentos da ciência da computação, como as estruturas de dados e os algoritmos. 5. TERMINE utilizando as melhores práticas de codificação, aprendendo dicas de como trabalhar em equipe e recebendo aconselhamento para conseguir uma vaga de emprego em programação.

Compre agora e leia